Índice

1. ## Introducción a tipos en `Python`

Python tiene soporte para "anotaciones de tipos" opcionales (también llamadas "anotaciones de tipos").

Estas **"anotaciones de tipos"** o anotaciones son una sintaxis especial que permite declarar el tipo de una variable.

Al declarar tipos para tus variables, los editores y herramientas te pueden proporcionar un mejor soporte.

Este es solo un **tutorial rápido / recordatorio** sobre las anotaciones de tipos en Python. Cubre solo lo mínimo necesario para usarlas con **FastAPI**, que en realidad es muy poco.

FastAPI se basa completamente en estas anotaciones de tipos, dándole muchas ventajas y beneficios.

Pero incluso si nunca usas **FastAPI**, te beneficiaría aprender un poco sobre ellas.

> **Nota**
> Si eres un experto en Python, y ya sabes todo sobre las anotaciones de tipos, salta al siguiente capítulo.

Motivación

Comencemos con un ejemplo simple:

```
def get_full_name(first_name, last_name):
    full_name = first_name.title() + " " + last_name.title()
    return full_name

print(get_full_name("john", "doe"))
```

Llamar a este programa genera:

```
John Doe
```

La función hace lo siguiente:

- Toma un `first_name` y `last_name`.
- Convierte la primera letra de cada uno a mayúsculas con `title()`.
- Concatena ambos con un espacio en el medio.

Edítalo

Es un programa muy simple.

Pero ahora imagina que lo escribieras desde cero.

En algún momento empiezas a definir la función, y tienes los parámetros listos...

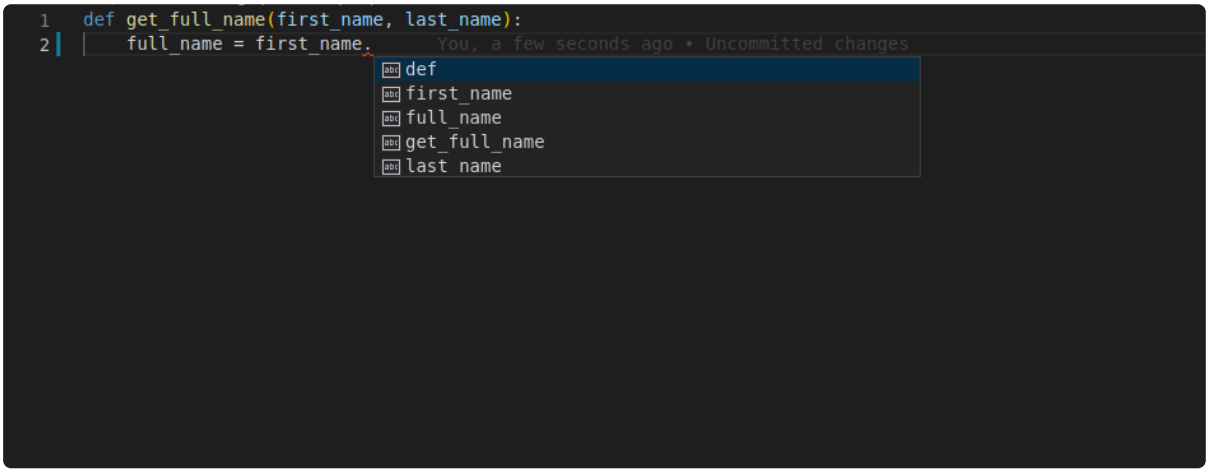
Pero luego tienes que llamar "ese método que convierte la primera letra a mayúscula".

¿Era `upper`? ¿Era `uppercase`? `first_uppercase`? `capitalize`?

Entonces, pruebas con el amigo del viejo programador, el autocompletado del editor.

Escribes el primer parámetro de la función, `first_name`, luego un punto (`.`) y luego presionas `Ctrl+Espacio` para activar el autocompletado.

Pero, tristemente, no obtienes nada útil:



Añade tipos

Modifiquemos una sola línea de la versión anterior.

Cambiaremos exactamente este fragmento, los parámetros de la función, de:

```
first_name, last_name
```

a:

```
first_name: str, last_name: str
```

Eso es todo.

Esas son las "anotaciones de tipos":

```
def get_full_name(first_name: str, last_name: str):
    full_name = first_name.title() + " " + last_name.title()
    return full_name

print(get_full_name("john", "doe"))
```

Eso no es lo mismo que declarar valores por defecto como sería con:

```
first_name="john", last_name="doe"
```

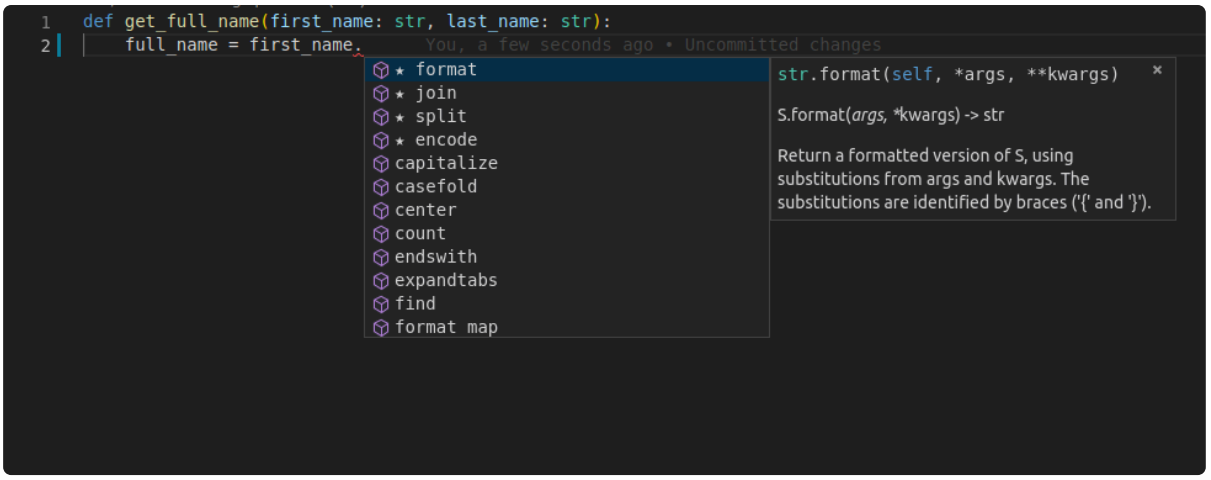
Es una cosa diferente.

Estamos usando dos puntos ":", no igualdades "=".

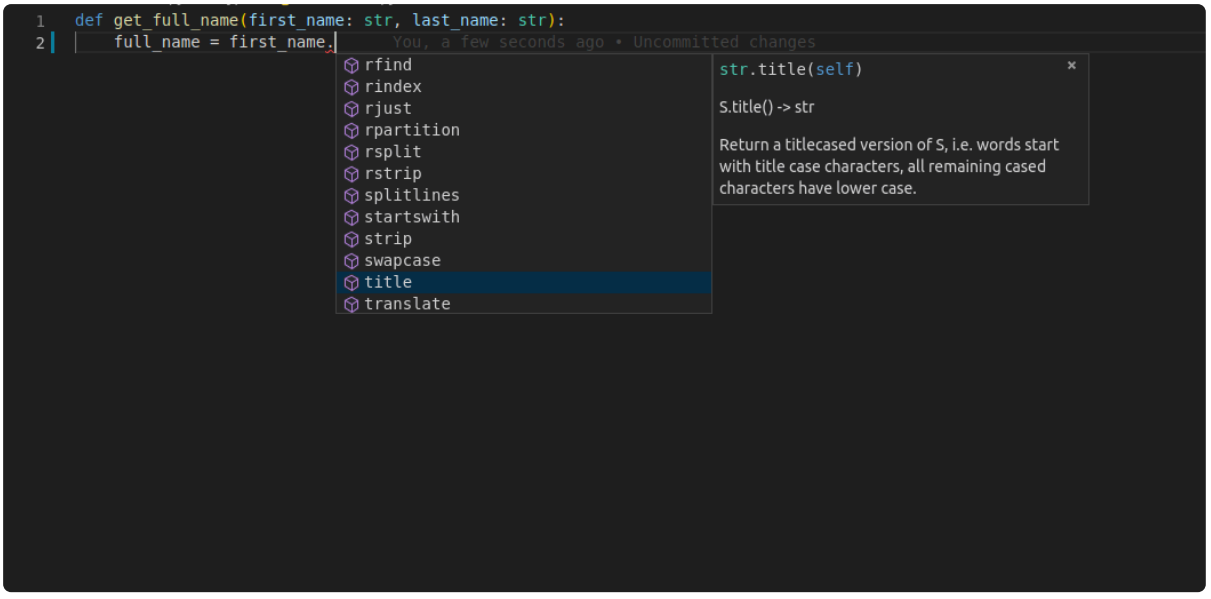
Y agregar anotaciones de tipos normalmente no cambia lo que sucede de lo que ocurriría sin ellas.

Pero ahora, imagina que nuevamente estás en medio de la creación de esa función, pero con anotaciones de tipos.

En el mismo punto, intentas activar el autocompletado con `Ctrl+Espacio` y ves:



Con eso, puedes desplazarte, viendo las opciones, hasta que encuentres la que "te suene":

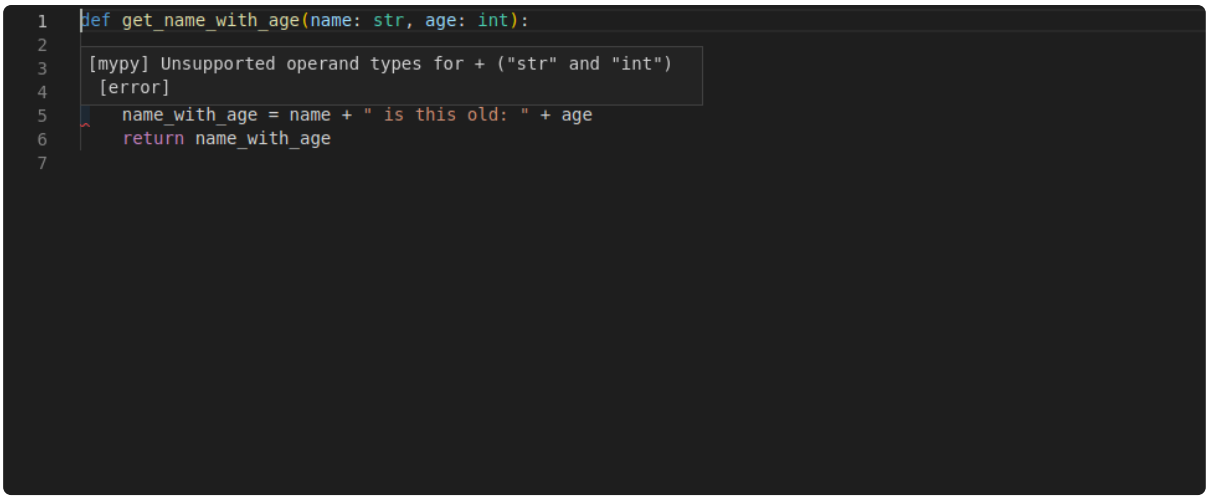


Más motivación

Revisa esta función, ya tiene anotaciones de tipos:

```
def get_name_with_age(name: str, age: int):
    name_with_age = name + " is this old: " + age
    return name_with_age
```

Porque el editor conoce los tipos de las variables, no solo obtienes autocompletado, también obtienes chequeo de errores:



Ahora sabes que debes corregirlo, convertir `age` a un `string` con `str(age)`:

```
def get_name_with_age(name: str, age: int):
    name_with_age = name + " is this old: " + str(age)
    return name_with_age
```

Declaración de tipos

Acabas de ver el lugar principal para declarar anotaciones de tipos. Como parámetros de función.

Este también es el lugar principal donde las utilizarías con **FastAPI**.

Tipos simples

Puedes declarar todos los tipos estándar de Python, no solo `str`.

Puedes usar, por ejemplo:

- `int`
- `float`
- `bool`
- `bytes`

```
def get_items(item_a: str, item_b: int, item_c: float, item_d:
bool, item_e: bytes):
    return item_a, item_b, item_c, item_d, item_e
```

Módulo typing

Para algunos casos adicionales, podrías necesitar importar algunas cosas del módulo `typing` del paquete estándar, por ejemplo cuando quieres declarar que algo tiene "cualquier tipo", puedes usar `Any` de `typing`:

```
from typing import Any

def some_function(data: Any):
    print(data)
```

Tipos genéricos

Algunos tipos pueden tomar "parámetros de tipo" entre corchetes, para definir sus tipos internos, por ejemplo una "list de strings" se declararía `list[str]`.

Estos tipos que pueden tomar parámetros de tipo se llaman **Tipos Genéricos** o **Genéricos**.

Puedes usar los mismos tipos integrados como genéricos (con corchetes y tipos dentro):

- `list`
- `tuple`
- `set`
- `dict`

List

Por ejemplo, vamos a definir una variable para ser una `list` de `str`.

Declara la variable, con la misma sintaxis de dos puntos (`:`).

Como tipo, pon `list`.

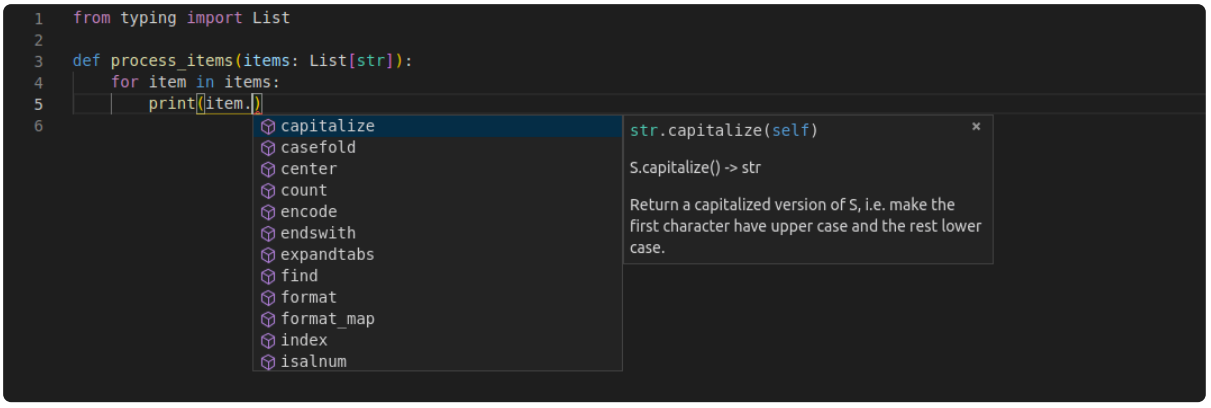
Como la `list` es un tipo que contiene algunos tipos internos, los pones entre corchetes:

```python
def process_items(items: list[str]):
    for item in items:
        print(item)
```

> **Nota**
> Esos tipos internos en los corchetes se denominan "parámetros de tipo".
>
> En este caso, `str` es el parámetro de tipo pasado a `list`.

Eso significa: "la variable `items` es una `list`, y cada uno de los ítems en esta `list` es un `str` ".

Al hacer eso, tu editor puede proporcionar soporte incluso mientras procesa elementos de la `list`:



Sin tipos, eso es casi imposible de lograr.

Nota que la variable `item` es uno de los elementos en la `list` `items`.

Y aún así, el editor sabe que es un `str` y proporciona soporte para eso.

Tuple y Set

Harías lo mismo para declarar `tuple` s y `set` s:

```python
def process_items(items_t: tuple[int, int, str], items_s:
set[bytes]):
    return items_t, items_s
```

Esto significa:

- La variable `items_t` es un `tuple` con 3 ítems, un `int`, otro `int`, y un `str`.
- La variable `items_s` es un `set`, y cada uno de sus ítems es del tipo `bytes`.

Dict

Para definir un `dict`, pasas 2 parámetros de tipo, separados por comas.

El primer parámetro de tipo es para las claves del `dict`.

El segundo parámetro de tipo es para los valores del `dict`:

```
def process_items(prices: dict[str, float]):
    for item_name, item_price in prices.items():
        print(item_name)
        print(item_price)
```

Esto significa:

- La variable `prices` es un `dict`:
- Las claves de este `dict` son del tipo `str` (digamos, el nombre de cada ítem).
- Los valores de este `dict` son del tipo `float` (digamos, el precio de cada ítem).

Union

Puedes declarar que una variable puede ser cualquiera de **varios tipos**, por ejemplo, un `int` o un `str`.

Para definirlo usas la barra vertical (`|`) para separar ambos tipos.

Esto se llama una "unión", porque la variable puede ser cualquiera en la unión de esos dos conjuntos de tipos.

```python
def process_item(item: int | str):
    print(item)
```

Esto significa que `item` podría ser un `int` o un `str`.

Posiblemente None

Puedes declarar que un valor podría tener un tipo, como `str`, pero que también podría ser `None`.

```python
def say_hi(name: str | None = None):
    if name is not None:
        print(f"Hey {name}!")
    else:
        print("Hello World")
```

Usar `str | None` en lugar de solo `str` te permitirá al editor ayudarte a detectar errores donde podrías estar asumiendo que un valor siempre es un `str`, cuando en realidad también podría ser `None`.

Clases como tipos

También puedes declarar una clase como el tipo de una variable.

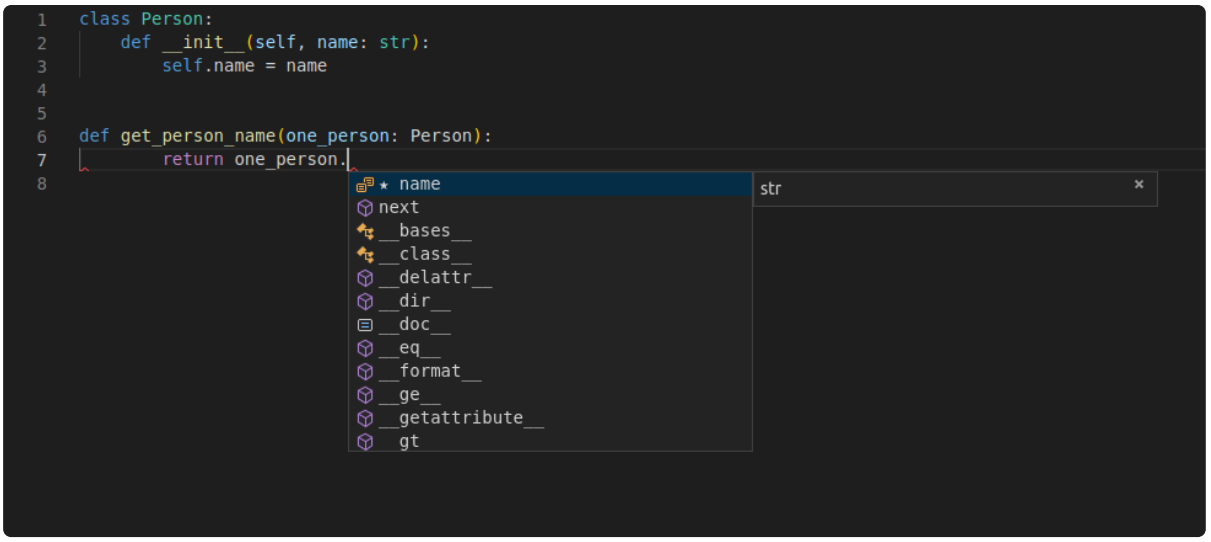
Digamos que tienes una clase `Person`, con un nombre:

```python
class Person:
    def __init__(self, name: str):
        self.name = name
```

Luego puedes declarar una variable para que sea de tipo `Person`:

```python
def get_person_name(one_person: Person):
    return one_person.name
```

Y luego, nuevamente, obtienes todo el soporte del editor:



Nota que esto significa " `one_person` es una **instance** de la clase `Person` ".

No significa " `one_person` es la **clase** llamada `Person` ".

Modelos Pydantic

Pydantic es un paquete de Python para realizar la validación de datos.

Declaras la "forma" de los datos como clases con atributos.

Y cada atributo tiene un tipo.

Entonces creas un instancia de esa clase con algunos valores y validará los valores, los convertirá al tipo adecuado (si es el caso) y te dará un objeto con todos los datos.

Y obtienes todo el soporte del editor con ese objeto resultante.

Un ejemplo de la documentación oficial de Pydantic:

```
from datetime import datetime

from pydantic import BaseModel

class User(BaseModel):
    id: int
    name: str = "John Doe"
    signup_ts: datetime | None = None
    friends: list[int] = []
```

```
external_data = {
    "id": "123",
    "signup_ts": "2017-06-01 12:22",
    "friends": [1, "2", b"3"],
}
user = User(**external_data)
print(user)
# > User id=123 name='John Doe'
signup_ts=datetime.datetime(2017, 6, 1, 12, 22) friends=[1, 2,
3]
print(user.id)
# > 123
```

> **Nota**
> Para saber más sobre Pydantic, revisa su documentación.

FastAPI está completamente basado en Pydantic.

Verás mucho más de todo esto en práctica en el Tutorial - Guía del Usuario.

Anotaciones de tipos con metadata

Python también tiene una funcionalidad que permite poner **metadata/información adicional** en estas anotaciones de tipos usando `Annotated`.

Puedes importar `Annotated` desde `typing`.

```
from typing import Annotated

def say_hello(name: Annotated[str, "this is just metadata"]) -
> str:
    return f"Hello {name}"
```

Python en sí no hace nada con este `Annotated`. Y para los editores y otras herramientas, el tipo sigue siendo `str`.

Pero puedes usar este espacio en `Annotated` para proporcionar a **FastAPI** información adicional sobre cómo quieres que se comporte tu aplicación.

Lo importante a recordar es que **el primer *parámetro de tipo*** que pasas a `Annotated` es el **tipo real**. El resto es solo información para otras herramientas.

Por ahora, solo necesitas saber que `Annotated` existe, y que es Python estándar.

Luego verás lo **poderoso** que puede ser.

> **Consejo**
> El hecho de que esto sea **Python estándar** significa que seguirás obteniendo la **mejor experiencia de desarrollador posible** en tu editor, con las herramientas que usas para analizar y refactorizar tu código, etc.
>
> Y también que tu código será muy compatible con muchas otras herramientas y paquetes de Python.

Anotaciones de tipos en FastAPI

FastAPI aprovecha estas anotaciones de tipos para hacer varias cosas.

Con **FastAPI** declaras parámetros con anotaciones de tipos y obtienes:

- **Soporte del editor**.
- **Chequeo de tipos**.

...y **FastAPI** usa las mismas declaraciones para:

- **Definir requerimientos**: de parámetros de path de la request, parámetros de query, headers, bodies, dependencias, etc.
- **Convertir datos**: de la request al tipo requerido.
- **Validar datos**: provenientes de cada request:

- Generando **errores automáticos** devueltos al cliente cuando los datos son inválidos.

- **Documentar** la API usando OpenAPI:
  - Que luego es usada por las interfaces de documentación interactiva automática.

Todo esto puede sonar abstracto. No te preocupes. Verás todo esto en acción en el Tutorial - Guía del Usuario.

Lo importante es que al usar tipos estándar de Python, en un solo lugar (en lugar de agregar más clases, decoradores, etc.), **FastAPI** hará gran parte del trabajo por ti.

> **Nota**
> Si ya revisaste todo el tutorial y volviste para ver más sobre tipos, un buen recurso es la "cheat sheet" de `mypy`.

2. ## Concurrencia y `async/await`

Detalles sobre la sintaxis `async def` para *`path operation functions`* y algunos antecedentes sobre el código asíncrono, la concurrencia y el paralelismo.

¿Con prisa?

TL;DR:

Si estás usando paquetes de terceros que te dicen que los llames con `await`, como:

```
results = await some_library()
```

Entonces, declara tus *`path operation functions`* con `async def` así:

```
@app.get('/')
async def read_results():
    results = await some_library()
    return results
```

> **Nota**
> Solo puedes usar `await` dentro de funciones creadas con `async def`.

Si estás usando un paquete de terceros que se comunica con algo (una base de datos, una `API`, el sistema de archivos, etc.) y no tiene soporte para usar `await` (este es actualmente el caso para la mayoría de los paquetes de base de datos), entonces declara tus *`path operation functions`* como normalmente, usando simplemente `def`, así:

```
@app.get('/')
def results():
    results = some_library()
    return results
```

Si tu aplicación (de alguna manera) no tiene que comunicarse con nada más y esperar a que responda, usa `async def`, incluso si no necesitas usar `await` dentro.

Si simplemente no lo sabes, usa `def` normal.

Nota: Puedes mezclar `def` y `async def` en tus *`path operation functions`* tanto como necesites y definir cada una utilizando la mejor opción para ti. `FastAPI` hará lo correcto con ellas.

De todos modos, en cualquiera de los casos anteriores, `FastAPI` seguirá funcionando de forma asíncrona y será extremadamente rápido.

Pero al seguir los pasos anteriores, podrá hacer algunas optimizaciones de rendimiento.

Detalles Técnicos

Las versiones modernas de `Python` tienen soporte para **"código asíncrono"** utilizando algo llamado **"`coroutines`"**, con la sintaxis **`async` y `await`**.

Veamos esa frase por partes en las secciones a continuación:

- **Código Asíncrono**
- `async` y `await`
- **Coroutines**

Código Asíncrono

El código asíncrono simplemente significa que el lenguaje tiene una forma de decirle a la computadora / programa que en algún momento del código, tendrá que esperar que *otra cosa* termine en otro lugar. Digamos que esa *otra cosa* se llama "archivo-lento".

Entonces, durante ese tiempo, la computadora puede ir y hacer algún otro trabajo, mientras "archivo-lento" termina.

Luego la computadora/programa volverá cada vez que tenga una oportunidad porque está esperando nuevamente, o siempre que haya terminado todo el trabajo que tenía en ese punto. Y comprobará si alguna de las tareas que estaba esperando ya se han completado, haciendo lo que tenía que hacer.

Después, el programa toma la primera tarea que termine (digamos, nuestro "archivo-lento") y continúa con lo que tenía que hacer con ella.

Ese "esperar otra cosa" normalmente se refiere a las operaciones de `I/O` que son relativamente "lentas" (comparadas con la velocidad del procesador y la memoria `RAM`), como esperar:

- que los datos del cliente se envíen a través de la red
- que los datos enviados por tu programa sean recibidos por el cliente a través de la red
- que el contenido de un archivo en el disco sea leído por el sistema y entregado a tu programa
- que el contenido que tu programa entregó al sistema sea escrito en el disco
- una operación de `API` remota
- que una operación de base de datos termine

- que una query de base de datos devuelva los resultados
- etc.

Como el tiempo de ejecución se consume principalmente esperando operaciones de `I/O`, las llaman operaciones "`I/O bound`".

Se llama "asíncrono" porque la computadora / programa no tiene que estar "sincronizado" con la tarea lenta, esperando el momento exacto en que la tarea termine, sin hacer nada, para poder tomar el resultado de la tarea y continuar el trabajo.

En lugar de eso, al ser un sistema "asíncrono", una vez terminado, la tarea puede esperar un poquito en la cola (algunos microsegundos) para que la computadora / programa termine lo que salió a hacer, y luego regrese para tomar los resultados y continuar trabajando con ellos.

Para el "sincrónico" (contrario al "asíncrono") comúnmente también usan el término "secuencial", porque la computadora / programa sigue todos los pasos en secuencia antes de cambiar a una tarea diferente, incluso si esos pasos implican esperar.

Concurrencia y Hamburguesas

Esta idea de código **asíncrono** descrita anteriormente a veces también se llama **"concurrencia"**. Es diferente del **"paralelismo"**.

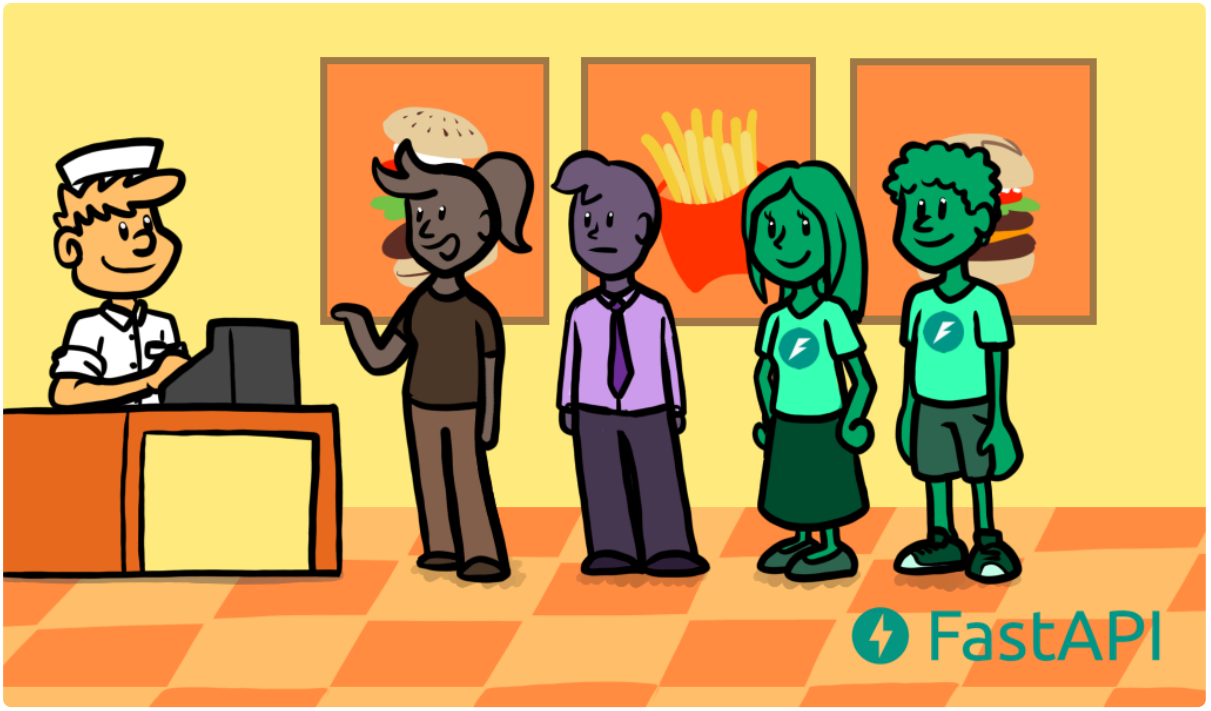
Concurrencia y **paralelismo** ambos se relacionan con "diferentes cosas sucediendo más o menos al mismo tiempo".

Pero los detalles entre *concurrencia* y *paralelismo* son bastante diferentes.

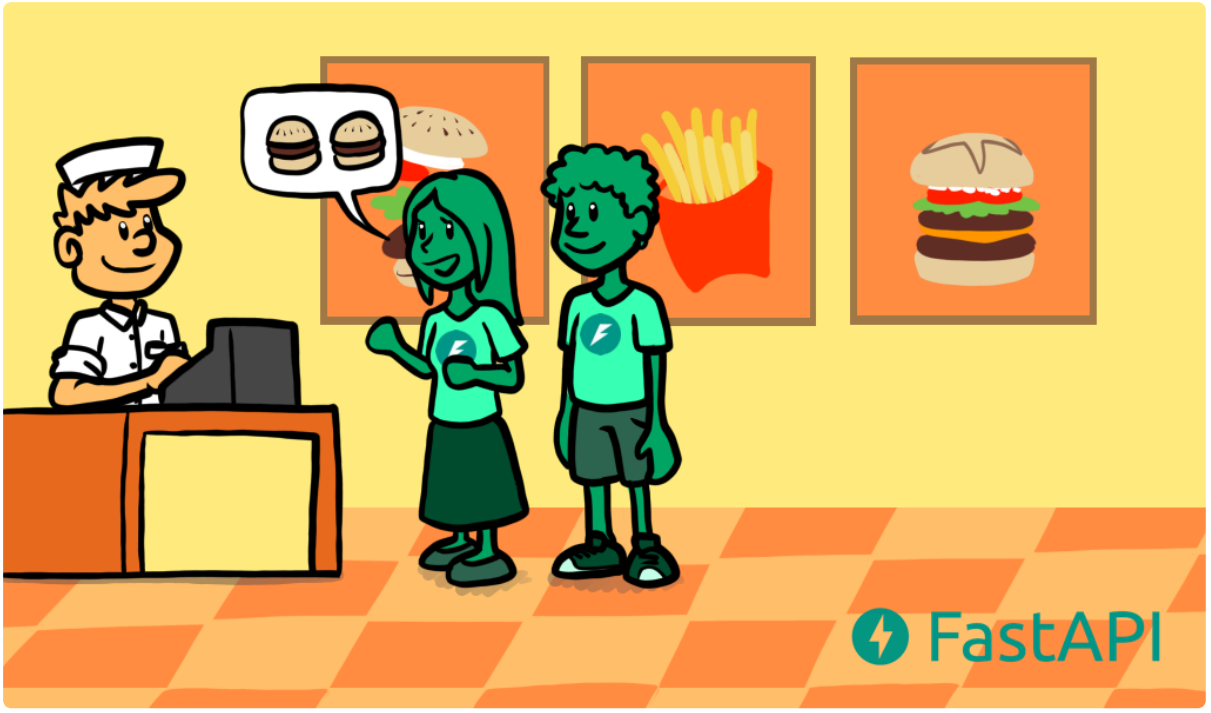
Para ver la diferencia, imagina la siguiente historia sobre hamburguesas:

Hamburguesas Concurrentes

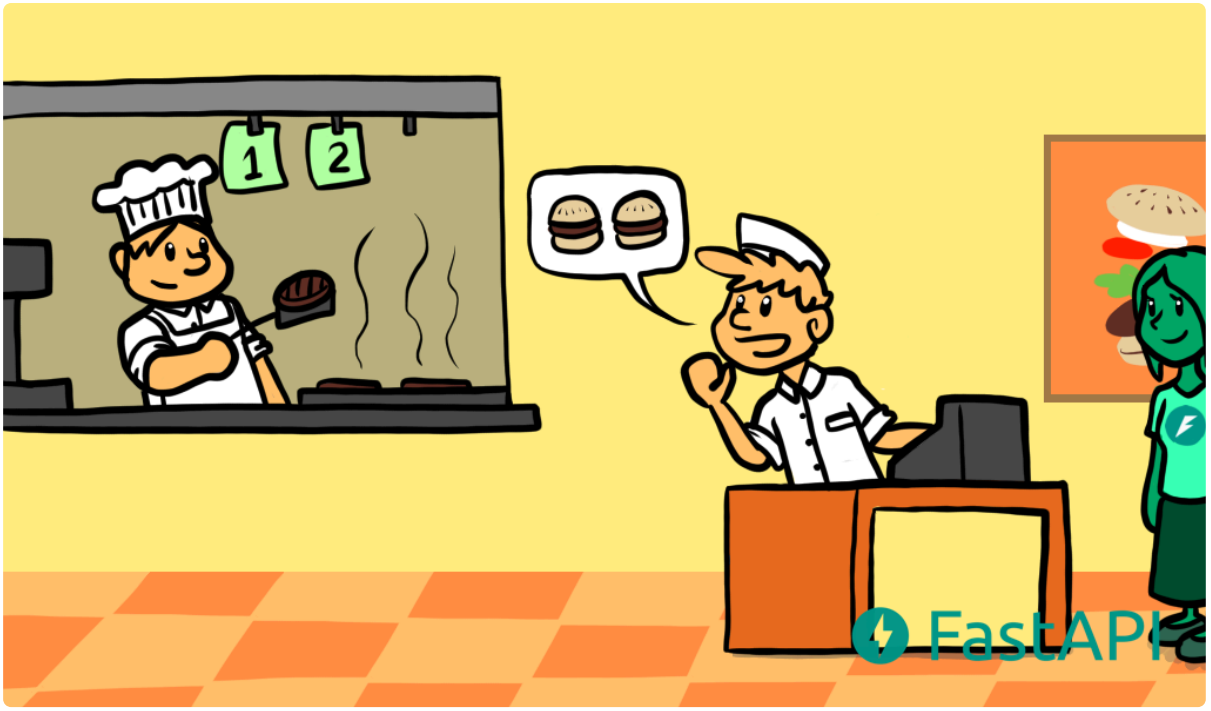
Vas con tu `crush` a conseguir comida rápida, te pones en fila mientras el cajero toma los pedidos de las personas frente a ti.



Luego es tu turno, haces tu pedido de 2 hamburguesas muy sofisticadas para tu `crush` y para ti.

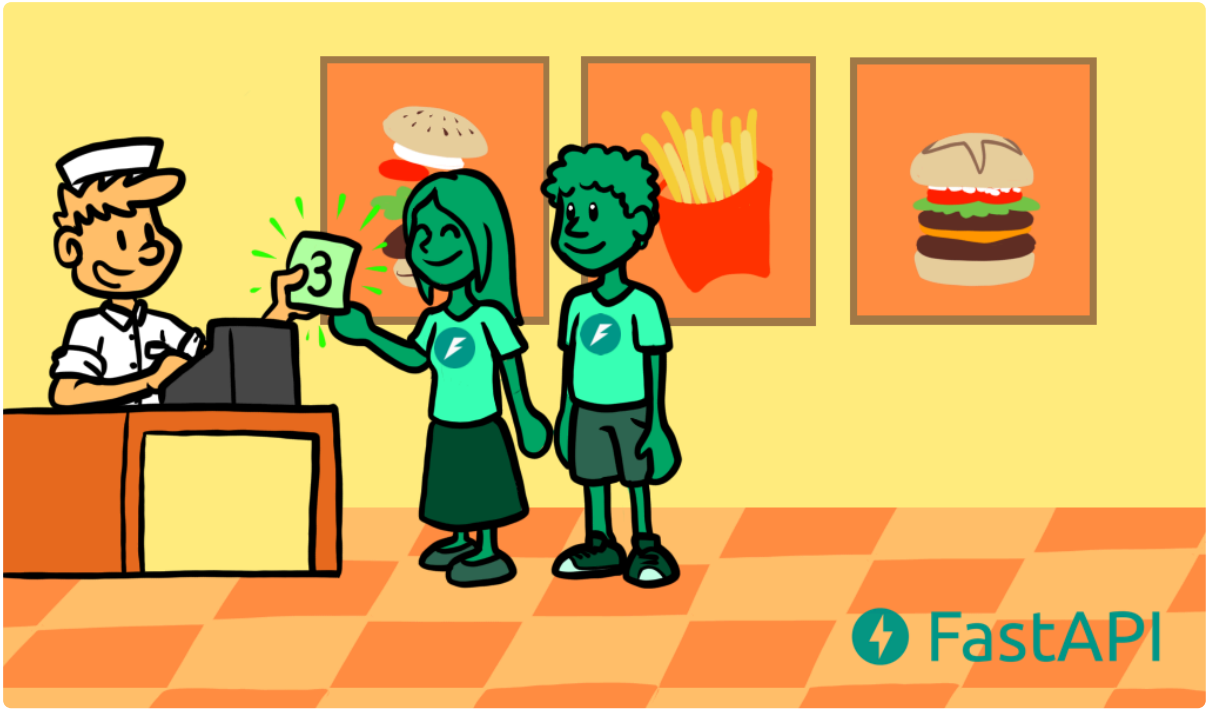


El cajero dice algo al cocinero en la cocina para que sepan que tienen que preparar tus hamburguesas (aunque actualmente están preparando las de los clientes anteriores).



Pagas.

El cajero te da el número de tu turno.



Mientras esperas, vas con tu `crush` y eliges una mesa, te sientas y hablas con tu `crush` por un largo rato (ya que tus hamburguesas son muy sofisticadas y toman un tiempo en prepararse).

Mientras estás sentado en la mesa con tu `crush`, mientras esperas las hamburguesas, puedes pasar ese tiempo admirando lo increíble, lindo e inteligente que es tu `crush`

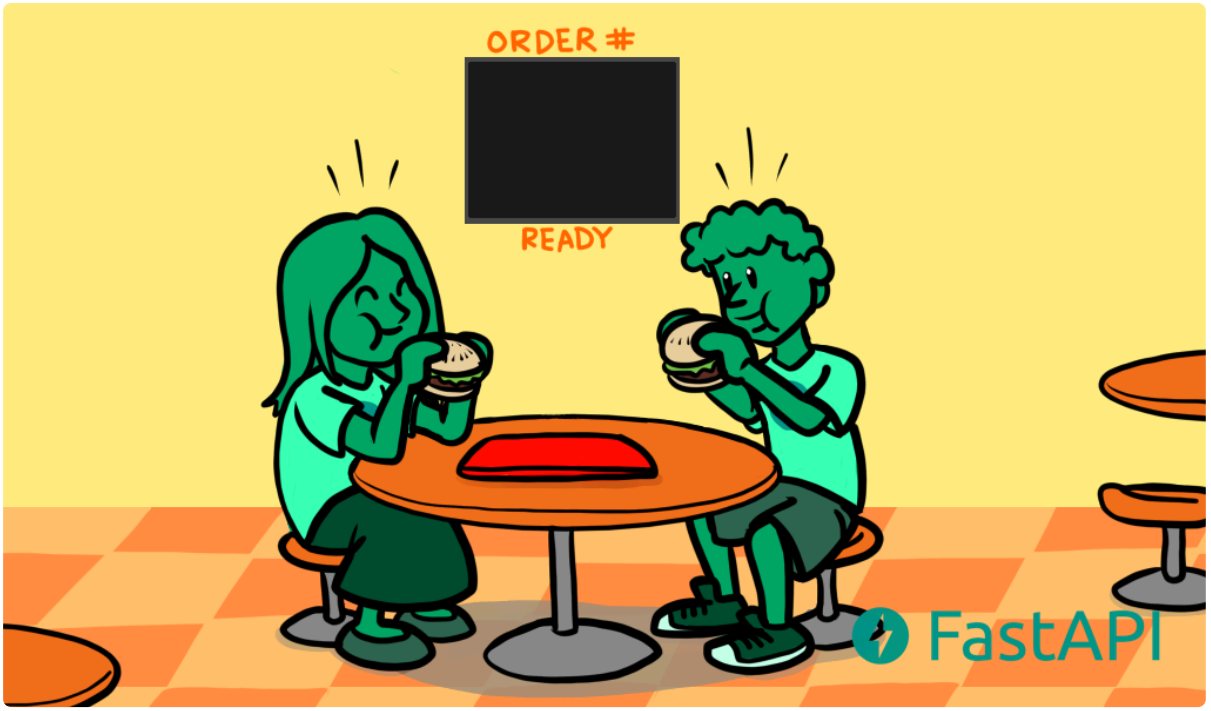


Mientras esperas y hablas con tu `crush`, de vez en cuando revisas el número mostrado en el mostrador para ver si ya es tu turno.

Luego, en algún momento, finalmente es tu turno. Vas al mostrador, obtienes tus hamburguesas y vuelves a la mesa.



Tú y tu `crush` comen las hamburguesas y pasan un buen rato.



Imagina que eres la computadora/programa en esa historia.

Mientras estás en la fila, estás inactivo, esperando tu turno, sin hacer nada muy "productivo". Pero la fila es rápida porque el cajero solo está tomando los pedidos (no preparándolos), así que está bien.

Luego, cuando es tu turno, haces un trabajo realmente "productivo", procesas el menú, decides lo que quieres, obtienes la elección de tu `crush`, pagas, revisas que das el billete o tarjeta correctos, revisas que te cobren correctamente, revisas que el pedido tenga los artículos correctos, etc.

Pero luego, aunque todavía no tienes tus hamburguesas, tu trabajo con el cajero está "en pausa", porque tienes que esperar a que tus hamburguesas estén listas.

Pero como te alejas del mostrador y te sientas en la mesa con un número para tu turno, puedes cambiar tu atención a tu `crush`, y "trabajar" en eso. Luego, nuevamente estás haciendo algo muy "productivo" como es coquetear con tu `crush`.

Luego el cajero dice "he terminado de hacer las hamburguesas" al poner tu número en el mostrador, pero no saltas como loco inmediatamente cuando el número mostrado cambia a tu número de

turno. Sabes que nadie robará tus hamburguesas porque tienes el número de tu turno, y ellos tienen el suyo.

Así que esperas a que tu `crush` termine la historia (termine el trabajo/tarea actual que se está procesando), sonríes amablemente y dices que vas por las hamburguesas.

Luego vas al mostrador, a la tarea inicial que ahora está terminada, recoges las hamburguesas, das las gracias y las llevas a la mesa. Eso termina ese paso/tarea de interacción con el mostrador. Eso a su vez, crea una nueva tarea, de "comer hamburguesas", pero la anterior de "obtener hamburguesas" ha terminado.

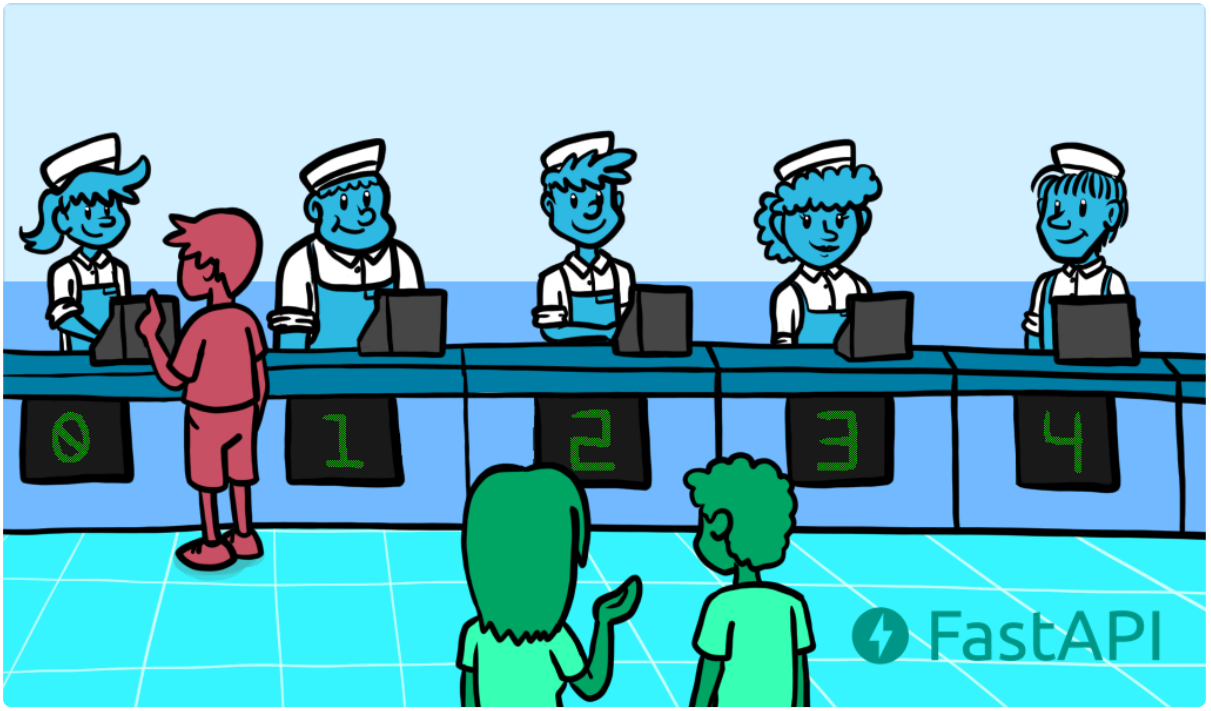
Hamburguesas Paralelas

Ahora imaginemos que estas no son "Hamburguesas Concurrentes", sino "Hamburguesas Paralelas".

Vas con tu `crush` a obtener comida rápida paralela.

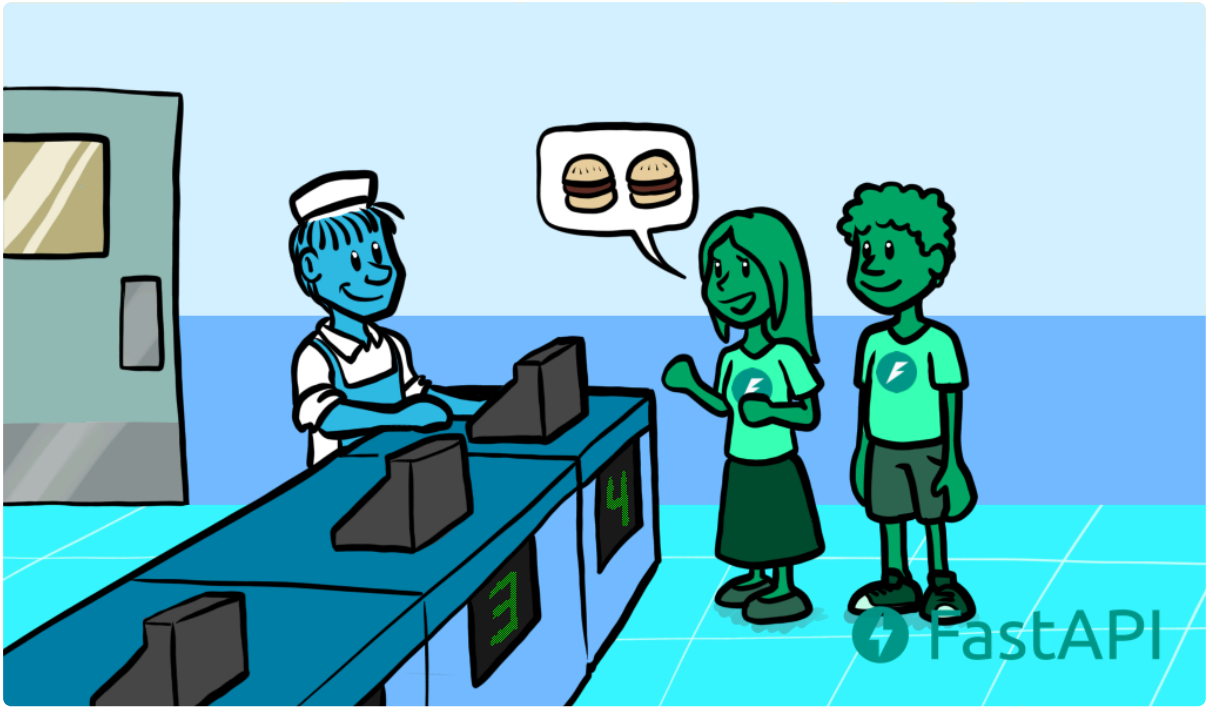
Te pones en fila mientras varios (digamos 8) cajeros que al mismo tiempo son cocineros toman los pedidos de las personas frente a ti.

Todos antes que tú están esperando a que sus hamburguesas estén listas antes de dejar el mostrador porque cada uno de los 8 cajeros va y prepara la hamburguesa de inmediato antes de obtener el siguiente pedido.



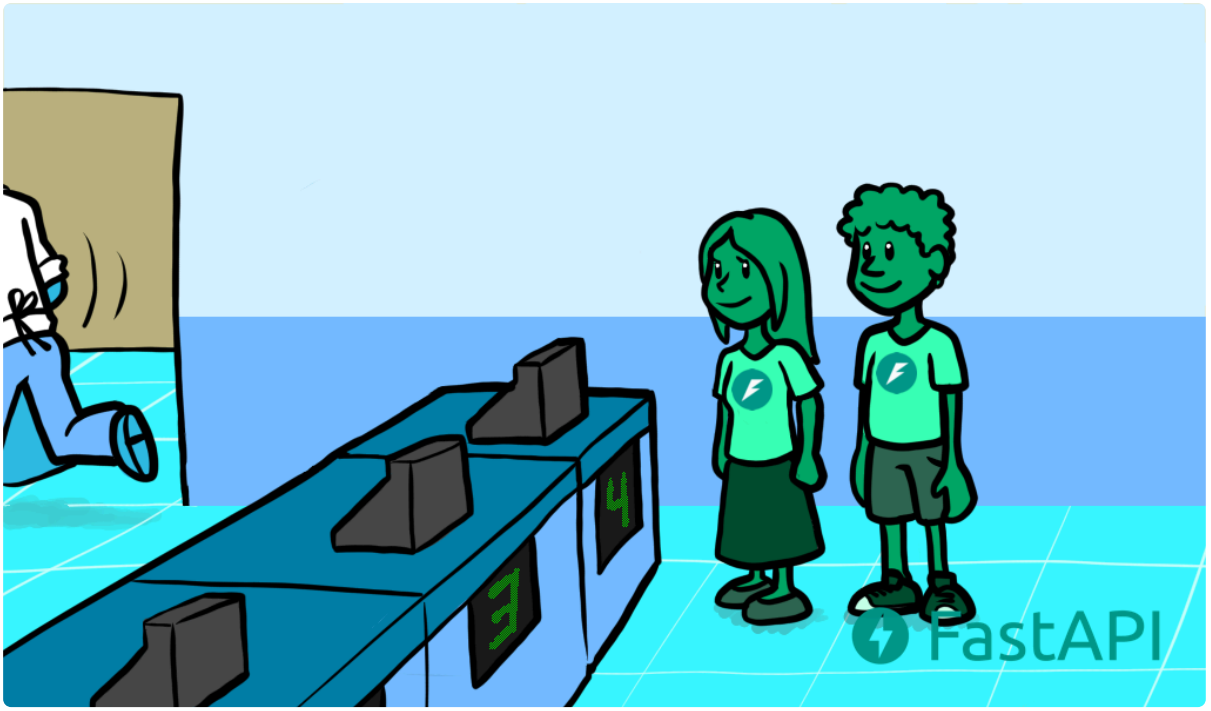
Luego, finalmente es tu turno, haces tu pedido de 2 hamburguesas muy sofisticadas para tu `crush` y para ti.

Pagas.



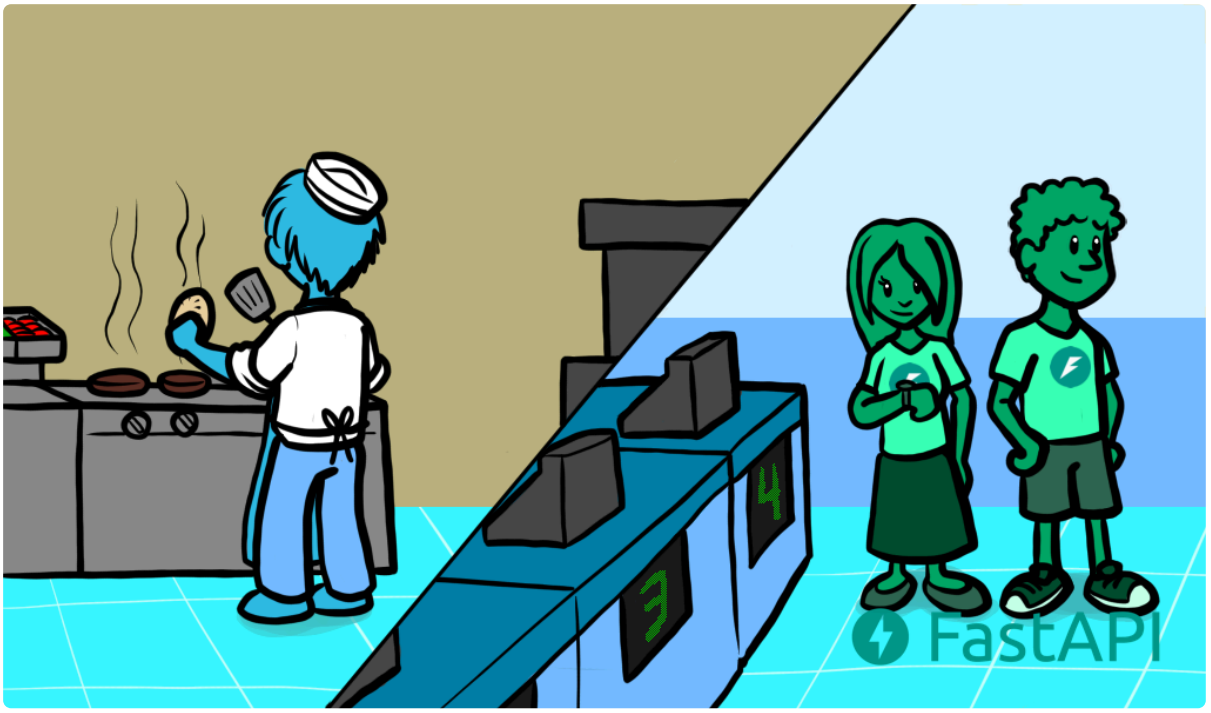
El cajero va a la cocina.

Esperas, de pie frente al mostrador, para que nadie más tome tus hamburguesas antes que tú, ya que no hay números para los turnos.

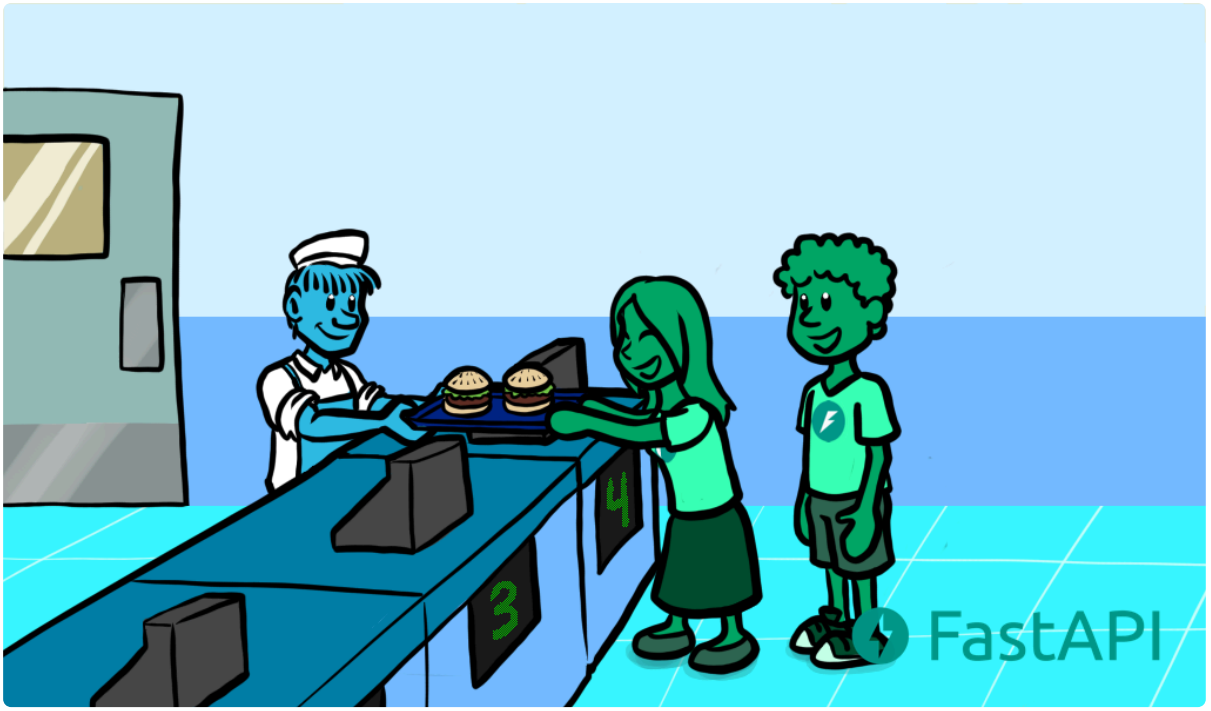


Como tú y tu `crush` están ocupados no dejando que nadie se interponga y tome tus hamburguesas cuando lleguen, no puedes prestar atención a tu `crush`.

Este es un trabajo "sincrónico", estás "sincronizado" con el cajero/cocinero. Tienes que esperar y estar allí en el momento exacto en que el cajero/cocinero termine las hamburguesas y te las entregue, o de lo contrario, alguien más podría tomarlas.



Luego tu cajero/cocinero finalmente regresa con tus hamburguesas, después de mucho tiempo esperando allí frente al mostrador.



Tomas tus hamburguesas y vas a la mesa con tu `crush`.

Simplemente las comes, y has terminado.



No hubo mucho hablar o coquetear ya que la mayor parte del tiempo se dedicó a esperar frente al mostrador.

En este escenario de las hamburguesas paralelas, eres una computadora/programa con dos procesadores (tú y tu `crush`), ambos

esperando y dedicando su atención a estar "esperando en el mostrador" por mucho tiempo.

La tienda de comida rápida tiene 8 procesadores (cajeros/cocineros). Mientras que la tienda de hamburguesas concurrentes podría haber tenido solo 2 (un cajero y un cocinero).

Pero aún así, la experiencia final no es la mejor.

Esta sería la historia equivalente de las hamburguesas paralelas.

Para un ejemplo más "de la vida real" de esto, imagina un banco.

Hasta hace poco, la mayoría de los bancos tenían múltiples cajeros y una gran fila.

Todos los cajeros haciendo todo el trabajo con un cliente tras otro.

Y tienes que esperar en la fila por mucho tiempo o pierdes tu turno.

Probablemente no querrías llevar a tu `crush` contigo a hacer trámites en el banco.

Conclusión de las Hamburguesas

En este escenario de "hamburguesas de comida rápida con tu `crush`", como hay mucha espera, tiene mucho más sentido tener un sistema concurrente.

Este es el caso para la mayoría de las aplicaciones web.

Muchos, muchos usuarios, pero tu servidor está esperando su conexión no tan buena para enviar sus `requests`.

Y luego esperar nuevamente a que los `responses` retornen.

Esta "espera" se mide en microsegundos, pero aún así, sumándolo todo, es mucha espera al final.

Por eso tiene mucho sentido usar código asíncrono para las `APIs web`.

Este tipo de asincronía es lo que hizo popular a `NodeJS` (aunque `NodeJS` no es paralelo) y esa es la fortaleza de `Go` como lenguaje de programación.

Y ese es el mismo nivel de rendimiento que obtienes con `FastAPI`.

Y como puedes tener paralelismo y asincronía al mismo tiempo, obtienes un mayor rendimiento que la mayoría de los `frameworks` de `NodeJS` probados y a la par con `Go`, que es un lenguaje compilado más cercano a `C` (todo gracias a `Starlette`).

¿Es la concurrencia mejor que el paralelismo?

¡No! Esa no es la moraleja de la historia.

La concurrencia es diferente del paralelismo. Y es mejor en escenarios **específicos** que implican mucha espera. Debido a eso, generalmente es mucho mejor que el paralelismo para el desarrollo de aplicaciones web. Pero no para todo.

Así que, para equilibrar eso, imagina la siguiente historia corta:

> Tienes que limpiar una casa grande y sucia.

Sí, esa es toda la historia.

No hay esperas en ninguna parte, solo mucho trabajo por hacer, en múltiples lugares de la casa.

Podrías tener turnos como en el ejemplo de las hamburguesas, primero la sala de estar, luego la cocina, pero como no estás esperando nada, solo limpiando y limpiando, los turnos no afectarían nada.

Tomaría la misma cantidad de tiempo terminar con o sin turnos (concurrencia) y habrías hecho la misma cantidad de trabajo.

Pero en este caso, si pudieras traer a los 8 ex-cajeros/cocineros/ahora-limpiadores, y cada uno de ellos (más tú) pudiera tomar una zona de la

casa para limpiarla, podrías hacer todo el trabajo en **paralelo**, con la ayuda extra, y terminar mucho antes.

En este escenario, cada uno de los limpiadores (incluyéndote) sería un procesador, haciendo su parte del trabajo.

Y como la mayor parte del tiempo de ejecución se dedica al trabajo real (en lugar de esperar), y el trabajo en una computadora lo realiza una `CPU`, llaman a estos problemas "`CPU bound`".

Ejemplos comunes de operaciones limitadas por la `CPU` son cosas que requieren procesamiento matemático complejo.

Por ejemplo:

- **Procesamiento de audio** o **imágenes**.
- **Visión por computadora**: una imagen está compuesta de millones de píxeles, cada píxel tiene 3 valores / colores, procesar eso normalmente requiere calcular algo en esos píxeles, todos al mismo tiempo.
- `Machine Learning`: normalmente requiere muchas multiplicaciones de "matrices" y "vectores". Piensa en una enorme hoja de cálculo con números y multiplicando todos juntos al mismo tiempo.
- `Deep Learning`: este es un subcampo de `Machine Learning`, por lo tanto, se aplica lo mismo. Es solo que no hay una sola hoja de cálculo de números para multiplicar, sino un enorme conjunto de ellas, y en muchos casos, usas un procesador especial para construir y / o usar esos modelos.

Concurrencia + Paralelismo: Web + `Machine Learning`

Con `FastAPI` puedes aprovechar la concurrencia que es muy común para el desarrollo web (la misma atracción principal de `NodeJS`).

Pero también puedes explotar los beneficios del paralelismo y la multiprocesamiento (tener múltiples procesos ejecutándose en

paralelo) para cargas de trabajo **`CPU bound`** como las de los sistemas de `Machine Learning`.

Eso, más el simple hecho de que `Python` es el lenguaje principal para **`Data Science`**, `Machine Learning` y especialmente `Deep Learning`, hacen de `FastAPI` una muy buena opción para `APIs` web de `Data Scienc`e / `Machine Learning` y aplicaciones (entre muchas otras).

Para ver cómo lograr este paralelismo en producción, consulta la sección sobre Despliegue.

`async` y `await`

Las versiones modernas de `Python` tienen una forma muy intuitiva de definir código asíncrono. Esto hace que se vea igual que el código "secuencial" normal y hace el "wait" por ti en los momentos adecuados.

Cuando hay una operación que requerirá esperar antes de dar los resultados y tiene soporte para estas nuevas funcionalidades de `Python`, puedes programarlo así:

```
burgers = await get_burgers(2)
```

La clave aquí es el `await`. Dice a `Python` que tiene que esperar a que `get_burgers(2)` termine de hacer su cosa antes de almacenar los resultados en `burgers`. Con eso, `Python` sabrá que puede ir y hacer algo más mientras tanto (como recibir otro `request`).

Para que `await` funcione, tiene que estar dentro de una función que soporte esta asincronía. Para hacer eso, solo declara la función con `async def`:

```
async def get_burgers(number: int):
    # Hacer algunas cosas asíncronas para crear las
hamburguesas
    return burgers
```

...en lugar de `def`:

```
# Esto no es asíncrono
def get_sequential_burgers(number: int):
    # Hacer algunas cosas secuenciales para crear las
hamburguesas
    return burgers
```

Con `async def`, `Python` sabe que, dentro de esa función, tiene que estar atento a las expresiones `await`, y que puede "pausar" la ejecución de esa función e ir a hacer algo más antes de regresar.

Cuando deseas llamar a una función `async def`, tienes que "await" dicha función. Así que, esto no funcionará:

```
# Esto no funcionará, porque get_burgers fue definido con:
async def
burgers = get_burgers(2)
```

Así que, si estás usando un paquete que te dice que puedes llamarlo con `await`, necesitas crear las *`path operation functions`* que lo usen con `async def`, como en:

```
@app.get('/burgers')
async def read_burgers():
    burgers = await get_burgers(2)
    return burgers
```

Más detalles técnicos

Podrías haber notado que `await` solo se puede usar dentro de funciones definidas con `async def`.

Pero al mismo tiempo, las funciones definidas con `async def` deben ser "`awaited`". Por lo tanto, las funciones con `async def` solo se pueden llamar dentro de funciones definidas con `async def` también.

Entonces, sobre el huevo y la gallina, ¿cómo llamas a la primera función `async`?

Si estás trabajando con **`FastAPI`** no tienes que preocuparte por eso, porque esa "primera" función será tu *`path operation function`*, y `FastAPI` sabrá cómo hacer lo correcto.

Pero si deseas usar `async` / `await` sin `FastAPI`, también puedes hacerlo.

Escribe tu propio código `async`

`Starlette` (y **`FastAPI`**) están basados en `AnyIO`, lo que lo hace compatible tanto con el asyncio del paquete estándar de `Python` como con `Trio`.

En particular, puedes usar directamente `AnyIO` para tus casos de uso avanzados de concurrencia que requieran patrones más avanzados en tu propio código.

E incluso si no estuvieras usando `FastAPI`, también podrías escribir tus propias aplicaciones asíncronas con `AnyIO` para ser altamente compatibles y obtener sus beneficios (p.ej. *concurrencia estructurada*).

Creé otro paquete sobre `AnyIO`, como una capa delgada, para mejorar un poco las anotaciones de tipos y obtener mejor **autocompletado**, **errores en línea**, etc. También tiene una introducción amigable y tutorial para ayudarte a **entender** y escribir **tu propio código** **`async`**: `Asyncer`. Sería particularmente útil si necesitas **combinar código** **`async`** **con regular** (bloqueante/sincrónico).

Otras formas de código asíncrono

Este estilo de usar `async` y `await` es relativamente nuevo en el lenguaje.

Pero hace que trabajar con código asíncrono sea mucho más fácil.

Esta misma sintaxis (o casi idéntica) también se incluyó recientemente en las versiones modernas de `JavaScript` (en el Navegador y

`NodeJS`).

Pero antes de eso, manejar el código asíncrono era mucho más complejo y difícil.

En versiones previas de `Python` , podrías haber usado hilos o `Gevent` . Pero el código es mucho más complejo de entender, depurar y razonar.

En versiones previas de `NodeJS` / `JavaScript` en el Navegador, habrías usado " `callbacks` ". Lo que lleva al " `callback hell` ".

`Coroutines`

`Coroutine` es simplemente el término muy elegante para la cosa que devuelve una función `async def` . `Python` sabe que es algo parecido a una función, que puede comenzar y que terminará en algún momento, pero que podría pausar ⏸ internamente también, siempre que haya un `await` dentro de él.

Pero toda esta funcionalidad de usar código asíncrono con `async` y `await` a menudo se resume como utilizar " `coroutines` ". Es comparable a la funcionalidad clave principal de `Go` , las " `Goroutines` ".

Conclusión

Veamos la misma frase de arriba:

> Las versiones modernas de `Python` tienen soporte para **"código asíncrono"** utilizando algo llamado **" `coroutines` "**, con la sintaxis **`async`** y **`await`** .

Eso debería tener más sentido ahora.

Todo eso es lo que impulsa `FastAPI` (a través de `Starlette`) y lo que hace que tenga un rendimiento tan impresionante.

Detalles Muy Técnicos

⚠ **Advertencia**
Probablemente puedas saltarte esto.

Estos son detalles muy técnicos de cómo `FastAPI` funciona en su interior.

Si tienes bastante conocimiento técnico (`coroutines` , hilos, bloqueo, etc.) y tienes curiosidad sobre cómo `FastAPI` maneja `async def` vs `def` normal, adelante.

`Path operation functions`

Cuando declaras una *`path operation function`* con `def` normal en lugar de `async def` , se ejecuta en un `threadpool` externo que luego es esperado, en lugar de ser llamado directamente (ya que bloquearía el servidor).

Si vienes de otro `framework` `async` que no funciona de la manera descrita anteriormente y estás acostumbrado a definir *`path operation functions`* solo de cómputo trivial con `def` normal para una pequeña ganancia de rendimiento (alrededor de 100 nanosegundos), ten en cuenta que en `FastAPI` el efecto sería bastante opuesto. En estos casos, es mejor usar `async def` a menos que tus *`path operation functions`* usen código que realice `I/O` de bloqueo.

Aun así, en ambas situaciones, es probable que `FastAPI` siga siendo más rápida que (o al menos comparable a) tu `framework` anterior.

Dependencias

Lo mismo aplica para las dependencias. Si una dependencia es una función estándar `def` en lugar de `async def` , se ejecuta en el `threadpool` externo.

Sub-dependencias

Puedes tener múltiples dependencias y sub-dependencias requiriéndose mutuamente (como parámetros de las definiciones de

funciones), algunas de ellas podrían ser creadas con `async def` y algunas con `def` normal. Aun funcionará, y las que fueron creadas con `def` normal serían llamadas en un hilo externo (del `threadpool`) en lugar de ser "`awaited`".

Otras funciones de utilidad

Cualquier otra función de utilidad que llames directamente puede ser creada con `def` normal o `async def` y `FastAPI` no afectará la forma en que la llames.

Esto contrasta con las funciones que `FastAPI` llama por ti: *`path operation functions`* y dependencias.

Si tu función de utilidad es una función normal con `def`, será llamada directamente (como la escribas en tu código), no en un `threadpool`; si la función es creada con `async def` entonces deberías "`await`" por esa función cuando la llames en tu código.

Nuevamente, estos son detalles muy técnicos que probablemente serían útiles si los buscaste.

De lo contrario, deberías estar bien con las pautas de la sección anterior: ¿Con prisa?.

3. **Tutorial - Guía del Usuario**

Este tutorial te muestra cómo usar `FastAPI` con la mayoría de sus funcionalidades, paso a paso.

Cada sección se basa gradualmente en las anteriores, pero está estructurada para separar temas, de manera que puedas ir directamente a cualquier sección específica para resolver tus necesidades específicas de `API`.

También está diseñado para funcionar como una referencia futura para que puedas volver y ver exactamente lo que necesitas.

Ejecuta el código

Todos los bloques de código pueden ser copiados y usados directamente (de hecho, son archivos `Python` probados).
Para ejecutar cualquiera de los ejemplos, copia el código a un archivo `main.py`, y comienza `fastapi dev` con `uv run`:

```
uv run fastapi dev
```

```
   FastAPI   Starting development server 🚀

             Searching for package file structure from
directories
             with __init__.py files
             Importing from /home/user/code/awesomeapp

    module   🐍 main.py

      code   Importing the FastAPI app object from the module
with
             the following code:

             from main import app

       app   Using import string: main:app

    server   Server started at http://127.0.0.1:8000
    server   Documentation at http://127.0.0.1:8000/docs

       tip   Running in development mode, for production use:
             fastapi run

             Logs:

      INFO   Will watch for changes in these directories:
             ['/home/user/code/awesomeapp']
      INFO   Uvicorn running on http://127.0.0.1:8000 (Press
CTRL+C
             to quit)
      INFO   Started reloader process [383138] using
WatchFiles
      INFO   Started server process [383153]
```

```
        INFO   Waiting for application startup.
        INFO   Application startup complete.
```

Es **ALTAMENTE recomendable** que escribas o copies el código, lo edites y lo ejecutes localmente.

Usarlo en tu editor es lo que realmente te muestra los beneficios de `FastAP`, al ver cuán poco código tienes que escribir, todo el chequeo de tipos, autocompletado, etc.

Instalar `FastAPI`

El primer paso es configurar tu proyecto y añadir `FastAPI`.

Instala `uv`, luego crea un proyecto y añade `FastAPI`:

```
uv init awesome-project --bare
cd awesome-project
uv add "fastapi[standard]"
████████████████████████████████████████ 100%
```

`uv add` crea el entorno virtual del proyecto en `.venv`, añade `FastAPI` a `pyproject.toml`, y crea `uv.lock` para que se puedan instalar las mismas versiones de paquetes más adelante.

Qué hacen estos comandos

- `uv init`: crea un nuevo proyecto Python.
- `awesome-project`: crea el proyecto en un nuevo directorio con este nombre.
- `--bare`: crea solo el archivo mínimo `pyproject.toml`, sin generar un `main.py`, `README.md`, u otros archivos de ejemplo. Tú crearás los archivos de la aplicación en los siguientes pasos de este tutorial.

Luego `cd awesome-project` entra al nuevo directorio del proyecto antes de añadir `FastAPI`.

`uv` usará una versión compatible de `Python` ya instalada en tu sistema, o descargará una si es necesario.

Cuando ejecutas `uv add`, selecciona versiones compatibles de `FastAPI` y todos los paquetes de los que depende `FastAPI`. Registra las versiones exactas en `uv.lock`, haciendo posible instalar las mismas versiones de paquetes más adelante en otra computadora o al hacer el despliegue de la aplicación.

Crear o actualizar este archivo se llama hacer **locking** de las dependencias del proyecto. `uv` hace esto automáticamente cuando añades un paquete.

Opciones de instalación de `FastAPI`

Cuando instalas con `uv add "fastapi[standard]"` viene con algunas dependencias opcionales estándar por defecto, incluyendo `fastapi-cloud-cli`, que te permite hacer despliegues en `FastAPI Cloud`.

Si no quieres tener esas dependencias opcionales, en su lugar puedes instalar `uv add fastapi`.

Si quieres instalar las dependencias estándar pero sin `fastapi-cloud-cli`, puedes instalar con `uv add "fastapi[standard-no-fastapi-cloud-cli]"`.

Usar `pip` en su lugar

Si prefieres gestionar un entorno virtual y paquetes manualmente, crea y activa un entorno virtual y luego instala `FastAPI` con `pip install "fastapi[standard]"`.

Lee la guía de Entornos Virtuales para ver los pasos detallados.

Habilidades de agentes de `IA`

`FastAPI` incluye una habilidad oficial para agentes de programación con `IA`. Viene incluida con el paquete, por lo que su guía se mantiene alineada con la versión de `FastAPI` instalada en tu proyecto y se actualiza cuando actualizas `FastAPI`.

Después de instalar `FastAPI` en tu proyecto, puedes instalar la habilidad con `Library Skills`:

```
uvx library-skills
```

> **Nota**
> `uvx` es un alias de `uv tool run`. Ejecuta `Library Skills` en un entorno temporal y aislado mientras `Library Skills` escanea los paquetes instalados en tu proyecto.

La habilidad es compatible con `Codex`, `Claude Code`, `Cursor`, `GitHub Copilot`, `Gemini CLI`, `Pi`, `OpenCode`, y la mayoría de otros agentes de programación. Para `Claude Code`, selecciona `.claude/skills` cuando se te pregunte dónde instalar la habilidad.

Guía avanzada del usuario

También hay una **Guía Avanzada del Usuario** que puedes leer después de este **Tutorial - Guía del Usuario**.

La **Guía Avanzada del Usuario** se basa en esta, utiliza los mismos conceptos y te enseña algunas funcionalidades adicionales.

Pero primero deberías leer la **Tutorial - Guía del Usuario** (lo que estás leyendo ahora mismo).

Está diseñada para que puedas construir una aplicación completa solo con la **Tutorial - Guía del Usuario**, y luego extenderla de diferentes maneras, dependiendo de tus necesidades, utilizando algunas de las ideas adicionales de la **Guía Avanzada del Usuario**.

- # Primeros pasos

  El archivo `FastAPI` más simple podría verse así:

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
```

```python
async def root():
    return {"message": "Hello World"}
```

Copia eso en un archivo `main.py`.

> **Consejo**
> `FastAPI` tiene una extensión oficial para VS Code (y `Cursor`), que proporciona muchas funcionalidades, incluyendo un explorador de `path operations`, búsqueda de `path operations`, navegación `CodeLens` en tests (saltar a la definición desde los tests), y despliegue y logs de `FastAPI Cloud`, todo desde tu editor.

Ejecuta el servidor en vivo:

```
uv run fastapi dev

   FastAPI   Starting development server 🚀

             Searching for package file structure from
directories
             with __init__.py files
             Importing from /home/user/code/awesomeapp

    module   🐍 main.py

      code   Importing the `FastAPI` app object from the
module with
             the following code:

             from main import app

       app   Using import string: main:app

    server   Server started at http://127.0.0.1:8000
    server   Documentation at http://127.0.0.1:8000/docs

       tip   Running in development mode, for production
use:
             FastAPI run
```

```
            Logs:

      INFO   Will watch for changes in these directories:
             ['/home/user/code/awesomeapp']
      INFO   Uvicorn running on http://127.0.0.1:8000
(Press CTRL+C
             to quit)
      INFO   Started reloader process [383138] using
WatchFiles
      INFO   Started server process [383153]
      INFO   Waiting for application startup.
      INFO   Application startup complete.
```

En el resultado, hay una línea con algo como:

```
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press
CTRL+C to quit)
```

Esa línea muestra la `URL` donde tu aplicación está siendo servida, en tu máquina local.

Revisa

Abre tu navegador en http://127.0.0.1:8000.

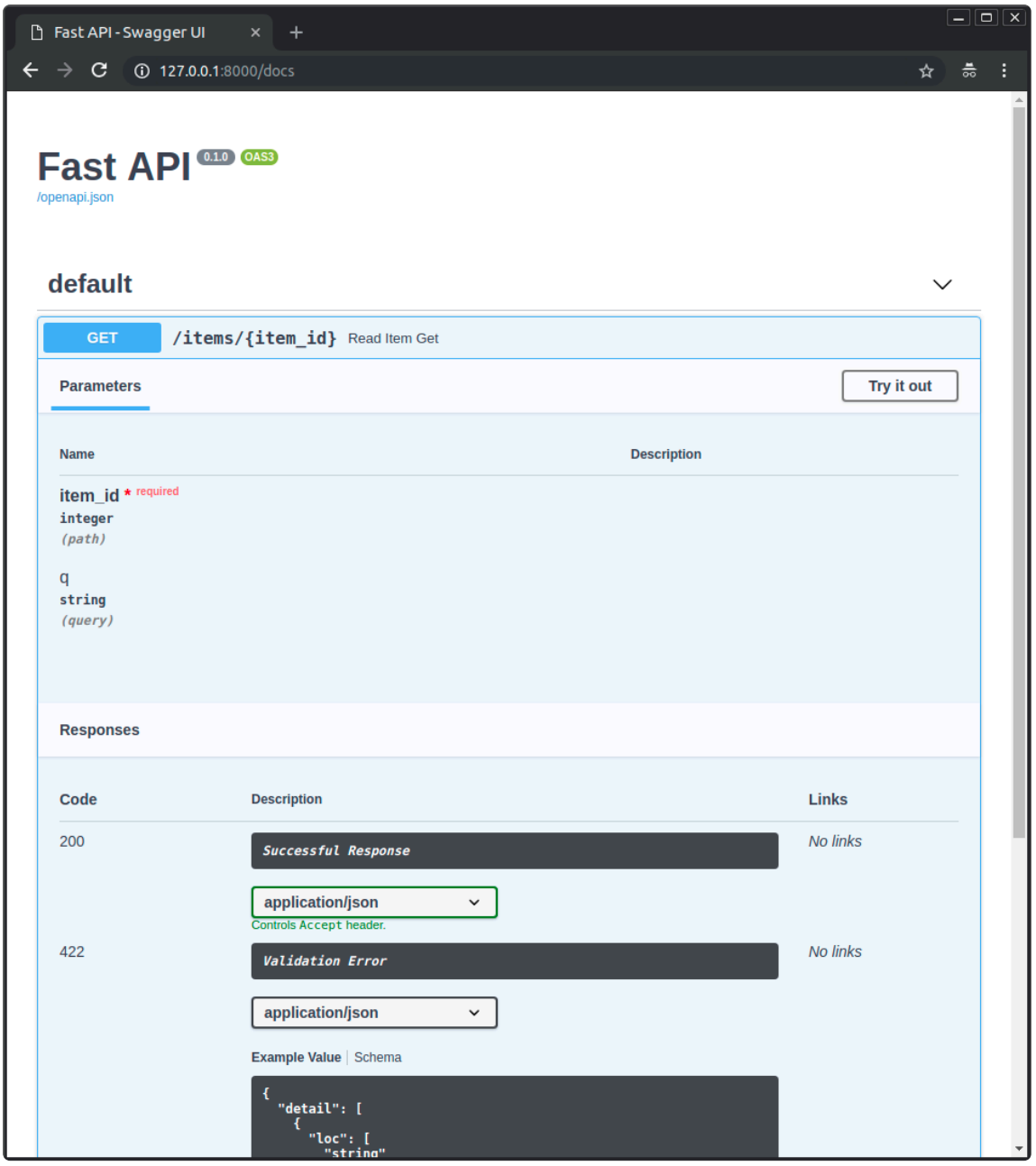
Verás el response `JSON` como:

```
{"message": "Hello World"}
```

Documentación interactiva de la `API`

Ahora ve a http://127.0.0.1:8000/docs.

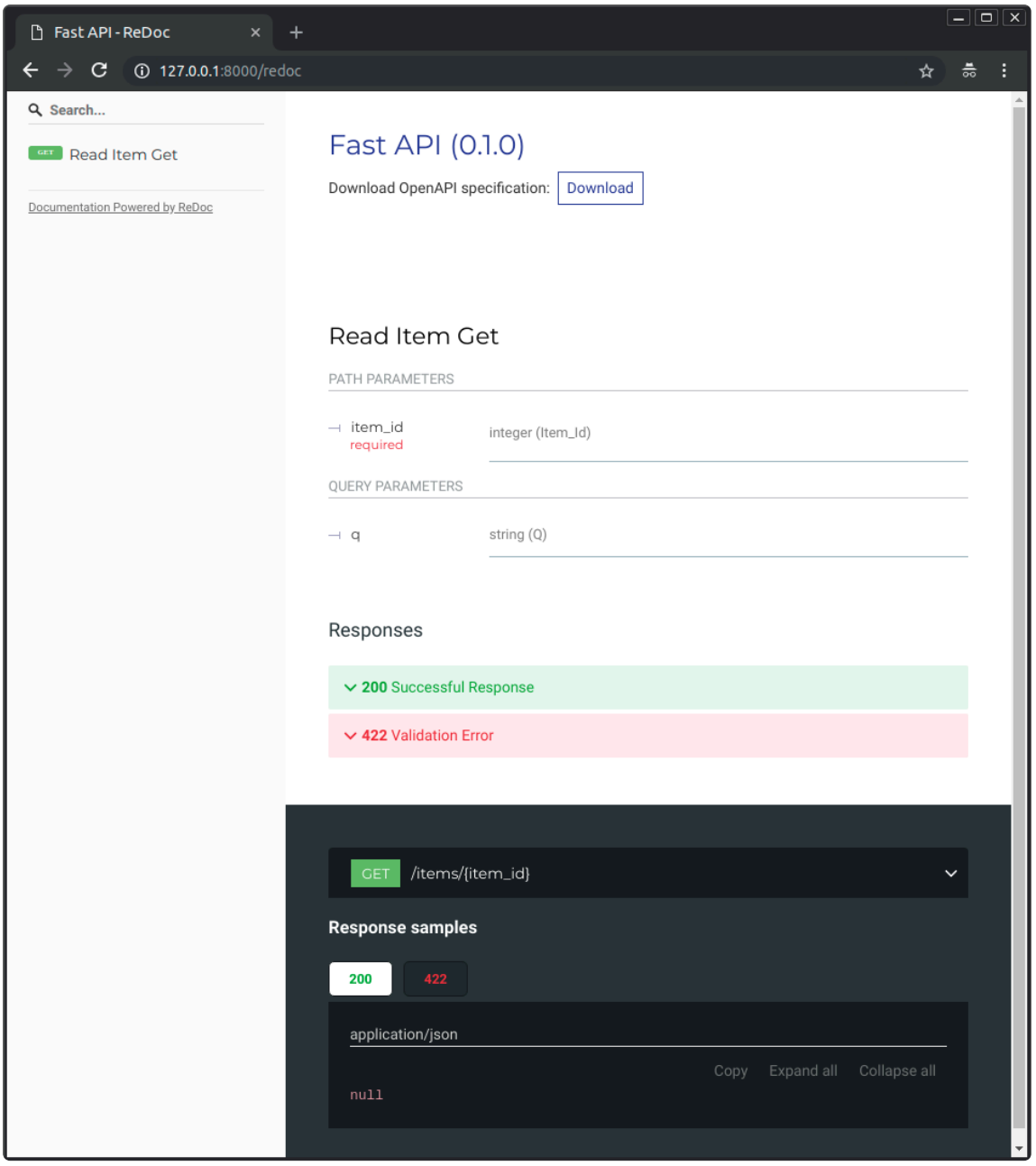
Verás la documentación interactiva automática de la `API` (proporcionada por Swagger UI):



Documentación alternativa de la `API`

Y ahora, ve a http://127.0.0.1:8000/redoc.

Verás la documentación alternativa automática (proporcionada por ReDoc):



`OpenAPI`

`FastAPI` genera un "esquema" con toda tu `API` utilizando el estándar `OpenAPI` para definir `API`s.

"Esquema"

Un "esquema" es una definición o descripción de algo. No el código que lo implementa, sino solo una descripción abstracta.

"Esquema" de la `API`

En este caso, `OpenAPI` es una especificación que dicta cómo definir un esquema de tu `API`.

Esta definición de esquema incluye los paths de tu `API`, los posibles parámetros que toman, etc.

Esquema de datos

El término "esquema" también podría referirse a la forma de algunos datos, como el contenido `JSON`.

En ese caso, significaría los atributos del `JSON`, los tipos de datos que tienen, etc.

`OpenAPI` y `JSON Schema`

`OpenAPI` define un esquema de `API` para tu `API`. Y ese esquema incluye definiciones (o "esquemas") de los datos enviados y recibidos por tu `API` utilizando **`JSON Schema`**, el estándar para esquemas de datos `JSON`.

Revisa el `OpenAPI.JSON`

Si tienes curiosidad por cómo se ve el esquema `OpenAPI` en bruto, `FastAPI` automáticamente genera un `JSON` (esquema) con las descripciones de toda tu `API`.

Puedes verlo directamente en: http://127.0.0.1:8000/openapi.json.

Mostrará un `JSON` que empieza con algo como:

```
{
    "OpenAPI": "3.1.0",
    "info": {
        "title": "FastAPI",
        "version": "0.1.0"
    },
    "paths": {
        "/items/": {
```

```
            "get": {
                "responses": {
                    "200": {
                        "description": "Successful
Response",
                        "content": {
                            "application/JSON": {

...
```

Para qué sirve `OpenAPI`

El esquema `OpenAPI` es lo que impulsa los dos sistemas de documentación interactiva incluidos.

Y hay docenas de alternativas, todas basadas en `OpenAPI`. Podrías añadir fácilmente cualquiera de esas alternativas a tu aplicación construida con `FastAPI`.

También podrías usarlo para generar código automáticamente, para clientes que se comuniquen con tu `API`. Por ejemplo, aplicaciones `frontend`, móviles o `IoT`.

Configura el `entrypoint` de la app en `pyproject.toml`

Puedes configurar dónde está tu app en un archivo `pyproject.toml` así:

```
[tool.fastAPI]
entrypoint = "main:app"
```

Ese `entrypoint` le dirá al comando `fastapi` que debe hacer el import de la app así:

```
from main import app
```

Si tu código estuviera estructurado así:

```
.
├── backend
│   ├── main.py
│   ├── __init__.py
```

Entonces pondrías el `entrypoint` como:

```toml
[tool.fastapi]
entrypoint = "backend.main:app"
```

lo cual sería equivalente a:

```
from backend.main import app
```

`fastapi dev` con `path` o con la opción de `CLI` `--entrypoint`

También puedes pasar el `path` del archivo al comando `fastapi dev`, y adivinará el objeto app de `FastAPI` que debe usar:

```
$ uv run fastapi dev main.py
```

O, también puedes pasar la opción `--entrypoint` al comando `fastapi dev`:

```
$ uv run fastapi dev --entrypoint main:app
```

Pero tendrías que recordar pasar el `path` `entrypoint` correcto cada vez que llames al comando `fastapi`.

Además, otras herramientas podrían no ser capaces de encontrarlo, por ejemplo la Extensión de VS Code o `FastAPI Cloud`, así que se recomienda usar el `entrypoint` en `pyproject.toml`.

Despliega tu app (opcional)

Opcionalmente puedes desplegar tu app de `FastAPI` en `FastAPI` Cloud con un solo comando.

```
uv run fastapi deploy

Deploying to FastAPI Cloud...

✅ Deployment successful!

🐔 Ready the chicken! Your app is ready at
https://myapp.fastapicloud.dev
```

La `CLI` detectará automáticamente tu aplicación de `FastAPI` y la desplegará en la nube. Si no has iniciado sesión, se abrirá tu navegador para completar el proceso de autenticación.

¡Eso es todo! Ahora puedes acceder a tu app en esa `URL`.

Recapitulación, paso a paso

Paso 1: importa `FastAPI`

```
from fastapi import FastAPI
```

`FastAPI` es una clase de `Python` que proporciona toda la funcionalidad para tu `API`.

> **Detalles técnicos**
> `FastAPI` es una clase que hereda directamente de `Starlette`.
>
> Puedes usar toda la funcionalidad de `Starlette` con `FastAPI` también.

Paso 2: crea una instancia de `FastAPI`

```
app = FastAPI()
```

Aquí la variable `app` será una instancia de la clase `FastAPI`.

Este será el punto principal de interacción para crear toda tu `API`.

Paso 3: crea una `path operation`

Path

`Path` aquí se refiere a la última parte de la `URL` empezando desde la primera `/`.

Así que, en una `URL` como:

```
https://example.com/items/foo
```

...el `path` sería:

```
/items/foo
```

> **Nota**
> Un `path` también es comúnmente llamado `endpoint` o "ruta".

Operación

"Operación" aquí se refiere a uno de los "métodos" HTTP.

Uno de:

- `POST`
- `GET`
- `PUT`
- `DELETE`

...y los más exóticos:

- `OPTIONS`
- `HEAD`
- `PATCH`
- `TRACE`

En el protocolo `HTTP`, puedes comunicarte con cada `path` usando uno, o más, de estos "métodos".

Al construir `API`s, normalmente usas estos métodos `HTTP` específicos para realizar una acción específica.

Normalmente usas:

- `POST`: para crear datos.
- `GET`: para leer datos.
- `PUT`: para actualizar datos.
- `DELETE`: para eliminar datos.

Así que, en `OpenAPI`, cada uno de los métodos `HTTP` se llama una "operación".

Vamos a llamarlas "**operaciones**" también.

Define un `path operation decorator`

```
@app.get("/")
```

El `@app.get("/")` le dice a `FastAPI` que la función justo debajo se encarga de manejar las peticiones que vayan a:

- el path `/`
- usando una `get` operación

> **Información sobre @decorator**
> Esa sintaxis `@algo` en Python se llama un "decorador".
>
> Lo pones encima de una función. Como un bonito sombrero decorativo (supongo que de ahí viene el término).
>
> Un "decorador" toma la función de abajo y hace algo con ella.
>
> En nuestro caso, este decorador le dice a `FastAPI` que la función de abajo corresponde al **path** `/` con una **operation**

`get`.

Es el **`path operation decorator`**.

También puedes usar las otras operaciones:

- `@app.post()`
- `@app.put()`
- `@app.delete()`

Y los más exóticos:

- `@app.options()`
- `@app.head()`
- `@app.patch()`
- `@app.trace()`

Consejo

Eres libre de usar cada operación (método `HTTP`) como quieras.

`FastAPI` no fuerza ningún significado específico.

La información aquí se presenta como una guía, no un requisito.

Por ejemplo, cuando usas `GraphQL` normalmente realizas todas las acciones usando solo operaciones `POST`.

Paso 4: define la `path operation function`

Esta es nuestra "**`path operation function`**":

- **`path`**: es `/`.
- **`operation`**: es `get`.
- **`function`**: es la función debajo del "decorador" (debajo de `@app.get("/")`).

```
async def root():
    return {"message": "Hello World"}
```

Esta es una función de `Python`.

Será llamada por `FastAPI` cuando reciba una solicitud en la `URL` "`/`" usando una operación `GET`.

En este caso, es una función `async`.

También podrías definirla como una función normal en lugar de `async def`:

```
def root():
    return {"message": "Hello World"}
```

> **Nota**
> Si no sabes la diferencia, Revisa la sección *Async: "¿Tienes prisa?"*.

Paso 5: retorna el contenido

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello World"}
```

Puedes retornar un `dict`, `list`, valores singulares como `str`, `int`, etc.

También puedes retornar modelos de `Pydantic` (verás más sobre eso más adelante).

Hay muchos otros objetos y modelos que serán automáticamente convertidos a `JSON` (incluyendo `ORMs`, etc). Intenta usar tus favoritos, es altamente probable que ya sean compatibles.

Paso 6: Despliégalo

Despliega tu app en **`FastAPI` Cloud** con un solo comando: `fastapi deploy`

Sobre `FastAPI Cloud`

`FastAPI Cloud` está construido por el mismo autor y equipo detrás de **`FastAPI`**.

Agiliza el proceso de **construir**, **desplegar** y **acceder** a una `API` con el mínimo esfuerzo.

Trae la misma **experiencia de desarrollador** de construir apps con `FastAPI` a **desplegarlas** en la nube.

`FastAPI Cloud` es el sponsor principal y proveedor de financiación para los proyectos `open source` de `FastAPI and friends`.

Despliega en otros proveedores `cloud`

`FastAPI` es `open source` y basado en estándares. Puedes desplegar apps de `FastAPI` en cualquier proveedor `cloud` que elijas.

Sigue las guías de tu proveedor `cloud` para desplegar apps de `FastAPI` con ellos.

Recapitulación

- Importa `FastAPI`.
- Crea una instancia `app`.
- Escribe un **`path operation decorator`** usando decoradores como `@app.get("/")`.

- Define una **`path operation function`**; por ejemplo, `def root(): ...`.
- Ejecuta el servidor de desarrollo usando el comando `fastapi dev`.
- Opcionalmente, despliega tu app con `fastapi deploy`.

Parámetros de `Path`

Puedes declarar "parámetros" o "variables" de `path` con la misma sintaxis que se usa en los `format strings` de `Python`:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
async def read_item(item_id):
    return {"item_id": item_id}
```

El valor del parámetro de `path` `item_id` se pasará a tu función como el argumento `item_id`.

Así que, si ejecutas este ejemplo y vas a http://127.0.0.1:8000/items/foo, verás la respuesta:

```
{"item_id": "foo"}
```

Parámetros de `path` con tipos

Puedes declarar el tipo de un parámetro de `path` en la función, usando anotaciones de tipos estándar de `Python`:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
async def read_item(item_id: int):
    return {"item_id": item_id}
```

En este caso, `item_id` se declara como un `int`.

> **Consejo**
> Esto te dará soporte del editor dentro de tu función, con chequeo de errores, autocompletado, etc.

Conversión de datos

Si ejecutas este ejemplo y abres tu navegador en http://127.0.0.1:8000/items/3, verás la respuesta:

```
{"item_id": 3}
```

> **Consejo**
> Nota que el valor que tu función recibió (y devolvió) es `3`, como un `int` de `Python`, no un `string` `"3"`.
>
> Entonces, con esa declaración de tipo, `FastAPI` te ofrece "`parsing`" automático de la solicitud (`request`).

Validación de datos

Pero si vas al navegador en http://127.0.0.1:8000/items/foo, verás un bonito error `HTTP` de:

```
{
  "detail": [
    {
      "type": "int_parsing",
      "loc": [
        "path",
        "item_id"
      ],
      "msg": "Input should be a valid integer, unable to
parse string as an integer",
      "input": "foo"
    }
```

```
    ]
}
```

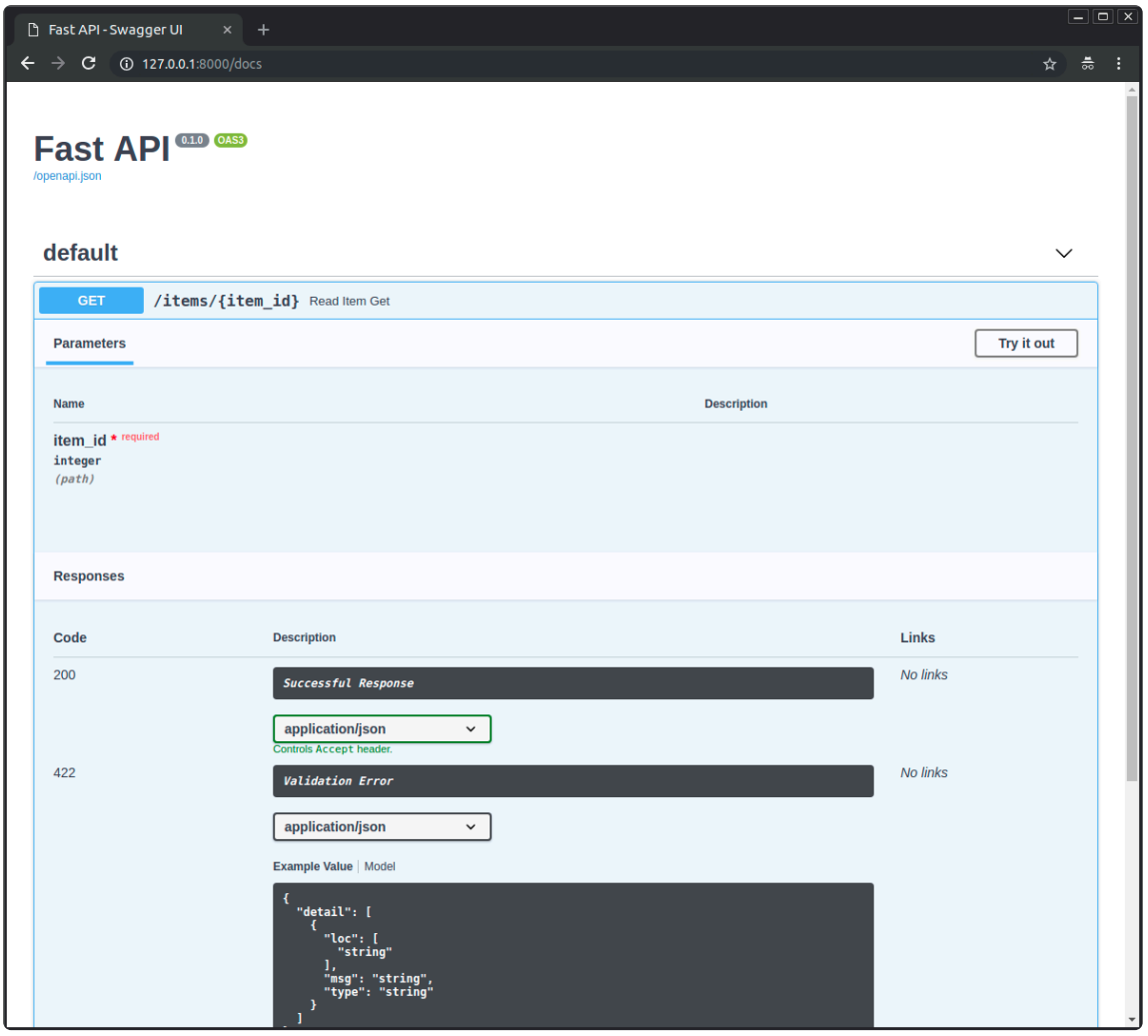
porque el parámetro de `path` `item_id` tenía un valor de `"foo"`, que no es un `int`.

El mismo error aparecería si proporcionaras un `float` en lugar de un `int`, como en: http://127.0.0.1:8000/items/4.2

> **Consejo**
> Entonces, con la misma declaración de tipo de `Python`, `FastAPI` te ofrece validación de datos.
>
> Nota que el error también indica claramente el punto exacto donde la validación falló.
>
> Esto es increíblemente útil mientras desarrollas y depuras código que interactúa con tu `API`.

Documentación

Y cuando abras tu navegador en http://127.0.0.1:8000/docs, verás una documentación de `API` automática e interactiva como:



Consejo

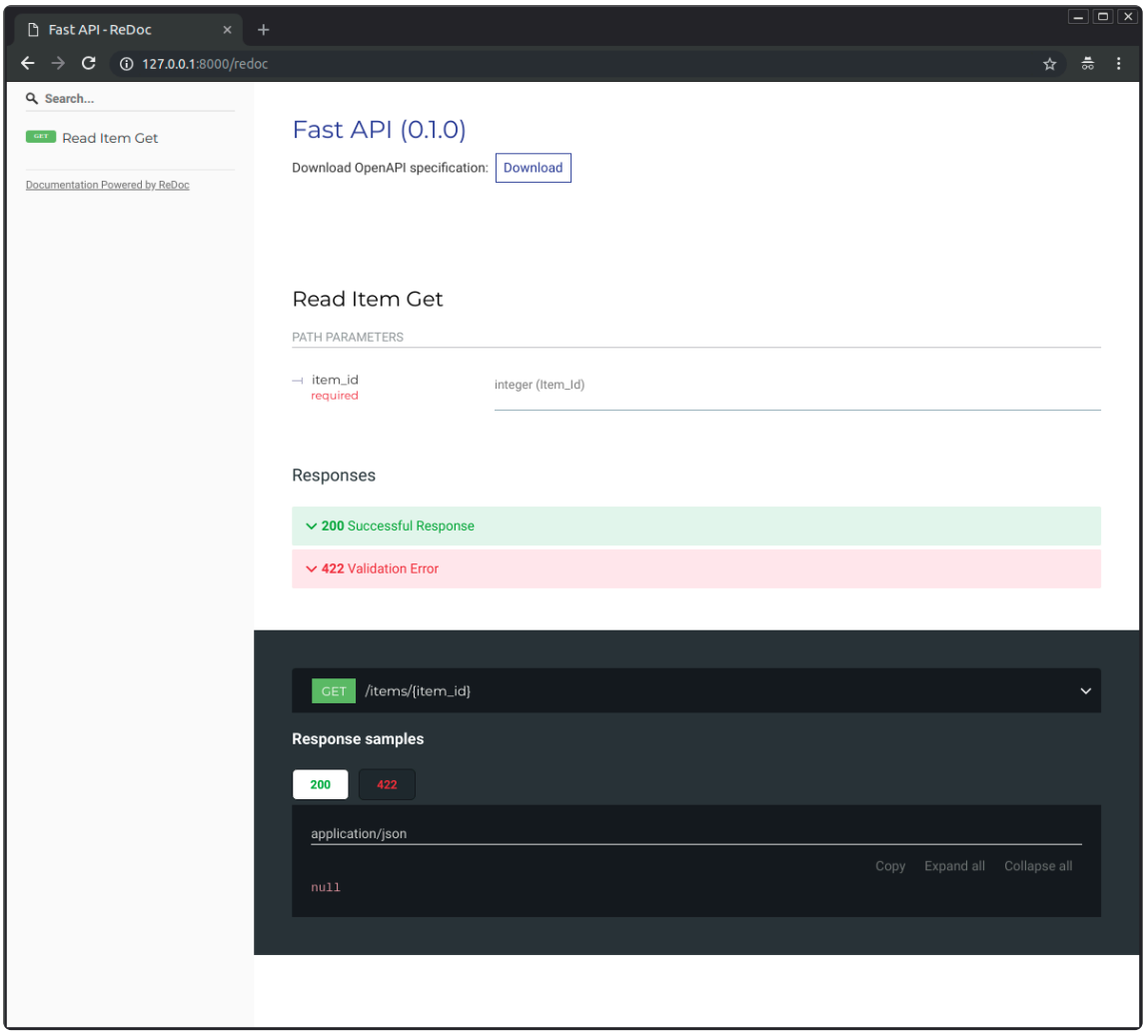
Nuevamente, solo con esa misma declaración de tipo de Python, `FastAPI` te ofrece documentación automática e interactiva (integrando `Swagger UI`).

Nota que el parámetro de `path` está declarado como un entero.

Beneficios basados en estándares, documentación alternativa

Y porque el esquema generado es del estándar `OpenAPI`, hay muchas herramientas compatibles.

Debido a esto, el propio `FastAPI` proporciona una documentación de `API` alternativa (usando `ReDoc`), a la cual puedes acceder en http://127.0.0.1:8000/redoc:



De la misma manera, hay muchas herramientas compatibles. Incluyendo herramientas de generación de código para muchos lenguajes.

`Pydantic`

Toda la validación de datos se realiza internamente con `Pydantic`, así que obtienes todos los beneficios de esta. Y sabes que estás en buenas manos.

Puedes usar las mismas declaraciones de tipo con `str`, `float`, `bool` y muchos otros tipos de datos complejos.

Varios de estos se exploran en los siguientes capítulos del tutorial.

El orden importa

Al crear `path operations`, puedes encontrarte en situaciones donde tienes un path fijo.

Como `/users/me`, imaginemos que es para obtener datos sobre el usuario actual.

Y luego también puedes tener un `path` `/users/{user_id}` para obtener datos sobre un usuario específico por algún `ID` de usuario.

Debido a que las `path operations` se evalúan en orden, necesitas asegurarte de que el path para `/users/me` se declara antes que el de `/users/{user_id}`:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users/me")
async def read_user_me():
    return {"user_id": "the current user"}

@app.get("/users/{user_id}")
async def read_user(user_id: str):
    return {"user_id": user_id}
```

De lo contrario, el `path` para `/users/{user_id}` también coincidiría para `/users/me`, "pensando" que está recibiendo un parámetro `user_id` con un valor de `"me"`.

De manera similar, no puedes redefinir una `path operation`:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users")
async def read_users():
    return ["Rick", "Morty"]

@app.get("/users")
async def read_users2():
    return ["Bean", "Elfo"]
```

La primera siempre será utilizada ya que el `path` coincide primero.

Valores predefinidos

Si tienes una `path operation` que recibe un `path parameter`, pero quieres que los valores posibles válidos del `path parameter` estén predefinidos, puedes usar un `Enum` estándar de `Python`.

Crear una clase `Enum`

Importa `Enum` y crea una subclase que herede de `str` y de `Enum`.

Al heredar de `str`, la documentación de la `API` podrá saber que los valores deben ser de tipo `string` y podrá representarlos correctamente.

Luego crea atributos de clase con valores fijos, que serán los valores válidos disponibles:

```
from enum import Enum

from fastapi import FastAPI

class ModelName(str, Enum):
    alexnet = "alexnet"
    resnet = "resnet"
    lenet = "lenet"

app = FastAPI()
```

> **Consejo**
> Si te estás preguntando, "AlexNet", "ResNet" y "LeNet" son solo nombres de modelos de Machine Learning.

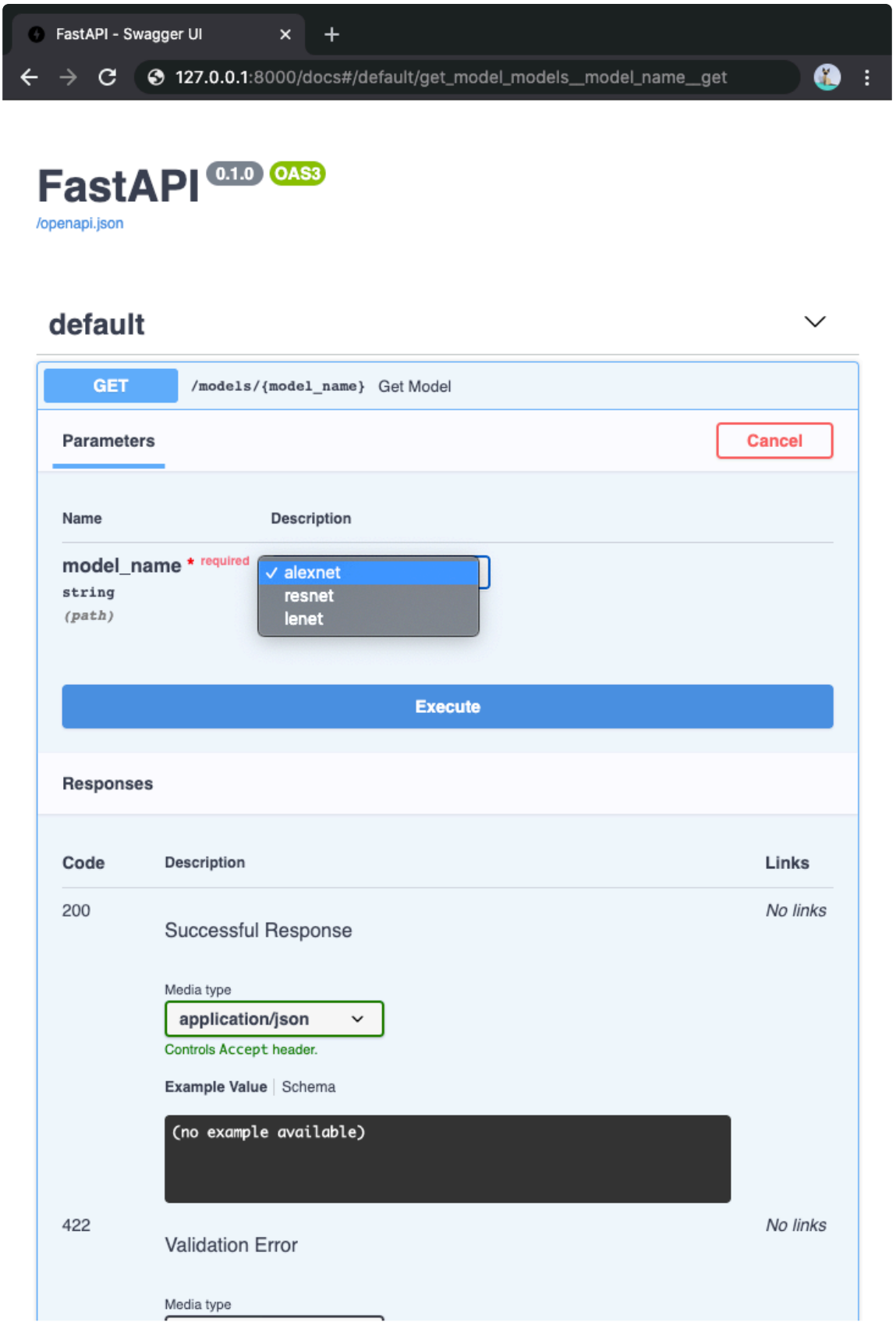
Declarar un `path parameter`

Luego crea un `path parameter` con una anotación de tipo usando la clase `enum` que creaste (`ModelName`):

```python
@app.get("/models/{model_name}")
async def get_model(model_name: ModelName):
```

Revisa la documentación

Como los valores disponibles para el `path parameter` están predefinidos, la documentación interactiva puede mostrarlos de manera ordenada:



Trabajando con enumeraciones de `Python`

El valor del `path parameter` será un miembro de enumeración.

Comparar miembros de enumeraciones

Puedes compararlo con el miembro de enumeración en tu `enum` creada `ModelName`:

```
    if model_name is ModelName.alexnet:
        return {"model_name": model_name, "message": "Deep
Learning FTW!"}

    if model_name.value == "lenet":
        return {"model_name": model_name, "message":
"LeCNN all the images"}

    return {"model_name": model_name, "message": "Have
some residuals"}
```

Obtener el valor de enumeración

Puedes obtener el valor actual (un `str` en este caso) usando `model_name.value`, o en general, `your_enum_member.value`:

```
async def get_model(model_name: ModelName):
    if model_name is ModelName.alexnet:
        return {"model_name": model_name, "message": "Deep
Learning FTW!"}

    if model_name.value == "lenet":
        return {"model_name": model_name, "message":
"LeCNN all the images"}

    return {"model_name": model_name, "message": "Have
some residuals"}
```

> **Consejo**
> También podrías acceder al valor `"lenet"` con `ModelName.lenet.value`.

Devolver miembros de enumeración

Puedes devolver miembros de `enum` desde tu `path operation`, incluso anidados en un cuerpo `JSON` (por ejemplo, un `dict`).

Serán convertidos a sus valores correspondientes (cadenas en este caso) antes de devolverlos al cliente:

```
from enum import Enum

from fastapi import FastAPI

class ModelName(str, Enum):
    alexnet = "alexnet"
    resnet = "resnet"
    lenet = "lenet"

app = FastAPI()

@app.get("/models/{model_name}")
async def get_model(model_name: ModelName):
    if model_name is ModelName.alexnet:
        return {"model_name": model_name, "message": "Deep
Learning FTW!"}

    if model_name.value == "lenet":
        return {"model_name": model_name, "message":
"LeCNN all the images"}

    return {"model_name": model_name, "message": "Have
some residuals"}
```

En tu cliente recibirás un response `JSON` como:

```
{
  "model_name": "alexnet",
  "message": "Deep Learning FTW!"
}
```

Parámetros de `path` conteniendo `paths`

Imaginemos que tienes una `path operation` con un `path` `/files/{file_path}`.

Pero necesitas que `file_path` en sí mismo contenga un `path`, como `home/johndoe/myfile.txt`.

Entonces, la `URL` para ese archivo sería algo como: `/files/home/johndoe/myfile.txt`.

Soporte de `OpenAPI`

`OpenAPI` no soporta una manera de declarar un `path parameter` para que contenga un `path` dentro, ya que eso podría llevar a escenarios que son difíciles de probar y definir.

Sin embargo, todavía puedes hacerlo en `FastAPI`, usando una de las herramientas internas de `Starlette`.

Y la documentación seguiría funcionando, aunque no agregue ninguna documentación indicando que el parámetro debe contener un `path`.

Convertidor de `Path`

Usando una opción directamente de `Starlette` puedes declarar un `path parameter` conteniendo un `path` usando una `URL` como:

```
/files/{file_path:path}
```

En este caso, el nombre del parámetro es `file_path`, y la última parte, `:path`, indica que el parámetro debería coincidir con cualquier `path`.

Así que, puedes usarlo con:

```
from fastapi import FastAPI

app = FastAPI()
```

```
@app.get("/files/{file_path:path}")
async def read_file(file_path: str):
    return {"file_path": file_path}
```

> **Consejo**
>
> Podrías necesitar que el parámetro contenga `/home/johndoe/myfile.txt`, con una barra inclinada (`/`) inicial.
>
> En ese caso, la `URL` sería: `/files//home/johndoe/myfile.txt`, con una doble barra inclinada (`//`) entre `files` y `home`.

Resumen

Con **`FastAPI`**, al usar declaraciones de tipo estándar de `Python`, cortas e intuitivas, obtienes:

- Soporte del editor: chequeo de errores, autocompletado, etc.
- "`parsing`" de datos
- Validación de datos
- Anotación de `API` y documentación automática

Y solo tienes que declararlos una vez.

Probablemente esa sea la principal ventaja visible de **`FastAPI`** en comparación con otros `frameworks` alternativos (aparte del rendimiento bruto).

Parámetros de `Query`

Cuando declaras otros parámetros de función que no son parte de los parámetros de `path`, son automáticamente interpretados como parámetros de "`query`".

```
from fastapi import FastAPI

app = FastAPI()

fake_items_db = [{"item_name": "Foo"}, {"item_name":
"Bar"}, {"item_name": "Baz"}]

@app.get("/items/")
```

```
async def read_item(skip: int = 0, limit: int = 10):
    return fake_items_db[skip : skip + limit]
```

La `query` es el conjunto de pares `clave=valor` que van después del `?` en una `URL`, separados por caracteres `&`.

Por ejemplo, en la `URL`:

```
http://127.0.0.1:8000/items/?skip=0&limit=10
```

...los parámetros de `query` son:

- `skip`: con un valor de `0`
- `limit`: con un valor de `10`

Como son parte de la `URL`, son "naturalmente" `strings`.

Pero cuando los declaras con tipos de `Python` (en el ejemplo anterior, como `int`), son convertidos a ese tipo y validados respecto a él.

Todo el mismo proceso que se aplica para los parámetros de `path` también se aplica para los parámetros de `query`:

- Soporte del editor (obviamente)
- "`parsing`" de datos
- Validación de datos
- Documentación automática

Valores por defecto

Como los parámetros de `query` no son una parte fija de un `path`, pueden ser opcionales y pueden tener valores por defecto.

En el ejemplo anterior, tienen valores por defecto de `skip=0` y `limit=10`.

Entonces, ir a la `URL`:

```
http://127.0.0.1:8000/items/
```

sería lo mismo que ir a:

```
http://127.0.0.1:8000/items/?skip=0&limit=10
```

Pero si vas a, por ejemplo:

```
http://127.0.0.1:8000/items/?skip=20
```

Los valores de los parámetros en tu función serán:

- `skip=20`: porque lo configuraste en la URL
- `limit=10`: porque ese era el valor por defecto

Parámetros opcionales

De la misma manera, puedes declarar parámetros de `query` opcionales, estableciendo su valor por defecto en `None`:

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
async def read_item(item_id: str, q: str | None = None):
    if q:
        return {"item_id": item_id, "q": q}
    return {"item_id": item_id}
```

En este caso, el parámetro de función `q` será opcional y será `None` por defecto.

> **Consejo**
> Además, nota que `FastAPI` es lo suficientemente inteligente para notar que el parámetro de `path` `item_id` es un

parámetro de `path` y `q` no lo es, por lo tanto, es un parámetro de `query`.

Conversión de tipos en parámetros de `query`

También puedes declarar tipos `bool`, y serán convertidos:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
async def read_item(item_id: str, q: str | None = None,
short: bool = False):
    item = {"item_id": item_id}
    if q:
        item.update({"q": q})
    if not short:
        item.update(
            {"description": "This is an amazing item that
has a long description"}
        )
    return item
```

En este caso, si vas a:

```
http://127.0.0.1:8000/items/foo?short=1
```

o

```
http://127.0.0.1:8000/items/foo?short=True
```

o

```
http://127.0.0.1:8000/items/foo?short=true
```

o

```
http://127.0.0.1:8000/items/foo?short=on
```

o

```
http://127.0.0.1:8000/items/foo?short=yes
```

o cualquier otra variación (mayúsculas, primera letra en mayúscula, etc.), tu función verá el parámetro `short` con un valor `bool` de `True`. De lo contrario, será `False`.

Múltiples parámetros de `path` y de `query`

Puedes declarar múltiples parámetros de `path` y de `query` al mismo tiempo, `FastAPI` sabe cuál es cuál.

Y no tienes que declararlos en un orden específico.

Serán detectados por nombre:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users/{user_id}/items/{item_id}")
async def read_user_item(
    user_id: int,
    item_id: str,
    q: str | None = None,
    short: bool = False
):
    item = {"item_id": item_id, "owner_id": user_id}
    if q:
        item.update({"q": q})
    if not short:
        item.update(
            {"description": "This is an amazing item that
has a long description"}
        )
    return item
```

Parámetros de `query` requeridos

Cuando declaras un valor por defecto para parámetros que no son de `path` (por ahora, solo hemos visto parámetros de `query`), entonces no es requerido.

Si no quieres agregar un valor específico pero solo hacer que sea opcional, establece el valor por defecto como `None`.

Pero cuando quieres hacer un parámetro de `query` requerido, simplemente no declares ningún valor por defecto:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
async def read_user_item(item_id: str, needy: str):
    item = {"item_id": item_id, "needy": needy}
    return item
```

Aquí el parámetro de `query` `needy` es un parámetro de query requerido de tipo `str`.

Si abres en tu navegador una `URL` como:

```
http://127.0.0.1:8000/items/foo-item
```

...sin agregar el parámetro requerido `needy`, verás un error como:

```
{
  "detail": [
    {
      "type": "missing",
      "loc": [
        "query",
        "needy"
      ],
      "msg": "Field required",
      "input": null
```

```
    }
  ]
}
```

Como `needy` es un parámetro requerido, necesitarías establecerlo en la `URL`:

```
http://127.0.0.1:8000/items/foo-item?needy=sooooneedy
```

...esto funcionaría:

```
{
    "item_id": "foo-item",
    "needy": "sooooneedy"
}
```

Y por supuesto, puedes definir algunos parámetros como requeridos, algunos con un valor por defecto, y algunos enteramente opcionales:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
async def read_user_item(
    item_id: str, needy: str, skip: int = 0, limit: int |
None = None
):
    item = {"item_id": item_id, "needy": needy, "skip":
skip, "limit": limit}
    return item
```

En este caso, hay 3 parámetros de `query`:

- `needy`, un `str` requerido.
- `skip`, un `int` con un valor por defecto de `0`.
- `limit`, un `int` opcional.

> **Consejo**
> También podrías usar `Enum` de la misma manera que con parámetros de path

- ## `Request Body`

Cuando necesitas enviar datos desde un cliente (digamos, un navegador) a tu `API`, los envías como un **`request body`**.

Un **`request body`** es un dato enviado por el cliente a tu `API`. Un **`response body`** es el dato que tu `API` envía al cliente.

Tu `API` casi siempre tiene que enviar un **`response body`**. Pero los clientes no necesariamente necesitan enviar **`request bodies`** todo el tiempo, a veces solo solicitan un `path`, quizás con algunos parámetros de `query`, pero no envían un `body`.

Para declarar un **`request body`**, usas modelos de `Pydantic` con todo su poder y beneficios.

> **Nota**
> Para enviar datos, deberías usar uno de estos métodos: `POST` (el más común), `PUT`, `DELETE` o `PATCH`.
>
> Enviar un `body` con un `request` `GET` tiene un comportamiento indefinido en las especificaciones, no obstante, es soportado por `FastAPI`, solo para casos de uso muy complejos/extremos.
>
> Como no se recomienda, la documentación interactiva con `Swagger UI` no mostrará la documentación para el `body` cuando se usa `GET`, y los `proxies` intermedios podrían no soportarlo.

Importar `BaseModel` de `Pydantic`

Primero, necesitas importar `BaseModel` de `pydantic`:

```
from fastapi import FastAPI
from pydantic import BaseModel
```

Crea tu modelo de datos

Luego, declaras tu modelo de datos como una clase que hereda de `BaseModel`.

Usa tipos estándar de `Python` para todos los atributos:

```
class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
```

Al igual que al declarar parámetros de `query`, cuando un atributo del modelo tiene un valor por defecto, no es obligatorio. De lo contrario, es obligatorio. Usa `None` para hacerlo solo opcional.

Por ejemplo, el modelo anterior declara un objeto `JSON` (o `dict` en `Python`) como:

```
{
    "name": "Foo",
    "description": "An optional description",
    "price": 45.2,
    "tax": 3.5
}
```

...dado que `description` y `tax` son opcionales (con un valor por defecto de `None`), este objeto `JSON` también sería válido:

```
{
    "name": "Foo",
    "price": 45.2
}
```

Decláralo como un parámetro

Para añadirlo a tu *`path operation`*, decláralo de la misma manera que declaraste parámetros de `path` y `query`:

```
app = FastAPI()

@app.post("/items/")
async def create_item(item: Item):
    return item
```

...y declara su tipo como el modelo que creaste, `Item`.

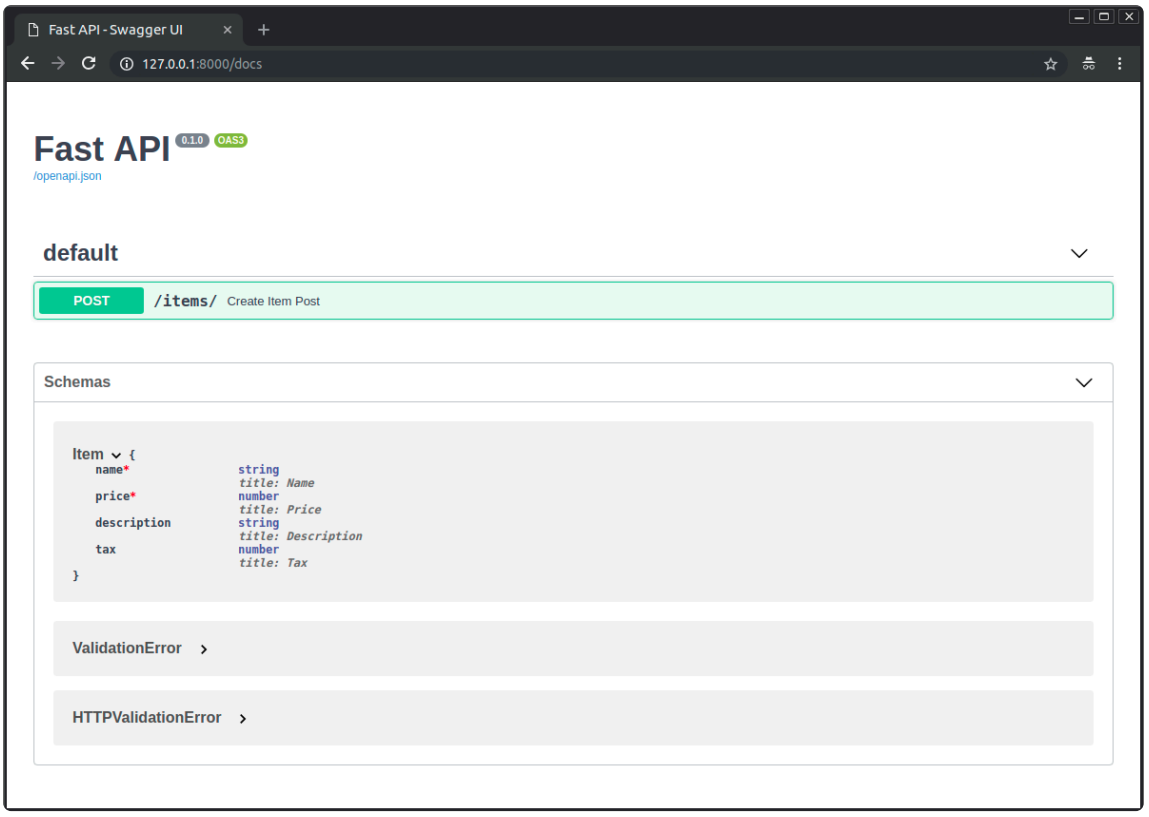
Resultados

Con solo esa declaración de tipo en `Python`, **`FastAPI`** hará lo siguiente:

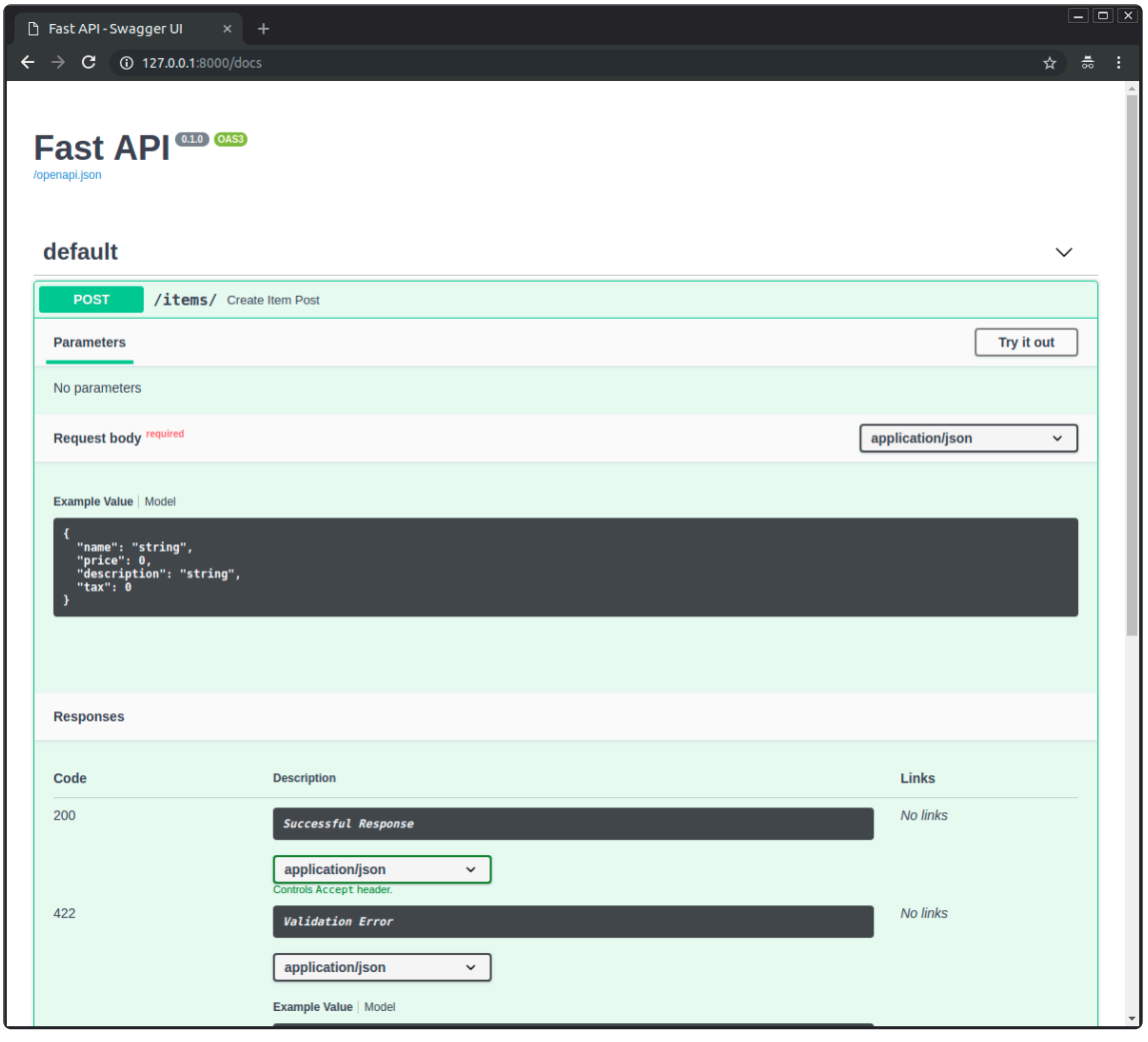
- Leer el `body` del `request` como `JSON`.
- Convertir los tipos correspondientes (si es necesario).
- Validar los datos.
    - Si los datos son inválidos, devolverá un error claro e indicado, señalando exactamente dónde y qué fue lo incorrecto.
- Proporcionar los datos recibidos en el parámetro `item`.
    - Como lo declaraste en la función como de tipo `Item`, también tendrás todo el soporte del editor (autocompletado, etc.) para todos los atributos y sus tipos.
- Generar definiciones de `JSON Schema` para tu modelo, que también puedes usar en cualquier otro lugar si tiene sentido para tu proyecto.
- Esos esquemas serán parte del esquema de `OpenAPI` generado y usados por las `UIs` de documentación automática.

Documentación automática

Los `JSON Schemas` de tus modelos serán parte del esquema `OpenAPI` generado y se mostrarán en la documentación `API` interactiva:

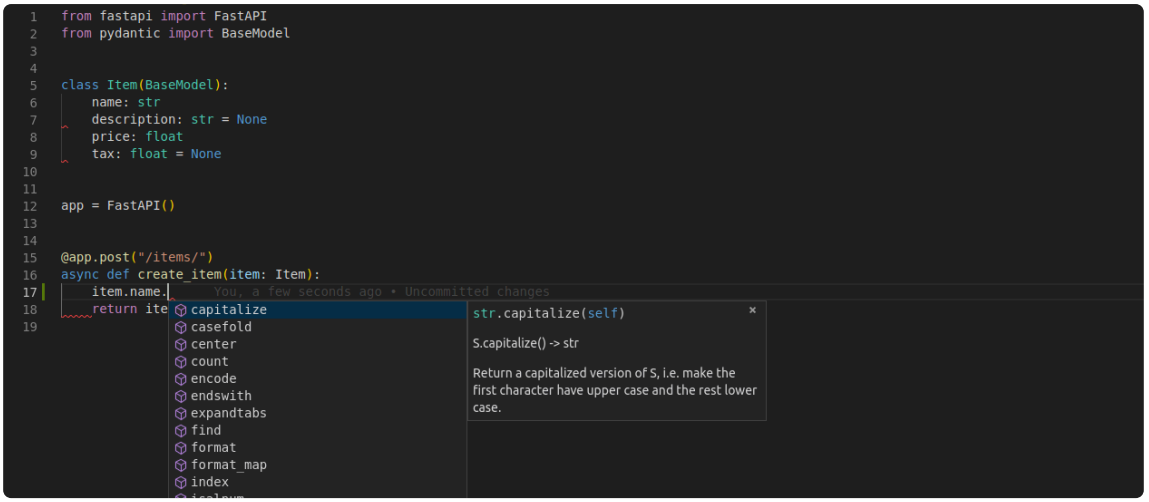


Y también se utilizarán en la documentación `API` dentro de cada *`path operation`* que los necesite:

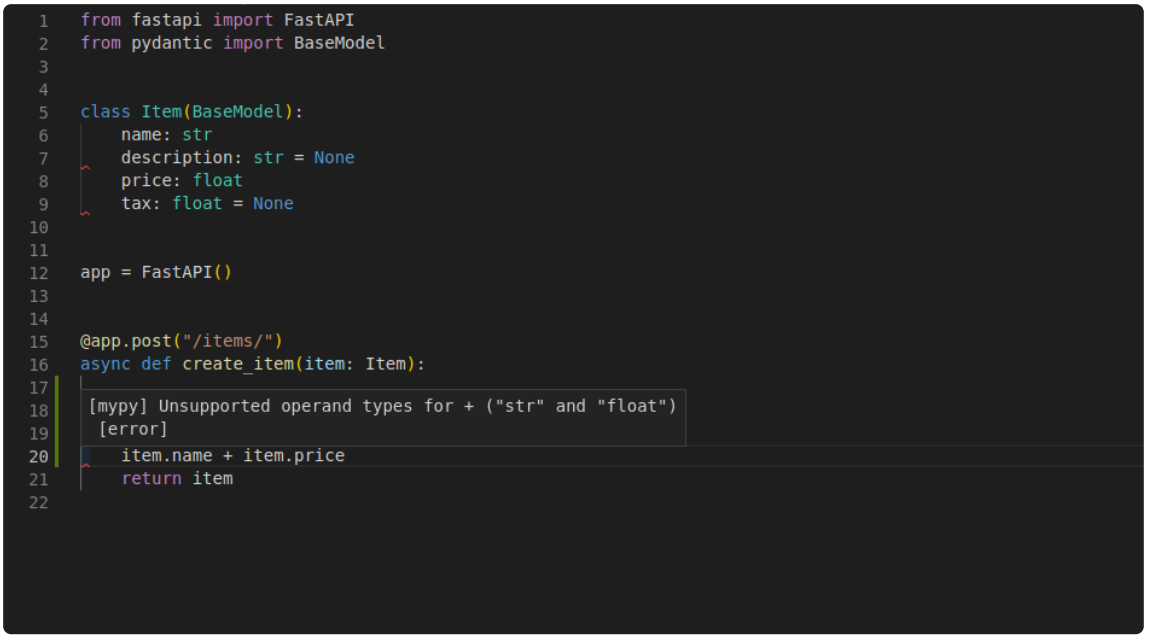


Soporte del editor

En tu editor, dentro de tu función, obtendrás anotaciones de tipos y autocompletado en todas partes (esto no sucedería si recibieras un `dict` en lugar de un modelo de `Pydantic`):



También recibirás chequeos de errores para operaciones de tipo incorrecto:



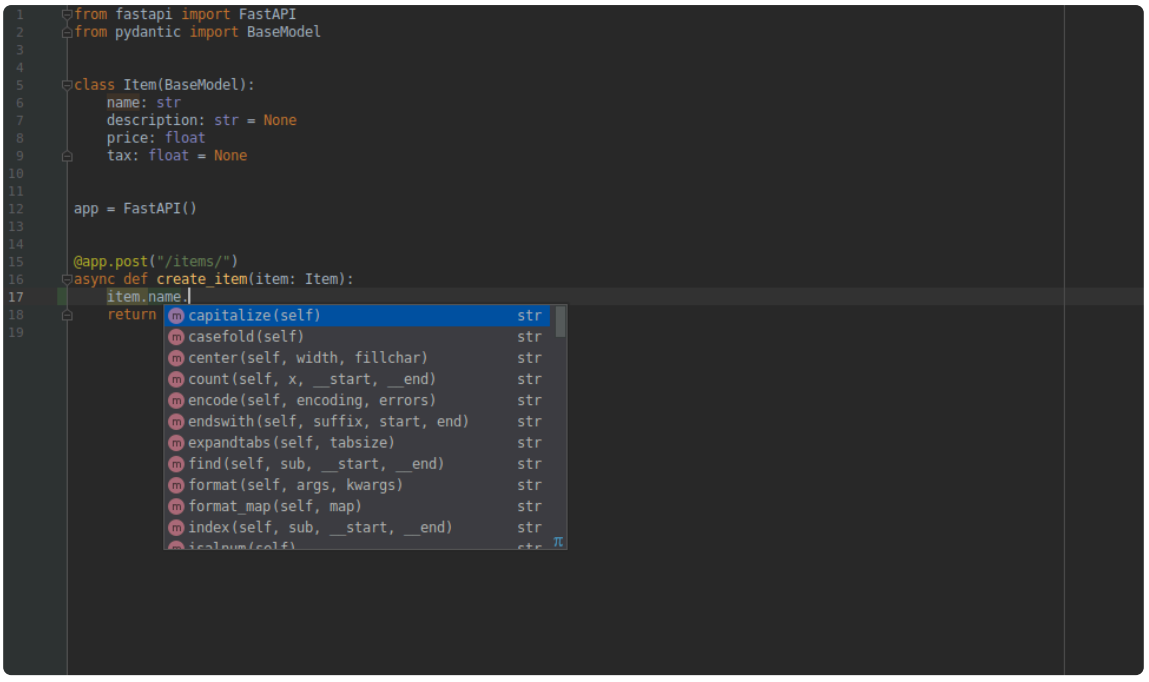
No es por casualidad, todo el `framework` fue construido alrededor de ese diseño.

Y fue rigurosamente probado en la fase de diseño, antes de cualquier implementación, para garantizar que funcionaría con todos los editores.

Incluso se hicieron algunos cambios en `Pydantic` para admitir esto.

Las capturas de pantalla anteriores se tomaron con Visual Studio Code.

Pero obtendrías el mismo soporte en el editor con `PyCharm` y la mayoría de los otros editores de `Python`:



Consejo

Si usas `PyCharm` como tu editor, puedes usar el `Pydantic PyCharm Plugin`.

Mejora el soporte del editor para modelos de `Pydantic`, con:

- autocompletado
- chequeo de tipos
- refactorización
- búsqueda
- inspecciones

Usa el modelo

Dentro de la función, puedes acceder a todos los atributos del objeto modelo directamente:

```
@app.post("/items/")
async def create_item(item: Item):
    item_dict = item.model_dump()
```

```
    if item.tax is not None:
        price_with_tax = item.price + item.tax
        item_dict.update({"price_with_tax":
price_with_tax})
    return item_dict
```

`Request body` + parámetros de `path`

Puedes declarar parámetros de `path` y `request body` al mismo tiempo.

`FastAPI` reconocerá que los parámetros de función que coinciden con los parámetros de `path` deben ser **tomados del `path`**, y que los parámetros de función que se declaran como modelos de `Pydantic` deben ser **tomados del `request body`**.

```
from fastapi import FastAPI
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

app = FastAPI()

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    return {"item_id": item_id, **item.model_dump()}
```

`Request body` + `path` + parámetros de `query`

También puedes declarar parámetros de **`body`**, **`path`** y **`query`**, todos al mismo tiempo.

`FastAPI` reconocerá cada uno de ellos y tomará los datos del lugar correcto.

```python
from fastapi import FastAPI
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

app = FastAPI()

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item, q: str |
None = None):
    result = {"item_id": item_id, **item.model_dump()}
    if q:
        result.update({"q": q})
    return result
```

Los parámetros de la función se reconocerán de la siguiente manera:

- Si el parámetro también se declara en el **`path`**, se utilizará como un parámetro de `path`.
- Si el parámetro es de un **tipo singular** (como `int`, `float`, `str`, `bool`, etc.), se interpretará como un parámetro de **`query`**.
- Si el parámetro se declara como del tipo de un **modelo de `Pydantic`**, se interpretará como un **`body`** de `request`.

> **Nota**
>
> `FastAPI` sabrá que el valor de `q` no es requerido debido al valor por defecto "= None".
>
> El `str | None` no es utilizado por `FastAPI` para determinar que el valor no es requerido, sabrá que no es requerido porque tiene un valor por defecto de `None`.

Pero agregar las anotaciones de tipos permitirá que tu editor te brinde un mejor soporte y detecte errores.

- # Parámetros de Query y Validaciones de String

`FastAPI` te permite declarar información adicional y validación para tus parámetros.

Tomemos esta aplicación como ejemplo:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/")
async def read_items(q: str | None = None):
    results = {"items": [{"item_id": "Foo"}, {"item_id":
"Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

El parámetro de `query` `q` es de tipo `str | None`, lo que significa que es de tipo `str` pero también podría ser `None`, y de hecho, el valor por defecto es `None`, así que `FastAPI` sabrá que no es requerido.

> **Nota**
> `FastAPI` sabrá que el valor de `q` no es requerido por el valor por defecto -.
>
> Tener `str | None` permitirá que tu editor te dé un mejor soporte y detecte errores.

Validaciones adicionales

Vamos a hacer que, aunque `q` sea opcional, siempre que se proporcione, **su longitud no exceda los 50 caracteres**.

Importar `Query` y `Annotated`

Para lograr eso, primero importa:

- `Query` desde `fastapi`
- `Annotated` desde `typing`

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items/")
async def read_items(q: Annotated[str | None,
Query(max_length=50)] = None):
    results = {"items": [{"item_id": "Foo"}, {"item_id":
"Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

> **Nota**
>
> `FastAPI` añadió soporte para `Annotated` (y empezó a recomendarlo) en la versión 0.95.0.
>
> Si tienes una versión más antigua, obtendrás errores al intentar usar `Annotated`.
>
> Asegúrate de Actualizar la versión de `FastAPI` a al menos 0.95.1 antes de usar `Annotated`.

Usar `Annotated` en el tipo del parámetro q

¿Recuerdas que te dije antes que `Annotated` puede usarse para agregar metadatos a tus parámetros en la Introducción a Tipos de Python?

Ahora es el momento de usarlo con `FastAPI`.

Teníamos esta anotación de tipo:

```
q: str | None = None
```

Lo que haremos es envolver eso con `Annotated`, para que se convierta en:

```
q: Annotated[str | None] = None
```

Ambas versiones significan lo mismo, `q` es un parámetro que puede ser un `str` o `None`, y por defecto, es `None`.

Ahora vamos a lo divertido.

Agregar `Query` a `Annotated` en el parámetro `q`

Ahora que tenemos este `Annotated` donde podemos poner más información (en este caso algunas validaciones adicionales), agrega `Query` dentro de `Annotated`, y establece el parámetro `max_length` a `50`:

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items/")
async def read_items(q: Annotated[str | None,
Query(max_length=50)] = None):
    results = {"items": [{"item_id": "Foo"}, {"item_id":
"Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

Nota que el valor por defecto sigue siendo `None`, por lo que el parámetro sigue siendo opcional.

Pero ahora, al tener `Query(max_length=50)` dentro de `Annotated`, le estamos diciendo a `FastAPI` que queremos que tenga **validación adicional** para este valor, queremos que tenga un máximo de 50 caracteres.

> **Consejo**
> Aquí estamos usando `Query()` porque este es un **parámetro de query**. Más adelante veremos otros como `Path()`, `Body()`, `Header()`, y `Cookie()`, que también aceptan los mismos argumentos que `Query()`.

`FastAPI` ahora:

- **Validará** los datos asegurándose de que la longitud máxima sea de 50 caracteres
- Mostrará un **error claro** para el cliente cuando los datos no sean válidos
- **Documentará** el parámetro en el esquema de `OpenAPI` *`path operation`* (así aparecerá en la `UI` **de documentación automática**)

Alternativa (antigua): `Query` como valor por defecto

Versiones anteriores de `FastAPI` (antes de 0.95.0) requerían que usaras `Query` como el valor por defecto de tu parámetro, en lugar de ponerlo en `Annotated`, hay una alta probabilidad de que veas código usándolo alrededor, así que te lo explicaré.

> **Consejo**
> Para nuevo código y siempre que sea posible, usa `Annotated` como se explicó arriba. Hay múltiples ventajas (explicadas a continuación) y no hay desventajas.

Así es como usarías `Query()` como el valor por defecto de tu parámetro de función, estableciendo el parámetro `max_length` a 50:

```
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items/")
async def read_items(q: str | None = Query(default=None,
max_length=50)):
    results = {"items": [{"item_id": "Foo"}, {"item_id":
"Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

Como en este caso (sin usar `Annotated`) debemos reemplazar el valor por defecto `None` en la función con `Query()`, ahora necesitamos establecer el valor por defecto con el parámetro `Query(default=None)`, esto sirve al mismo propósito de definir ese valor por defecto (al menos para `FastAPI`).

Entonces:

```
q: str | None = Query(default=None)
```

...hace que el parámetro sea opcional, con un valor por defecto de `None`, lo mismo que:

```
q: str | None = None
```

Pero la versión con `Query` lo declara explícitamente como un parámetro de query.

Luego, podemos pasar más parámetros a `Query`. En este caso, el parámetro `max_length` que se aplica a los strings:

```
q: str | None = Query(default=None, max_length=50)
```

Esto validará los datos, mostrará un error claro cuando los datos no sean válidos, y documentará el parámetro en el esquema de `OpenAPI` *`path operation`*.

`Query` como valor por defecto o en `Annotated`

Ten en cuenta que cuando uses `Query` dentro de `Annotated` no puedes usar el parámetro `default` para `Query`.

En su lugar utiliza el valor por defecto real del parámetro de la función. De lo contrario, sería inconsistente.

Por ejemplo, esto no está permitido:

```
q: Annotated[str, Query(default="rick")] = "morty"
```

...porque no está claro si el valor por defecto debería ser `"rick"` o `"morty"`.

Así que utilizarías (preferentemente):

```
q: Annotated[str, Query()] = "rick"
```

...o en code bases más antiguas encontrarás:

```
q: str = Query(default="rick")
```

Ventajas de `Annotated`

Usar `Annotated` es recomendado en lugar del valor por defecto en los parámetros de función, es **mejor** por múltiples razones.

El valor **por defecto** del **parámetro de función** es el **valor real por defecto**, eso es más intuitivo con `Python` en general.

Podrías **llamar** a esa misma función en **otros lugares** sin `FastAPI`, y **funcionaría como se espera**. Si hay un parámetro **requerido** (sin un valor por defecto), tu **editor** te avisará con un error, `Python` también se quejará si lo ejecutas sin pasar el parámetro requerido.

Cuando no usas `Annotated` y en su lugar usas el **estilo de valor por defecto (antiguo)**, si llamas a esa función sin `FastAPI` en **otros lugares**, tienes que **recordar** pasar los argumentos a la función para que funcione correctamente, de lo contrario, los valores serán diferentes de lo que esperas (por ejemplo, `QueryInfo` o algo similar en lugar de `str`). Y tu editor no se quejará, y `Python` no se quejará al ejecutar esa función, solo cuando las operaciones internas generen errores.

Dado que `Annotated` puede tener más de una anotación de metadato, ahora podrías incluso usar la misma función con otras herramientas, como Typer.

Agregar más validaciones

También puedes agregar un parámetro `min_length`:

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items/")
async def read_items(
    q: Annotated[str | None, Query(min_length=3,
max_length=50)] = None,
):
    results = {"items": [{"item_id": "Foo"}, {"item_id":
"Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

Agregar expresiones regulares

Puedes definir una expresión regular `pattern` que el parámetro debe coincidir:

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items/")
async def read_items(
    q: Annotated[
        str | None, Query(min_length=3, max_length=50,
pattern="^fixedquery$")
    ] = None,
):
    results = {"items": [{"item_id": "Foo"}, {"item_id":
"Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

Este patrón específico de expresión regular revisa que el valor recibido del parámetro:

- `^`: comienza con los siguientes caracteres, no tiene caracteres antes.
- `fixedquery`: tiene el valor exacto `fixedquery`.
- `$`: termina allí, no tiene más caracteres después de `fixedquery`.

Si te sientes perdido con todas estas ideas de **"expresión regular"**, no te preocupes. Son un tema difícil para muchas personas. Aún puedes hacer muchas cosas sin necesitar expresiones regulares todavía.

Ahora sabes que cuando las necesites puedes usarlas en `FastAPI`.

Valores por defecto

Puedes, por supuesto, usar valores por defecto diferentes de `None`.

Digamos que quieres declarar el parámetro de `query` `q` para que tenga un `min_length` de `3`, y para que tenga un valor por defecto de "`fixedquery`":

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items/")
async def read_items(q: Annotated[str,
Query(min_length=3)] = "fixedquery"):
    results = {"items": [{"item_id": "Foo"}, {"item_id":
"Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

> **Nota**
> Tener un valor por defecto de cualquier tipo, incluyendo `None`, hace que el parámetro sea opcional (no requerido).

Parámetros requeridos

Cuando no necesitamos declarar más validaciones o metadatos, podemos hacer que el parámetro de `query` `q` sea requerido simplemente no declarando un valor por defecto, como:

```
q: str
```

en lugar de:

```
q: str | None = None
```

Pero ahora lo estamos declarando con `Query`, por ejemplo, como:

```
q: Annotated[str | None, Query(min_length=3)] = None
```

Así que, cuando necesites declarar un valor como requerido mientras usas `Query`, simplemente puedes no declarar un valor por defecto:

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items/")
async def read_items(q: Annotated[str,
Query(min_length=3)]):
    results = {"items": [{"item_id": "Foo"}, {"item_id":
"Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

Requerido, puede ser `None`

Puedes declarar que un parámetro puede aceptar `None`, pero que aún así es requerido. Esto obligaría a los clientes a enviar un valor, incluso si el valor es `None`.

Para hacer eso, puedes declarar que `None` es un tipo válido pero simplemente no declarar un valor por defecto:

```
from typing import Annotated

from fastapi import FastAPI, Query
```

```python
app = FastAPI()

@app.get("/items/")
async def read_items(q: Annotated[str | None,
Query(min_length=3)]):
    results = {"items": [{"item_id": "Foo"}, {"item_id":
"Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

Lista de parámetros de `Query` / múltiples valores

Cuando defines un parámetro de `query` explícitamente con `Query` también puedes declararlo para recibir una lista de valores, o dicho de otra manera, para recibir múltiples valores.

Por ejemplo, para declarar un parámetro de `query` `q` que puede aparecer varias veces en la `URL`, puedes escribir:

```python
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items/")
async def read_items(q: Annotated[list[str] | None,
Query()] = None):
    query_items = {"q": q}
    return query_items
```

Entonces, con una URL como:

```
http://localhost:8000/items/?q=foo&q=bar
```

recibirías los múltiples valores de los *`query parameters`* `q` (`foo` y `bar`) en una `list` de `Python` dentro de tu *`path operation`*

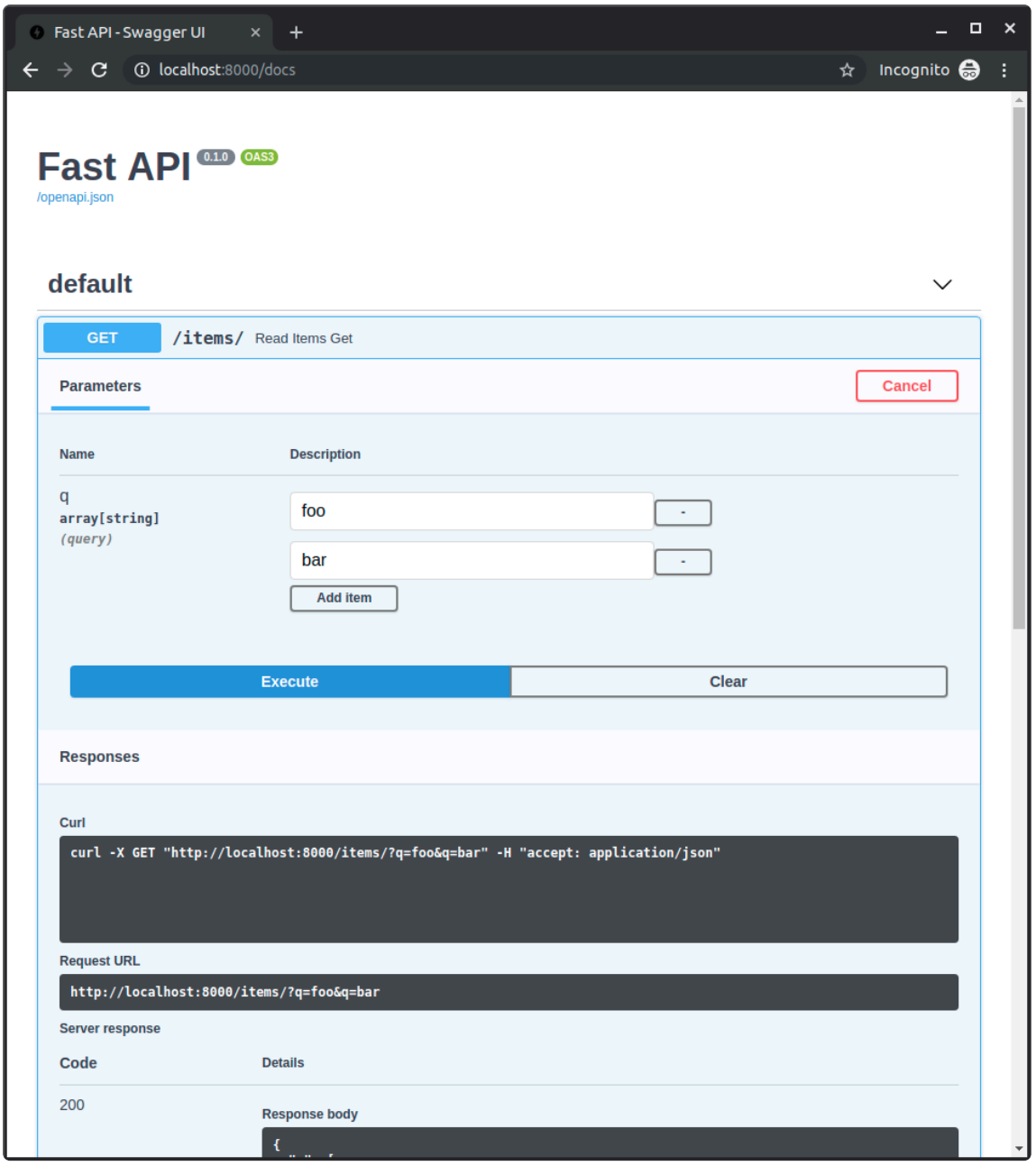
`function`, en el *parámetro de función* `q`.

Entonces, la respuesta a esa `URL` sería:

```
{
  "q": [
    "foo",
    "bar"
  ]
}
```

> **Consejo**
> Para declarar un parámetro de `query` con un tipo de `list`, como en el ejemplo anterior, necesitas usar explícitamente `Query`, de lo contrario sería interpretado como un `request body`.

La documentación interactiva de `API` se actualizará en consecuencia, para permitir múltiples valores:



Lista de parámetros de `Query` / múltiples valores con valores por defecto

También puedes definir un valor por defecto `list` de valores si no se proporciona ninguno:

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items/")
async def read_items(q: Annotated[list[str], Query()] =
```

```
["foo", "bar"]):
    query_items = {"q": q}
    return query_items
```

Si vas a:

```
http://localhost:8000/items/
```

el valor por defecto de `q` será: `["foo", "bar"]` y tu respuesta será:

```
{
  "q": [
    "foo",
    "bar"
  ]
}
```

Usando solo `list`

También puedes usar `list` directamente en lugar de `list[str]`:

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items/")
async def read_items(q: Annotated[list, Query()] = []):
    query_items = {"q": q}
    return query_items
```

> **Nota**
> Ten en cuenta que en este caso, `FastAPI` no revisará el contenido de la `list`.

Por ejemplo, `list[int]` revisaría (y documentaría) que el contenido de la `list` son enteros. Pero `list` sola no lo haría.

Declarar más metadatos

Puedes agregar más información sobre el parámetro.

Esa información se incluirá en el `OpenAPI` generado y será utilizada por las interfaces de usuario de documentación y herramientas externas.

> **Nota**
> Ten en cuenta que diferentes herramientas podrían tener diferentes niveles de soporte de `OpenAPI`.
>
> Algunas de ellas podrían no mostrar toda la información extra declarada todavía, aunque en la mayoría de los casos, la funcionalidad faltante ya está planificada para desarrollo.

Puedes agregar un `title`:

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items/")
async def read_items(
    q: Annotated[str | None, Query(title="Query string",
min_length=3)] = None,
):
    results = {"items": [{"item_id": "Foo"}, {"item_id":
"Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

Y una `description`:

```python
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items/")
async def read_items(
    q: Annotated[
        str | None,
        Query(
            title="Query string",
            description="Query string for the items to
search in the database that have a good match",
            min_length=3,
        ),
    ] = None,
):
    results = {"items": [{"item_id": "Foo"}, {"item_id":
"Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

Alias para parámetros

Imagina que quieres que el parámetro sea `item-query`.

Como en:

```
http://127.0.0.1:8000/items/?item-query=foobaritems
```

Pero `item-query` no es un nombre de variable válido en `Python`.

Lo más cercano sería `item_query`.

Pero aún necesitas que sea exactamente `item-query`...

Entonces puedes declarar un `alias`, y ese alias será usado para encontrar el valor del parámetro:

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items/")
async def read_items(q: Annotated[str | None,
Query(alias="item-query")] = None):
    results = {"items": [{"item_id": "Foo"}, {"item_id":
"Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

Declarar parámetros obsoletos

Ahora digamos que ya no te gusta este parámetro.

Tienes que dejarlo allí por un tiempo porque hay clientes usándolo, pero quieres que la documentación lo muestre claramente como obsoleto.

Luego pasa el parámetro `deprecated=True` a `Query`:

```
from typing import Annotated

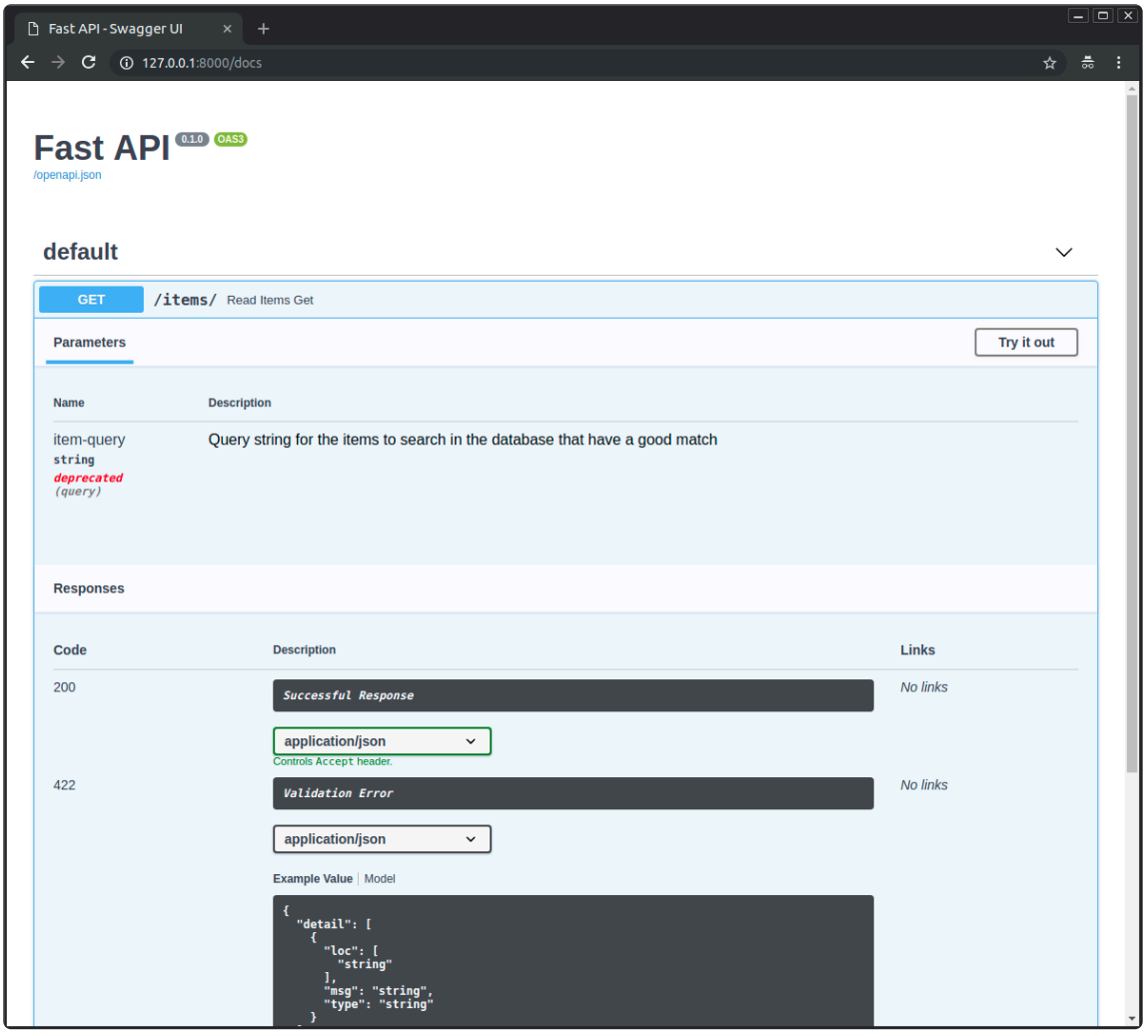
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items/")
async def read_items(
    q: Annotated[
        str | None,
        Query(
            alias="item-query",
            title="Query string",
            description="Query string for the items to
search in the database that have a good match",
```

```
            min_length=3,
            max_length=50,
            pattern="^fixedquery$",
            deprecated=True,
        ),
    ] = None,
):
    results = {"items": [{"item_id": "Foo"}, {"item_id":
"Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

La documentación lo mostrará así:



Excluir parámetros de `OpenAPI`

Para excluir un parámetro de `query` del esquema de `OpenAPI` generado (y por lo tanto, de los sistemas de documentación

automática), establece el parámetro `include_in_schema` de `Query` a `False`:

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items/")
async def read_items(
    hidden_query: Annotated[str | None,
Query(include_in_schema=False)] = None,
):
    if hidden_query:
        return {"hidden_query": hidden_query}
    else:
        return {"hidden_query": "Not found"}
```

Validación personalizada

Podría haber casos donde necesites hacer alguna **validación personalizada** que no puede hacerse con los parámetros mostrados arriba.

En esos casos, puedes usar una **función validadora personalizada** que se aplique después de la validación normal (por ejemplo, después de validar que el valor es un `str`).

Puedes lograr eso usando `AfterValidator` de `Pydantic` dentro de `Annotated`.

Consejo

`Pydantic` también tiene `BeforeValidator` y otros.
0
Por ejemplo, este validador personalizado revisa que el `ID` del ítem empiece con `isbn-` para un número de libro `ISBN` o con `imdb-` para un `ID` de `URL` de película de `IMDB`:

```python
import random
from typing import Annotated

from fastapi import FastAPI
from pydantic import AfterValidator

app = FastAPI()

data = {
    "isbn-9781529046137": "The Hitchhiker's Guide to the Galaxy",
    "imdb-tt0371724": "The Hitchhiker's Guide to the Galaxy",
    "isbn-9781439512982": "Isaac Asimov: The Complete Stories, Vol. 2",
}

def check_valid_id(id: str):
    if not id.startswith(("isbn-", "imdb-")):
        raise ValueError('Invalid ID format, it must start with "isbn-" or "imdb-"')
    return id

@app.get("/items/")
async def read_items(
    id: Annotated[str | None, AfterValidator(check_valid_id)] = None,
):
    if id:
        item = data.get(id)
    else:
        id, item = random.choice(list(data.items()))
    return {"id": id, "name": item}
```

Nota

Esto está disponible con `Pydantic` versión 2 o superior.

Consejo

Si necesitas hacer cualquier tipo de validación que requiera comunicarte con algún **componente externo**, como una base de datos u otra `API`, deberías usar **Dependencias de `FastAPI`**, las aprenderás más adelante.

Estos validadores personalizados son para cosas que pueden revisarse **solo** con los **mismos datos** provistos en el `request`.

Entiende ese código

El punto importante es solo usar `AfterValidator` **con una función dentro de** `Annotated`. Si quieres, sáltate esta parte.

Pero si te da curiosidad este ejemplo de código específico y sigues entretenido, aquí tienes algunos detalles extra.

`String` con `value.startswith()`

¿Lo notaste? Un `string` usando `value.startswith()` puede recibir una tupla, y revisará cada valor en la tupla:

```
# Code above omitted

def check_valid_id(id: str):
    if not id.startswith(("isbn-", "imdb-")):
        raise ValueError('Invalid ID format, it must start
with "isbn-" or "imdb-"')
    return id

# Code below omitted
```

```
import random
from typing import Annotated

from fastapi import FastAPI
from pydantic import AfterValidator
```

```
app = FastAPI()

data = {
    "isbn-9781529046137": "The Hitchhiker's Guide to the
Galaxy",
    "imdb-tt0371724": "The Hitchhiker's Guide to the
Galaxy",
    "isbn-9781439512982": "Isaac Asimov: The Complete
Stories, Vol. 2",
}

def check_valid_id(id: str):
    if not id.startswith(("isbn-", "imdb-")):
        raise ValueError('Invalid ID format, it must start
with "isbn-" or "imdb-"')
    return id

@app.get("/items/")
async def read_items(
    id: Annotated[str | None,
AfterValidator(check_valid_id)] = None,
):
    if id:
        item = data.get(id)
    else:
        id, item = random.choice(list(data.items()))
    return {"id": id, "name": item}
```

Un ítem aleatorio

Con `data.items()` obtenemos un objeto iterable con tuplas que contienen la clave y el valor para cada elemento del diccionario.

Convertimos este objeto iterable en una `list` propiamente dicha con `list(data.items())`.

Luego con `random.choice()` podemos obtener un **valor aleatorio** de la `list`, así que obtenemos una tupla con `(id, name)`. Será algo como `("imdb-tt0371724", "The Hitchhiker's Guide to the Galaxy")`.

Luego **asignamos esos dos valores** de la tupla a las variables `id` y `name`.

Así, si el usuario no proporcionó un `ID` de ítem, aún recibirá una sugerencia aleatoria.

...hacemos todo esto en una **sola línea simple**. ¿No te encanta `Python`?

```
# Code above omitted

@app.get("/items/")
async def read_items(
    id: Annotated[str | None,
AfterValidator(check_valid_id)] = None,
):
    if id:
        item = data.get(id)
    else:
        id, item = random.choice(list(data.items()))
    return {"id": id, "name": item}
```

```
import random
from typing import Annotated

from fastapi import FastAPI
from pydantic import AfterValidator

app = FastAPI()

data = {
    "isbn-9781529046137": "The Hitchhiker's Guide to the
Galaxy",
    "imdb-tt0371724": "The Hitchhiker's Guide to the
Galaxy",
    "isbn-9781439512982": "Isaac Asimov: The Complete
Stories, Vol. 2",
}

def check_valid_id(id: str):
    if not id.startswith(("isbn-", "imdb-")):
```

```
        raise ValueError('Invalid ID format, it must start
with "isbn-" or "imdb-"')
    return id

@app.get("/items/")
async def read_items(
    id: Annotated[str | None,
AfterValidator(check_valid_id)] = None,
):
    if id:
        item = data.get(id)
    else:
        id, item = random.choice(list(data.items()))
    return {"id": id, "name": item}
```

Recapitulación

Puedes declarar validaciones y metadatos adicionales para tus parámetros.

Validaciones genéricas y metadatos:

- `alias`
- `title`
- `description`
- `deprecated`

Validaciones específicas para strings:

- `min_length`
- `max_length`
- `pattern`

Validaciones personalizadas usando `AfterValidator`.

En estos ejemplos viste cómo declarar validaciones para valores de tipo `str`.

Mira los siguientes capítulos para aprender cómo declarar validaciones para otros tipos, como números.

Parámetros de Path y Validaciones Numéricas

De la misma manera que puedes declarar más validaciones y metadatos para los parámetros de query con `Query`, puedes declarar el mismo tipo de validaciones y metadatos para los parámetros de path con `Path`.

Importar Path

Primero, importa `Path` de `fastapi`, e importa `Annotated`:

```
from typing import Annotated

from fastapi import FastAPI, Path, Query

app = FastAPI()
```

> **Nota**
>
> FastAPI agregó soporte para `Annotated` (y comenzó a recomendar su uso) en la versión 0.95.0.
>
> Si tienes una versión anterior, obtendrás errores al intentar usar `Annotated`.
>
> Asegúrate de Actualizar la versión de FastAPI a al menos la 0.95.1 antes de usar `Annotated`.

Declarar metadatos

Puedes declarar todos los mismos parámetros que para `Query`.

Por ejemplo, para declarar un valor de metadato `title` para el parámetro de path `item_id` puedes escribir:

```
@app.get("/items/{item_id}")
async def read_items(
    item_id: Annotated[int, Path(title="The ID of the item
to get")],
```

```
    q: Annotated[str | None, Query(alias="item-query")] =
None,
):
    results = {"item_id": item_id}
    if q:
        results.update({"q": q})
    return results
```

> **Nota**
> Un parámetro de path siempre es requerido ya que tiene que formar parte del path. Incluso si lo declaras con `None` o le asignas un valor por defecto, no afectará en nada, siempre será requerido.

Ordena los parámetros como necesites

> **Consejo**
> Esto probablemente no es tan importante o necesario si usas `Annotated`.

Supongamos que quieres declarar el parámetro de query `q` como un `str` requerido.

Y no necesitas declarar nada más para ese parámetro, así que realmente no necesitas usar `Query`.

Pero aún necesitas usar `Path` para el parámetro de path `item_id`. Y no quieres usar `Annotated` por alguna razón.

Python se quejará si pones un valor con "por defecto" antes de un valor que no tenga "por defecto".

Pero puedes reordenarlos y poner el valor sin un valor por defecto (el parámetro de query `q`) primero.

No importa para **FastAPI**. Detectará los parámetros por sus nombres, tipos y declaraciones por defecto (`Query`, `Path`, etc), no le importa el orden.

Así que puedes declarar tu función como:

```
from fastapi import FastAPI, Path

app = FastAPI()

@app.get("/items/{item_id}")
async def read_items(q: str, item_id: int =
Path(title="The ID of the item to get")):
    results = {"item_id": item_id}
    if q:
        results.update({"q": q})
    return results
```

Pero ten en cuenta que si usas `Annotated`, no tendrás este problema, no importará ya que no estás usando los valores por defecto de los parámetros de la función para `Query()` o `Path()`.

```
from typing import Annotated

from fastapi import FastAPI, Path

app = FastAPI()

@app.get("/items/{item_id}")
async def read_items(
    q: str, item_id: Annotated[int, Path(title="The ID of
the item to get")]
):
    results = {"item_id": item_id}
    if q:
        results.update({"q": q})
    return results
```

Ordena los parámetros como necesites, trucos

Consejo

Esto probablemente no es tan importante o necesario si usas `Annotated`.

Aquí hay un **pequeño truco** que puede ser útil, pero no lo necesitarás a menudo.

Si quieres:

- declarar el parámetro de query `q` sin un `Query` ni ningún valor por defecto
- declarar el parámetro de path `item_id` usando `Path`
- tenerlos en un orden diferente
- no usar `Annotated`

...Python tiene una sintaxis especial para eso.

Pasa `*`, como el primer parámetro de la función.

Python no hará nada con ese `*`, pero sabrá que todos los parámetros siguientes deben ser llamados como argumentos de palabras clave (parejas key-value), también conocidos como `kwargs`. Incluso si no tienen un valor por defecto.

```
from fastapi import FastAPI, Path

app = FastAPI()

@app.get("/items/{item_id}")
async def read_items(*, item_id: int = Path(title="The ID
of the item to get"), q: str):
    results = {"item_id": item_id}
    if q:
        results.update({"q": q})
    return results
```

Mejor con Annotated

Ten en cuenta que si usas `Annotated`, como no estás usando valores por defecto de los parámetros de la función, no tendrás

este problema y probablemente no necesitarás usar `*`.

```
from typing import Annotated

from fastapi import FastAPI, Path

app = FastAPI()

@app.get("/items/{item_id}")
async def read_items(
    item_id: Annotated[int, Path(title="The ID of the item
to get")], q: str
):
    results = {"item_id": item_id}
    if q:
        results.update({"q": q})
    return results
```

Validaciones numéricas: mayor o igual

Con `Query` y `Path` (y otros que verás más adelante) puedes declarar restricciones numéricas.

Aquí, con `ge=1`, `item_id` necesitará ser un número entero " `g` reater than or `e` qual" a `1`.

```
from typing import Annotated

from fastapi import FastAPI, Path

app = FastAPI()

@app.get("/items/{item_id}")
async def read_items(
    item_id: Annotated[int, Path(title="The ID of the item
to get", ge=1)], q: str
):
    results = {"item_id": item_id}
    if q:
```

```
        results.update({"q": q})
    return results
```

Validaciones numéricas: mayor que y menor o igual

Lo mismo aplica para:

- `gt` : `g` reater `t` han
- `le` : `l` ess than or `e` qual

```
from typing import Annotated

from fastapi import FastAPI, Path

app = FastAPI()

@app.get("/items/{item_id}")
async def read_items(
    item_id: Annotated[int, Path(title="The ID of the item to get", gt=0, le=1000)],
    q: str,
):
    results = {"item_id": item_id}
    if q:
        results.update({"q": q})
    return results
```

Validaciones numéricas: flotantes, mayor y menor

Las validaciones numéricas también funcionan para valores `float` .

Aquí es donde se convierte en importante poder declarar `gt` y no solo `ge` . Ya que con esto puedes requerir, por ejemplo, que un valor sea mayor que `0` , incluso si es menor que `1` .

Así, `0.5` sería un valor válido. Pero `0.0` o `0` no lo serían.

Y lo mismo para `lt`.

```
from typing import Annotated

from fastapi import FastAPI, Path, Query

app = FastAPI()

@app.get("/items/{item_id}")
async def read_items(
    *,
    item_id: Annotated[int, Path(title="The ID of the item
to get", ge=0, le=1000)],
    q: str,
    size: Annotated[float, Query(gt=0, lt=10.5)],
):
    results = {"item_id": item_id}
    if q:
        results.update({"q": q})
    if size:
        results.update({"size": size})
    return results
```

Resumen

Con `Query`, `Path` (y otros que aún no has visto) puedes declarar metadatos y validaciones de string de las mismas maneras que con Parámetros de Query y Validaciones de String.

Y también puedes declarar validaciones numéricas:

- `gt`: `g` reater `t` han
- `ge`: `g` reater than or `e` qual
- `lt`: `l` ess `t` han
- `le`: `l` ess than or `e` qual

> **Nota**
> `Query`, `Path` y otras clases que verás más adelante son subclases de una clase común `Param`.

Todas ellas comparten los mismos parámetros para validación adicional y metadatos que has visto.

Detalles técnicos

Cuando importas `Query`, `Path` y otros de `fastapi`, en realidad son funciones.

Que cuando se llaman, retornan instancias de clases con el mismo nombre.

Así que importas `Query`, que es una función. Y cuando la llamas, retorna una instancia de una clase también llamada `Query`.

Estas funciones están allí (en lugar de usar simplemente las clases directamente) para que tu editor no marque errores sobre sus tipos.

De esa forma puedes usar tu editor y herramientas de programación normales sin tener que agregar configuraciones personalizadas para omitir esos errores.

Modelos de Parámetros Query

Si tienes un grupo de **parámetros query** que están relacionados, puedes crear un **modelo de Pydantic** para declararlos.

Esto te permitiría **reutilizar el modelo** en **múltiples lugares** y también declarar validaciones y metadatos para todos los parámetros de una vez.

Nota

Esto es compatible desde la versión `0.115.0` de FastAPI.

Parámetros Query con un Modelo Pydantic

Declara los **parámetros query** que necesitas en un **modelo de Pydantic**, y luego declara el parámetro como `Query`:

```python
from typing import Annotated, Literal

from fastapi import FastAPI, Query
from pydantic import BaseModel, Field

app = FastAPI()

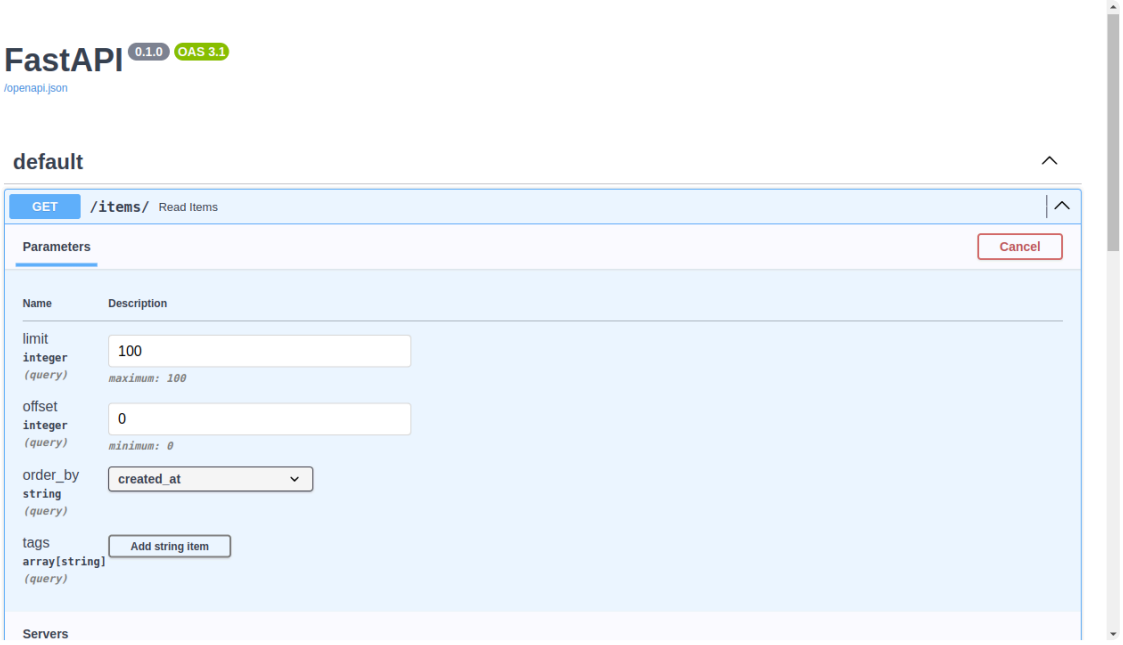
class FilterParams(BaseModel):
    limit: int = Field(100, gt=0, le=100)
    offset: int = Field(0, ge=0)
    order_by: Literal["created_at", "updated_at"] =
"created_at"
    tags: list[str] = []

@app.get("/items/")
async def read_items(filter_query: Annotated[FilterParams,
Query()]):
    return filter_query
```

FastAPI extraerá los datos para **cada campo** de los **parámetros query** en el request y te proporcionará el modelo de Pydantic que definiste.

Revisa la Documentación

Puedes ver los parámetros query en la UI de documentación en `/docs`:



Prohibir Parámetros Query Extras

En algunos casos de uso especiales (probablemente no muy comunes), podrías querer **restringir** los parámetros query que deseas recibir.

Puedes usar la configuración del modelo de Pydantic para `forbid` cualquier campo `extra`:

```
from typing import Annotated, Literal

from fastapi import FastAPI, Query
from pydantic import BaseModel, Field

app = FastAPI()

class FilterParams(BaseModel):
    model_config = {"extra": "forbid"}

    limit: int = Field(100, gt=0, le=100)
    offset: int = Field(0, ge=0)
    order_by: Literal["created_at", "updated_at"] =
"created_at"
    tags: list[str] = []

@app.get("/items/")
async def read_items(filter_query: Annotated[FilterParams,
```

```
Query()]):
    return filter_query
```

Si un cliente intenta enviar algunos datos **extra** en los **parámetros query**, recibirán un response de **error**.

Por ejemplo, si el cliente intenta enviar un parámetro query `tool` con un valor de `plumbus`, como:

```
https://example.com/items/?limit=10&tool=plumbus
```

Recibirán un response de **error** que les indica que el parámetro query `tool` no está permitido:

```
{
    "detail": [
        {
            "type": "extra_forbidden",
            "loc": ["query", "tool"],
            "msg": "Extra inputs are not permitted",
            "input": "plumbus"
        }
    ]
}
```

Resumen

Puedes usar **modelos de Pydantic** para declarar **parámetros query** en **FastAPI**.

> **Consejo**
> Alerta de spoiler: también puedes usar modelos de Pydantic para declarar cookies y headers, pero leerás sobre eso más adelante en el tutorial.

Cuerpo - Múltiples Parámetros

Ahora que hemos visto cómo usar `Path` y `Query`, veamos usos más avanzados de las declaraciones del request body.

Mezclar Path, Query y parámetros del cuerpo

Primero, por supuesto, puedes mezclar las declaraciones de parámetros de `Path`, `Query` y del request body libremente y **FastAPI** sabrá qué hacer.

Y también puedes declarar parámetros del cuerpo como opcionales, estableciendo el valor por defecto a `None`:

```
from typing import Annotated

from fastapi import FastAPI, Path
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

@app.put("/items/{item_id}")
async def update_item(
    item_id: Annotated[int, Path(title="The ID of the item
to get", ge=0, le=1000)],
    q: str | None = None,
    item: Item | None = None,
):
    results = {"item_id": item_id}
    if q:
        results.update({"q": q})
    if item:
        results.update({"item": item})
    return results
```

Nota

Ten en cuenta que, en este caso, el `item` que se tomaría del cuerpo es opcional. Ya que tiene un valor por defecto de `None`.

Múltiples parámetros del cuerpo

En el ejemplo anterior, las *path operations* esperarían un cuerpo JSON con los atributos de un `Item`, como:

```
{
    "name": "Foo",
    "description": "The pretender",
    "price": 42.0,
    "tax": 3.2
}
```

Pero también puedes declarar múltiples parámetros del cuerpo, por ejemplo `item` y `user`:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

class User(BaseModel):
    username: str
    full_name: str | None = None

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item, user:
User):
    results = {"item_id": item_id, "item": item, "user":
user}
    return results
```

En este caso, **FastAPI** notará que hay más de un parámetro del cuerpo en la función (hay dos parámetros que son modelos de Pydantic).

Entonces, usará los nombres de los parámetros como claves (nombres de campo) en el cuerpo, y esperará un cuerpo como:

```
{
    "item": {
        "name": "Foo",
        "description": "The pretender",
        "price": 42.0,
        "tax": 3.2
    },
    "user": {
        "username": "dave",
        "full_name": "Dave Grohl"
    }
}
```

> **Nota**
> Ten en cuenta que aunque el `item` se declaró de la misma manera que antes, ahora se espera que esté dentro del cuerpo con una clave `item`.

FastAPI hará la conversión automática del request, de modo que el parámetro `item` reciba su contenido específico y lo mismo para `user`.

Realizará la validación de los datos compuestos, y los documentará así para el esquema de OpenAPI y la documentación automática.

Valores singulares en el cuerpo

De la misma manera que hay un `Query` y `Path` para definir datos extra para parámetros de query y path, **FastAPI** proporciona un equivalente `Body`.

Por ejemplo, ampliando el modelo anterior, podrías decidir que deseas tener otra clave `importance` en el mismo cuerpo, además de `item` y `user`.

Si lo declaras tal cual, debido a que es un valor singular, **FastAPI** asumirá que es un parámetro de query.

Pero puedes instruir a **FastAPI** para que lo trate como otra clave del cuerpo usando `Body`:

```
from typing import Annotated

from fastapi import Body, FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

class User(BaseModel):
    username: str
    full_name: str | None = None

@app.put("/items/{item_id}")
async def update_item(
    item_id: int, item: Item, user: User, importance:
Annotated[int, Body()]
):
    results = {"item_id": item_id, "item": item, "user":
user, "importance": importance}
    return results
```

En este caso, **FastAPI** esperará un cuerpo como:

```
{
    "item": {
        "name": "Foo",
```

```
        "description": "The pretender",
        "price": 42.0,
        "tax": 3.2
    },
    "user": {
        "username": "dave",
        "full_name": "Dave Grohl"
    },
    "importance": 5
}
```

Nuevamente, convertirá los tipos de datos, validará, documentará, etc.

Múltiples parámetros de cuerpo y query

Por supuesto, también puedes declarar parámetros adicionales de query siempre que lo necesites, además de cualquier parámetro del cuerpo.

Como, por defecto, los valores singulares se interpretan como parámetros de query, no tienes que añadir explícitamente un `Query`, solo puedes hacer:

```
q: str | None = None
```

Por ejemplo:

```
from typing import Annotated

from fastapi import Body, FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
```

```
    tax: float | None = None

class User(BaseModel):
    username: str
    full_name: str | None = None

@app.put("/items/{item_id}")
async def update_item(
    *,
    item_id: int,
    item: Item,
    user: User,
    importance: Annotated[int, Body(gt=0)],
    q: str | None = None,
):
    results = {"item_id": item_id, "item": item, "user":
user, "importance": importance}
    if q:
        results.update({"q": q})
    return results
```

> **Nota**
> `Body` también tiene todos los mismos parámetros de validación y metadatos extras que `Query`, `Path` y otros que verás luego.

Embeber un solo parámetro de cuerpo

Supongamos que solo tienes un único parámetro de cuerpo `item` de un modelo Pydantic `Item`.

Por defecto, **FastAPI** esperará su cuerpo directamente.

Pero si deseas que espere un JSON con una clave `item` y dentro de ella los contenidos del modelo, como lo hace cuando declaras parámetros de cuerpo extra, puedes usar el parámetro especial `Body` `embed`:

```
item: Annotated[Item, Body(embed=True)]
```

como en:

```python
from typing import Annotated

from fastapi import Body, FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Annotated[Item,
Body(embed=True)]):
    results = {"item_id": item_id, "item": item}
    return results
```

En este caso, **FastAPI** esperará un cuerpo como:

```json
{
    "item": {
        "name": "Foo",
        "description": "The pretender",
        "price": 42.0,
        "tax": 3.2
    }
}
```

en lugar de:

```json
{
    "name": "Foo",
    "description": "The pretender",
    "price": 42.0,
```

```
    "tax": 3.2
}
```

Resumen

Puedes añadir múltiples parámetros de cuerpo a tu *path operation function*, aunque un request solo puede tener un único cuerpo.

Pero **FastAPI** lo manejará, te dará los datos correctos en tu función, y validará y documentará el esquema correcto en la *path operation*.

También puedes declarar valores singulares para ser recibidos como parte del cuerpo.

Y puedes instruir a **FastAPI** para embeber el cuerpo en una clave incluso cuando solo hay un único parámetro declarado.

Body - Campos

De la misma manera que puedes declarar validaciones adicionales y metadatos en los parámetros de las *path operation function* con `Query`, `Path` y `Body`, puedes declarar validaciones y metadatos dentro de los modelos de Pydantic usando `Field` de Pydantic.

Importar Field

Primero, tienes que importarlo:

```
from typing import Annotated

from fastapi import Body, FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = Field(
        default=None, title="The description of the item",
max_length=300
```

```
    )
    price: float = Field(gt=0, description="The price must
be greater than zero")
    tax: float | None = None

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Annotated[Item,
Body(embed=True)]):
    results = {"item_id": item_id, "item": item}
    return results
```

> ⚠ **Advertencia**
> Fíjate que `Field` se importa directamente desde `pydantic`, no desde `fastapi` como el resto (`Query`, `Path`, `Body`, etc).

Declarar atributos del modelo

Después puedes utilizar `Field` con los atributos del modelo:

```
from typing import Annotated

from fastapi import Body, FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = Field(
        default=None, title="The description of the item",
max_length=300
    )
    price: float = Field(gt=0, description="The price must
be greater than zero")
    tax: float | None = None

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Annotated[Item,
Body(embed=True)]):
```

```
    results = {"item_id": item_id, "item": item}
    return results
```

`Field` funciona de la misma manera que `Query`, `Path` y `Body`, tiene todos los mismos parámetros, etc.

Detalles técnicos

En realidad, `Query`, `Path` y otros que verás a continuación crean objetos de subclases de una clase común `Param`, que es a su vez una subclase de la clase `FieldInfo` de Pydantic.

Y `Field` de Pydantic también devuelve una instance de `FieldInfo`.

`Body` también devuelve objetos de una subclase de `FieldInfo` directamente. Y hay otros que verás más adelante que son subclases de la clase `Body`.

Recuerda que cuando importas `Query`, `Path`, y otros desde `fastapi`, en realidad son funciones que devuelven clases especiales.

Consejo

Observa cómo cada atributo del modelo con un tipo, un valor por defecto y `Field` tiene la misma estructura que un parámetro de una *path operation function*, con `Field` en lugar de `Path`, `Query` y `Body`.

Agregar información extra

Puedes declarar información extra en `Field`, `Query`, `Body`, etc. Y será incluida en el JSON Schema generado.

Aprenderás más sobre cómo agregar información extra más adelante en la documentación, cuando aprendamos a declarar ejemplos.

Advertencia

Las claves extra pasadas a `Field` también estarán presentes en el esquema de OpenAPI resultante para tu aplicación. Como estas claves no necesariamente tienen que ser parte de la especificación de OpenAPI, algunas herramientas de OpenAPI, por ejemplo el validador de OpenAPI, podrían no funcionar con tu esquema generado.

Resumen

Puedes utilizar `Field` de Pydantic para declarar validaciones adicionales y metadatos para los atributos del modelo.

También puedes usar los argumentos de palabra clave extra para pasar metadatos adicionales del JSON Schema.

Cuerpo - Modelo Anidados

Con **FastAPI**, puedes definir, validar, documentar y usar modelos profundamente anidados de manera arbitraria (gracias a Pydantic).

Campos de lista

Puedes definir un atributo como un subtipo. Por ejemplo, una `list` en Python:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: list = []

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
```

```
    results = {"item_id": item_id, "item": item}
    return results
```

Esto hará que `tags` sea una lista, aunque no declare el tipo de los elementos de la lista.

Campos de lista con parámetro de tipo

Pero Python tiene una forma específica de declarar listas con tipos internos, o "parámetros de tipo":

Declarar una list con un parámetro de tipo

Para declarar tipos que tienen parámetros de tipo (tipos internos), como `list`, `dict`, `tuple`, pasa el/los tipo(s) interno(s) como "parámetros de tipo" usando corchetes: `[` y `]`

```
my_list: list[str]
```

Esa es toda la sintaxis estándar de Python para declaraciones de tipo.

Usa esa misma sintaxis estándar para atributos de modelos con tipos internos.

Así, en nuestro ejemplo, podemos hacer que `tags` sea específicamente una "lista de strings":

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: list[str] = []
```

```python
@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    results = {"item_id": item_id, "item": item}
    return results
```

Tipos de conjunto

Pero luego pensamos en ello, y nos damos cuenta de que los tags no deberían repetirse, probablemente serían strings únicos.

Y Python tiene un tipo de datos especial para conjuntos de elementos únicos, el `set`.

Entonces podemos declarar `tags` como un conjunto de strings:

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    results = {"item_id": item_id, "item": item}
    return results
```

Con esto, incluso si recibes un request con datos duplicados, se convertirá en un conjunto de elementos únicos.

Y siempre que emitas esos datos, incluso si la fuente tenía duplicados, se emitirá como un conjunto de elementos únicos.

Y también se anotará/documentará en consecuencia.

Modelos Anidados

Cada atributo de un modelo Pydantic tiene un tipo.

Pero ese tipo puede ser en sí mismo otro modelo Pydantic.

Así que, puedes declarar "objetos" JSON anidados profundamente con nombres de atributos específicos, tipos y validaciones.

Todo eso, de manera arbitraria.

Definir un submodelo

Por ejemplo, podemos definir un modelo `Image`:

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Image(BaseModel):
    url: str
    name: str

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()
    image: Image | None = None

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    results = {"item_id": item_id, "item": item}
    return results
```

Usar el submodelo como tipo

Y luego podemos usarlo como el tipo de un atributo:

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Image(BaseModel):
    url: str
    name: str

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()
    image: Image | None = None

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    results = {"item_id": item_id, "item": item}
    return results
```

Esto significaría que **FastAPI** esperaría un cuerpo similar a:

```json
{
    "name": "Foo",
    "description": "The pretender",
    "price": 42.0,
    "tax": 3.2,
    "tags": ["rock", "metal", "bar"],
    "image": {
        "url": "http://example.com/baz.jpg",
        "name": "The Foo live"
    }
}
```

Nuevamente, haciendo solo esa declaración, con **FastAPI** obtienes:

- Soporte de editor (autocompletado, etc.), incluso para modelos anidados
- Conversión de datos
- Validación de datos
- Documentación automática

Tipos especiales y validación

Además de tipos singulares normales como `str`, `int`, `float`, etc., puedes usar tipos singulares más complejos que heredan de `str`.

Para ver todas las opciones que tienes, Revisa Resumen de tipos de Pydantic. Verás algunos ejemplos en el siguiente capítulo.

Por ejemplo, como en el modelo `Image` tenemos un campo `url`, podemos declararlo como una instance de `HttpUrl` de Pydantic en lugar de un `str`:

```python
from fastapi import FastAPI
from pydantic import BaseModel, HttpUrl

app = FastAPI()

class Image(BaseModel):
    url: HttpUrl
    name: str

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()
    image: Image | None = None

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
```

```
    results = {"item_id": item_id, "item": item}
    return results
```

El string será verificado para ser una URL válida, y documentado en JSON Schema / OpenAPI como tal.

Atributos con listas de submodelos

También puedes usar modelos Pydantic como subtipos de `list`, `set`, etc.:

```
from fastapi import FastAPI
from pydantic import BaseModel, HttpUrl

app = FastAPI()

class Image(BaseModel):
    url: HttpUrl
    name: str

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()
    images: list[Image] | None = None

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    results = {"item_id": item_id, "item": item}
    return results
```

Esto esperará (convertirá, validará, documentará, etc.) un cuerpo JSON como:

```
{
    "name": "Foo",
    "description": "The pretender",
```

```
    "price": 42.0,
    "tax": 3.2,
    "tags": [
        "rock",
        "metal",
        "bar"
    ],
    "images": [
        {
            "url": "http://example.com/baz.jpg",
            "name": "The Foo live"
        },
        {
            "url": "http://example.com/dave.jpg",
            "name": "The Baz"
        }
    ]
}
```

Nota

Nota cómo la clave `images` ahora tiene una lista de objetos de imagen.

Modelos anidados profundamente

Puedes definir modelos anidados tan profundamente como desees:

```
from fastapi import FastAPI
from pydantic import BaseModel, HttpUrl

app = FastAPI()

class Image(BaseModel):
    url: HttpUrl
    name: str

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
```

```
    tax: float | None = None
    tags: set[str] = set()
    images: list[Image] | None = None

class Offer(BaseModel):
    name: str
    description: str | None = None
    price: float
    items: list[Item]

@app.post("/offers/")
async def create_offer(offer: Offer):
    return offer
```

> **Nota**
> Observa cómo `Offer` tiene una lista de `Item` s, que a su vez tienen una lista opcional de `Image` s

Cuerpos de listas puras

Si el valor superior del cuerpo JSON que esperas es un `array` JSON (una `list` en Python), puedes declarar el tipo en el parámetro de la función, al igual que en los modelos Pydantic:

```
images: list[Image]
```

como en:

```
from fastapi import FastAPI
from pydantic import BaseModel, HttpUrl

app = FastAPI()

class Image(BaseModel):
    url: HttpUrl
    name: str

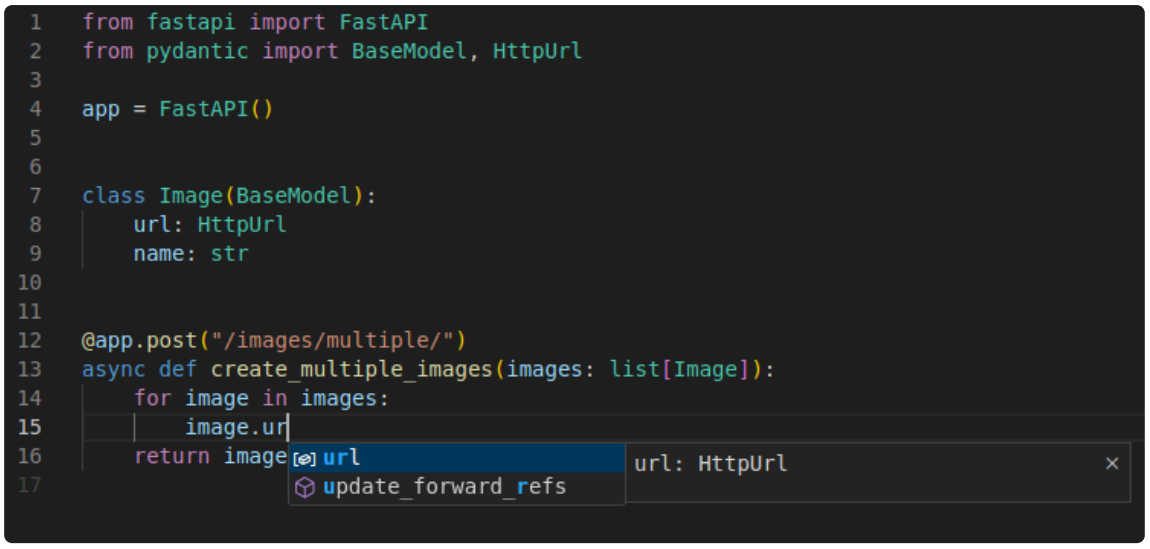
@app.post("/images/multiple/")
```

```python
async def create_multiple_images(images: list[Image]):
    return images
```

Soporte de editor en todas partes

Y obtienes soporte de editor en todas partes.

Incluso para elementos dentro de listas:



No podrías obtener este tipo de soporte de editor si estuvieras trabajando directamente con `dict` en lugar de modelos Pydantic.

Pero tampoco tienes que preocuparte por ellos, los `dicts` entrantes se convierten automáticamente y tu salida se convierte automáticamente a JSON también.

Cuerpos de dicts arbitrarios

También puedes declarar un cuerpo como un `dict` con claves de algún tipo y valores de algún otro tipo.

De esta manera, no tienes que saber de antemano cuáles son los nombres válidos de campo/atributo (como sería el caso con modelos Pydantic).

Esto sería útil si deseas recibir claves que aún no conoces.

Otro caso útil es cuando deseas tener claves de otro tipo (por ejemplo, `int`).

Eso es lo que vamos a ver aquí.

En este caso, aceptarías cualquier `dict` siempre que tenga claves `int` con valores `float`:

```
from fastapi import FastAPI

app = FastAPI()

@app.post("/index-weights/")
async def create_index_weights(weights: dict[int, float]):
    return weights
```

> **Consejo**
>
> Ten en cuenta que JSON solo admite `str` como claves.
>
> Pero Pydantic tiene conversión automática de datos.
>
> Esto significa que, aunque tus clientes de API solo pueden enviar strings como claves, mientras esos strings contengan enteros puros, Pydantic los convertirá y validará.
>
> Y el `dict` que recibas como `weights` tendrá realmente claves `int` y valores `float`.

- # Declara Datos de Ejemplo de Request

Puedes declarar ejemplos de los datos que tu aplicación puede recibir.

Aquí tienes varias formas de hacerlo.

Datos extra de JSON Schema en modelos de Pydantic

Puedes declarar `examples` para un modelo de Pydantic que se añadirá al JSON Schema generado.

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

    model_config = {
        "json_schema_extra": {
            "examples": [
                {
                    "name": "Foo",
                    "description": "A very nice Item",
                    "price": 35.4,
                    "tax": 3.2,
                }
            ]
        }
    }

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    results = {"item_id": item_id, "item": item}
    return results
```

Esa información extra se añadirá tal cual al **JSON Schema** resultante para ese modelo, y se usará en la documentación de la API.

Puedes usar el atributo `model_config` que toma un `dict` como se describe en Documentación de Pydantic: Configuración.

Puedes establecer `"json_schema_extra"` con un `dict` que contenga cualquier dato adicional que te gustaría que aparezca en el JSON Schema generado, incluyendo `examples`.

Consejo

Podrías usar la misma técnica para extender el JSON Schema y añadir tu propia información extra personalizada.

Por ejemplo, podrías usarlo para añadir metadatos para una interfaz de usuario frontend, etc.

Nota

OpenAPI 3.1.0 (usado desde FastAPI 0.99.0) añadió soporte para `examples`, que es parte del estándar de **JSON Schema**.

Antes de eso, solo soportaba la palabra clave `example` con un solo ejemplo. Eso aún es soportado por OpenAPI 3.1.0, pero está obsoleto y no es parte del estándar de JSON Schema. Así que se te anima a migrar `example` a `examples`.

Puedes leer más al final de esta página.

Argumentos adicionales en Field

Cuando usas `Field()` con modelos de Pydantic, también puedes declarar `examples` adicionales:

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class Item(BaseModel):
    name: str = Field(examples=["Foo"])
    description: str | None = Field(default=None,
examples=["A very nice Item"])
    price: float = Field(examples=[35.4])
    tax: float | None = Field(default=None, examples=
[3.2])

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
```

```
    results = {"item_id": item_id, "item": item}
    return results
```

examples en JSON Schema - OpenAPI

Cuando usas cualquiera de:

- `Path()`
- `Query()`
- `Header()`
- `Cookie()`
- `Body()`
- `Form()`
- `File()`

también puedes declarar un grupo de `examples` con información adicional que se añadirá a sus **JSON Schemas** dentro de **OpenAPI**.

Body con examples

Aquí pasamos `examples` que contiene un ejemplo de los datos esperados en `Body()`:

```
from typing import Annotated

from fastapi import Body, FastAPI
from pydantic import BaseModel

app = FastAPI()

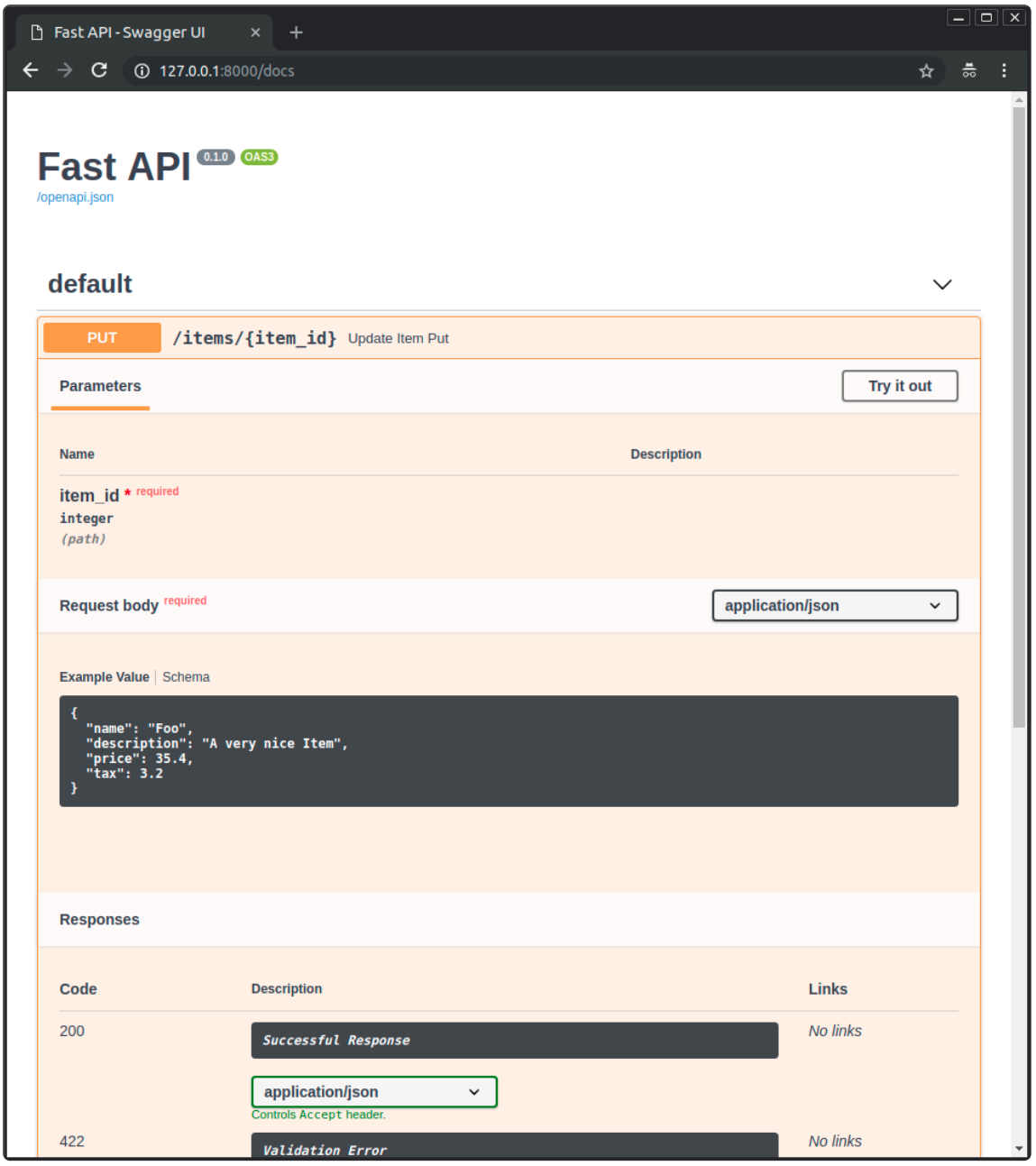
class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

@app.put("/items/{item_id}")
```

```python
async def update_item(
    item_id: int,
    item: Annotated[
        Item,
        Body(
            examples=[
                {
                    "name": "Foo",
                    "description": "A very nice Item",
                    "price": 35.4,
                    "tax": 3.2,
                }
            ],
        ),
    ],
):
    results = {"item_id": item_id, "item": item}
    return results
```

Ejemplo en la interfaz de documentación

Con cualquiera de los métodos anteriores se vería así en los `/docs`:



Body con múltiples examples

Por supuesto, también puedes pasar múltiples `examples`:

```python
from typing import Annotated

from fastapi import Body, FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
```

```
    price: float
    tax: float | None = None

@app.put("/items/{item_id}")
async def update_item(
    *,
    item_id: int,
    item: Annotated[
        Item,
        Body(
            examples=[
                {
                    "name": "Foo",
                    "description": "A very nice Item",
                    "price": 35.4,
                    "tax": 3.2,
                },
                {
                    "name": "Bar",
                    "price": "35.4",
                },
                {
                    "name": "Baz",
                    "price": "thirty five point four",
                },
            ],
        ),
    ],
):
    results = {"item_id": item_id, "item": item}
    return results
```

Cuando haces esto, los ejemplos serán parte del **JSON Schema** interno para esos datos del body.

Sin embargo, al momento de escribir esto, Swagger UI, la herramienta encargada de mostrar la interfaz de documentación, no soporta mostrar múltiples ejemplos para los datos en **JSON Schema**. Pero lee más abajo para una solución alternativa.

examples específicos de OpenAPI

Desde antes de que **JSON Schema** soportara `examples`, OpenAPI tenía soporte para un campo diferente también llamado `examples`.

Estos `examples` específicos de **OpenAPI** van en otra sección en la especificación de OpenAPI. Van en los **detalles para cada *path operation***, no dentro de cada JSON Schema.

Y Swagger UI ha soportado este campo particular de `examples` por un tiempo. Así que, puedes usarlo para **mostrar** diferentes **ejemplos en la interfaz de documentación**.

La forma de este campo específico de OpenAPI `examples` es un `dict` con **múltiples ejemplos** (en lugar de una `list`), cada uno con información adicional que también se añadirá a **OpenAPI**.

Esto no va dentro de cada JSON Schema contenido en OpenAPI, esto va afuera, directamente en la *path operation*.

Usando el Parámetro openapi_examples

Puedes declarar los `examples` específicos de OpenAPI en FastAPI con el parámetro `openapi_examples` para:

- `Path()`
- `Query()`
- `Header()`
- `Cookie()`
- `Body()`
- `Form()`
- `File()`

Las claves del `dict` identifican cada ejemplo, y cada valor es otro `dict`.

Cada `dict` específico del ejemplo en los `examples` puede contener:

- `summary`: Descripción corta del ejemplo.

- `description`: Una descripción larga que puede contener texto Markdown.
- `value`: Este es el ejemplo real mostrado, e.g. un `dict`.
- `externalValue`: alternativa a `value`, una URL que apunta al ejemplo. Aunque esto puede no ser soportado por tantas herramientas como `value`.

Puedes usarlo así:

```
from typing import Annotated

from fastapi import Body, FastAPI
from pydantic import BaseModel

app = FastAPI()

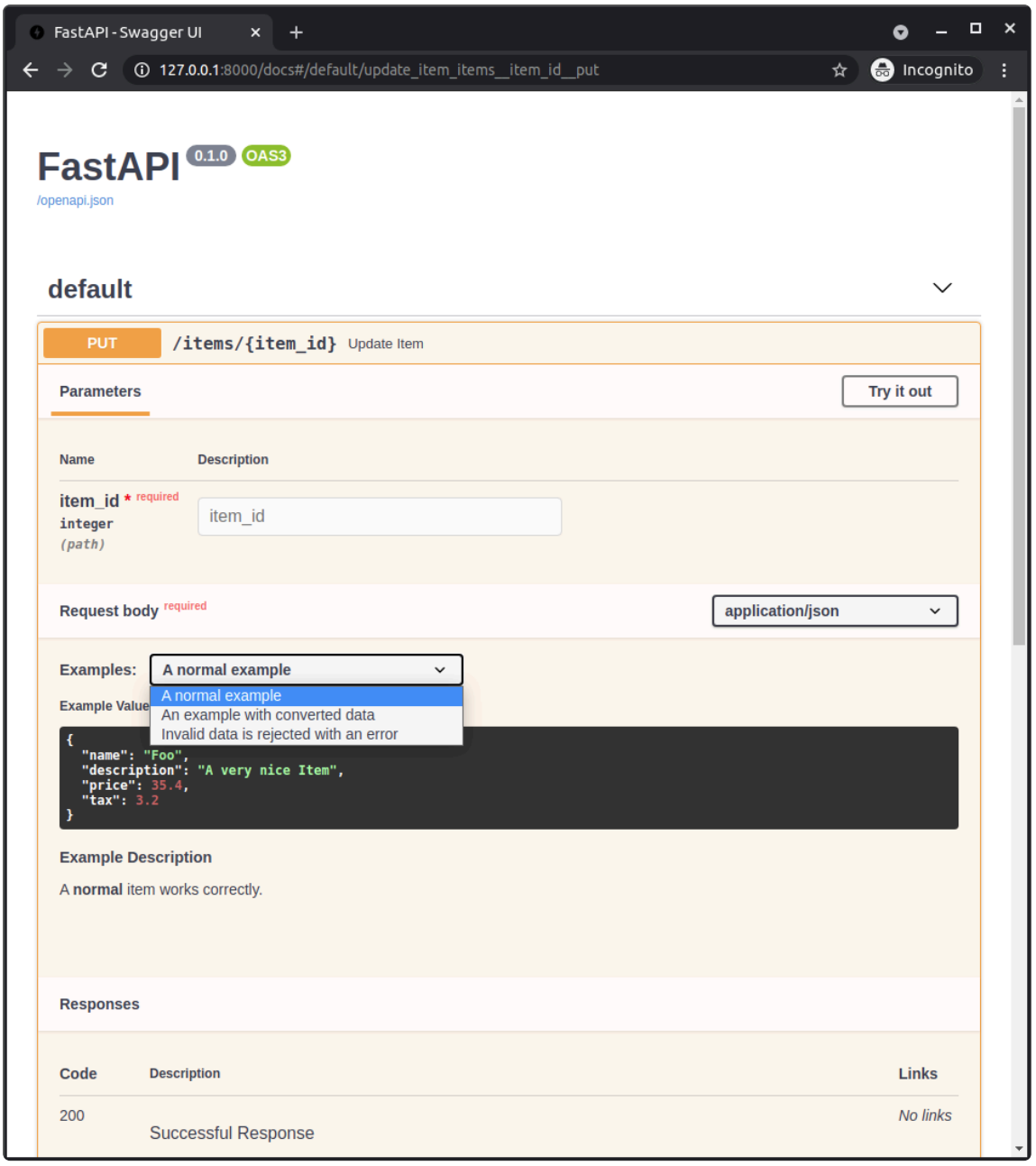
class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

@app.put("/items/{item_id}")
async def update_item(
    *,
    item_id: int,
    item: Annotated[
        Item,
        Body(
            openapi_examples={
                "normal": {
                    "summary": "A normal example",
                    "description": "A **normal** item
works correctly.",
                    "value": {
                        "name": "Foo",
                        "description": "A very nice Item",
                        "price": 35.4,
                        "tax": 3.2,
                    },
```

```
                },
                "converted": {
                    "summary": "An example with converted
data",
                    "description": "FastAPI can convert
price \`strings\` to actual \`numbers\` automatically",
                    "value": {
                        "name": "Bar",
                        "price": "35.4",
                    },
                },
                "invalid": {
                    "summary": "Invalid data is rejected
with an error",
                    "value": {
                        "name": "Baz",
                        "price": "thirty five point four",
                    },
                },
            },
        ),
    ],
):
    results = {"item_id": item_id, "item": item}
    return results
```

Ejemplos de OpenAPI en la Interfaz de Documentación

Con `openapi_examples` añadido a `Body()`, los `/docs` se verían así:



Detalles Técnicos

Consejo

Si ya estás usando la versión **0.99.0 o superior** de **FastAPI**, probablemente puedes **omitir** estos detalles.

Son más relevantes para versiones más antiguas, antes de que OpenAPI 3.1.0 estuviera disponible.

Puedes considerar esto una breve lección de **historia** de OpenAPI y JSON Schema.

Advertencia

Estos son detalles muy técnicos sobre los estándares **JSON Schema** y **OpenAPI**.

Si las ideas anteriores ya funcionan para ti, eso podría ser suficiente, y probablemente no necesites estos detalles, siéntete libre de omitirlos.

Antes de OpenAPI 3.1.0, OpenAPI usaba una versión más antigua y modificada de **JSON Schema**.

JSON Schema no tenía `examples`, así que OpenAPI añadió su propio campo `example` a su versión modificada.

OpenAPI también añadió los campos `example` y `examples` a otras partes de la especificación:

- `Parameter Object` (en la especificación) que era usado por FastAPI:
    - `Path()`
        - `Query()`
        - `Header()`
        - `Cookie()`
- `Request Body Object`, en el campo `content`, sobre el `Media Type Object` (en la especificación) que era usado por FastAPI:
    - `Body()`
        - `File()`
        - `Form()`

/ Nota

Este viejo parámetro `examples` específico de OpenAPI ahora es `openapi_examples` desde FastAPI `0.103.0`.

Campo examples de JSON Schema

Pero luego JSON Schema añadió un campo `examples` a una nueva versión de la especificación.

Y entonces el nuevo OpenAPI 3.1.0 se basó en la última versión (JSON Schema 2020-12) que incluía este nuevo campo `examples`.

Y ahora este nuevo campo `examples` tiene precedencia sobre el viejo campo único (y personalizado) `example`, que ahora está obsoleto.

Este nuevo campo `examples` en JSON Schema es **solo una** `list` de ejemplos, no un dict con metadatos adicionales como en los otros lugares en OpenAPI (descritos arriba).

> **Nota**
> Incluso después de que OpenAPI 3.1.0 fue lanzado con esta nueva integración más sencilla con JSON Schema, por un tiempo, Swagger UI, la herramienta que proporciona la documentación automática, no soportaba OpenAPI 3.1.0 (lo hace desde la versión 5.0.0 🎉).
>
> Debido a eso, las versiones de FastAPI anteriores a 0.99.0 todavía usaban versiones de OpenAPI menores a 3.1.0.

examples de Pydantic y FastAPI

Cuando añades `examples` dentro de un modelo de Pydantic, usando `schema_extra` o `Field(examples=["something"])`, ese ejemplo se añade al **JSON Schema** para ese modelo de Pydantic.

Y ese **JSON Schema** del modelo de Pydantic se incluye en el **OpenAPI** de tu API, y luego se usa en la interfaz de documentación.

En las versiones de FastAPI antes de 0.99.0 (0.99.0 y superiores usan el nuevo OpenAPI 3.1.0) cuando usabas `example` o `examples` con cualquiera de las otras utilidades (`Query()`, `Body()`, etc.) esos ejemplos no se añadían al JSON Schema que describe esos datos (ni siquiera a la propia versión de JSON Schema de OpenAPI), se añadían directamente a la declaración de la *path operation* en OpenAPI (fuera de las partes de OpenAPI que usan JSON Schema).

Pero ahora que FastAPI 0.99.0 y superiores usa OpenAPI 3.1.0, que usa JSON Schema 2020-12, y Swagger UI 5.0.0 y superiores, todo es más consistente y los ejemplos se incluyen en JSON Schema.

Swagger UI y examples específicos de OpenAPI

Ahora, como Swagger UI no soportaba múltiples ejemplos de JSON Schema (a fecha de 2023-08-26), los usuarios no tenían una forma de mostrar múltiples ejemplos en la documentación.

Para resolver eso, FastAPI `0.103.0` **añadió soporte** para declarar el mismo viejo campo **específico de OpenAPI** `examples` con el nuevo parámetro `openapi_examples`.

Resumen

Solía decir que no me gustaba mucho la historia... y mírame ahora dando lecciones de "historia tecnológica".

En resumen, **actualiza a FastAPI 0.99.0 o superior**, y las cosas son mucho **más simples, consistentes e intuitivas**, y no necesitas conocer todos estos detalles históricos.

- # Tipos de Datos Extra

Hasta ahora, has estado usando tipos de datos comunes, como:

- `int`
- `float`
- `str`
- `bool`

Pero también puedes usar tipos de datos más complejos.

Y seguirás teniendo las mismas funcionalidades como hasta ahora:

- Gran soporte de editor.
- Conversión de datos de requests entrantes.
- Conversión de datos para datos de response.

- Validación de datos.
- Anotación y documentación automática.

Otros tipos de datos

Aquí hay algunos de los tipos de datos adicionales que puedes usar:

- `UUID`:
    - Un "Identificador Universalmente Único" estándar, común como un ID en muchas bases de datos y sistemas.
        - En requests y responses se representará como un `str`.
- `datetime.datetime`:
    - Un `datetime.datetime` de Python.
        - En requests y responses se representará como un `str` en formato ISO 8601, como: `2008-09-15T15:53:00+05:00`.
- `datetime.date`:
    - `datetime.date` de Python.
        - En requests y responses se representará como un `str` en formato ISO 8601, como: `2008-09-15`.
- `datetime.time`:
    - Un `datetime.time` de Python.
        - En requests y responses se representará como un `str` en formato ISO 8601, como: `14:23:55.003`.
- `datetime.timedelta`:
    - Un `datetime.timedelta` de Python.
        - En requests y responses se representará como un `float` de segundos totales.
        - Pydantic también permite representarlo como una "codificación de diferencia horaria ISO 8601", consulta la documentación para más información.
- `frozenset`:
    - En requests y responses, tratado igual que un `set`:

        - En requests, se leerá una list, eliminando duplicados y convirtiéndola en un `set`.
        - En responses, el `set` se convertirá en una `list`.
        - El esquema generado especificará que los valores del `set` son únicos (usando `uniqueItems` de JSON Schema).
- `bytes`:
    - `bytes` estándar de Python.
        - En requests y responses se tratará como `str`.
        - El esquema generado especificará que es un `str` con `binary` como "format".
- `Decimal`:
    - `Decimal` estándar de Python.
        - En requests y responses, manejado igual que un `float`.
- Puedes revisar todos los tipos de datos válidos de Pydantic aquí: Tipos de datos de Pydantic.

Ejemplo

Aquí tienes un ejemplo de una *path operation* con parámetros usando algunos de los tipos anteriores.

```
from datetime import datetime, time, timedelta
from typing import Annotated
from uuid import UUID

from fastapi import Body, FastAPI

app = FastAPI()

@app.put("/items/{item_id}")
async def read_items(
    item_id: UUID,
    start_datetime: Annotated[datetime, Body()],
    end_datetime: Annotated[datetime, Body()],
    process_after: Annotated[timedelta, Body()],
```

```
    repeat_at: Annotated[time | None, Body()] = None,
):
    start_process = start_datetime + process_after
    duration = end_datetime - start_process
    return {
        "item_id": item_id,
        "start_datetime": start_datetime,
        "end_datetime": end_datetime,
        "process_after": process_after,
        "repeat_at": repeat_at,
        "start_process": start_process,
        "duration": duration,
    }
```

Nota que los parámetros dentro de la función tienen su tipo de dato natural, y puedes, por ejemplo, realizar manipulaciones de fechas normales, como:

```
from datetime import datetime, time, timedelta
from typing import Annotated
from uuid import UUID

from fastapi import Body, FastAPI

app = FastAPI()

@app.put("/items/{item_id}")
async def read_items(
    item_id: UUID,
    start_datetime: Annotated[datetime, Body()],
    end_datetime: Annotated[datetime, Body()],
    process_after: Annotated[timedelta, Body()],
    repeat_at: Annotated[time | None, Body()] = None,
):
    start_process = start_datetime + process_after
    duration = end_datetime - start_process
    return {
        "item_id": item_id,
        "start_datetime": start_datetime,
        "end_datetime": end_datetime,
        "process_after": process_after,
```

```
        "repeat_at": repeat_at,
        "start_process": start_process,
        "duration": duration,
    }
```

Parámetros de Cookie

Puedes definir parámetros de Cookie de la misma manera que defines los parámetros `Query` y `Path`.

Importar Cookie

Primero importa `Cookie`:

```
from typing import Annotated

from fastapi import Cookie, FastAPI
```

Declarar parámetros de Cookie

Luego declara los parámetros de cookie usando la misma estructura que con `Path` y `Query`.

Puedes definir el valor por defecto así como toda la validación extra o los parámetros de anotación:

```
app = FastAPI()

@app.get("/items/")
async def read_items(ads_id: Annotated[str | None,
Cookie()] = None):
    return {"ads_id": ads_id}
```

Detalles Técnicos

`Cookie` es una clase "hermana" de `Path` y `Query`. También hereda de la misma clase común `Param`.

Pero recuerda que cuando importas `Query`, `Path`, `Cookie` y otros desde `fastapi`, en realidad son funciones que

devuelven clases especiales.

Nota
Para declarar cookies, necesitas usar `Cookie`, porque de lo contrario los parámetros serían interpretados como parámetros de query.

Nota
Ten en cuenta que, como **los navegadores manejan las cookies** de formas especiales y por detrás, **no** permiten fácilmente que **JavaScript** las toque.

Si vas a la **UI de la documentación de la API** en `/docs` podrás ver la **documentación** de cookies para tus *path operations*.

Pero incluso si **rellenas los datos** y haces clic en "Execute", como la UI de la documentación funciona con **JavaScript**, las cookies no se enviarán y verás un mensaje de **error** como si no hubieras escrito ningún valor.

Resumen

Declara cookies con `Cookie`, usando el mismo patrón común que `Query` y `Path`.

Parámetros de Header

Puedes definir los parámetros de Header de la misma manera que defines los parámetros de `Query`, `Path` y `Cookie`.

Importar Header

Primero importa `Header`:

```
from typing import Annotated

from fastapi import FastAPI, Header
```

Declarar parámetros de Header

Luego declara los parámetros de header usando la misma estructura que con `Path`, `Query` y `Cookie`.

Puedes definir el valor por defecto así como toda la validación extra o los parámetros de anotaciones:

```
app = FastAPI()

@app.get("/items/")
async def read_items(user_agent: Annotated[str | None,
Header()] = None):
    return {"User-Agent": user_agent}
```

> **Detalles Técnicos**
> `Header` es una clase "hermana" de `Path`, `Query` y `Cookie`. También hereda de la misma clase común `Param`.
>
> Pero recuerda que cuando importas `Query`, `Path`, `Header`, y otros de `fastapi`, en realidad son funciones que retornan clases especiales.

> **Nota**
> Para declarar headers, necesitas usar `Header`, porque de otra forma los parámetros serían interpretados como parámetros de query.

Conversión automática

`Header` tiene un poquito de funcionalidad extra además de lo que proporcionan `Path`, `Query` y `Cookie`.

La mayoría de los headers estándar están separados por un carácter "guion", también conocido como el "símbolo menos" (`-`).

Pero una variable como `user-agent` es inválida en Python.

Así que, por defecto, `Header` convertirá los caracteres de los nombres de los parámetros de guion bajo (`_`) a guion (`-`) para extraer y documentar los headers.

Además, los headers HTTP no diferencian entre mayúsculas y minúsculas, por lo que los puedes declarar con el estilo estándar de Python (también conocido como `snake_case`).

Así que, puedes usar `user_agent` como normalmente lo harías en código Python, en lugar de necesitar capitalizar las primeras letras como `User_Agent` o algo similar.

Si por alguna razón necesitas desactivar la conversión automática de guiones bajos a guiones, establece el parámetro `convert_underscores` de `Header` a `False` :

```
from typing import Annotated

from fastapi import FastAPI, Header

app = FastAPI()

@app.get("/items/")
async def read_items(
    strange_header: Annotated[str | None,
Header(convert_underscores=False)] = None,
):
    return {"strange_header": strange_header}
```

⚠ Advertencia
Antes de establecer `convert_underscores` a `False` , ten en cuenta que algunos proxies y servidores HTTP no permiten el uso de headers con guiones bajos.

Headers duplicados

Es posible recibir headers duplicados. Eso significa, el mismo header con múltiples valores.

Puedes definir esos casos usando una list en la declaración del tipo.

Recibirás todos los valores del header duplicado como una `list` de Python.

Por ejemplo, para declarar un header de `X-Token` que puede aparecer más de una vez, puedes escribir:

```
from typing import Annotated

from fastapi import FastAPI, Header

app = FastAPI()

@app.get("/items/")
async def read_items(x_token: Annotated[list[str] | None,
Header()] = None):
    return {"X-Token values": x_token}
```

Si te comunicas con esa *path operation* enviando dos headers HTTP como:

```
X-Token: foo
X-Token: bar
```

El response sería como:

```
{
    "X-Token values": [
        "bar",
        "foo"
    ]
}
```

Recapitulación

Declara headers con `Header`, usando el mismo patrón común que `Query`, `Path` y `Cookie`.

Y no te preocupes por los guiones bajos en tus variables, **FastAPI** se encargará de convertirlos.

- # Modelos de Cookies

Si tienes un grupo de **cookies** que están relacionadas, puedes crear un **modelo de Pydantic** para declararlas.

Esto te permitirá **reutilizar el modelo** en **múltiples lugares** y también declarar validaciones y metadatos para todos los parámetros a la vez.

> **Nota**
> Esto es compatible desde la versión `0.115.0` de FastAPI.

> **Consejo**
> Esta misma técnica se aplica a `Query`, `Cookie`, y `Header`.

Cookies con un Modelo de Pydantic

Declara los parámetros de **cookie** que necesites en un **modelo de Pydantic**, y luego declara el parámetro como `Cookie`:

```
from typing import Annotated

from fastapi import Cookie, FastAPI
from pydantic import BaseModel

app = FastAPI()

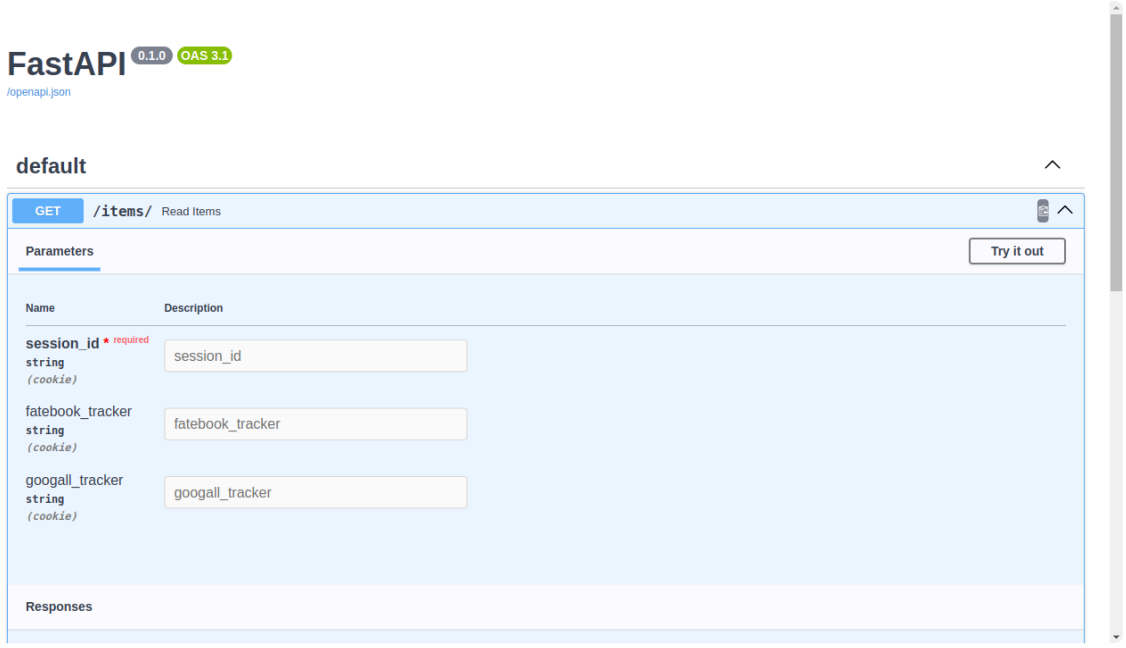
class Cookies(BaseModel):
    session_id: str
    fatebook_tracker: str | None = None
    googall_tracker: str | None = None

@app.get("/items/")
async def read_items(cookies: Annotated[Cookies,
Cookie()]):
    return cookies
```

FastAPI extraerá los datos para **cada campo** de las **cookies** recibidas en el request y te entregará el modelo de Pydantic que definiste.

Revisa la Documentación

Puedes ver las cookies definidas en la UI de la documentación en `/docs`:



Nota

Ten en cuenta que, como los **navegadores manejan las cookies** de maneras especiales y detrás de escenas, **no** permiten fácilmente que **JavaScript** las toque.

Si vas a la **UI de la documentación de la API** en `/docs` podrás ver la **documentación** de las cookies para tus *path operations*.

Pero incluso si **rellenas los datos** y haces clic en "Execute", como la UI de la documentación funciona con **JavaScript**, las cookies no serán enviadas y verás un **mensaje de error** como si no hubieras escrito ningún valor.

Prohibir Cookies Extra

En algunos casos de uso especiales (probablemente no muy comunes), podrías querer **restringir** las cookies que deseas recibir.

Tu API ahora tiene el poder de controlar su propio consentimiento de cookies.

Puedes usar la configuración del modelo de Pydantic para `prohibir` cualquier campo `extra`:

```
from typing import Annotated

from fastapi import Cookie, FastAPI
from pydantic import BaseModel

app = FastAPI()

class Cookies(BaseModel):
    model_config = {"extra": "forbid"}

    session_id: str
    fatebook_tracker: str | None = None
    googall_tracker: str | None = None

@app.get("/items/")
async def read_items(cookies: Annotated[Cookies,
Cookie()]):
    return cookies
```

Si un cliente intenta enviar algunas **cookies extra**, recibirán un response de **error**.

Pobres banners de cookies con todo su esfuerzo para obtener tu consentimiento para que la API lo rechace.

Por ejemplo, si el cliente intenta enviar una cookie `santa_tracker` con un valor de `good-list-please`, el cliente recibirá un response de **error** que le informa que la `santa_tracker` cookie no está permitida:

```
{
    "detail": [
        {
            "type": "extra_forbidden",
            "loc": ["cookie", "santa_tracker"],
            "msg": "Extra inputs are not permitted",
            "input": "good-list-please",
        }
    ]
}
```

Resumen

Puedes usar **modelos de Pydantic** para declarar **cookies** en **FastAPI**.

Modelos de Parámetros de Header

Si tienes un grupo de **parámetros de header** relacionados, puedes crear un **modelo Pydantic** para declararlos.

Esto te permitirá **reutilizar el modelo** en **múltiples lugares** y también declarar validaciones y metadatos para todos los parámetros al mismo tiempo.

> **Nota**
> Esto es compatible desde la versión `0.115.0` de FastAPI.

Parámetros de Header con un Modelo Pydantic

Declara los **parámetros de header** que necesitas en un **modelo Pydantic**, y luego declara el parámetro como `Header`:

```
from typing import Annotated

from fastapi import FastAPI, Header
from pydantic import BaseModel

app = FastAPI()
```

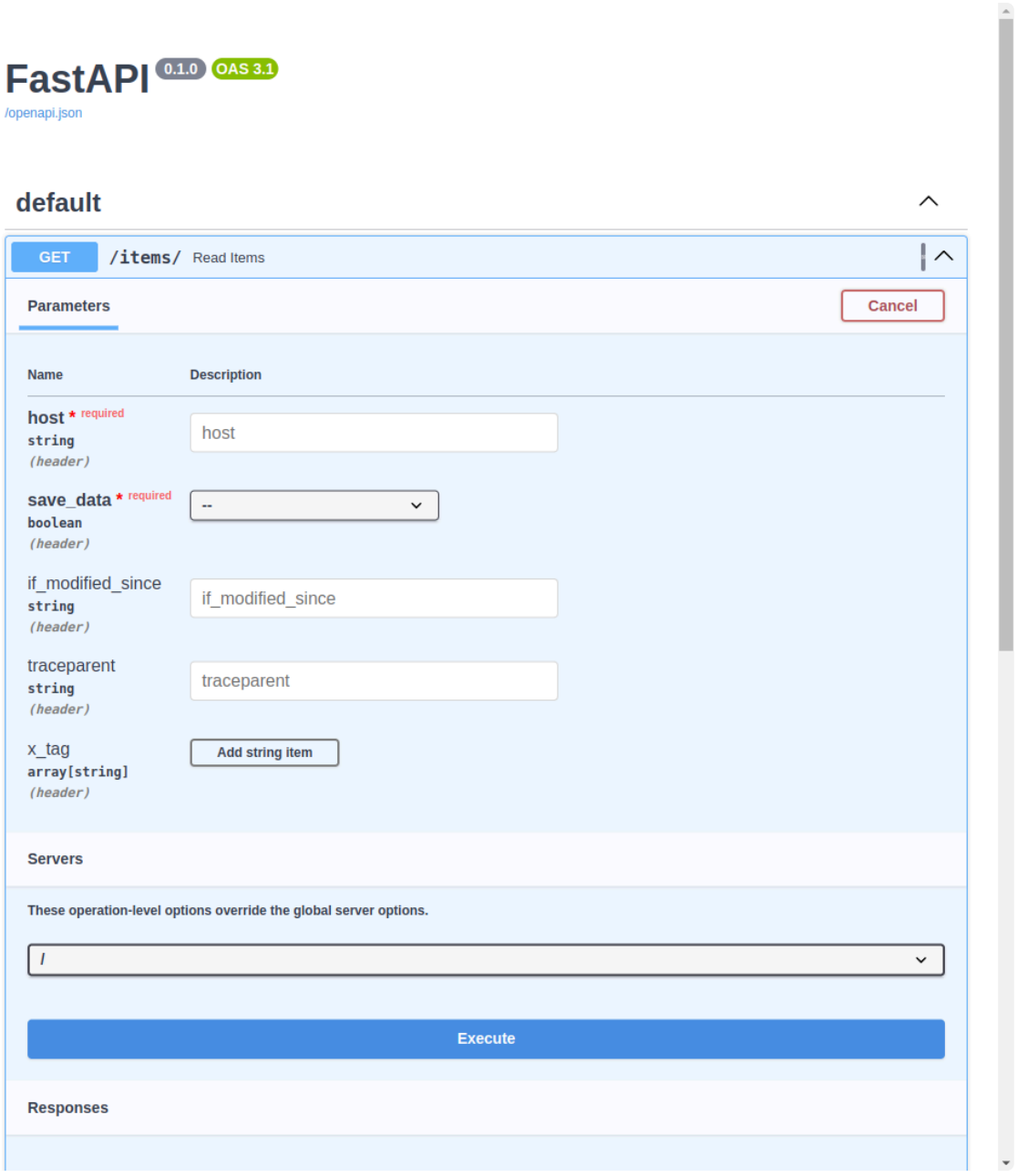
```python
class CommonHeaders(BaseModel):
    host: str
    save_data: bool
    if_modified_since: str | None = None
    traceparent: str | None = None
    x_tag: list[str] = []

@app.get("/items/")
async def read_items(headers: Annotated[CommonHeaders,
Header()]):
    return headers
```

FastAPI extraerá los datos para **cada campo** de los **headers** en el request y te dará el modelo Pydantic que definiste.

Revisa la Documentación

Puedes ver los headers requeridos en la interfaz de documentación en `/docs`:



Prohibir Headers Extra

En algunos casos de uso especiales (probablemente no muy comunes), podrías querer **restringir** los headers que deseas recibir.

Puedes usar la configuración del modelo de Pydantic para `prohibir` cualquier campo `extra`:

```
from typing import Annotated

from fastapi import FastAPI, Header
from pydantic import BaseModel
```

```python
app = FastAPI()

class CommonHeaders(BaseModel):
    model_config = {"extra": "forbid"}

    host: str
    save_data: bool
    if_modified_since: str | None = None
    traceparent: str | None = None
    x_tag: list[str] = []

@app.get("/items/")
async def read_items(headers: Annotated[CommonHeaders,
Header()]):
    return headers
```

Si un cliente intenta enviar algunos **headers extra**, recibirán un response de **error**.

Por ejemplo, si el cliente intenta enviar un header `tool` con un valor de `plumbus`, recibirán un response de **error** indicando que el parámetro de header `tool` no está permitido:

```
{
    "detail": [
        {
            "type": "extra_forbidden",
            "loc": ["header", "tool"],
            "msg": "Extra inputs are not permitted",
            "input": "plumbus",
        }
    ]
}
```

Desactivar la conversión de guiones bajos

De la misma forma que con los parámetros de header normales, cuando tienes caracteres de guion bajo en los nombres de los parámetros, se **convierten automáticamente en guiones**.

Por ejemplo, si tienes un parámetro de header `save_data` en el código, el header HTTP esperado será `save-data`, y aparecerá así en la documentación.

Si por alguna razón necesitas desactivar esta conversión automática, también puedes hacerlo para los modelos Pydantic de parámetros de header.

```
from typing import Annotated

from fastapi import FastAPI, Header
from pydantic import BaseModel

app = FastAPI()

class CommonHeaders(BaseModel):
    host: str
    save_data: bool
    if_modified_since: str | None = None
    traceparent: str | None = None
    x_tag: list[str] = []

@app.get("/items/")
async def read_items(
    headers: Annotated[CommonHeaders,
Header(convert_underscores=False)],
):
    return headers
```

⚠ Advertencia
Antes de establecer `convert_underscores` a `False`, ten en cuenta que algunos proxies y servidores HTTP no permiten el uso de headers con guiones bajos.

Resumen

Puedes usar **modelos Pydantic** para declarar **headers** en **FastAPI**.

Modelo de Response - Tipo de Retorno

Puedes declarar el tipo utilizado para el response anotando el **tipo de retorno** de la *path operation function*.

Puedes utilizar **anotaciones de tipos** de la misma manera que lo harías para datos de entrada en **parámetros** de función, puedes utilizar modelos de Pydantic, lists, diccionarios, valores escalares como enteros, booleanos, etc.

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: list[str] = []

@app.post("/items/")
async def create_item(item: Item) -> Item:
    return item

@app.get("/items/")
async def read_items() -> list[Item]:
    return [
        Item(name="Portal Gun", price=42.0),
        Item(name="Plumbus", price=32.0),
    ]
```

FastAPI usará este tipo de retorno para:

- **Validar** los datos devueltos.
    - Si los datos son inválidos (por ejemplo, falta un campo), significa que el código de *tu* aplicación está defectuoso, no devolviendo lo que debería, y retornará un error del servidor en lugar de devolver datos incorrectos. De esta manera, tú y tus clientes pueden estar seguros de que recibirán los datos y la forma de los datos esperada.

- Agregar un **JSON Schema** para el response, en la *path operation* de OpenAPI.
    - Esto será utilizado por la **documentación automática**.
        - También será utilizado por herramientas de generación automática de código de cliente.
- **Serializar** los datos devueltos a JSON usando Pydantic, que está escrito en **Rust**, por lo que será **mucho más rápido**.

Pero lo más importante:

- **Limitará y filtrará** los datos de salida a lo que se define en el tipo de retorno.
    - Esto es particularmente importante para la **seguridad**, veremos más sobre eso a continuación.

Parámetro response_model

Hay algunos casos en los que necesitas o quieres devolver algunos datos que no son exactamente lo que declara el tipo.

Por ejemplo, podrías querer **devolver un diccionario** u objeto de base de datos, pero **declararlo como un modelo de Pydantic**. De esta manera el modelo de Pydantic haría toda la documentación de datos, validación, etc. para el objeto que devolviste (por ejemplo, un diccionario u objeto de base de datos).

Si añadiste la anotación del tipo de retorno, las herramientas y editores se quejarían con un error (correcto) diciéndote que tu función está devolviendo un tipo (por ejemplo, un dict) que es diferente de lo que declaraste (por ejemplo, un modelo de Pydantic).

En esos casos, puedes usar el parámetro del *decorador de path operation* `response_model` en lugar del tipo de retorno.

Puedes usar el parámetro `response_model` en cualquiera de las *path operations*:

- `@app.get()`

- `@app.post()`
- `@app.put()`
- `@app.delete()`
- etc.

```python
from typing import Any

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: list[str] = []

@app.post("/items/", response_model=Item)
async def create_item(item: Item) -> Any:
    return item

@app.get("/items/", response_model=list[Item])
async def read_items() -> Any:
    return [
        {"name": "Portal Gun", "price": 42.0},
        {"name": "Plumbus", "price": 32.0},
    ]
```

> **Nota**
> Observa que `response_model` es un parámetro del método "decorador" (`get` , `post` , etc). No de tu *path operation function*, como todos los parámetros y el cuerpo.

`response_model` recibe el mismo tipo que declararías para un campo de modelo Pydantic, por lo que puede ser un modelo de

Pydantic, pero también puede ser, por ejemplo, un `list` de modelos de Pydantic, como `List[Item]`.

FastAPI usará este `response_model` para hacer toda la documentación de datos, validación, etc. y también para **convertir y filtrar los datos de salida** a su declaración de tipo.

> **Consejo**
> Si tienes chequeo de tipos estricto en tu editor, mypy, etc., puedes declarar el tipo de retorno de la función como `Any`.
>
> De esa manera le dices al editor que intencionalmente estás devolviendo cualquier cosa. Pero FastAPI todavía hará la documentación de datos, validación, filtrado, etc. con `response_model`.

Prioridad del response_model

Si declaras tanto un tipo de retorno como un `response_model`, el `response_model` tomará prioridad y será utilizado por FastAPI.

De esta manera puedes añadir anotaciones de tipos correctas a tus funciones incluso cuando estás devolviendo un tipo diferente al modelo de response, para ser utilizado por el editor y herramientas como mypy. Y aún así puedes hacer que FastAPI realice la validación de datos, documentación, etc. usando el `response_model`.

También puedes usar `response_model=None` para desactivar la creación de un modelo de response para esa *path operation*, podrías necesitar hacerlo si estás añadiendo anotaciones de tipos para cosas que no son campos válidos de Pydantic, verás un ejemplo de eso en una de las secciones a continuación.

Devolver los mismos datos de entrada

Aquí estamos declarando un modelo `UserIn`, contendrá una contraseña en texto plano:

```python
from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

app = FastAPI()

class UserIn(BaseModel):
    username: str
    password: str
    email: EmailStr
    full_name: str | None = None

# Don't do this in production!
@app.post("/user/")
async def create_user(user: UserIn) -> UserIn:
    return user
```

Nota

Para usar `EmailStr`, primero instala `email-validator`.

Añádelo a tu proyecto:

```
$ uv add email-validator
```

o con:

```
$ uv add "pydantic[email]"
```

Y estamos usando este modelo para declarar nuestra entrada y el mismo modelo para declarar nuestra salida:

```python
from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

app = FastAPI()

class UserIn(BaseModel):
    username: str
    password: str
```

```
    email: EmailStr
    full_name: str | None = None

# Don't do this in production!
@app.post("/user/")
async def create_user(user: UserIn) -> UserIn:
    return user
```

Ahora, cada vez que un navegador esté creando un usuario con una contraseña, la API devolverá la misma contraseña en el response.

En este caso, podría no ser un problema, porque es el mismo usuario que envía la contraseña.

Pero si usamos el mismo modelo para otra *path operation*, podríamos estar enviando las contraseñas de nuestros usuarios a cada cliente.

> **Peligro**
> Nunca almacenes la contraseña en texto plano de un usuario ni la envíes en un response como esta, a menos que conozcas todas las advertencias y sepas lo que estás haciendo.

Añadir un modelo de salida

Podemos en cambio crear un modelo de entrada con la contraseña en texto plano y un modelo de salida sin ella:

```
from typing import Any

from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

app = FastAPI()

class UserIn(BaseModel):
    username: str
    password: str
    email: EmailStr
```

```
    full_name: str | None = None

class UserOut(BaseModel):
    username: str
    email: EmailStr
    full_name: str | None = None

@app.post("/user/", response_model=UserOut)
async def create_user(user: UserIn) -> Any:
    return user
```

Aquí, aunque nuestra *path operation function* está devolviendo el mismo usuario de entrada que contiene la contraseña:

```
from typing import Any

from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

app = FastAPI()

class UserIn(BaseModel):
    username: str
    password: str
    email: EmailStr
    full_name: str | None = None

class UserOut(BaseModel):
    username: str
    email: EmailStr
    full_name: str | None = None

@app.post("/user/", response_model=UserOut)
async def create_user(user: UserIn) -> Any:
    return user
```

...hemos declarado el `response_model` para ser nuestro modelo `UserOut`, que no incluye la contraseña:

```python
from typing import Any

from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

app = FastAPI()

class UserIn(BaseModel):
    username: str
    password: str
    email: EmailStr
    full_name: str | None = None

class UserOut(BaseModel):
    username: str
    email: EmailStr
    full_name: str | None = None

@app.post("/user/", response_model=UserOut)
async def create_user(user: UserIn) -> Any:
    return user
```

Entonces, **FastAPI** se encargará de filtrar todos los datos que no estén declarados en el modelo de salida (usando Pydantic).

response_model o Tipo de Retorno

En este caso, como los dos modelos son diferentes, si anotáramos el tipo de retorno de la función como `UserOut`, el editor y las herramientas se quejarían de que estamos devolviendo un tipo inválido, ya que son clases diferentes.

Por eso en este ejemplo tenemos que declararlo en el parámetro `response_model`.

...pero sigue leyendo abajo para ver cómo superar eso.

Tipo de Retorno y Filtrado de Datos

Continuemos con el ejemplo anterior. Queríamos **anotar la función con un tipo**, pero queríamos poder devolver desde la función algo que en realidad incluya **más datos**.

Queremos que FastAPI continúe **filtrando** los datos usando el modelo de response. Para que, incluso cuando la función devuelva más datos, el response solo incluya los campos declarados en el modelo de response.

En el ejemplo anterior, debido a que las clases eran diferentes, tuvimos que usar el parámetro `response_model`. Pero eso también significa que no obtenemos el soporte del editor y las herramientas verificando el tipo de retorno de la función.

Pero en la mayoría de los casos en los que necesitamos hacer algo como esto, queremos que el modelo solo **filtre/elimine** algunos de los datos como en este ejemplo.

Y en esos casos, podemos usar clases y herencia para aprovechar las **anotaciones de tipos** de funciones para obtener mejor soporte en el editor y herramientas, y aún así obtener el **filtrado de datos** de FastAPI.

```python
from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

app = FastAPI()

class BaseUser(BaseModel):
    username: str
    email: EmailStr
    full_name: str | None = None

class UserIn(BaseUser):
    password: str

@app.post("/user/")
async def create_user(user: UserIn) -> BaseUser:
    return user
```

Con esto, obtenemos soporte de las herramientas, de los editores y mypy ya que este código es correcto en términos de tipos, pero también obtenemos el filtrado de datos de FastAPI.

¿Cómo funciona esto? Vamos a echarle un vistazo.

Anotaciones de Tipos y Herramientas

Primero vamos a ver cómo los editores, mypy y otras herramientas verían esto.

`BaseUser` tiene los campos base. Luego `UserIn` hereda de `BaseUser` y añade el campo `password`, por lo que incluirá todos los campos de ambos modelos.

Anotamos el tipo de retorno de la función como `BaseUser`, pero en realidad estamos devolviendo un `UserIn` instance.

El editor, mypy y otras herramientas no se quejarán de esto porque, en términos de tipificación, `UserIn` es una subclase de `BaseUser`, lo que significa que es un tipo *válido* cuando se espera algo que es un `BaseUser`.

Filtrado de Datos en FastAPI

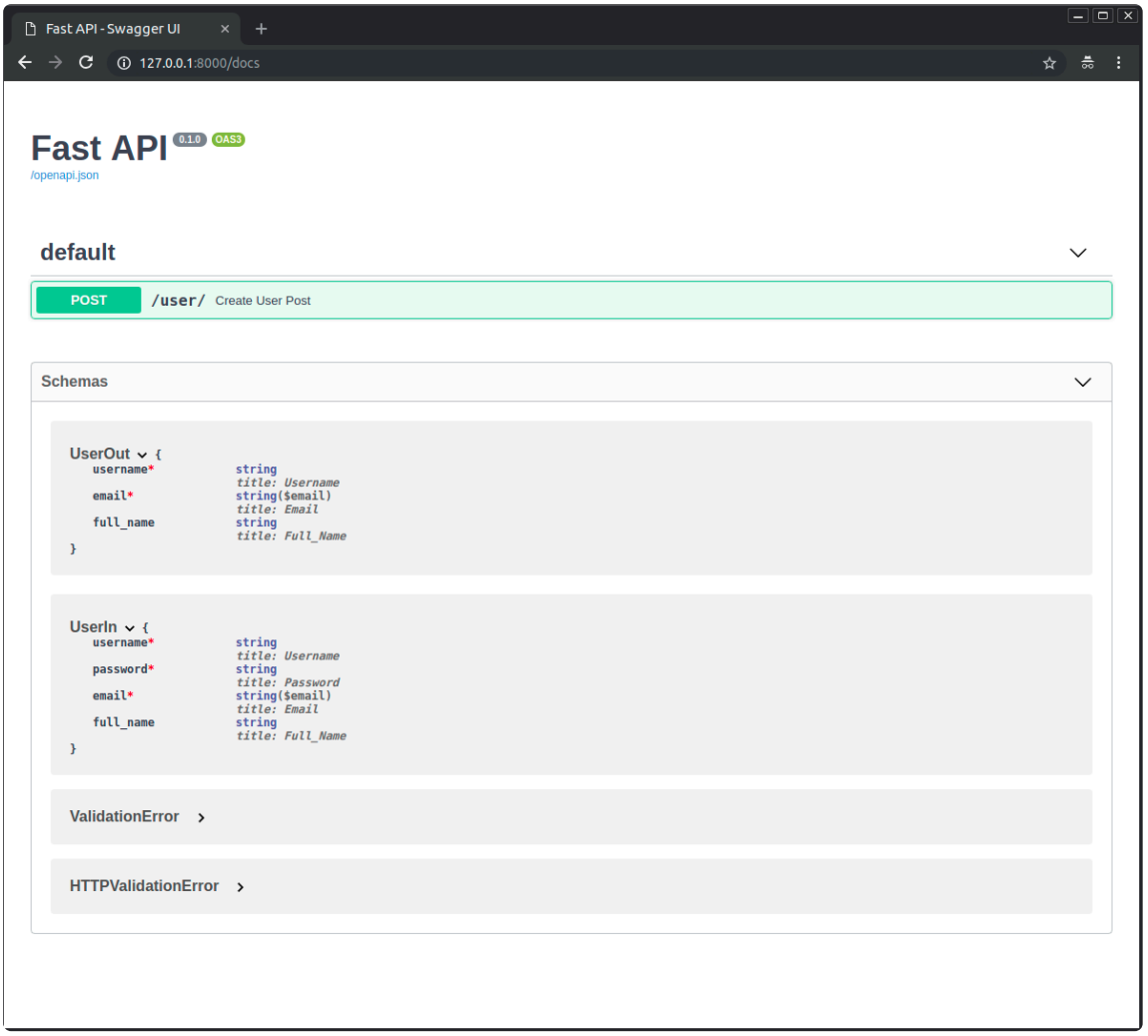
Ahora, para FastAPI, verá el tipo de retorno y se asegurará de que lo que devuelves incluya **solo** los campos que están declarados en el tipo.

FastAPI realiza varias cosas internamente con Pydantic para asegurarse de que esas mismas reglas de herencia de clases no se utilicen para el filtrado de datos devueltos, de lo contrario, podrías terminar devolviendo muchos más datos de los que esperabas.

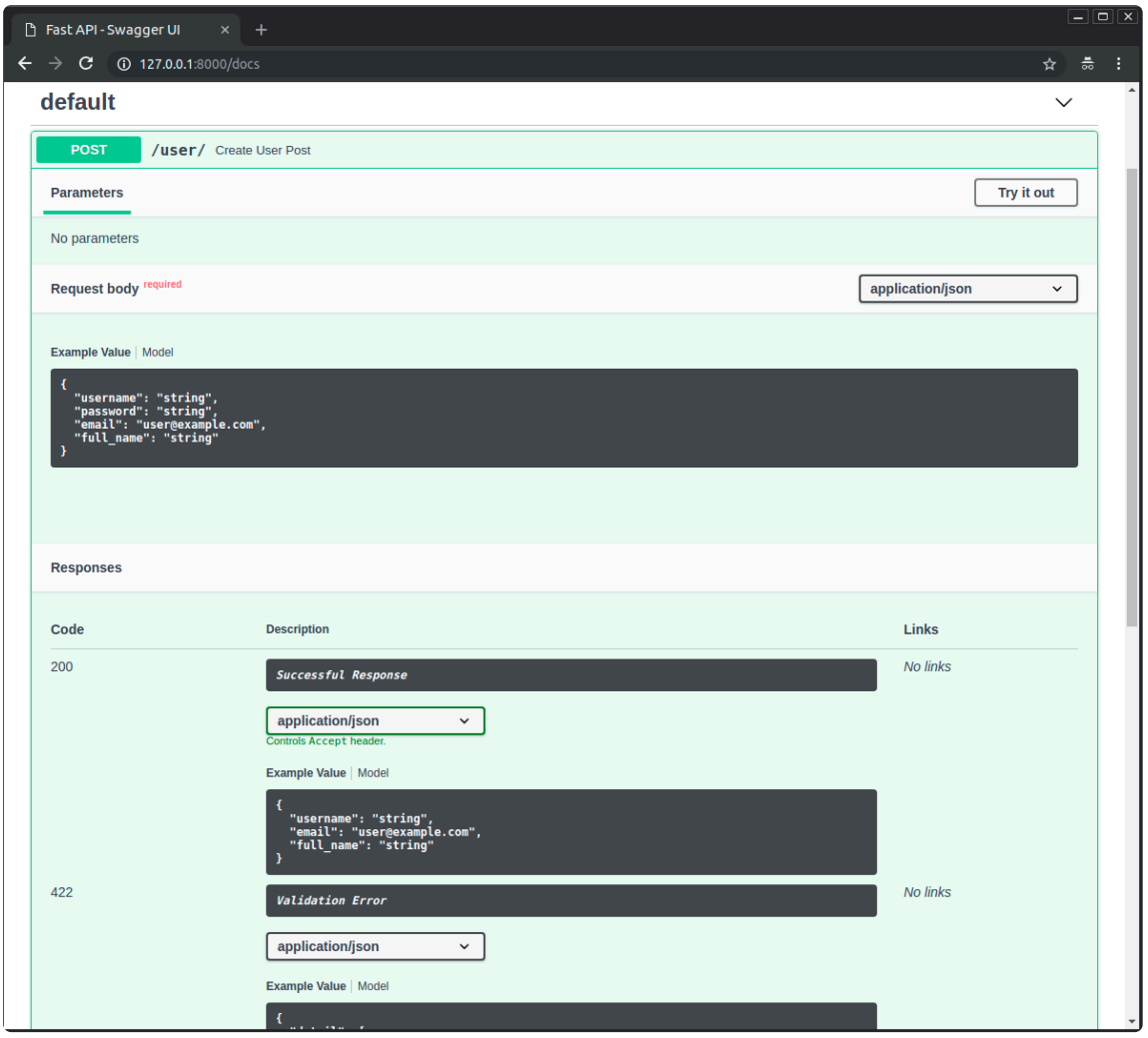
De esta manera, puedes obtener lo mejor de ambos mundos: anotaciones de tipos con **soporte de herramientas** y **filtrado de datos**.

Verlo en la documentación

Cuando veas la documentación automática, puedes verificar que el modelo de entrada y el modelo de salida tendrán cada uno su propio JSON Schema:



Y ambos modelos se utilizarán para la documentación interactiva de la API:



Otras Anotaciones de Tipos de Retorno

Podría haber casos en los que devuelvas algo que no es un campo válido de Pydantic y lo anotes en la función, solo para obtener el soporte proporcionado por las herramientas (el editor, mypy, etc).

Devolver un Response Directamente

El caso más común sería devolver un Response directamente como se explica más adelante en la documentación avanzada.

```
from fastapi import FastAPI, Response
from fastapi.responses import JSONResponse,
RedirectResponse

app = FastAPI()

@app.get("/portal")
```

```
async def get_portal(teleport: bool = False) -> Response:
    if teleport:
        return
RedirectResponse(url="https://www.youtube.com/watch?
v=dQw4w9WgXcQ")
    return JSONResponse(content={"message": "Here's your
interdimensional portal."})
```

Este caso simple es manejado automáticamente por FastAPI porque la anotación del tipo de retorno es la clase (o una subclase de) `Response`.

Y las herramientas también estarán felices porque tanto `RedirectResponse` como `JSONResponse` son subclases de `Response`, por lo que la anotación del tipo es correcta.

Anotar una Subclase de Response

También puedes usar una subclase de `Response` en la anotación del tipo:

```
from fastapi import FastAPI
from fastapi.responses import RedirectResponse

app = FastAPI()

@app.get("/teleport")
async def get_teleport() -> RedirectResponse:
    return
RedirectResponse(url="https://www.youtube.com/watch?
v=dQw4w9WgXcQ")
```

Esto también funcionará porque `RedirectResponse` es una subclase de `Response`, y FastAPI manejará automáticamente este caso simple.

Anotaciones de Tipos de Retorno Inválidas

Pero cuando devuelves algún otro objeto arbitrario que no es un tipo válido de Pydantic (por ejemplo, un objeto de base de datos) y lo anotas así en la función, FastAPI intentará crear un modelo de response de Pydantic a partir de esa anotación de tipo, y fallará.

Lo mismo sucedería si tuvieras algo como una unión entre diferentes tipos donde uno o más de ellos no son tipos válidos de Pydantic, por ejemplo esto fallaría:

```
from fastapi import FastAPI, Response
from fastapi.responses import RedirectResponse

app = FastAPI()

@app.get("/portal")
async def get_portal(teleport: bool = False) -> Response |
dict:
    if teleport:
        return
RedirectResponse(url="https://www.youtube.com/watch?
v=dQw4w9WgXcQ")
    return {"message": "Here's your interdimensional
portal."}
```

...esto falla porque la anotación de tipo no es un tipo de Pydantic y no es solo una sola clase `Response` o subclase, es una unión (cualquiera de los dos) entre una `Response` y un `dict`.

Desactivar el Modelo de Response

Continuando con el ejemplo anterior, puede que no quieras tener la validación de datos por defecto, documentación, filtrado, etc. que realiza FastAPI.

Pero puedes querer mantener la anotación del tipo de retorno en la función para obtener el soporte de herramientas como editores y verificadores de tipos (por ejemplo, mypy).

En este caso, puedes desactivar la generación del modelo de response configurando `response_model=None`:

```
from fastapi import FastAPI, Response
from fastapi.responses import RedirectResponse

app = FastAPI()

@app.get("/portal", response_model=None)
async def get_portal(teleport: bool = False) -> Response |
dict:
    if teleport:
        return
RedirectResponse(url="https://www.youtube.com/watch?
v=dQw4w9WgXcQ")
    return {"message": "Here's your interdimensional
portal."}
```

Esto hará que FastAPI omita la generación del modelo de response y de esa manera puedes tener cualquier anotación de tipo de retorno que necesites sin que afecte a tu aplicación FastAPI.

Parámetros de codificación del Modelo de Response

Tu modelo de response podría tener valores por defecto, como:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float = 10.5
    tags: list[str] = []
```

```
items = {
    "foo": {"name": "Foo", "price": 50.2},
    "bar": {"name": "Bar", "description": "The
bartenders", "price": 62, "tax": 20.2},
    "baz": {"name": "Baz", "description": None, "price":
50.2, "tax": 10.5, "tags": []},
}

@app.get("/items/{item_id}", response_model=Item,
response_model_exclude_unset=True)
async def read_item(item_id: str):
    return items[item_id]
```

- `description: Union[str, None] = None` (o `str | None = None` en Python 3.10) tiene un valor por defecto de `None`.
- `tax: float = 10.5` tiene un valor por defecto de `10.5`.
- `tags: List[str] = []` tiene un valor por defecto de una list vacía: `[]`.

pero podrías querer omitirlos del resultado si no fueron en realidad almacenados.

Por ejemplo, si tienes modelos con muchos atributos opcionales en una base de datos NoSQL, pero no quieres enviar responses JSON muy largos llenos de valores por defecto.

Usa el parámetro response_model_exclude_unset

Puedes configurar el parámetro del decorador de path operation `response_model_exclude_unset=True`:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
```

```
    name: str
    description: str | None = None
    price: float
    tax: float = 10.5
    tags: list[str] = []

items = {
    "foo": {"name": "Foo", "price": 50.2},
    "bar": {"name": "Bar", "description": "The
bartenders", "price": 62, "tax": 20.2},
    "baz": {"name": "Baz", "description": None, "price":
50.2, "tax": 10.5, "tags": []},
}

@app.get("/items/{item_id}", response_model=Item,
response_model_exclude_unset=True)
async def read_item(item_id: str):
    return items[item_id]
```

y esos valores por defecto no serán incluidos en el response, solo los valores realmente establecidos.

Entonces, si envías un request a esa *path operation* para el ítem con ID `foo`, el response (no incluyendo valores por defecto) será:

```
{
    "name": "Foo",
    "price": 50.2
}
```

Nota

También puedes usar:

- `response_model_exclude_defaults=True`
- `response_model_exclude_none=True`

como se describe en la documentación de Pydantic para `exclude_defaults` y `exclude_none`.

Datos con valores para campos con valores por defecto

Pero si tus datos tienen valores para los campos del modelo con valores por defecto, como el artículo con ID `bar`:

```
{
    "name": "Bar",
    "description": "The bartenders",
    "price": 62,
    "tax": 20.2
}
```

serán incluidos en el response.

Datos con los mismos valores que los valores por defecto

Si los datos tienen los mismos valores que los valores por defecto, como el artículo con ID `baz`:

```
{
    "name": "Baz",
    "description": None,
    "price": 50.2,
    "tax": 10.5,
    "tags": []
}
```

FastAPI es lo suficientemente inteligente (de hecho, Pydantic es lo suficientemente inteligente) para darse cuenta de que, a pesar de que `description`, `tax` y `tags` tienen los mismos valores que los valores por defecto, fueron establecidos explícitamente (en lugar de tomados de los valores por defecto).

Por lo tanto, se incluirán en el response JSON.

Consejo

Ten en cuenta que los valores por defecto pueden ser cualquier cosa, no solo `None`.

Pueden ser una list (`[]`), un `float` de `10.5`, etc.

response_model_include y response_model_exclude

También puedes usar los parámetros del decorador de path operation `response_model_include` y `response_model_exclude`.

Aceptan un `set` de `str` con el nombre de los atributos a incluir (omitiendo el resto) o excluir (incluyendo el resto).

Esto se puede usar como un atajo rápido si solo tienes un modelo de Pydantic y quieres eliminar algunos datos de la salida.

Consejo

Pero todavía se recomienda usar las ideas anteriores, usando múltiples clases, en lugar de estos parámetros.

Esto se debe a que el JSON Schema generado en el OpenAPI de tu aplicación (y la documentación) aún será el del modelo completo, incluso si usas `response_model_include` o `response_model_exclude` para omitir algunos atributos.

Esto también se aplica a `response_model_by_alias` que funciona de manera similar.

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float = 10.5
```

```
items = {
    "foo": {"name": "Foo", "price": 50.2},
    "bar": {"name": "Bar", "description": "The Bar
fighters", "price": 62, "tax": 20.2},
    "baz": {
        "name": "Baz",
        "description": "There goes my baz",
        "price": 50.2,
        "tax": 10.5,
    },
}

@app.get(
    "/items/{item_id}/name",
    response_model=Item,
    response_model_include={"name", "description"},
)
async def read_item_name(item_id: str):
    return items[item_id]

@app.get("/items/{item_id}/public", response_model=Item,
response_model_exclude={"tax"})
async def read_item_public_data(item_id: str):
    return items[item_id]
```

Consejo

La sintaxis `{"name", "description"}` crea un `set` con esos dos valores.

Es equivalente a `set(["name", "description"])`.

Usar lists en lugar de sets

Si olvidas usar un `set` y usas un `list` o `tuple` en su lugar, FastAPI todavía lo convertirá a un `set` y funcionará correctamente:

```
from fastapi import FastAPI
from pydantic import BaseModel
```

```python
app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float = 10.5

items = {
    "foo": {"name": "Foo", "price": 50.2},
    "bar": {"name": "Bar", "description": "The Bar
fighters", "price": 62, "tax": 20.2},
    "baz": {
        "name": "Baz",
        "description": "There goes my baz",
        "price": 50.2,
        "tax": 10.5,
    },
}

@app.get(
    "/items/{item_id}/name",
    response_model=Item,
    response_model_include=["name", "description"],
)
async def read_item_name(item_id: str):
    return items[item_id]

@app.get("/items/{item_id}/public", response_model=Item,
response_model_exclude=["tax"])
async def read_item_public_data(item_id: str):
    return items[item_id]
```

Resumen

Usa el parámetro `response_model` del *decorador de path operation* para definir modelos de response y especialmente para asegurarte de que los datos privados sean filtrados.

Usa `response_model_exclude_unset` para devolver solo los valores establecidos explícitamente.

- # Modelos Extra

Continuando con el ejemplo anterior, será común tener más de un modelo relacionado.

Esto es especialmente el caso para los modelos de usuario, porque:

- El **modelo de entrada** necesita poder tener una contraseña.
- El **modelo de salida** no debería tener una contraseña.
- El **modelo de base de datos** probablemente necesitaría tener una contraseña hasheada.

> **Peligro**
> Nunca almacenes contraseñas de usuarios en texto plano. Siempre almacena un "hash seguro" que puedas verificar luego.
>
> Si no lo sabes, aprenderás qué es un "hash de contraseña" en los capítulos de seguridad.

Múltiples modelos

Aquí tienes una idea general de cómo podrían ser los modelos con sus campos de contraseña y los lugares donde se utilizan:

```
from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

app = FastAPI()

class UserIn(BaseModel):
    username: str
    password: str
    email: EmailStr
    full_name: str | None = None

class UserOut(BaseModel):
```

```
    username: str
    email: EmailStr
    full_name: str | None = None

class UserInDB(BaseModel):
    username: str
    hashed_password: str
    email: EmailStr
    full_name: str | None = None

def fake_password_hasher(raw_password: str):
    return "supersecret" + raw_password

def fake_save_user(user_in: UserIn):
    hashed_password =
fake_password_hasher(user_in.password)
    user_in_db = UserInDB(**user_in.model_dump(),
hashed_password=hashed_password)
    print("User saved! ..not really")
    return user_in_db

@app.post("/user/", response_model=UserOut)
async def create_user(user_in: UserIn):
    user_saved = fake_save_user(user_in)
    return user_saved
```

Acerca de user_in.model_dump()
.model_dump() de Pydantic

`user_in` es un modelo Pydantic de la clase `UserIn`.

Los modelos Pydantic tienen un método `.model_dump()` que devuelve un `dict` con los datos del modelo.

Así que, si creamos un objeto Pydantic `user_in` como:

```
user_in = UserIn(username="john", password="secret",
email="john.doe@example.com")
```

y luego llamamos a:

```
user_dict = user_in.model_dump()
```

ahora tenemos un `dict` con los datos en la variable `user_dict` (es un `dict` en lugar de un objeto modelo Pydantic).

Y si llamamos a:

```
print(user_dict)
```

obtendríamos un `dict` de Python con:

```
{
    'username': 'john',
    'password': 'secret',
    'email': 'john.doe@example.com',
    'full_name': None,
}
```

Desempaquetando un dict

Si tomamos un `dict` como `user_dict` y lo pasamos a una función (o clase) con `**user_dict`, Python lo "desempaquetará". Pasará las claves y valores del `user_dict` directamente como argumentos clave-valor.

Así que, continuando con el `user_dict` anterior, escribir:

```
UserInDB(**user_dict)
```

sería equivalente a algo como:

```
UserInDB(
    username="john",
    password="secret",
    email="john.doe@example.com",
    full_name=None,
)
```

O más exactamente, usando `user_dict` directamente, con cualquier contenido que pueda tener en el futuro:

```
UserInDB(
    username = user_dict["username"],
    password = user_dict["password"],
    email = user_dict["email"],
    full_name = user_dict["full_name"],
)
```

Un modelo Pydantic a partir del contenido de otro

Como en el ejemplo anterior obtuvimos `user_dict` de `user_in.model_dump()`, este código:

```
user_dict = user_in.model_dump()
UserInDB(**user_dict)
```

sería equivalente a:

```
UserInDB(**user_in.model_dump())
```

...porque `user_in.model_dump()` es un `dict`, y luego hacemos que Python lo "desempaquete" al pasarlo a `UserInDB` con el prefijo `**`.

Así, obtenemos un modelo Pydantic a partir de los datos en otro modelo Pydantic.

Desempaquetando un dict y palabras clave adicionales

Y luego agregando el argumento de palabra clave adicional `hashed_password=hashed_password`, como en:

```
UserInDB(**user_in.model_dump(),
hashed_password=hashed_password)
```

...termina siendo como:

```
UserInDB(
    username = user_dict["username"],
    password = user_dict["password"],
    email = user_dict["email"],
    full_name = user_dict["full_name"],
    hashed_password = hashed_password,
)
```

⚠ **Advertencia**
Las funciones adicionales de soporte `fake_password_hasher` y `fake_save_user` son solo para demostrar un posible flujo de datos, pero por supuesto no proporcionan ninguna seguridad real.

Reducir duplicación

Reducir la duplicación de código es una de las ideas centrales en **FastAPI**.

Ya que la duplicación de código incrementa las posibilidades de bugs, problemas de seguridad, problemas de desincronización de código (cuando actualizas en un lugar pero no en los otros), etc.

Y estos modelos están compartiendo muchos de los datos y duplicando nombres y tipos de atributos.

Podríamos hacerlo mejor.

Podemos declarar un modelo `UserBase` que sirva como base para nuestros otros modelos. Y luego podemos hacer subclases de ese modelo que heredan sus atributos (declaraciones de tipos, validación, etc).

Toda la conversión de datos, validación, documentación, etc. seguirá funcionando normalmente.

De esa manera, podemos declarar solo las diferencias entre los modelos (con `password` en texto plano, con `hashed_password` y sin

contraseña):

```
from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

app = FastAPI()

class UserBase(BaseModel):
    username: str
    email: EmailStr
    full_name: str | None = None

class UserIn(UserBase):
    password: str

class UserOut(UserBase):
    pass

class UserInDB(UserBase):
    hashed_password: str

def fake_password_hasher(raw_password: str):
    return "supersecret" + raw_password

def fake_save_user(user_in: UserIn):
    hashed_password =
fake_password_hasher(user_in.password)
    user_in_db = UserInDB(**user_in.model_dump(),
hashed_password=hashed_password)
    print("User saved! ..not really")
    return user_in_db

@app.post("/user/", response_model=UserOut)
async def create_user(user_in: UserIn):
    user_saved = fake_save_user(user_in)
    return user_saved
```

Union o anyOf

Puedes declarar un response que sea la `Union` de dos o más tipos, eso significa que el response sería cualquiera de ellos.

Se definirá en OpenAPI con `anyOf`.

Para hacerlo, usa la anotación de tipos estándar de Python `typing.Union`:

> **Nota**
> Al definir una `Union`, incluye el tipo más específico primero, seguido por el tipo menos específico. En el ejemplo a continuación, el más específico `PlaneItem` viene antes de `CarItem` en `Union[PlaneItem, CarItem]`.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class BaseItem(BaseModel):
    description: str
    type: str

class CarItem(BaseItem):
    type: str = "car"

class PlaneItem(BaseItem):
    type: str = "plane"
    size: int

items = {
    "item1": {"description": "All my friends drive a low
rider", "type": "car"},
    "item2": {
        "description": "Music is my aeroplane, it's my
aeroplane",
        "type": "plane",
        "size": 5,
    },
}
```

```
@app.get("/items/{item_id}", response_model=PlaneItem |
CarItem)
async def read_item(item_id: str):
    return items[item_id]
```

Union en Python 3.10

En este ejemplo pasamos `Union[PlaneItem, CarItem]` como el valor del argumento `response_model`.

Porque lo estamos pasando como un **valor a un argumento** en lugar de ponerlo en una **anotación de tipos**, tenemos que usar `Union` incluso en Python 3.10.

Si estuviera en una anotación de tipos podríamos haber usado la barra vertical, como:

```
some_variable: PlaneItem | CarItem
```

Pero si ponemos eso en la asignación `response_model=PlaneItem | CarItem` obtendríamos un error, porque Python intentaría realizar una **operación inválida** entre `PlaneItem` y `CarItem` en lugar de interpretar eso como una anotación de tipos.

Lista de modelos

De la misma manera, puedes declarar responses de listas de objetos.

Para eso, usa la `list` estándar de Python:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
```

```
    description: str

items = [
    {"name": "Foo", "description": "There comes my hero"},
    {"name": "Red", "description": "It's my aeroplane"},
]

@app.get("/items/", response_model=list[Item])
async def read_items():
    return items
```

Response con dict arbitrario

También puedes declarar un response usando un `dict` arbitrario plano, declarando solo el tipo de las claves y valores, sin usar un modelo Pydantic.

Esto es útil si no conoces los nombres de los campos/atributos válidos (que serían necesarios para un modelo Pydantic) de antemano.

En este caso, puedes usar `dict`:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/keyword-weights/", response_model=dict[str,
float])
async def read_keyword_weights():
    return {"foo": 2.3, "bar": 3.4}
```

Recapitulación

Usa múltiples modelos Pydantic y hereda libremente para cada caso.

No necesitas tener un solo modelo de datos por entidad si esa entidad debe poder tener diferentes "estados". La "entidad"

usuario es un ejemplo, con estados que incluyen `password`, `password_hash` o ninguna contraseña.

- # Código de Estado del Response

De la misma manera que puedes especificar un modelo de response, también puedes declarar el código de estado HTTP usado para el response con el parámetro `status_code` en cualquiera de las *path operations*:

- `@app.get()`
- `@app.post()`
- `@app.put()`
- `@app.delete()`
- etc.

```
from fastapi import FastAPI

app = FastAPI()

@app.post("/items/", status_code=201)
async def create_item(name: str):
    return {"name": name}
```

> **Nota**
> Observa que `status_code` es un parámetro del método "decorador" (`get`, `post`, etc). No de tu *path operation function*, como todos los parámetros y body.

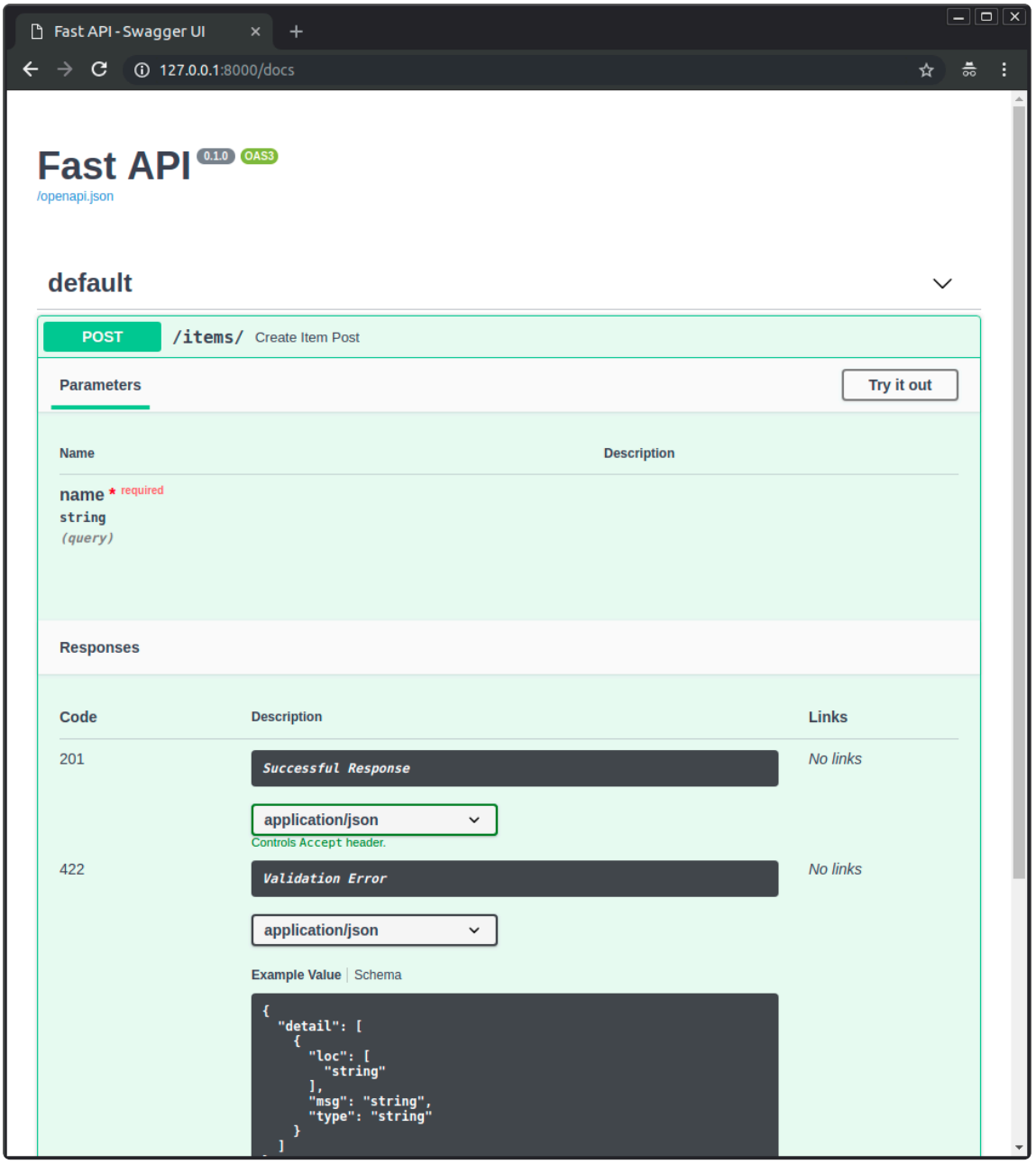
El parámetro `status_code` recibe un número con el código de estado HTTP.

> **Nota**
> `status_code` también puede recibir un `IntEnum`, como por ejemplo el `http.HTTPStatus` de Python.

Esto hará:

- Devolver ese código de estado en el response.

- Documentarlo como tal en el esquema de OpenAPI (y por lo tanto, en las interfaces de usuario):



Nota

Algunos códigos de response (ver la siguiente sección) indican que el response no tiene un body.

FastAPI sabe esto, y producirá documentación OpenAPI que establece que no hay un response body.

Acerca de los códigos de estado HTTP

Nota

Si ya sabes qué son los códigos de estado HTTP, salta a la siguiente sección.

En HTTP, envías un código de estado numérico de 3 dígitos como parte del response.

Estos códigos de estado tienen un nombre asociado para reconocerlos, pero la parte importante es el número.

En breve:

- `100 - 199` son para "Información". Rara vez los usas directamente. Los responses con estos códigos de estado no pueden tener un body.
- **`200 - 299`** son para responses "Exitosos". Estos son los que usarías más.
    - `200` es el código de estado por defecto, lo que significa que todo estaba "OK".
        - Otro ejemplo sería `201`, "Created". Comúnmente se usa después de crear un nuevo registro en la base de datos.
        - Un caso especial es `204`, "No Content". Este response se usa cuando no hay contenido para devolver al cliente, por lo tanto, el response no debe tener un body.
- **`300 - 399`** son para "Redirección". Los responses con estos códigos de estado pueden o no tener un body, excepto `304`, "Not Modified", que no debe tener uno.
- **`400 - 499`** son para responses de "Error del Cliente". Este es el segundo tipo que probablemente más usarías.
    - Un ejemplo es `404`, para un response "Not Found".
        - Para errores genéricos del cliente, puedes usar simplemente `400`.
- `500 - 599` son para errores del servidor. Casi nunca los usas directamente. Cuando algo sale mal en alguna parte de tu código de aplicación, o del servidor, automáticamente devolverá uno de estos códigos de estado.

> **Consejo**
> Para saber más sobre cada código de estado y qué código es para qué, revisa la documentación de MDN sobre códigos de estado HTTP.

Atajo para recordar los nombres

Veamos de nuevo el ejemplo anterior:

```
from fastapi import FastAPI

app = FastAPI()

@app.post("/items/", status_code=201)
async def create_item(name: str):
    return {"name": name}
```

`201` es el código de estado para "Created".

Pero no tienes que memorizar lo que significa cada uno de estos códigos.

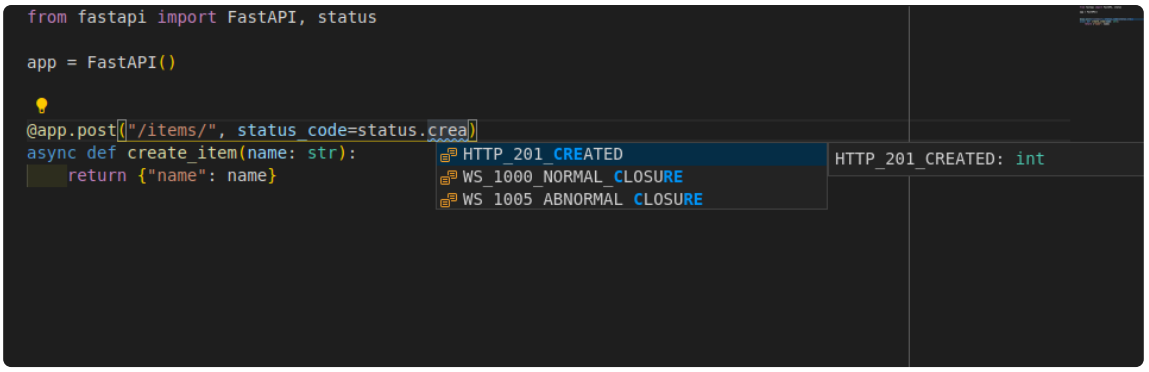
Puedes usar las variables de conveniencia de `fastapi.status`.

```
from fastapi import FastAPI, status

app = FastAPI()

@app.post("/items/", status_code=status.HTTP_201_CREATED)
async def create_item(name: str):
    return {"name": name}
```

Son solo una conveniencia, mantienen el mismo número, pero de esa manera puedes usar el autocompletado del editor para encontrarlos:



Detalles técnicos

También podrías usar `from starlette import status`.

FastAPI proporciona el mismo `starlette.status` como `fastapi.status` solo como una conveniencia para ti, el desarrollador. Pero proviene directamente de Starlette.

Form Data

Cuando necesitas recibir campos de formulario en lugar de JSON, puedes usar `Form`.

Nota

Para usar formularios, primero instala `python-multipart`.

Añádelo a tu proyecto:

```
$ uv add python-multipart
```

Importa Form

Importa `Form` desde `fastapi`:

```
from typing import Annotated

from fastapi import FastAPI, Form

app = FastAPI()

@app.post("/login/")
async def login(username: Annotated[str, Form()],
```

```
password: Annotated[str, Form()]):
    return {"username": username}
```

Define parámetros de Form

Crea parámetros de formulario de la misma manera que lo harías para `Body` o `Query`:

```
from typing import Annotated

from fastapi import FastAPI, Form

app = FastAPI()

@app.post("/login/")
async def login(username: Annotated[str, Form()],
password: Annotated[str, Form()]):
    return {"username": username}
```

Por ejemplo, en una de las formas en las que se puede usar la especificación OAuth2 (llamada "password flow") se requiere enviar un `username` y `password` como campos de formulario.

La especificación requiere que los campos se llamen exactamente `username` y `password`, y que se envíen como campos de formulario, no JSON.

Con `Form` puedes declarar las mismas configuraciones que con `Body` (y `Query`, `Path`, `Cookie`), incluyendo validación, ejemplos, un alias (por ejemplo, `user-name` en lugar de `username`), etc.

> **Nota**
> `Form` es una clase que hereda directamente de `Body`.

> **Consejo**
> Para declarar bodies de formularios, necesitas usar `Form` explícitamente, porque sin él, los parámetros se interpretarían

como parámetros de query o como parámetros de body (JSON).

Sobre "Campos de formulario"

La manera en que los formularios HTML (`<form></form>`) envían los datos al servidor normalmente usa una codificación "especial" para esos datos, es diferente de JSON.

FastAPI se encargará de leer esos datos del lugar correcto en lugar de JSON.

Detalles técnicos

Los datos de formularios normalmente se codifican usando el "media type" `application/x-www-form-urlencoded` .

Pero cuando el formulario incluye archivos, se codifica como `multipart/form-data` . Leerás sobre la gestión de archivos en el próximo capítulo.

Si quieres leer más sobre estas codificaciones y campos de formulario, dirígete a las MDN web docs para `POST` .

Advertencia

Puedes declarar múltiples parámetros `Form` en una *path operation*, pero no puedes también declarar campos `Body` que esperas recibir como JSON, ya que el request tendrá el body codificado usando `application/x-www-form-urlencoded` en lugar de `application/json` .

Esto no es una limitación de **FastAPI**, es parte del protocolo HTTP.

Recapitulación

Usa `Form` para declarar parámetros de entrada de form data.

Modelos de Formulario

Puedes usar **modelos de Pydantic** para declarar **campos de formulario** en FastAPI.

> **Nota**
> Para usar formularios, primero instala `python-multipart`.
>
> Agrégalo a tu proyecto:
>
> ```
> $ uv add python-multipart
> ```

> **Nota**
> Esto es compatible desde la versión `0.113.0` de FastAPI.

Modelos de Pydantic para Formularios

Solo necesitas declarar un **modelo de Pydantic** con los campos que quieres recibir como **campos de formulario**, y luego declarar el parámetro como `Form`:

```
from typing import Annotated

from fastapi import FastAPI, Form
from pydantic import BaseModel

app = FastAPI()

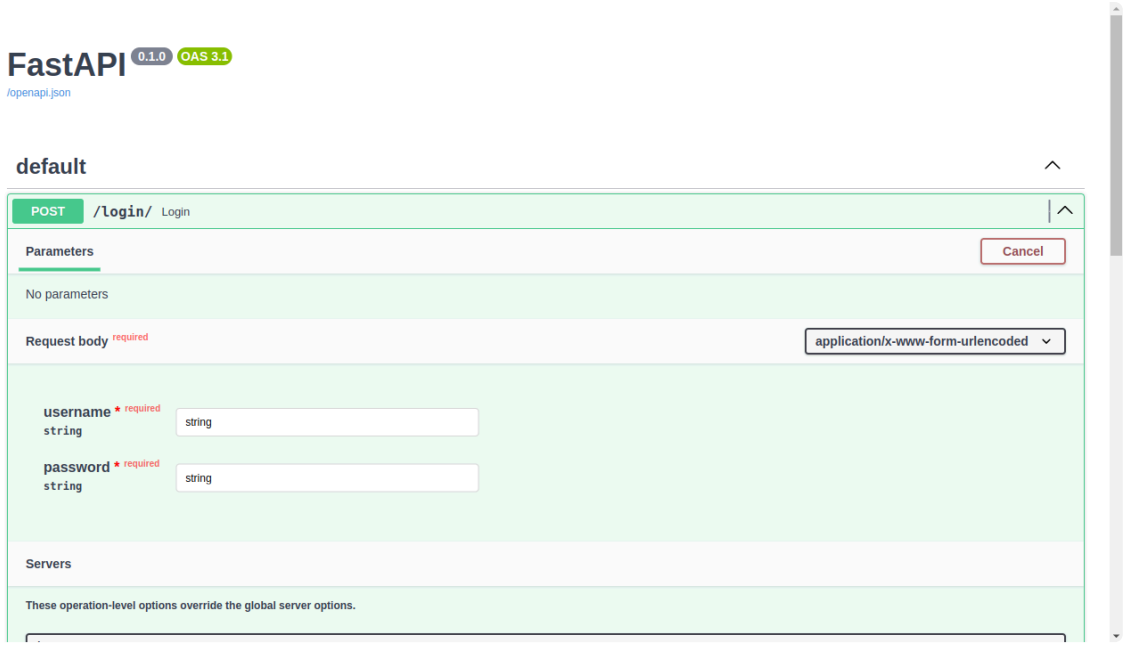
class FormData(BaseModel):
    username: str
    password: str

@app.post("/login/")
async def login(data: Annotated[FormData, Form()]):
    return data
```

FastAPI extraerá los datos de **cada campo** de los **form data** en el request y te dará el modelo de Pydantic que definiste.

Revisa la Documentación

Puedes verificarlo en la interfaz de documentación en `/docs`:



Prohibir Campos de Formulario Extra

En algunos casos de uso especiales (probablemente no muy comunes), podrías querer **restringir** los campos de formulario a solo aquellos declarados en el modelo de Pydantic. Y **prohibir** cualquier campo **extra**.

> **Nota**
> Esto es compatible desde la versión `0.114.0` de FastAPI. 😳

Puedes usar la configuración del modelo de Pydantic para `forbid` cualquier campo `extra`:

```
from typing import Annotated

from fastapi import FastAPI, Form
from pydantic import BaseModel

app = FastAPI()

class FormData(BaseModel):
    username: str
```

```
    password: str
    model_config = {"extra": "forbid"}

@app.post("/login/")
async def login(data: Annotated[FormData, Form()]):
    return data
```

Si un cliente intenta enviar datos extra, recibirá un response de **error**.

Por ejemplo, si el cliente intenta enviar los campos de formulario:

- `username`: `Rick`
- `password`: `Portal Gun`
- `extra`: `Mr. Poopybutthole`

Recibirá un response de error indicando que el campo `extra` no está permitido:

```
{
    "detail": [
        {
            "type": "extra_forbidden",
            "loc": ["body", "extra"],
            "msg": "Extra inputs are not permitted",
            "input": "Mr. Poopybutthole"
        }
    ]
}
```

Resumen

Puedes usar modelos de Pydantic para declarar campos de formulario en FastAPI.

Archivos de Request

Puedes definir archivos que serán subidos por el cliente utilizando `File`.

Nota

Para recibir archivos subidos, primero instala `python-multipart`.

Agrégalo a tu proyecto:

```
$ uv add python-multipart
```

Esto es porque los archivos subidos se envían como "form data".

Importa File

Importa `File` y `UploadFile` desde `fastapi`:

```python
from typing import Annotated

from fastapi import FastAPI, File, UploadFile

app = FastAPI()

@app.post("/files/")
async def create_file(file: Annotated[bytes, File()]):
    return {"file_size": len(file)}

@app.post("/uploadfile/")
async def create_upload_file(file: UploadFile):
    return {"filename": file.filename}
```

Define Parámetros File

Crea parámetros de archivo de la misma manera que lo harías para `Body` o `Form`:

```python
from typing import Annotated

from fastapi import FastAPI, File, UploadFile

app = FastAPI()
```

```python
@app.post("/files/")
async def create_file(file: Annotated[bytes, File()]):
    return {"file_size": len(file)}

@app.post("/uploadfile/")
async def create_upload_file(file: UploadFile):
    return {"filename": file.filename}
```

Nota

`File` es una clase que hereda directamente de `Form`.

Pero recuerda que cuando importas `Query`, `Path`, `File` y otros desde `fastapi`, esos son en realidad funciones que devuelven clases especiales.

Consejo

Para declarar cuerpos de File, necesitas usar `File`, porque de otra manera los parámetros serían interpretados como parámetros query o parámetros de cuerpo (JSON).

Los archivos se subirán como "form data".

Si declaras el tipo de tu parámetro de *path operation function* como `bytes`, **FastAPI** leerá el archivo por ti y recibirás el contenido como `bytes`.

Ten en cuenta que esto significa que todo el contenido se almacenará en memoria. Esto funcionará bien para archivos pequeños.

Pero hay varios casos en los que podrías beneficiarte de usar `UploadFile`.

Parámetros de Archivo con UploadFile

Define un parámetro de archivo con un tipo de `UploadFile`:

```python
from typing import Annotated

from fastapi import FastAPI, File, UploadFile

app = FastAPI()

@app.post("/files/")
async def create_file(file: Annotated[bytes, File()]):
    return {"file_size": len(file)}

@app.post("/uploadfile/")
async def create_upload_file(file: UploadFile):
    return {"filename": file.filename}
```

Usar `UploadFile` tiene varias ventajas sobre `bytes`:

- No tienes que usar `File()` en el valor por defecto del parámetro.
- Usa un archivo "spooled":
    - Un archivo almacenado en memoria hasta un límite de tamaño máximo, y después de superar este límite, se almacenará en el disco.
- Esto significa que funcionará bien para archivos grandes como imágenes, videos, binarios grandes, etc. sin consumir toda la memoria.
- Puedes obtener metadatos del archivo subido.
- Tiene una interfaz `async` parecida a un archivo.
- Expone un objeto Python real `SpooledTemporaryFile` que puedes pasar directamente a otros paquetes que esperan un objeto parecido a un archivo.

UploadFile

`UploadFile` tiene los siguientes atributos:

- `filename`: Un `str` con el nombre original del archivo que fue subido (por ejemplo, `myimage.jpg`).

- `content_type`: Un `str` con el tipo de contenido (MIME type / media type) (por ejemplo, `image/jpeg`).
- `file`: Un `SpooledTemporaryFile` (un objeto parecido a un archivo). Este es el objeto de archivo Python real que puedes pasar directamente a otras funciones o paquetes que esperan un objeto "parecido a un archivo".

`UploadFile` tiene los siguientes métodos `async`. Todos ellos llaman a los métodos correspondientes del archivo por debajo (usando el `SpooledTemporaryFile` interno).

- `write(data)`: Escribe `data` (`str` o `bytes`) en el archivo.
- `read(size)`: Lee `size` (`int`) bytes/caracteres del archivo.
- `seek(offset)`: Va a la posición de bytes `offset` (`int`) en el archivo.
    - Por ejemplo, `await myfile.seek(0)` iría al inicio del archivo.
        - Esto es especialmente útil si ejecutas `await myfile.read()` una vez y luego necesitas leer el contenido nuevamente.
- `close()`: Cierra el archivo.

Como todos estos métodos son métodos `async`, necesitas "await" para ellos.

Por ejemplo, dentro de una *path operation function* `async` puedes obtener los contenidos con:

```
contents = await myfile.read()
```

Si estás dentro de una *path operation function* normal `def`, puedes acceder al `UploadFile.file` directamente, por ejemplo:

```
contents = myfile.file.read()
```

Detalles Técnicos de async

Cuando usas los métodos `async`, **FastAPI** ejecuta los métodos del archivo en un threadpool y los espera.

/// info | Detalles Técnicos de Starlette

El `UploadFile` de **FastAPI** hereda directamente del `UploadFile` de **Starlette**, pero añade algunas partes necesarias para hacerlo compatible con **Pydantic** y las otras partes de FastAPI.

///

Qué es "Form Data"

La manera en que los formularios de HTML (`<form></form>`) envían los datos al servidor normalmente utiliza una codificación "especial" para esos datos, es diferente de JSON.

FastAPI se asegurará de leer esos datos del lugar correcto en lugar de JSON.

/// info | Detalles Técnicos

Los datos de los formularios normalmente se codifican usando el "media type" `application/x-www-form-urlencoded` cuando no incluyen archivos.

Pero cuando el formulario incluye archivos, se codifica como `multipart/form-data`. Si usas `File`, **FastAPI** sabrá que tiene que obtener los archivos de la parte correcta del cuerpo.

Si deseas leer más sobre estas codificaciones y campos de formularios, dirígete a la MDN web docs para `POST`.

///

/// warning | Advertencia

Puedes declarar múltiples parámetros `File` y `Form` en una *path operation*, pero no puedes declarar campos `Body` que esperas recibir como JSON, ya que el request tendrá el cuerpo codificado usando `multipart/form-data` en lugar de `application/json`.

///

Esto no es una limitación de **FastAPI**, es parte del protocolo HTTP.

Subida de Archivos Opcional

Puedes hacer un archivo opcional utilizando anotaciones de tipos estándar y estableciendo un valor por defecto de `None`:

```
from typing import Annotated

from fastapi import FastAPI, File, UploadFile

app = FastAPI()

@app.post("/files/")
async def create_file(file: Annotated[bytes | None,
File()] = None):
    if not file:
        return {"message": "No file sent"}
    else:
        return {"file_size": len(file)}

@app.post("/uploadfile/")
async def create_upload_file(file: UploadFile | None =
None):
    if not file:
        return {"message": "No upload file sent"}
    else:
        return {"filename": file.filename}
```

UploadFile con Metadatos Adicionales

También puedes usar `File()` con `UploadFile`, por ejemplo, para establecer metadatos adicionales:

```
from typing import Annotated

from fastapi import FastAPI, File, UploadFile
```

```
app = FastAPI()

@app.post("/files/")
async def create_file(file: Annotated[bytes,
File(description="A file read as bytes")]):
    return {"file_size": len(file)}

@app.post("/uploadfile/")
async def create_upload_file(
    file: Annotated[UploadFile, File(description="A file
read as UploadFile")],
):
    return {"filename": file.filename}
```

Subidas de Múltiples Archivos

Es posible subir varios archivos al mismo tiempo.

Estarían asociados al mismo "campo de formulario" enviado usando "form data".

Para usar eso, declara una `list` de `bytes` o `UploadFile`:

```
from typing import Annotated

from fastapi import FastAPI, File, UploadFile
from fastapi.responses import HTMLResponse

app = FastAPI()

@app.post("/files/")
async def create_files(files: Annotated[list[bytes],
File()]):
    return {"file_sizes": [len(file) for file in files]}

@app.post("/uploadfiles/")
async def create_upload_files(files: list[UploadFile]):
    return {"filenames": [file.filename for file in
files]}

@app.get("/")
```

```
async def main():
    content = """
<body>
<form action="/files/" enctype="multipart/form-data"
method="post">
<input name="files" type="file" multiple>
<input type="submit">
</form>
<form action="/uploadfiles/" enctype="multipart/form-data"
method="post">
<input name="files" type="file" multiple>
<input type="submit">
</form>
</body>
    """
    return HTMLResponse(content=content)
```

Recibirás, como se declaró, una `list` de `bytes` o `UploadFile` s.

> **Detalles Técnicos**
> También podrías usar `from starlette.responses import HTMLResponse`.
>
> **FastAPI** proporciona las mismas `starlette.responses` como `fastapi.responses` solo como una conveniencia para ti, el desarrollador. Pero la mayoría de los responses disponibles vienen directamente de Starlette.

Subidas de Múltiples Archivos con Metadatos Adicionales

Y de la misma manera que antes, puedes usar `File()` para establecer parámetros adicionales, incluso para `UploadFile`:

```
from typing import Annotated

from fastapi import FastAPI, File, UploadFile
from fastapi.responses import HTMLResponse

app = FastAPI()
```

```python
@app.post("/files/")
async def create_files(
    files: Annotated[list[bytes],
File(description="Multiple files as bytes")],
):
    return {"file_sizes": [len(file) for file in files]}

@app.post("/uploadfiles/")
async def create_upload_files(
    files: Annotated[
        list[UploadFile], File(description="Multiple files
as UploadFile")
    ],
):
    return {"filenames": [file.filename for file in
files]}

@app.get("/")
async def main():
    content = """
<body>
<form action="/files/" enctype="multipart/form-data"
method="post">
<input name="files" type="file" multiple>
<input type="submit">
</form>
<form action="/uploadfiles/" enctype="multipart/form-data"
method="post">
<input name="files" type="file" multiple>
<input type="submit">
</form>
</body>
    """
    return HTMLResponse(content=content)
```

Recapitulación

Usa `File`, `bytes` y `UploadFile` para declarar archivos que se subirán en el request, enviados como form data.

Formularios y archivos del request

Puedes definir archivos y campos de formulario al mismo tiempo usando `File` y `Form`.

> **Nota**
> Para recibir archivos subidos y/o form data, primero instala `python-multipart`.
>
> Añádelo a tu proyecto:
>
> ```
> $ uv add python-multipart
> ```

Importa File y Form

```python
from typing import Annotated

from fastapi import FastAPI, File, Form, UploadFile

app = FastAPI()

@app.post("/files/")
async def create_file(
    file: Annotated[bytes, File()],
    fileb: Annotated[UploadFile, File()],
    token: Annotated[str, Form()],
):
    return {
        "file_size": len(file),
        "token": token,
        "fileb_content_type": fileb.content_type,
    }
```

Define parámetros File y Form

Crea parámetros de archivo y formulario de la misma manera que lo harías para `Body` o `Query`:

```python
from typing import Annotated

from fastapi import FastAPI, File, Form, UploadFile

app = FastAPI()

@app.post("/files/")
async def create_file(
    file: Annotated[bytes, File()],
    fileb: Annotated[UploadFile, File()],
    token: Annotated[str, Form()],
):
    return {
        "file_size": len(file),
        "token": token,
        "fileb_content_type": fileb.content_type,
    }
```

Los archivos y campos de formulario se subirán como form data y recibirás los archivos y campos de formulario.

Y puedes declarar algunos de los archivos como `bytes` y algunos como `UploadFile`.

> ⚠ **Advertencia**
> Puedes declarar múltiples parámetros `File` y `Form` en una *path operation*, pero no puedes también declarar campos `Body` que esperas recibir como JSON, ya que el request tendrá el body codificado usando `multipart/form-data` en lugar de `application/json`.
>
> Esto no es una limitación de **FastAPI**, es parte del protocolo HTTP.

Resumen

Usa `File` y `Form` juntos cuando necesites recibir datos y archivos en el mismo request.

- # Manejo de Errores

Existen muchas situaciones en las que necesitas notificar un error a un cliente que está usando tu API.

Este cliente podría ser un navegador con un frontend, un código de otra persona, un dispositivo IoT, etc.

Podrías necesitar decirle al cliente que:

- El cliente no tiene suficientes privilegios para esa operación.
- El cliente no tiene acceso a ese recurso.
- El ítem al que el cliente intentaba acceder no existe.
- etc.

En estos casos, normalmente devolverías un **código de estado HTTP** en el rango de **400** (de 400 a 499).

Esto es similar a los códigos de estado HTTP 200 (de 200 a 299). Esos códigos de estado "200" significan que de alguna manera hubo un "éxito" en el request.

Los códigos de estado en el rango de 400 significan que hubo un error por parte del cliente.

¿Recuerdas todos esos errores de **"404 Not Found"** (y chistes)?

Usa HTTPException

Para devolver responses HTTP con errores al cliente, usa `HTTPException`.

Importa HTTPException

```
from fastapi import FastAPI, HTTPException

app = FastAPI()

items = {"foo": "The Foo Wrestlers"}

@app.get("/items/{item_id}")
async def read_item(item_id: str):
```

```
    if item_id not in items:
        raise HTTPException(status_code=404, detail="Item
not found")
    return {"item": items[item_id]}
```

Lanza un HTTPException en tu código

`HTTPException` es una excepción de Python normal con datos adicionales relevantes para APIs.

Debido a que es una excepción de Python, no la `return`, sino que la `raise`.

Esto también significa que si estás dentro de una función de utilidad que estás llamando dentro de tu *path operation function*, y lanzas el `HTTPException` desde dentro de esa función de utilidad, no se ejecutará el resto del código en la *path operation function*, terminará ese request de inmediato y enviará el error HTTP del `HTTPException` al cliente.

El beneficio de lanzar una excepción en lugar de `return` un valor será más evidente en la sección sobre Dependencias y Seguridad.

En este ejemplo, cuando el cliente solicita un ítem por un ID que no existe, lanza una excepción con un código de estado de `404`:

```
from fastapi import FastAPI, HTTPException

app = FastAPI()

items = {"foo": "The Foo Wrestlers"}

@app.get("/items/{item_id}")
async def read_item(item_id: str):
    if item_id not in items:
        raise HTTPException(status_code=404, detail="Item
not found")
    return {"item": items[item_id]}
```

El response resultante

Si el cliente solicita `http://example.com/items/foo` (un `item_id` `"foo"`), ese cliente recibirá un código de estado HTTP de 200, y un response JSON de:

```
{
  "item": "The Foo Wrestlers"
}
```

Pero si el cliente solicita `http://example.com/items/bar` (un `item_id` inexistente `"bar"`), ese cliente recibirá un código de estado HTTP de 404 (el error "no encontrado"), y un response JSON de:

```
{
  "detail": "Item not found"
}
```

> **Consejo**
> Cuando lanzas un `HTTPException`, puedes pasar cualquier valor que pueda convertirse a JSON como el parámetro `detail`, no solo `str`.
>
> Podrías pasar un `dict`, un `list`, etc.
>
> Son manejados automáticamente por **FastAPI** y convertidos a JSON.

Agrega headers personalizados

Existen algunas situaciones en las que es útil poder agregar headers personalizados al error HTTP. Por ejemplo, para algunos tipos de seguridad.

Probablemente no necesitarás usarlos directamente en tu código.

Pero en caso de que los necesites para un escenario avanzado, puedes agregar headers personalizados:

```python
from fastapi import FastAPI, HTTPException

app = FastAPI()

items = {"foo": "The Foo Wrestlers"}

@app.get("/items-header/{item_id}")
async def read_item_header(item_id: str):
    if item_id not in items:
        raise HTTPException(
            status_code=404,
            detail="Item not found",
            headers={"X-Error": "There goes my error"},
        )
    return {"item": items[item_id]}
```

Instalar manejadores de excepciones personalizados

Puedes agregar manejadores de excepciones personalizados con las mismas utilidades de excepciones de Starlette.

Supongamos que tienes una excepción personalizada `UnicornException` que tú (o un paquete que usas) podrías lanzar.

Y quieres manejar esta excepción globalmente con FastAPI.

Podrías agregar un manejador de excepciones personalizado con `@app.exception_handler()`:

```python
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

class UnicornException(Exception):
    def __init__(self, name: str):
        self.name = name

app = FastAPI()

@app.exception_handler(UnicornException)
```

```python
async def unicorn_exception_handler(request: Request, exc:
UnicornException):
    return JSONResponse(
        status_code=418,
        content={"message": f"Oops! {exc.name} did
something. There goes a rainbow..."},
    )

@app.get("/unicorns/{name}")
async def read_unicorn(name: str):
    if name == "yolo":
        raise UnicornException(name=name)
    return {"unicorn_name": name}
```

Aquí, si solicitas `/unicorns/yolo`, la *path operation* hará `raise` de un `UnicornException`.

Pero será manejado por el `unicorn_exception_handler`.

Así que recibirás un error limpio, con un código de estado HTTP de `418` y un contenido JSON de:

```
{"message": "Oops! yolo did something. There goes a
rainbow..."}
```

> **Detalles Técnicos**
> También podrías usar `from starlette.requests import Request` y `from starlette.responses import JSONResponse`.
>
> **FastAPI** ofrece las mismas `starlette.responses` como `fastapi.responses` solo como una conveniencia para ti, el desarrollador. Pero la mayoría de los responses disponibles vienen directamente de Starlette. Lo mismo con `Request`.

Sobrescribir los manejadores de excepciones por defecto

FastAPI tiene algunos manejadores de excepciones por defecto.

Estos manejadores se encargan de devolver los responses JSON por defecto cuando lanzas un `HTTPException` y cuando el request tiene datos inválidos.

Puedes sobrescribir estos manejadores de excepciones con los tuyos propios.

Sobrescribir excepciones de validación de request

Cuando un request contiene datos inválidos, **FastAPI** lanza internamente un `RequestValidationError`.

Y también incluye un manejador de excepciones por defecto para ello.

Para sobrescribirlo, importa el `RequestValidationError` y úsalo con `@app.exception_handler(RequestValidationError)` para decorar el manejador de excepciones.

El manejador de excepciones recibirá un `Request` y la excepción.

```
from fastapi import FastAPI, HTTPException
from fastapi.exceptions import RequestValidationError
from fastapi.responses import PlainTextResponse
from starlette.exceptions import HTTPException as
StarletteHTTPException

app = FastAPI()

@app.exception_handler(StarletteHTTPException)
async def http_exception_handler(request, exc):
    return PlainTextResponse(str(exc.detail),
status_code=exc.status_code)

@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request, exc:
RequestValidationError):
    message = "Validation errors:"
    for error in exc.errors():
```

```
        message += f"\nField: {error['loc']}, Error:
{error['msg']}"
    return PlainTextResponse(message, status_code=400)

@app.get("/items/{item_id}")
async def read_item(item_id: int):
    if item_id == 3:
        raise HTTPException(status_code=418, detail="Nope!
I don't like 3.")
    return {"item_id": item_id}
```

Ahora, si vas a `/items/foo`, en lugar de obtener el error JSON por defecto con:

```
{
    "detail": [
        {
            "loc": [
                "path",
                "item_id"
            ],
            "msg": "value is not a valid integer",
            "type": "type_error.integer"
        }
    ]
}
```

obtendrás una versión en texto, con:

```
Validation errors:
Field: ('path', 'item_id'), Error: Input should be a valid
integer, unable to parse string as an integer
```

Sobrescribir el manejador de errores de HTTPException

De la misma manera, puedes sobrescribir el manejador de `HTTPException`.

Por ejemplo, podrías querer devolver un response de texto plano en lugar de JSON para estos errores:

```
from fastapi import FastAPI, HTTPException
from fastapi.exceptions import RequestValidationError
from fastapi.responses import PlainTextResponse
from starlette.exceptions import HTTPException as
StarletteHTTPException

app = FastAPI()

@app.exception_handler(StarletteHTTPException)
async def http_exception_handler(request, exc):
    return PlainTextResponse(str(exc.detail),
status_code=exc.status_code)

@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request, exc:
RequestValidationError):
    message = "Validation errors:"
    for error in exc.errors():
        message += f"\nField: {error['loc']}, Error:
{error['msg']}"
    return PlainTextResponse(message, status_code=400)

@app.get("/items/{item_id}")
async def read_item(item_id: int):
    if item_id == 3:
        raise HTTPException(status_code=418, detail="Nope!
I don't like 3.")
    return {"item_id": item_id}
```

Detalles Técnicos

También podrías usar `from starlette.responses import PlainTextResponse`.

FastAPI ofrece las mismas `starlette.responses` como `fastapi.responses` solo como una conveniencia para ti, el

desarrollador. Pero la mayoría de los responses disponibles vienen directamente de Starlette.

⚠ Advertencia

Ten en cuenta que `RequestValidationError` contiene la información del nombre de archivo y la línea donde ocurre el error de validación, para que puedas mostrarla en tus logs con la información relevante si quieres.

Pero eso significa que si simplemente lo conviertes a un string y devuelves esa información directamente, podrías estar filtrando un poquito de información sobre tu sistema, por eso aquí el código extrae y muestra cada error de forma independiente.

Usar el body de RequestValidationError

El `RequestValidationError` contiene el `body` que recibió con datos inválidos.

Podrías usarlo mientras desarrollas tu aplicación para registrar el body y depurarlo, devolverlo al usuario, etc.

```
from fastapi import FastAPI, Request
from fastapi.encoders import jsonable_encoder
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from pydantic import BaseModel

app = FastAPI()

@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request: Request,
exc: RequestValidationError):
    return JSONResponse(
        status_code=422,
        content=jsonable_encoder({"detail": exc.errors(),
"body": exc.body}),
    )
```

```python
class Item(BaseModel):
    title: str
    size: int

@app.post("/items/")
async def create_item(item: Item):
    return item
```

Ahora intenta enviar un ítem inválido como:

```
{
  "title": "towel",
  "size": "XL"
}
```

Recibirás un response que te dirá que los datos son inválidos conteniendo el body recibido:

```
{
  "detail": [
    {
      "loc": [
        "body",
        "size"
      ],
      "msg": "value is not a valid integer",
      "type": "type_error.integer"
    }
  ],
  "body": {
    "title": "towel",
    "size": "XL"
  }
}
```

HTTPException de FastAPI vs HTTPException de Starlette

FastAPI tiene su propio `HTTPException`.

Y la clase de error `HTTPException` de **FastAPI** hereda de la clase de error `HTTPException` de Starlette.

La única diferencia es que el `HTTPException` de **FastAPI** acepta cualquier dato JSON-able para el campo `detail`, mientras que el `HTTPException` de Starlette solo acepta strings para ello.

Así que puedes seguir lanzando un `HTTPException` de **FastAPI** como de costumbre en tu código.

Pero cuando registras un manejador de excepciones, deberías registrarlo para el `HTTPException` de Starlette.

De esta manera, si alguna parte del código interno de Starlette, o una extensión o plug-in de Starlette, lanza un `HTTPException` de Starlette, tu manejador podrá capturarlo y manejarlo.

En este ejemplo, para poder tener ambos `HTTPException` en el mismo código, las excepciones de Starlette son renombradas a `StarletteHTTPException`:

```
from starlette.exceptions import HTTPException as
StarletteHTTPException
```

Reutilizar los manejadores de excepciones de FastAPI

Si quieres usar la excepción junto con los mismos manejadores de excepciones por defecto de **FastAPI**, puedes importar y reutilizar los manejadores de excepciones por defecto de `fastapi.exception_handlers`:

```
from fastapi import FastAPI, HTTPException
from fastapi.exception_handlers import (
    http_exception_handler,
    request_validation_exception_handler,
)
```

```python
from fastapi.exceptions import RequestValidationError
from starlette.exceptions import HTTPException as
StarletteHTTPException

app = FastAPI()

@app.exception_handler(StarletteHTTPException)
async def custom_http_exception_handler(request, exc):
    print(f"OMG! An HTTP error!: {repr(exc)}")
    return await http_exception_handler(request, exc)

@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request, exc):
    print(f"OMG! The client sent invalid data!: {exc}")
    return await
request_validation_exception_handler(request, exc)

@app.get("/items/{item_id}")
async def read_item(item_id: int):
    if item_id == 3:
        raise HTTPException(status_code=418, detail="Nope!
I don't like 3.")
    return {"item_id": item_id}
```

En este ejemplo solo estás `print` eando el error con un mensaje muy expresivo, pero te haces una idea. Puedes usar la excepción y luego simplemente reutilizar los manejadores de excepciones por defecto.

Configuración de Path Operation

Hay varios parámetros que puedes pasar a tu *path operation decorator* para configurarlo.

> ⚠ **Advertencia**
> Ten en cuenta que estos parámetros se pasan directamente al *path operation decorator*, no a tu *path operation function*.

Código de Estado del Response

Puedes definir el `status_code` (HTTP) que se utilizará en el response de tu *path operation*.

Puedes pasar directamente el código `int`, como `404`.

Pero si no recuerdas para qué es cada código numérico, puedes usar las constantes atajo en `status`:

```
from fastapi import FastAPI, status
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()

@app.post("/items/", status_code=status.HTTP_201_CREATED)
async def create_item(item: Item) -> Item:
    return item
```

Ese código de estado se usará en el response y se añadirá al esquema de OpenAPI.

> **Detalles Técnicos**
> También podrías usar `from starlette import status`.
>
> **FastAPI** ofrece el mismo `starlette.status` como `fastapi.status` solo por conveniencia para ti, el desarrollador. Pero viene directamente de Starlette.

Puedes añadir tags a tu *path operation*, pasando el parámetro `tags` con un `list` de `str` (comúnmente solo una `str`):

```
from fastapi import FastAPI
from pydantic import BaseModel
```

```python
app = FastAPI()

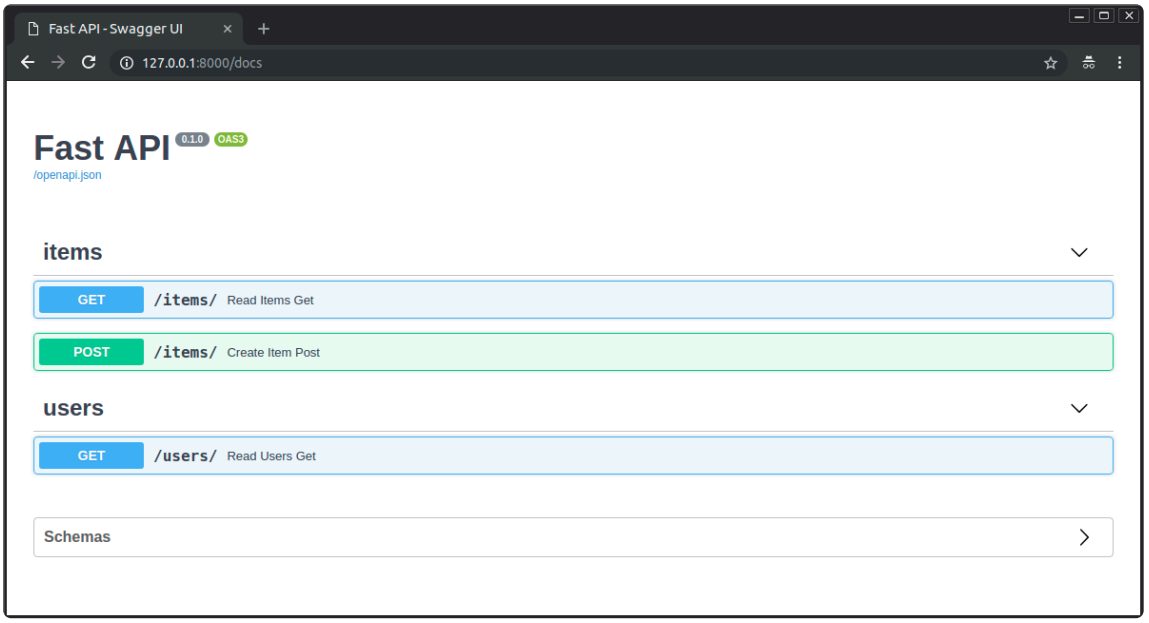
class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()

@app.post("/items/", tags=["items"])
async def create_item(item: Item) -> Item:
    return item

@app.get("/items/", tags=["items"])
async def read_items():
    return [{"name": "Foo", "price": 42}]

@app.get("/users/", tags=["users"])
async def read_users():
    return [{"username": "johndoe"}]
```

Serán añadidas al esquema de OpenAPI y usadas por las interfaces de documentación automática:



Tags con Enums

Si tienes una gran aplicación, podrías terminar acumulando **varias tags**, y querrías asegurarte de que siempre uses la **misma tag** para *path operations* relacionadas.

En estos casos, podría tener sentido almacenar las tags en un `Enum`.

FastAPI soporta eso de la misma manera que con strings normales:

```
from enum import Enum

from fastapi import FastAPI

app = FastAPI()

class Tags(Enum):
    items = "items"
    users = "users"

@app.get("/items/", tags=[Tags.items])
async def get_items():
    return ["Portal gun", "Plumbus"]

@app.get("/users/", tags=[Tags.users])
async def read_users():
    return ["Rick", "Morty"]
```

Resumen y Descripción

Puedes añadir un `summary` y `description`:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
```

```
    price: float
    tax: float | None = None
    tags: set[str] = set()

@app.post(
    "/items/",
    summary="Create an item",
    description="Create an item with all the information,
name, description, price, tax and a set of unique tags",
)
async def create_item(item: Item) -> Item:
    return item
```

Descripción desde docstring

Como las descripciones tienden a ser largas y cubrir múltiples líneas, puedes declarar la descripción de la *path operation* en la docstring de la función y **FastAPI** la leerá desde allí.

Puedes escribir Markdown en el docstring, se interpretará y mostrará correctamente (teniendo en cuenta la indentación del docstring).

```
from fastapi import FastAPI
from pydantic import BaseModel

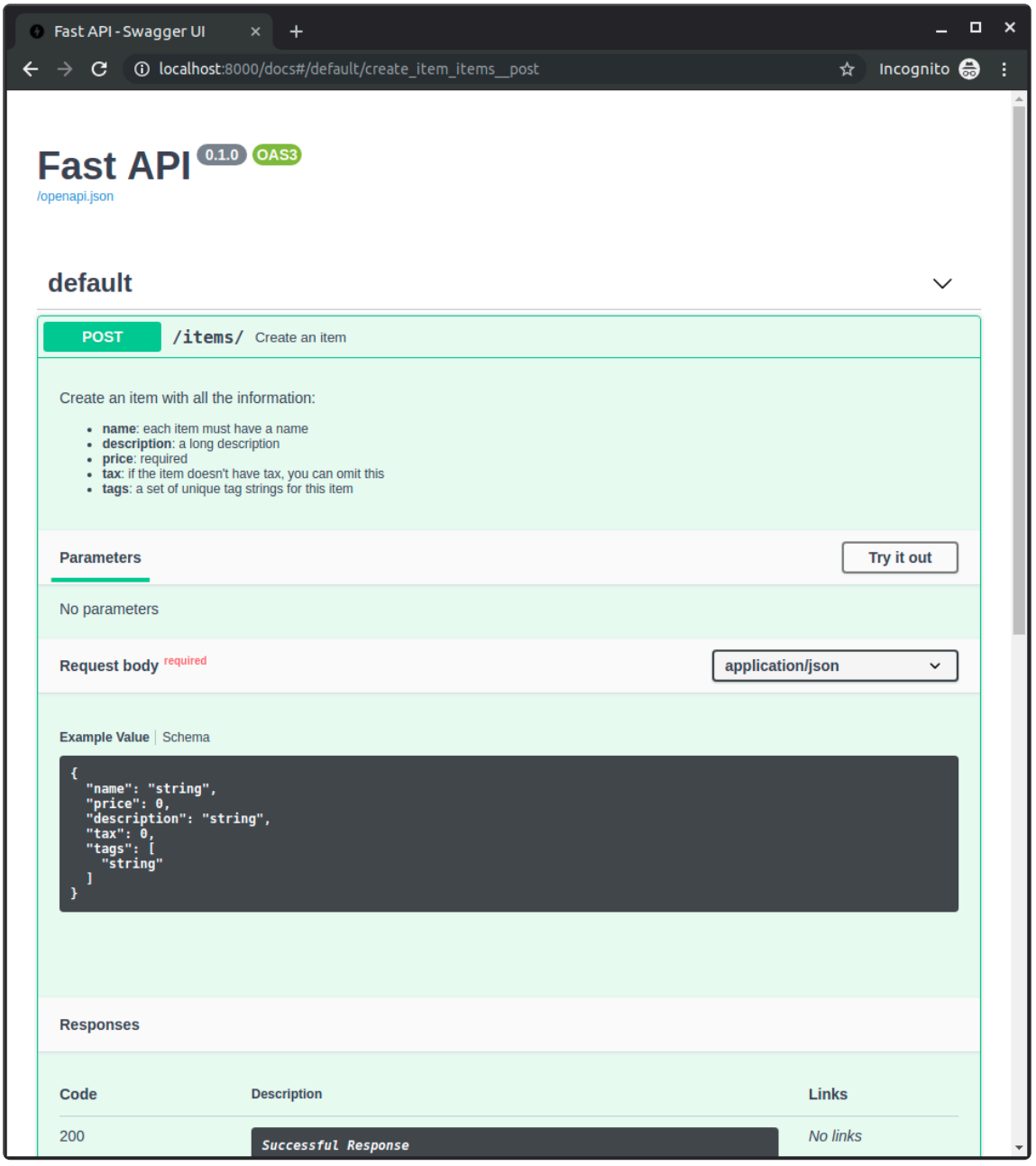
app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()

@app.post("/items/", summary="Create an item")
async def create_item(item: Item) -> Item:
    """
    Create an item with all the information:
```

```
    - **name**: each item must have a name
    - **description**: a long description
    - **price**: required
    - **tax**: if the item doesn't have tax, you can omit
this
    - **tags**: a set of unique tag strings for this item
    """
    return item
```

Será usado en la documentación interactiva:



Descripción del Response

Puedes especificar la descripción del response con el parámetro `response_description`:

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()

@app.post(
    "/items/",
    summary="Create an item",
    response_description="The created item",
)
async def create_item(item: Item) -> Item:
    """
    Create an item with all the information:

    - **name**: each item must have a name
    - **description**: a long description
    - **price**: required
    - **tax**: if the item doesn't have tax, you can omit this
    - **tags**: a set of unique tag strings for this item
    """
    return item
```

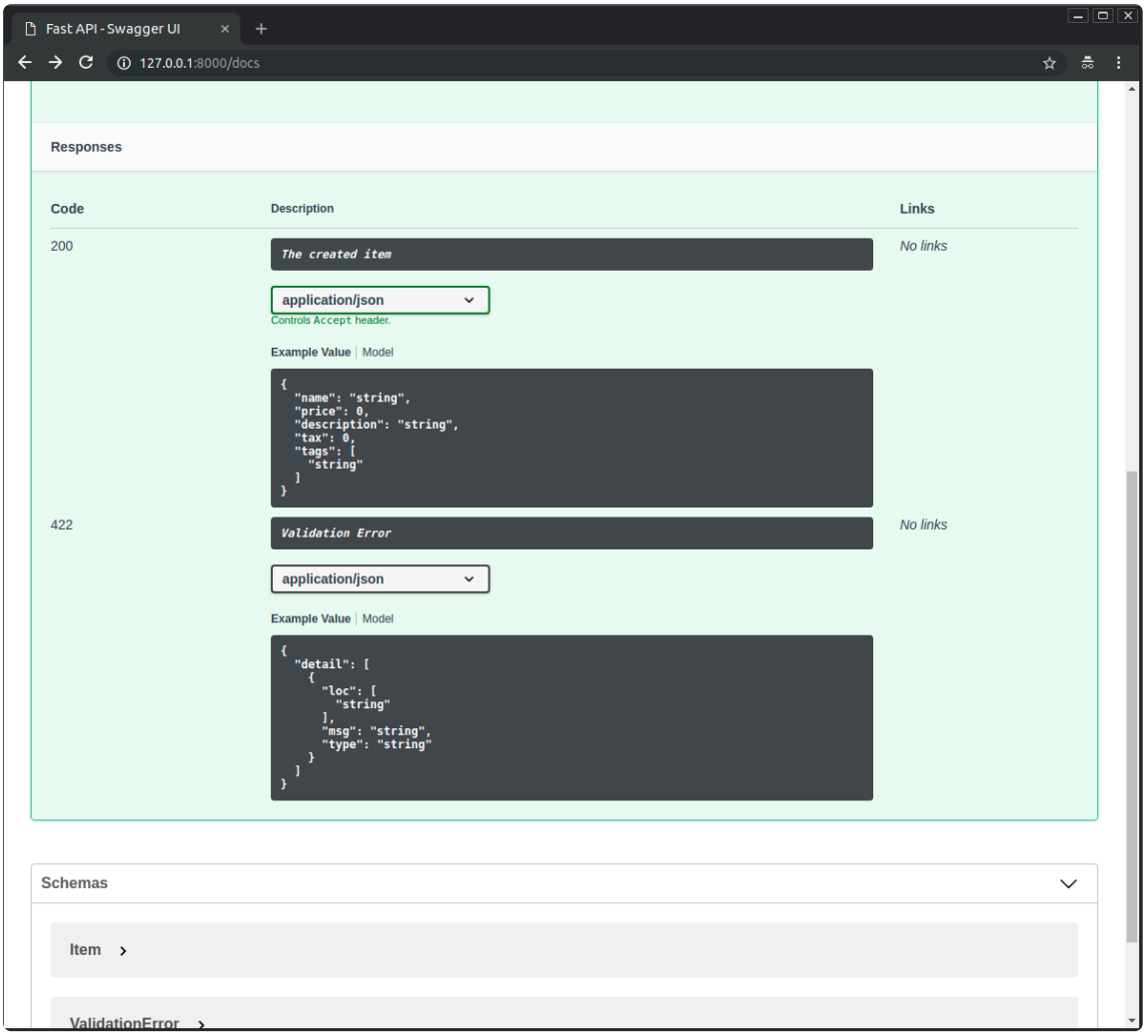
Nota

Ten en cuenta que `response_description` se refiere específicamente al response, mientras que `description` se refiere a la *path operation* en general.

Consejo

OpenAPI especifica que cada *path operation* requiere una descripción de response.

Entonces, si no proporcionas una, **FastAPI** generará automáticamente una de "Response exitoso".



Deprecar una path operation

Si necesitas marcar una *path operation* como obsoleta, pero sin eliminarla, pasa el parámetro `deprecated`:

```
from fastapi import FastAPI

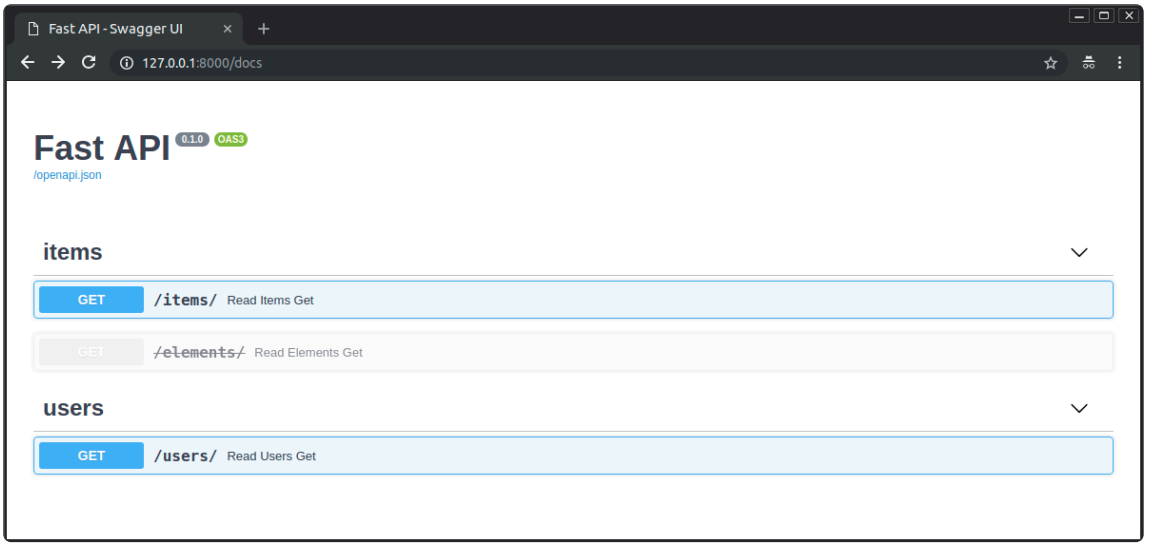
app = FastAPI()

@app.get("/items/", tags=["items"])
async def read_items():
    return [{"name": "Foo", "price": 42}]
```

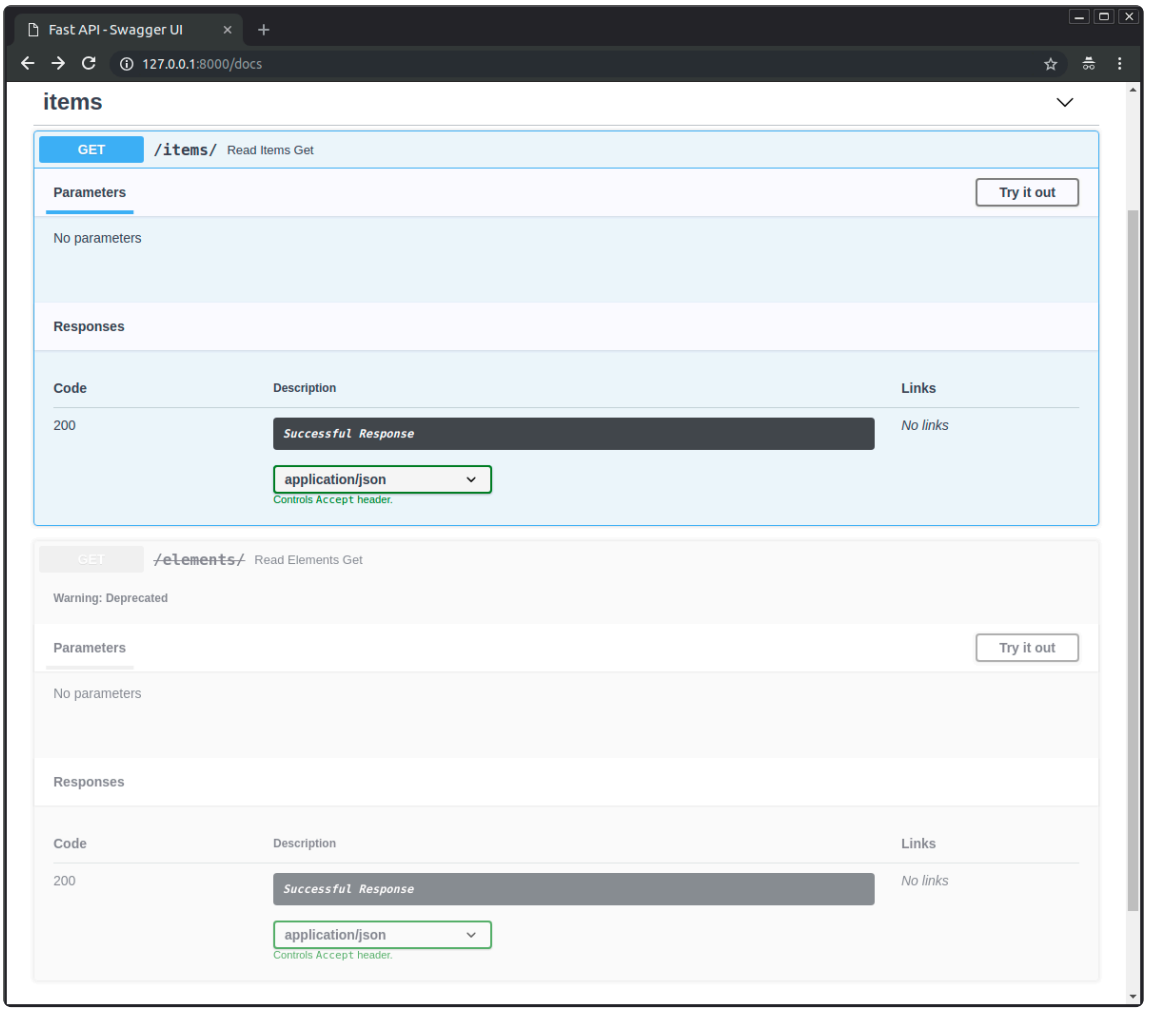
```python
@app.get("/users/", tags=["users"])
async def read_users():
    return [{"username": "johndoe"}]

@app.get("/elements/", tags=["items"], deprecated=True)
async def read_elements():
    return [{"item_id": "Foo"}]
```

Se marcará claramente como deprecated en la documentación interactiva:



Revisa cómo lucen las *path operations* deprecadas y no deprecadas:



Resumen

Puedes configurar y añadir metadatos a tus *path operations* fácilmente pasando parámetros a los *path operation decorators*.

Codificador compatible con JSON

Hay algunos casos en los que podrías necesitar convertir un tipo de dato (como un modelo de Pydantic) a algo compatible con JSON (como un `dict`, `list`, etc).

Por ejemplo, si necesitas almacenarlo en una base de datos.

Para eso, **FastAPI** proporciona una función `jsonable_encoder()`.

Usando el jsonable_encoder

Imaginemos que tienes una base de datos `fake_db` que solo recibe datos compatibles con JSON.

Por ejemplo, no recibe objetos `datetime`, ya que no son compatibles con JSON.

Entonces, un objeto `datetime` tendría que ser convertido a un `str` que contenga los datos en formato ISO.

De la misma manera, esta base de datos no recibiría un modelo de Pydantic (un objeto con atributos), solo un `dict`.

Puedes usar `jsonable_encoder` para eso.

Recibe un objeto, como un modelo de Pydantic, y devuelve una versión compatible con JSON:

```python
from datetime import datetime

from fastapi import FastAPI
from fastapi.encoders import jsonable_encoder
from pydantic import BaseModel

fake_db = {}

class Item(BaseModel):
    title: str
    timestamp: datetime
    description: str | None = None

app = FastAPI()

@app.put("/items/{id}")
def update_item(id: str, item: Item):
    json_compatible_item_data = jsonable_encoder(item)
    fake_db[id] = json_compatible_item_data
```

En este ejemplo, convertiría el modelo de Pydantic a un `dict`, y el `datetime` a un `str`.

El resultado de llamarlo es algo que puede ser codificado con la función estándar de Python `json.dumps()`.

No devuelve un gran `str` que contenga los datos en formato JSON (como un string). Devuelve una estructura de datos estándar de Python (por ejemplo, un `dict`) con valores y sub-valores que son todos compatibles con JSON.

> **Nota**
> `jsonable_encoder` es utilizado internamente por **FastAPI** para convertir datos. Pero es útil en muchos otros escenarios.

Body - Actualizaciones

Actualización reemplazando con PUT

Para actualizar un ítem puedes utilizar la operación de HTTP PUT.

Puedes usar el `jsonable_encoder` para convertir los datos de entrada en datos que se puedan almacenar como JSON (por ejemplo, con una base de datos NoSQL). Por ejemplo, convirtiendo `datetime` a `str`.

```
from fastapi import FastAPI
from fastapi.encoders import jsonable_encoder
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str | None = None
    description: str | None = None
    price: float | None = None
    tax: float = 10.5
    tags: list[str] = []

items = {
    "foo": {"name": "Foo", "price": 50.2},
    "bar": {"name": "Bar", "description": "The
bartenders", "price": 62, "tax": 20.2},
    "baz": {"name": "Baz", "description": None, "price":
50.2, "tax": 10.5, "tags": []},
}

@app.get("/items/{item_id}", response_model=Item)
```

```python
async def read_item(item_id: str):
    return items[item_id]

@app.put("/items/{item_id}", response_model=Item)
async def update_item(item_id: str, item: Item):
    update_item_encoded = jsonable_encoder(item)
    items[item_id] = update_item_encoded
    return update_item_encoded
```

`PUT` se usa para recibir datos que deben reemplazar los datos existentes.

Advertencia sobre el reemplazo

Esto significa que si quieres actualizar el ítem `bar` usando `PUT` con un body que contenga:

```
{
    "name": "Barz",
    "price": 3,
    "description": None,
}
```

debido a que no incluye el atributo ya almacenado `"tax": 20.2`, el modelo de entrada tomaría el valor por defecto de `"tax": 10.5`.

Y los datos se guardarían con ese "nuevo" `tax` de `10.5`.

Actualizaciones parciales con PATCH

También puedes usar la operación de HTTP `PATCH` para actualizar *parcialmente* datos.

Esto significa que puedes enviar solo los datos que deseas actualizar, dejando el resto intacto.

> **Nota**
> `PATCH` es menos usado y conocido que `PUT`.

Y muchos equipos utilizan solo `PUT`, incluso para actualizaciones parciales.

Eres **libre** de usarlos como desees, **FastAPI** no impone ninguna restricción.

Pero esta guía te muestra, más o menos, cómo se pretende que se usen.

Uso del parámetro exclude_unset de Pydantic

Si quieres recibir actualizaciones parciales, es muy útil usar el parámetro `exclude_unset` en el `.model_dump()` del modelo de Pydantic.

Como `item.model_dump(exclude_unset=True)`.

Eso generaría un `dict` solo con los datos que se establecieron al crear el modelo `item`, excluyendo los valores por defecto.

Luego puedes usar esto para generar un `dict` solo con los datos que se establecieron (enviados en el request), omitiendo los valores por defecto:

```
from fastapi import FastAPI
from fastapi.encoders import jsonable_encoder
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str | None = None
    description: str | None = None
    price: float | None = None
    tax: float = 10.5
    tags: list[str] = []

items = {
    "foo": {"name": "Foo", "price": 50.2},
    "bar": {"name": "Bar", "description": "The
bartenders", "price": 62, "tax": 20.2},
```

```
    "baz": {"name": "Baz", "description": None, "price":
50.2, "tax": 10.5, "tags": []},
}

@app.get("/items/{item_id}", response_model=Item)
async def read_item(item_id: str):
    return items[item_id]

@app.patch("/items/{item_id}")
async def update_item(item_id: str, item: Item) -> Item:
    stored_item_data = items[item_id]
    stored_item_model = Item(**stored_item_data)
    update_data = item.model_dump(exclude_unset=True)
    updated_item =
stored_item_model.model_copy(update=update_data)
    items[item_id] = jsonable_encoder(updated_item)
    return updated_item
```

Uso del parámetro update de Pydantic

Ahora, puedes crear una copia del modelo existente usando `.model_copy()`, y pasar el parámetro `update` con un `dict` que contenga los datos a actualizar.

Como `stored_item_model.model_copy(update=update_data)`:

```
from fastapi import FastAPI
from fastapi.encoders import jsonable_encoder
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str | None = None
    description: str | None = None
    price: float | None = None
    tax: float = 10.5
    tags: list[str] = []

items = {
```

```
    "foo": {"name": "Foo", "price": 50.2},
    "bar": {"name": "Bar", "description": "The
bartenders", "price": 62, "tax": 20.2},
    "baz": {"name": "Baz", "description": None, "price":
50.2, "tax": 10.5, "tags": []},
}

@app.get("/items/{item_id}", response_model=Item)
async def read_item(item_id: str):
    return items[item_id]

@app.patch("/items/{item_id}")
async def update_item(item_id: str, item: Item) -> Item:
    stored_item_data = items[item_id]
    stored_item_model = Item(**stored_item_data)
    update_data = item.model_dump(exclude_unset=True)
    updated_item =
stored_item_model.model_copy(update=update_data)
    items[item_id] = jsonable_encoder(updated_item)
    return updated_item
```

Resumen de actualizaciones parciales

En resumen, para aplicar actualizaciones parciales deberías:

- (Opcionalmente) usar `PATCH` en lugar de `PUT`.
- Recuperar los datos almacenados.
- Poner esos datos en un modelo de Pydantic.
- Generar un `dict` sin valores por defecto del modelo de entrada (usando `exclude_unset`).
    - De esta manera puedes actualizar solo los valores realmente establecidos por el usuario, en lugar de sobrescribir valores ya almacenados con valores por defecto en tu modelo.
- Crear una copia del modelo almacenado, actualizando sus atributos con las actualizaciones parciales recibidas (usando el parámetro `update`).
- Convertir el modelo copiado en algo que pueda almacenarse en tu DB (por ejemplo, usando el `jsonable_encoder`).

> - Esto es comparable a usar el método `.model_dump()` del modelo de nuevo, pero asegura (y convierte) los valores a tipos de datos que pueden convertirse a JSON, por ejemplo, `datetime` a `str`.

- Guardar los datos en tu DB.
- Devolver el modelo actualizado.

```
from fastapi import FastAPI
from fastapi.encoders import jsonable_encoder
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str | None = None
    description: str | None = None
    price: float | None = None
    tax: float = 10.5
    tags: list[str] = []

items = {
    "foo": {"name": "Foo", "price": 50.2},
    "bar": {"name": "Bar", "description": "The
bartenders", "price": 62, "tax": 20.2},
    "baz": {"name": "Baz", "description": None, "price":
50.2, "tax": 10.5, "tags": []},
}

@app.get("/items/{item_id}", response_model=Item)
async def read_item(item_id: str):
    return items[item_id]

@app.patch("/items/{item_id}")
async def update_item(item_id: str, item: Item) -> Item:
    stored_item_data = items[item_id]
    stored_item_model = Item(**stored_item_data)
    update_data = item.model_dump(exclude_unset=True)
    updated_item =
stored_item_model.model_copy(update=update_data)
```

```
    items[item_id] = jsonable_encoder(updated_item)
    return updated_item
```

Consejo

Puedes realmente usar esta misma técnica con una operación HTTP `PUT`.

Pero el ejemplo aquí usa `PATCH` porque fue creado para estos casos de uso.

Nota

Observa que el modelo de entrada sigue siendo validado.

Entonces, si deseas recibir actualizaciones parciales que puedan omitir todos los atributos, necesitas tener un modelo con todos los atributos marcados como opcionales (con valores por defecto o `None`).

Para distinguir entre los modelos con todos los valores opcionales para **actualizaciones** y modelos con valores requeridos para **creación**, puedes utilizar las ideas descritas en Modelos Extra.

Dependencias

FastAPI tiene un sistema de **Inyección de Dependencias** muy poderoso pero intuitivo.

Está diseñado para ser muy simple de usar, y para hacer que cualquier desarrollador integre otros componentes con **FastAPI** de forma muy sencilla.

Qué es la "Inyección de Dependencias"

"Inyección de Dependencias" significa, en programación, que hay una manera para que tu código (en este caso, tus *path operation functions*) declare las cosas que necesita para funcionar y utilizar: "dependencias".

Y luego, ese sistema (en este caso **FastAPI**) se encargará de hacer lo que sea necesario para proporcionar a tu código esas dependencias necesarias ("inyectar" las dependencias).

Esto es muy útil cuando necesitas:

- Tener lógica compartida (la misma lógica de código una y otra vez).
- Compartir conexiones a bases de datos.
- Imponer seguridad, autenticación, requisitos de roles, etc.
- Y muchas otras cosas...

Todo esto, mientras minimizas la repetición de código.

Primeros Pasos

Veamos un ejemplo muy simple. Será tan simple que no es muy útil, por ahora.

Pero de esta manera podemos enfocarnos en cómo funciona el sistema de **Inyección de Dependencias**.

Crear una dependencia, o "dependable"

Primero enfoquémonos en la dependencia.

Es solo una función que puede tomar todos los mismos parámetros que una *path operation function* puede tomar:

```
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()

async def common_parameters(q: str | None = None, skip:
int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

@app.get("/items/")
```

```
async def read_items(commons: Annotated[dict,
Depends(common_parameters)]):
    return commons

@app.get("/users/")
async def read_users(commons: Annotated[dict,
Depends(common_parameters)]):
    return commons
```

Eso es todo.

2 líneas.

Y tiene la misma forma y estructura que todas tus *path operation functions*.

Puedes pensar en ella como una *path operation function* sin el "decorador" (sin el `@app.get("/some-path")`).

Y puede devolver lo que quieras.

En este caso, esta dependencia espera:

- Un parámetro de query opcional `q` que es un `str`.
- Un parámetro de query opcional `skip` que es un `int`, y por defecto es `0`.
- Un parámetro de query opcional `limit` que es un `int`, y por defecto es `100`.

Y luego solo devuelve un `dict` que contiene esos valores.

> **Nota**
> FastAPI agregó soporte para `Annotated` (y comenzó a recomendarlo) en la versión 0.95.0.
>
> Si tienes una versión anterior, obtendrás errores al intentar usar `Annotated`.
>
> Asegúrate de Actualizar la versión de FastAPI al menos a la 0.95.1 antes de usar `Annotated`.

Importar Depends

```
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()

async def common_parameters(q: str | None = None, skip:
int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

@app.get("/items/")
async def read_items(commons: Annotated[dict,
Depends(common_parameters)]):
    return commons

@app.get("/users/")
async def read_users(commons: Annotated[dict,
Depends(common_parameters)]):
    return commons
```

Declarar la dependencia, en el "dependant"

De la misma forma en que usas `Body`, `Query`, etc. con los parámetros de tu *path operation function*, usa `Depends` con un nuevo parámetro:

```
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()

async def common_parameters(q: str | None = None, skip:
int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

@app.get("/items/")
```

```
async def read_items(commons: Annotated[dict,
Depends(common_parameters)]):
    return commons

@app.get("/users/")
async def read_users(commons: Annotated[dict,
Depends(common_parameters)]):
    return commons
```

Aunque usas `Depends` en los parámetros de tu función de la misma manera que usas `Body`, `Query`, etc., `Depends` funciona un poco diferente.

Le das a `Depends` un solo parámetro.

Este parámetro debe ser algo como una función.

No la llames directamente (no agregues los paréntesis al final), solo pásala como un parámetro a `Depends()`.

Y esa función toma parámetros de la misma manera que las *path operation functions*.

> **Consejo**
> Verás qué otras "cosas", además de funciones, pueden usarse como dependencias en el próximo capítulo.

Cada vez que llega un nuevo request, **FastAPI** se encargará de:

- Llamar a tu función de dependencia ("dependable") con los parámetros correctos.
- Obtener el resultado de tu función.
- Asignar ese resultado al parámetro en tu *path operation function*.

De esta manera escribes código compartido una vez y **FastAPI** se encarga de llamarlo para tus *path operations*.

> **Consejo**

Nota que no tienes que crear una clase especial y pasarla en algún lugar a **FastAPI** para "registrarla" o algo similar.

Solo la pasas a `Depends` y **FastAPI** sabe cómo hacer el resto.

Compartir dependencias Annotated

En los ejemplos anteriores, ves que hay un poquito de **duplicación de código**.

Cuando necesitas usar la dependencia `common_parameters()`, tienes que escribir todo el parámetro con la anotación de tipo y `Depends()`:

```
commons: Annotated[dict, Depends(common_parameters)]
```

Pero como estamos usando `Annotated`, podemos almacenar ese valor `Annotated` en una variable y usarlo en múltiples lugares:

```
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()

async def common_parameters(q: str | None = None, skip:
int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

CommonsDep = Annotated[dict, Depends(common_parameters)]

@app.get("/items/")
async def read_items(commons: CommonsDep):
    return commons

@app.get("/users/")
async def read_users(commons: CommonsDep):
    return commons
```

> **Consejo**
> Esto es solo Python estándar, se llama un "alias de tipo", en realidad no es específico de **FastAPI**.
>
> Pero porque **FastAPI** está basado en los estándares de Python, incluido `Annotated`, puedes usar este truco en tu código.

Las dependencias seguirán funcionando como se esperaba, y la **mejor parte** es que la **información de tipo se preservará**, lo que significa que tu editor podrá seguir proporcionándote **autocompletado**, **errores en línea**, etc. Lo mismo para otras herramientas como `mypy`.

Esto será especialmente útil cuando lo uses en una **gran code base** donde uses **las mismas dependencias** una y otra vez en **muchas *path operations***.

Usar async o no usar async

Como las dependencias también serán llamadas por **FastAPI** (lo mismo que tus *path operation functions*), las mismas reglas aplican al definir tus funciones.

Puedes usar `async def` o `def` normal.

Y puedes declarar dependencias con `async def` dentro de *path operation functions* normales `def`, o dependencias `def` dentro de *path operation functions* `async def`, etc.

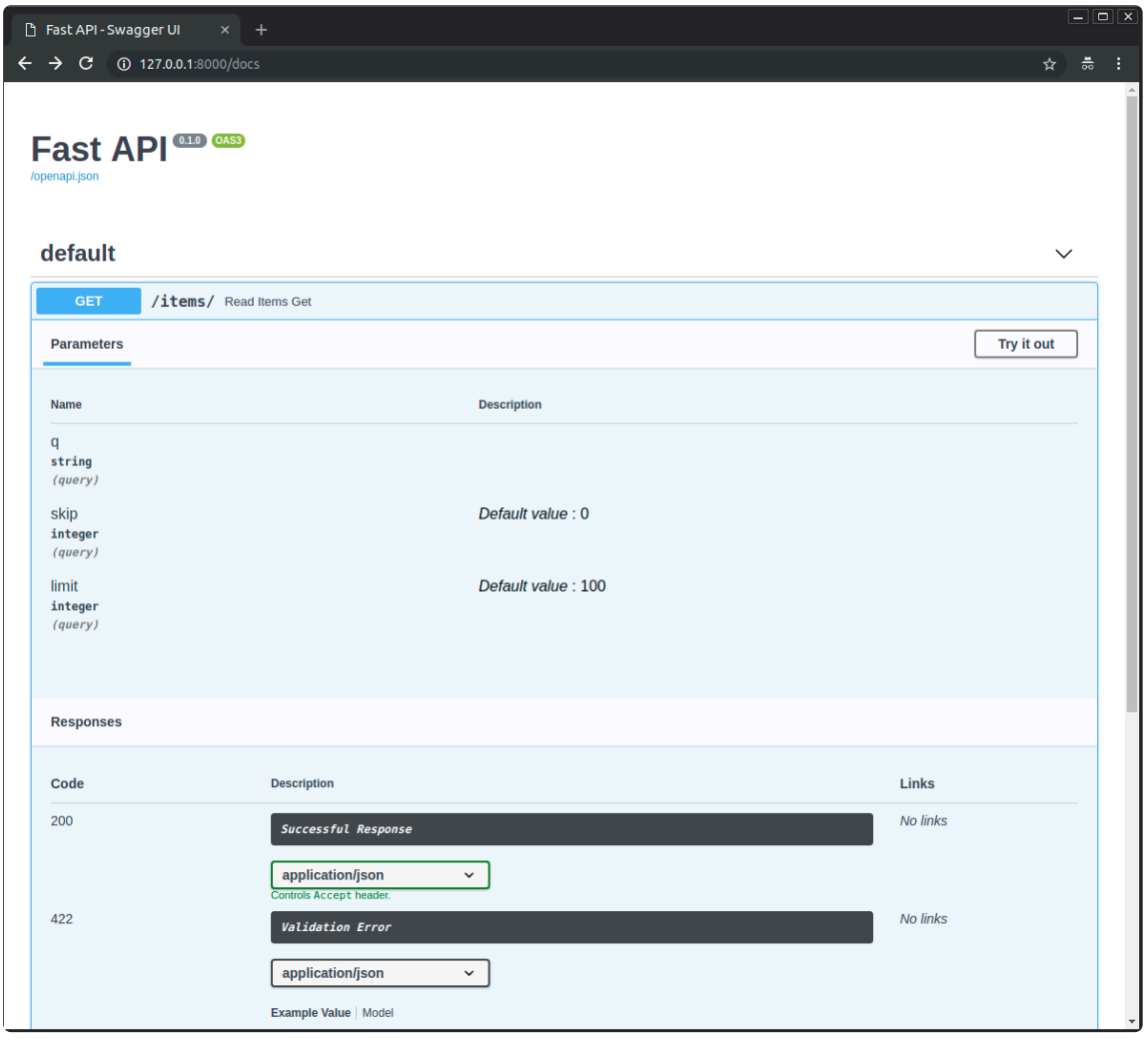
No importa. **FastAPI** sabrá qué hacer.

> **Nota**
> Si no lo sabes, revisa la sección Async: *"¿Con prisa?"* sobre `async` y `await` en la documentación.

Integración con OpenAPI

Todas las declaraciones de request, validaciones y requisitos de tus dependencias (y sub-dependencias) se integrarán en el mismo esquema de OpenAPI.

Así, la documentación interactiva tendrá toda la información de estas dependencias también:



Uso simple

Si lo ves, las *path operation functions* se declaran para ser usadas siempre que un *path* y una *operación* coincidan, y luego **FastAPI** se encarga de llamar la función con los parámetros correctos, extrayendo los datos del request.

En realidad, todos (o la mayoría) de los frameworks web funcionan de esta misma manera.

Nunca llamas directamente a esas funciones. Son llamadas por tu framework (en este caso, **FastAPI**).

Con el sistema de Inyección de Dependencias, también puedes decirle a **FastAPI** que tu *path operation function* también "depende" de algo más que debe ejecutarse antes que tu *path operation function*, y **FastAPI** se encargará de ejecutarlo e "inyectar" los resultados.

Otros términos comunes para esta misma idea de "inyección de dependencias" son:

- recursos
- proveedores
- servicios
- inyectables
- componentes

Plug-ins de FastAPI

Las integraciones y "plug-ins" pueden construirse usando el sistema de **Inyección de Dependencias**. Pero, de hecho, en realidad **no hay necesidad de crear "plug-ins"**, ya que al usar dependencias es posible declarar una cantidad infinita de integraciones e interacciones que se vuelven disponibles para tus *path operation functions*.

Y las dependencias se pueden crear de una manera muy simple e intuitiva que te permite simplemente importar los paquetes de Python que necesitas, e integrarlos con tus funciones de API en un par de líneas de código, *literalmente*.

Verás ejemplos de esto en los próximos capítulos, sobre bases de datos relacionales y NoSQL, seguridad, etc.

Compatibilidad de FastAPI

La simplicidad del sistema de inyección de dependencias hace que **FastAPI** sea compatible con:

- todas las bases de datos relacionales

- bases de datos NoSQL
- paquetes externos
- APIs externas
- sistemas de autenticación y autorización
- sistemas de monitoreo de uso de la API
- sistemas de inyección de datos de response
- etc.

Simple y Poderoso

Aunque el sistema de inyección de dependencias jerárquico es muy simple de definir y usar, sigue siendo muy poderoso.

Puedes definir dependencias que a su vez pueden definir dependencias ellas mismas.

Al final, se construye un árbol jerárquico de dependencias, y el sistema de **Inyección de Dependencias** se encarga de resolver todas estas dependencias por ti (y sus sub-dependencias) y proporcionar (inyectar) los resultados en cada paso.

Por ejemplo, digamos que tienes 4 endpoints de API (*path operations*):

- `/items/public/`
- `/items/private/`
- `/users/{user_id}/activate`
- `/items/pro/`

entonces podrías agregar diferentes requisitos de permiso para cada uno de ellos solo con dependencias y sub-dependencias:

Integrado con OpenAPI

Todas estas dependencias, al declarar sus requisitos, también añaden parámetros, validaciones, etc. a tus *path operations*.

FastAPI se encargará de agregar todo al esquema de OpenAPI, para que se muestre en los sistemas de documentación interactiva.

- # Clases como dependencias

  Antes de profundizar en el sistema de **Inyección de Dependencias**, vamos a mejorar el ejemplo anterior.

Un dict del ejemplo anterior

En el ejemplo anterior, estábamos devolviendo un `dict` de nuestra dependencia ("dependable"):

```
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()

async def common_parameters(q: str | None = None,
skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

@app.get("/items/")
async def read_items(commons: Annotated[dict,
Depends(common_parameters)]):
    return commons

@app.get("/users/")
async def read_users(commons: Annotated[dict,
Depends(common_parameters)]):
    return commons
```

Pero luego obtenemos un `dict` en el parámetro `commons` de la *path operation function*.

Y sabemos que los editores no pueden proporcionar mucho soporte (como autocompletado) para `dict` s, porque no pueden conocer sus claves y tipos de valor.

Podemos hacerlo mejor...

Qué hace a una dependencia

Hasta ahora has visto dependencias declaradas como funciones.

Pero esa no es la única forma de declarar dependencias (aunque probablemente sea la más común).

El factor clave es que una dependencia debe ser un "callable".

Un " **callable** " en Python es cualquier cosa que Python pueda "llamar" como una función.

Entonces, si tienes un objeto `something` (que podría *no* ser una función) y puedes "llamarlo" (ejecutarlo) como:

```
something()
```

o

```
something(some_argument, some_keyword_argument="foo")
```

entonces es un "callable".

Clases como dependencias

Puedes notar que para crear una instance de una clase en Python, utilizas esa misma sintaxis.

Por ejemplo:

```
class Cat:
    def __init__(self, name: str):
        self.name = name

fluffy = Cat(name="Mr Fluffy")
```

En este caso, `fluffy` es una instance de la clase `Cat`.

Y para crear `fluffy`, estás "llamando" a `Cat`.

Entonces, una clase en Python también es un **callable**.

Entonces, en **FastAPI**, podrías usar una clase de Python como una dependencia.

Lo que **FastAPI** realmente comprueba es que sea un "callable" (función, clase o cualquier otra cosa) y los parámetros definidos.

Si pasas un "callable" como dependencia en **FastAPI**, analizará los parámetros de ese "callable", y los procesará de la misma manera que los parámetros de una *path operation function*. Incluyendo sub-dependencias.

Eso también se aplica a los callables sin parámetros. Igual que sería para *path operation functions* sin parámetros.

Entonces, podemos cambiar la dependencia "dependable" `common_parameters` de arriba a la clase `CommonQueryParams`:

```
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()

fake_items_db = [{"item_name": "Foo"}, {"item_name":
"Bar"}, {"item_name": "Baz"}]

class CommonQueryParams:
    def __init__(self, q: str | None = None, skip: int
= 0, limit: int = 100):
        self.q = q
        self.skip = skip
        self.limit = limit

@app.get("/items/")
async def read_items(commons:
Annotated[CommonQueryParams,
Depends(CommonQueryParams)]):
    response = {}
```

```
    if commons.q:
        response.update({"q": commons.q})
    items = fake_items_db[commons.skip : commons.skip
+ commons.limit]
    response.update({"items": items})
    return response
```

Presta atención al método `__init__` usado para crear la instance de la clase:

```
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()

fake_items_db = [{"item_name": "Foo"}, {"item_name":
"Bar"}, {"item_name": "Baz"}]

class CommonQueryParams:
    def __init__(self, q: str | None = None, skip: int
= 0, limit: int = 100):
        self.q = q
        self.skip = skip
        self.limit = limit

@app.get("/items/")
async def read_items(commons:
Annotated[CommonQueryParams,
Depends(CommonQueryParams)]):
    response = {}
    if commons.q:
        response.update({"q": commons.q})
    items = fake_items_db[commons.skip : commons.skip
+ commons.limit]
    response.update({"items": items})
    return response
```

...tiene los mismos parámetros que nuestros `common_parameters` anteriores:

```python
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()

async def common_parameters(q: str | None = None,
skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

@app.get("/items/")
async def read_items(commons: Annotated[dict,
Depends(common_parameters)]):
    return commons

@app.get("/users/")
async def read_users(commons: Annotated[dict,
Depends(common_parameters)]):
    return commons
```

Esos parámetros son los que **FastAPI** usará para "resolver" la dependencia.

En ambos casos, tendrá:

- Un parámetro de query `q` opcional que es un `str`.
- Un parámetro de query `skip` que es un `int`, con un valor por defecto de `0`.
- Un parámetro de query `limit` que es un `int`, con un valor por defecto de `100`.

En ambos casos, los datos serán convertidos, validados, documentados en el esquema de OpenAPI, etc.

Úsalo

Ahora puedes declarar tu dependencia usando esta clase.

```python
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()

fake_items_db = [{"item_name": "Foo"}, {"item_name":
"Bar"}, {"item_name": "Baz"}]

class CommonQueryParams:
    def __init__(self, q: str | None = None, skip: int
= 0, limit: int = 100):
        self.q = q
        self.skip = skip
        self.limit = limit

@app.get("/items/")
async def read_items(commons:
Annotated[CommonQueryParams,
Depends(CommonQueryParams)]):
    response = {}
    if commons.q:
        response.update({"q": commons.q})
    items = fake_items_db[commons.skip : commons.skip
+ commons.limit]
    response.update({"items": items})
    return response
```

FastAPI llama a la clase `CommonQueryParams`. Esto crea una "instance" de esa clase y la instance será pasada como el parámetro `commons` a tu función.

Anotación de tipos vs Depends

Nota cómo escribimos `CommonQueryParams` dos veces en el código anterior:

```python
commons: Annotated[CommonQueryParams,
Depends(CommonQueryParams)]
```

Consejo

Prefiere usar la versión `Annotated` si es posible.

```
commons: CommonQueryParams =
Depends(CommonQueryParams)
```

El último `CommonQueryParams`, en:

```
... Depends(CommonQueryParams)
```

...es lo que **FastAPI** utilizará realmente para saber cuál es la dependencia.

Es a partir de este que **FastAPI** extraerá los parámetros declarados y es lo que **FastAPI** realmente llamará.

En este caso, el primer `CommonQueryParams`, en:

```
commons: Annotated[CommonQueryParams, ...
```

Consejo

Prefiere usar la versión `Annotated` si es posible.

```
commons: CommonQueryParams ...
```

...no tiene ningún significado especial para **FastAPI**. **FastAPI** no lo usará para la conversión de datos, validación, etc. (ya que está usando `Depends(CommonQueryParams)` para eso).

De hecho, podrías escribir simplemente:

```
commons: Annotated[Any, Depends(CommonQueryParams)]
```

Consejo

Prefiere usar la versión `Annotated` si es posible.

```
commons = Depends(CommonQueryParams)
```

...como en:

```
from typing import Annotated, Any

from fastapi import Depends, FastAPI

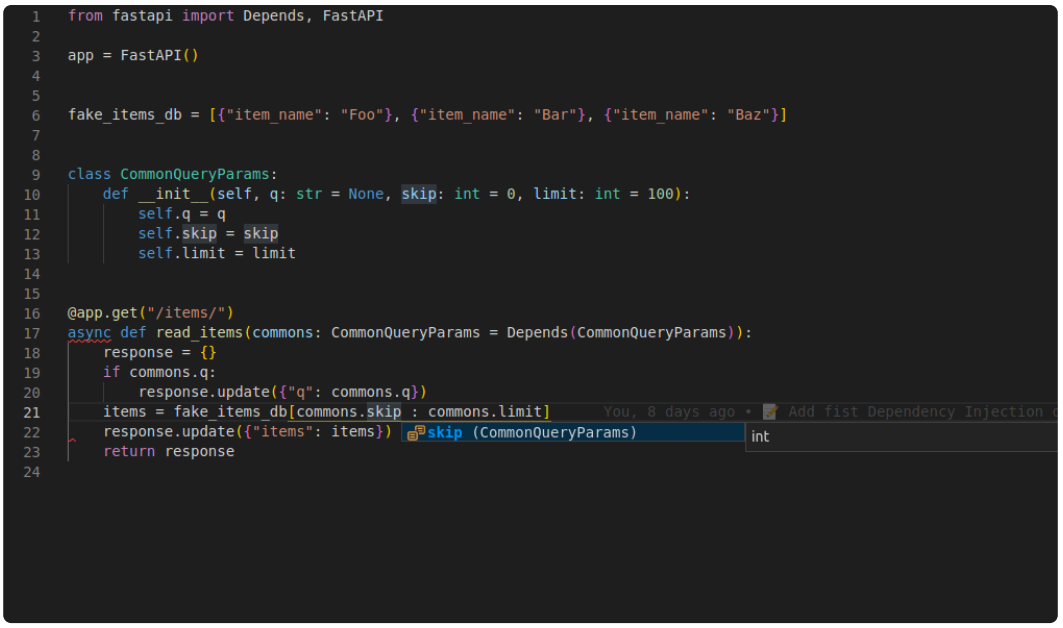
app = FastAPI()

fake_items_db = [{"item_name": "Foo"}, {"item_name":
"Bar"}, {"item_name": "Baz"}]

class CommonQueryParams:
    def __init__(self, q: str | None = None, skip: int
= 0, limit: int = 100):
        self.q = q
        self.skip = skip
        self.limit = limit

@app.get("/items/")
async def read_items(commons: Annotated[Any,
Depends(CommonQueryParams)]):
    response = {}
    if commons.q:
        response.update({"q": commons.q})
    items = fake_items_db[commons.skip : commons.skip
+ commons.limit]
    response.update({"items": items})
    return response
```

Pero declarar el tipo es recomendable, ya que de esa manera tu editor sabrá lo que se pasará como el parámetro `commons`, y entonces podrá ayudarte con el autocompletado, chequeo de tipos, etc:



Atajo

Pero ves que estamos teniendo algo de repetición de código aquí, escribiendo `CommonQueryParams` dos veces:

```
commons: Annotated[CommonQueryParams,
Depends(CommonQueryParams)]
```

> **Consejo**
> Prefiere usar la versión `Annotated` si es posible.

```
commons: CommonQueryParams =
Depends(CommonQueryParams)
```

FastAPI proporciona un atajo para estos casos, en donde la dependencia es *específicamente* una clase que **FastAPI** "llamará" para crear una instance de la clase misma.

Para esos casos específicos, puedes hacer lo siguiente:

En lugar de escribir:

```
commons: Annotated[CommonQueryParams,
Depends(CommonQueryParams)]
```

> **Consejo**
> Prefiere usar la versión `Annotated` si es posible.

```
commons: CommonQueryParams =
Depends(CommonQueryParams)
```

...escribes:

```
commons: Annotated[CommonQueryParams, Depends()]
```

> **Consejo**
> Prefiere usar la versión `Annotated` si es posible.

```
commons: CommonQueryParams = Depends()
```

Declaras la dependencia como el tipo del parámetro, y usas `Depends()` sin ningún parámetro, en lugar de tener que escribir la clase completa *otra vez* dentro de `Depends(CommonQueryParams)`.

El mismo ejemplo se vería entonces así:

```
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()

fake_items_db = [{"item_name": "Foo"}, {"item_name":
"Bar"}, {"item_name": "Baz"}]

class CommonQueryParams:
    def __init__(self, q: str | None = None, skip: int
= 0, limit: int = 100):
        self.q = q
        self.skip = skip
        self.limit = limit
```

```
@app.get("/items/")
async def read_items(commons:
Annotated[CommonQueryParams, Depends()]):
    response = {}
    if commons.q:
        response.update({"q": commons.q})
    items = fake_items_db[commons.skip : commons.skip
+ commons.limit]
    response.update({"items": items})
    return response
```

...y **FastAPI** sabrá qué hacer.

> **Consejo**
> Si eso parece más confuso que útil, ignóralo, no lo *necesitas*.
>
> Es solo un atajo. Porque a **FastAPI** le importa ayudarte a minimizar la repetición de código.

- # Sub-dependencias

Puedes crear dependencias que tengan **sub-dependencias**.

Pueden ser tan **profundas** como necesites.

FastAPI se encargará de resolverlas.

Primera dependencia "dependable"

Podrías crear una primera dependencia ("dependable") así:

```
from typing import Annotated

from fastapi import Cookie, Depends, FastAPI

app = FastAPI()

def query_extractor(q: str | None = None):
    return q
```

```
def query_or_cookie_extractor(
    q: Annotated[str, Depends(query_extractor)],
    last_query: Annotated[str | None, Cookie()] =
None,
):
    if not q:
        return last_query
    return q

@app.get("/items/")
async def read_query(
    query_or_default: Annotated[str,
Depends(query_or_cookie_extractor)],
):
    return {"q_or_cookie": query_or_default}
```

Declara un parámetro de query opcional `q` como un `str`, y luego simplemente lo devuelve.

Esto es bastante simple (no muy útil), pero nos ayudará a centrarnos en cómo funcionan las sub-dependencias.

Segunda dependencia, "dependable" y "dependant"

Luego puedes crear otra función de dependencia (un "dependable") que al mismo tiempo declare una dependencia propia (por lo que también es un "dependant"):

```
from typing import Annotated

from fastapi import Cookie, Depends, FastAPI

app = FastAPI()

def query_extractor(q: str | None = None):
    return q

def query_or_cookie_extractor(
```

```
    q: Annotated[str, Depends(query_extractor)],
    last_query: Annotated[str | None, Cookie()] =
None,
):
    if not q:
        return last_query
    return q

@app.get("/items/")
async def read_query(
    query_or_default: Annotated[str,
Depends(query_or_cookie_extractor)],
):
    return {"q_or_cookie": query_or_default}
```

Centrémonos en los parámetros declarados:

- Aunque esta función es una dependencia ("dependable") en sí misma, también declara otra dependencia (depende de algo más).
    - Depende del `query_extractor`, y asigna el valor que devuelve al parámetro `q`.
- También declara una `last_query` cookie opcional, como un `str`.
    - Si el usuario no proporcionó ningún query `q`, usamos el último query utilizado, que guardamos previamente en una cookie.

Usa la dependencia

Entonces podemos usar la dependencia con:

```
from typing import Annotated

from fastapi import Cookie, Depends, FastAPI

app = FastAPI()

def query_extractor(q: str | None = None):
```

```
    return q

def query_or_cookie_extractor(
    q: Annotated[str, Depends(query_extractor)],
    last_query: Annotated[str | None, Cookie()] =
None,
):
    if not q:
        return last_query
    return q

@app.get("/items/")
async def read_query(
    query_or_default: Annotated[str,
Depends(query_or_cookie_extractor)],
):
    return {"q_or_cookie": query_or_default}
```

/// Nota

Fíjate que solo estamos declarando una dependencia en la *path operation function*, `query_or_cookie_extractor`.

Pero **FastAPI** sabrá que tiene que resolver `query_extractor` primero, para pasar los resultados de eso a `query_or_cookie_extractor` al llamarlo.

///

Usando la misma dependencia múltiples veces

Si una de tus dependencias se declara varias veces para la misma *path operation*, por ejemplo, múltiples dependencias tienen una sub-dependencia común, **FastAPI** sabrá llamar a esa sub-dependencia solo una vez por request.

Y guardará el valor devuelto en un "caché" y lo pasará a todos los "dependants" que lo necesiten en ese request específico, en lugar de llamar a la dependencia varias veces para el mismo request.

En un escenario avanzado donde sabes que necesitas que la dependencia se llame en cada paso (posiblemente varias veces) en el mismo request en lugar de usar el valor "en caché", puedes establecer el parámetro `use_cache=False` al usar `Depends`:

```
async def needy_dependency(fresh_value: Annotated[str, Depends(get_value, use_cache=False)]):
    return {"fresh_value": fresh_value}
```

> **Consejo**
> Prefiere usar la versión `Annotated` si es posible.

```
async def needy_dependency(fresh_value: str = Depends(get_value, use_cache=False)):
    return {"fresh_value": fresh_value}
```

Resumen

Aparte de todas las palabras rimbombantes usadas aquí, el sistema de **Inyección de Dependencias** es bastante simple.

Solo son funciones que se ven igual que las *path operation functions*.

Pero aun así, es muy potente y te permite declarar "grafos" de dependencia anidados arbitrariamente profundos (árboles).

> **Consejo**
> Todo esto podría no parecer tan útil con estos ejemplos simples.
>
> Pero verás lo útil que es en los capítulos sobre **seguridad**.
>
> Y también verás la cantidad de código que te ahorrará.

- # Dependencias en decoradores de path operation

En algunos casos realmente no necesitas el valor de retorno de una dependencia dentro de tu *path operation function*.

O la dependencia no devuelve un valor.

Pero aún necesitas que sea ejecutada/resuelta.

Para esos casos, en lugar de declarar un parámetro de *path operation function* con `Depends`, puedes añadir una `list` de `dependencies` al decorador de *path operation*.

Agregar dependencies al decorador de path operation

El decorador de *path operation* recibe un argumento opcional `dependencies`.

Debe ser una `list` de `Depends()`:

```
from typing import Annotated

from fastapi import Depends, FastAPI, Header,
HTTPException

app = FastAPI()

async def verify_token(x_token: Annotated[str,
Header()]):
    if x_token != "fake-super-secret-token":
        raise HTTPException(status_code=400,
detail="X-Token header invalid")

async def verify_key(x_key: Annotated[str, Header()]):
    if x_key != "fake-super-secret-key":
        raise HTTPException(status_code=400,
detail="X-Key header invalid")
    return x_key
```

```
@app.get("/items/", dependencies=
[Depends(verify_token), Depends(verify_key)])
async def read_items():
    return [{"item": "Foo"}, {"item": "Bar"}]
```

Estas dependencias serán ejecutadas/resueltas de la misma manera que las dependencias normales. Pero su valor (si devuelven alguno) no será pasado a tu *path operation function*.

> **Consejo**
> Algunos editores revisan los parámetros de función no usados y los muestran como errores.
>
> Usando estas `dependencies` en el decorador de *path operation* puedes asegurarte de que se ejecutan mientras evitas errores en editores/herramientas.
>
> También puede ayudar a evitar confusiones para nuevos desarrolladores que vean un parámetro no usado en tu código y puedan pensar que es innecesario.

> **Nota**
> En este ejemplo usamos headers personalizados inventados `X-Key` y `X-Token`.
>
> Pero en casos reales, al implementar seguridad, obtendrías más beneficios usando las Utilidades de Seguridad integradas (el próximo capítulo).

Errores de dependencias y valores de retorno

Puedes usar las mismas *funciones* de dependencia que usas normalmente.

Requisitos de dependencia

Pueden declarar requisitos de request (como headers) u otras sub-dependencias:

```
from typing import Annotated

from fastapi import Depends, FastAPI, Header,
HTTPException

app = FastAPI()

async def verify_token(x_token: Annotated[str,
Header()]):
    if x_token != "fake-super-secret-token":
        raise HTTPException(status_code=400,
detail="X-Token header invalid")

async def verify_key(x_key: Annotated[str, Header()]):
    if x_key != "fake-super-secret-key":
        raise HTTPException(status_code=400,
detail="X-Key header invalid")
    return x_key

@app.get("/items/", dependencies=
[Depends(verify_token), Depends(verify_key)])
async def read_items():
    return [{"item": "Foo"}, {"item": "Bar"}]
```

Lanzar excepciones

Estas dependencias pueden `raise` excepciones, igual que las dependencias normales:

```
from typing import Annotated

from fastapi import Depends, FastAPI, Header,
HTTPException

app = FastAPI()

async def verify_token(x_token: Annotated[str,
```

```
Header()]):
    if x_token != "fake-super-secret-token":
        raise HTTPException(status_code=400,
detail="X-Token header invalid")

async def verify_key(x_key: Annotated[str, Header()]):
    if x_key != "fake-super-secret-key":
        raise HTTPException(status_code=400,
detail="X-Key header invalid")
    return x_key

@app.get("/items/", dependencies=
[Depends(verify_token), Depends(verify_key)])
async def read_items():
    return [{"item": "Foo"}, {"item": "Bar"}]
```

Valores de retorno

Y pueden devolver valores o no, los valores no serán usados.

Así que, puedes reutilizar una dependencia normal (que devuelve un valor) que ya uses en otro lugar, y aunque el valor no se use, la dependencia será ejecutada:

```
from typing import Annotated

from fastapi import Depends, FastAPI, Header,
HTTPException

app = FastAPI()

async def verify_token(x_token: Annotated[str,
Header()]):
    if x_token != "fake-super-secret-token":
        raise HTTPException(status_code=400,
detail="X-Token header invalid")

async def verify_key(x_key: Annotated[str, Header()]):
    if x_key != "fake-super-secret-key":
        raise HTTPException(status_code=400,
```

```
detail="X-Key header invalid")
    return x_key

@app.get("/items/", dependencies=
[Depends(verify_token), Depends(verify_key)])
async def read_items():
    return [{"item": "Foo"}, {"item": "Bar"}]
```

Dependencias para un grupo de path operations

Más adelante, cuando leas sobre cómo estructurar aplicaciones más grandes (Aplicaciones Más Grandes - Múltiples Archivos), posiblemente con múltiples archivos, aprenderás cómo declarar un único parámetro `dependencies` para un grupo de *path operations*.

Dependencias Globales

A continuación veremos cómo añadir dependencias a toda la aplicación `FastAPI`, de modo que se apliquen a cada *path operation*.

Dependencias Globales

Para algunos tipos de aplicaciones, podrías querer agregar dependencias a toda la aplicación.

Similar a como puedes agregar `dependencies` a los *path operation decorators*, puedes agregarlos a la aplicación de `FastAPI`.

En ese caso, se aplicarán a todas las *path operations* en la aplicación:

```
from typing import Annotated

from fastapi import Depends, FastAPI, Header,
HTTPException
```

```python
async def verify_token(x_token: Annotated[str,
Header()]):
    if x_token != "fake-super-secret-token":
        raise HTTPException(status_code=400,
detail="X-Token header invalid")

async def verify_key(x_key: Annotated[str, Header()]):
    if x_key != "fake-super-secret-key":
        raise HTTPException(status_code=400,
detail="X-Key header invalid")
    return x_key

app = FastAPI(dependencies=[Depends(verify_token),
Depends(verify_key)])

@app.get("/items/")
async def read_items():
    return [{"item": "Portal Gun"}, {"item":
"Plumbus"}]

@app.get("/users/")
async def read_users():
    return [{"username": "Rick"}, {"username":
"Morty"}]
```

Y todas las ideas en la sección sobre agregar `dependencies` a los *path operation decorators* siguen aplicándose, pero en este caso, a todas las *path operations* en la app.

Dependencias para grupos de path operations

Más adelante, al leer sobre cómo estructurar aplicaciones más grandes (Aplicaciones Más Grandes - Múltiples Archivos), posiblemente con múltiples archivos, aprenderás cómo declarar un solo parámetro de `dependencies` para un grupo de *path operations*.

- ## Dependencias con yield

FastAPI admite dependencias que realizan algunos pasos adicionales después de finalizar.

Para hacer esto, usa `yield` en lugar de `return`, y escribe los pasos adicionales (código) después.

> **Consejo**
> Asegúrate de usar `yield` una sola vez por dependencia.

> **Detalles técnicos**
> Cualquier función que sea válida para usar con:
>
> - `@contextlib.contextmanager` o
> - `@contextlib.asynccontextmanager`
>
> sería válida para usar como una dependencia en **FastAPI**.
>
> De hecho, FastAPI usa esos dos decoradores internamente.

Una dependencia de base de datos con yield

Por ejemplo, podrías usar esto para crear una sesión de base de datos y cerrarla después de finalizar.

Solo el código anterior e incluyendo el `yield` statement se ejecuta antes de crear un response:

```
async def get_db():
    db = DBSession()
    try:
        yield db
    finally:
        db.close()
```

El valor generado es lo que se inyecta en *path operations* y otras dependencias:

```
async def get_db():
    db = DBSession()
    try:
        yield db
    finally:
        db.close()
```

El código posterior al `yield` statement se ejecuta después del response:

```
async def get_db():
    db = DBSession()
    try:
        yield db
    finally:
        db.close()
```

> **Consejo**
> Puedes usar funciones `async` o regulares.
>
> **FastAPI** hará lo correcto con cada una, igual que con dependencias normales.

Una dependencia con yield y try

Si usas un bloque `try` en una dependencia con `yield`, recibirás cualquier excepción que se haya lanzado al usar la dependencia.

Por ejemplo, si algún código en algún punto intermedio, en otra dependencia o en una *path operation*, realiza un "rollback" en una transacción de base de datos o crea cualquier otro error, recibirás la excepción en tu dependencia.

Por lo tanto, puedes buscar esa excepción específica dentro de la dependencia con `except SomeException`.

Del mismo modo, puedes usar `finally` para asegurarte de que los pasos de salida se ejecuten, sin importar si hubo una

excepción o no.

```
async def get_db():
    db = DBSession()
    try:
        yield db
    finally:
        db.close()
```

Sub-dependencias con yield

Puedes tener sub-dependencias y "árboles" de sub-dependencias de cualquier tamaño y forma, y cualquiera o todas ellas pueden usar `yield`.

FastAPI se asegurará de que el "código de salida" en cada dependencia con `yield` se ejecute en el orden correcto.

Por ejemplo, `dependency_c` puede tener una dependencia de `dependency_b`, y `dependency_b` de `dependency_a`:

```
from typing import Annotated

from fastapi import Depends

async def dependency_a():
    dep_a = generate_dep_a()
    try:
        yield dep_a
    finally:
        dep_a.close()

async def dependency_b(dep_a: Annotated[DepA,
Depends(dependency_a)]):
    dep_b = generate_dep_b()
    try:
        yield dep_b
    finally:
        dep_b.close(dep_a)
```

```
async def dependency_c(dep_b: Annotated[DepB,
Depends(dependency_b)]):
    dep_c = generate_dep_c()
    try:
        yield dep_c
    finally:
        dep_c.close(dep_b)
```

Y todas ellas pueden usar `yield`.

En este caso, `dependency_c`, para ejecutar su código de salida, necesita que el valor de `dependency_b` (aquí llamado `dep_b`) todavía esté disponible.

Y, a su vez, `dependency_b` necesita que el valor de `dependency_a` (aquí llamado `dep_a`) esté disponible para su código de salida.

```
from typing import Annotated

from fastapi import Depends

async def dependency_a():
    dep_a = generate_dep_a()
    try:
        yield dep_a
    finally:
        dep_a.close()

async def dependency_b(dep_a: Annotated[DepA,
Depends(dependency_a)]):
    dep_b = generate_dep_b()
    try:
        yield dep_b
    finally:
        dep_b.close(dep_a)

async def dependency_c(dep_b: Annotated[DepB,
Depends(dependency_b)]):
    dep_c = generate_dep_c()
    try:
```

```
        yield dep_c
    finally:
        dep_c.close(dep_b)
```

De la misma manera, podrías tener algunas dependencias con `yield` y otras dependencias con `return`, y hacer que algunas de esas dependan de algunas de las otras.

Y podrías tener una sola dependencia que requiera varias otras dependencias con `yield`, etc.

Puedes tener cualquier combinación de dependencias que quieras.

FastAPI se asegurará de que todo se ejecute en el orden correcto.

> **Detalles técnicos**
> Esto funciona gracias a los Context Managers de Python.
>
> **FastAPI** los utiliza internamente para lograr esto.

Dependencias con yield y HTTPException

Viste que puedes usar dependencias con `yield` y tener bloques `try` que intentan ejecutar algo de código y luego ejecutar código de salida después de `finally`.

También puedes usar `except` para capturar la excepción que se lanzó y hacer algo con ella.

Por ejemplo, puedes lanzar una excepción diferente, como `HTTPException`.

> **Consejo**
> Esta es una técnica algo avanzada, y en la mayoría de los casos realmente no la necesitarás, ya que puedes lanzar excepciones (incluyendo `HTTPException`) desde

dentro del resto del código de tu aplicación, por ejemplo, en la *path operation function*.

Pero está ahí para ti si la necesitas.

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException

app = FastAPI()

data = {
    "plumbus": {"description": "Freshly pickled
plumbus", "owner": "Morty"},
    "portal-gun": {"description": "Gun to create
portals", "owner": "Rick"},
}

class OwnerError(Exception):
    pass

def get_username():
    try:
        yield "Rick"
    except OwnerError as e:
        raise HTTPException(status_code=400,
detail=f"Owner error: {e}")

@app.get("/items/{item_id}")
def get_item(item_id: str, username: Annotated[str,
Depends(get_username)]):
    if item_id not in data:
        raise HTTPException(status_code=404,
detail="Item not found")
    item = data[item_id]
    if item["owner"] != username:
        raise OwnerError(username)
    return item
```

Si quieres capturar excepciones y crear un response personalizado en base a eso, crea un Manejador de Excepciones Personalizado.

Dependencias con yield y except

Si capturas una excepción usando `except` en una dependencia con `yield` y no la lanzas nuevamente (o lanzas una nueva excepción), FastAPI no podrá notar que hubo una excepción, al igual que sucedería con Python normal:

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException

app = FastAPI()

class InternalError(Exception):
    pass

def get_username():
    try:
        yield "Rick"
    except InternalError:
        print("Oops, we didn't raise again, Britney
😱")

@app.get("/items/{item_id}")
def get_item(item_id: str, username: Annotated[str,
Depends(get_username)]):
    if item_id == "portal-gun":
        raise InternalError(
            f"The portal gun is too dangerous to be
owned by {username}"
        )
    if item_id != "plumbus":
        raise HTTPException(
            status_code=404, detail="Item not found,
there's only a plumbus here"
```

```
        )
    return item_id
```

En este caso, el cliente verá un response *HTTP 500 Internal Server Error* como debería, dado que no estamos lanzando una `HTTPException` o similar, pero el servidor **no tendrá ningún registro** ni ninguna otra indicación de cuál fue el error.

Siempre raise en Dependencias con yield y except

Si capturas una excepción en una dependencia con `yield`, a menos que estés lanzando otra `HTTPException` o similar, **deberías volver a lanzar la excepción original**.

Puedes volver a lanzar la misma excepción usando `raise`:

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException

app = FastAPI()

class InternalError(Exception):
    pass

def get_username():
    try:
        yield "Rick"
    except InternalError:
        print("We don't swallow the internal error
here, we raise again 😎")
        raise

@app.get("/items/{item_id}")
def get_item(item_id: str, username: Annotated[str,
Depends(get_username)]):
    if item_id == "portal-gun":
        raise InternalError(
```

```
            f"The portal gun is too dangerous to be
owned by {username}"
        )
    if item_id != "plumbus":
        raise HTTPException(
            status_code=404, detail="Item not found,
there's only a plumbus here"
        )
    return item_id
```

Ahora el cliente obtendrá el mismo response *HTTP 500 Internal Server Error*, pero el servidor tendrá nuestro `InternalError` personalizado en los registros.

Ejecución de dependencias con yield

La secuencia de ejecución es más o menos como este diagrama. El tiempo fluye de arriba a abajo. Y cada columna es una de las partes que interactúa o ejecuta código.

> **Nota**
> Solo **un response** será enviado al cliente. Podría ser uno de los responses de error o será el response de la *path operation*.
>
> Después de que se envíe uno de esos responses, no se podrá enviar ningún otro response.

> **Consejo**
> Si lanzas cualquier excepción en el código de la *path operation function*, se pasará a las dependencias con `yield`, incluyendo `HTTPException`. En la mayoría de los casos querrás volver a lanzar esa misma excepción o una nueva desde la dependencia con `yield` para asegurarte de que se maneje correctamente.

Salida temprana y scope

Normalmente, el código de salida de las dependencias con `yield` se ejecuta **después de que el response** se envía al cliente.

Pero si sabes que no necesitarás usar la dependencia después de regresar de la *path operation function*, puedes usar `Depends(scope="function")` para decirle a FastAPI que debe cerrar la dependencia después de que la *path operation function* regrese, pero **antes** de que se envíe el **response**.

```
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()

def get_username():
    try:
        yield "Rick"
    finally:
        print("Cleanup up before response is sent")

@app.get("/users/me")
def get_user_me(username: Annotated[str,
Depends(get_username, scope="function")]):
    return username
```

`Depends()` recibe un parámetro `scope` que puede ser:

- `"function"` : iniciar la dependencia antes de la *path operation function* que maneja el request, terminar la dependencia después de que termine la *path operation function*, pero **antes** de que el response se envíe de vuelta al cliente. Entonces, la función de dependencia se ejecutará **alrededor** de la *path operation **function***.
- `"request"` : iniciar la dependencia antes de la *path operation function* que maneja el request (similar a cuando se usa `"function"`), pero terminar **después** de que el response se envíe de vuelta al cliente. Entonces,

la función de dependencia se ejecutará **alrededor** del **request** y del ciclo del response.

Si no se especifica y la dependencia tiene `yield`, tendrá un `scope` de `"request"` por defecto.

scope para sub-dependencias

Cuando declaras una dependencia con `scope="request"` (el valor por defecto), cualquier sub-dependencia también necesita tener un `scope` de `"request"`.

Pero una dependencia con `scope` de `"function"` puede tener dependencias con `scope` de `"function"` y `scope` de `"request"`.

Esto es porque cualquier dependencia necesita poder ejecutar su código de salida antes que las sub-dependencias, ya que podría necesitar seguir usándolas durante su código de salida.

Dependencias con yield, HTTPException, except y Tareas en Background

Las dependencias con `yield` han evolucionado con el tiempo para cubrir diferentes casos de uso y corregir algunos problemas.

Si quieres ver qué ha cambiado en diferentes versiones de FastAPI, puedes leer más al respecto en la guía avanzada, en Dependencias avanzadas - Dependencias con `yield`, `HTTPException`, `except` y Tareas en Background.

Context Managers
Qué son los "Context Managers"

Los "Context Managers" son aquellos objetos de Python que puedes usar en un `with` statement.

Por ejemplo, puedes usar `with` para leer un archivo:

```
with open("./somefile.txt") as f:
    contents = f.read()
    print(contents)
```

Internamente, `open("./somefile.txt")` crea un objeto llamado "Context Manager".

Cuando el bloque `with` termina, se asegura de cerrar el archivo, incluso si hubo excepciones.

Cuando creas una dependencia con `yield`, **FastAPI** creará internamente un context manager para ella y lo combinará con algunas otras herramientas relacionadas.

Usando context managers en dependencias con yield

> ⚠ **Advertencia**
> Esto es, más o menos, una idea "avanzada".
>
> Si apenas estás comenzando con **FastAPI**, podrías querer omitirlo por ahora.

En Python, puedes crear Context Managers creando una clase con dos métodos: `__enter__()` y `__exit__()`.

También puedes usarlos dentro de las dependencias de **FastAPI** con `yield` usando `with` o `async with` en la función de dependencia:

```
class MySuperContextManager:
    def __init__(self):
        self.db = DBSession()
```

```
    def __enter__(self):
        return self.db

    def __exit__(self, exc_type, exc_value,
traceback):
        self.db.close()

async def get_db():
    with MySuperContextManager() as db:
        yield db
```

> **Consejo**
> Otra manera de crear un context manager es con:
>
> - `@contextlib.contextmanager` o
> - `@contextlib.asynccontextmanager`
>
> usándolos para decorar una función con un solo `yield`.
>
> Eso es lo que **FastAPI** usa internamente para dependencias con `yield`.
>
> Pero no tienes que usar los decoradores para las dependencias de FastAPI (y no deberías).
>
> FastAPI lo hará por ti internamente.

- # Seguridad

Hay muchas formas de manejar la seguridad, autenticación y autorización.

Y normalmente es un tema complejo y "difícil".

En muchos frameworks y sistemas, solo manejar la seguridad y autenticación requiere una gran cantidad de esfuerzo y código (en muchos casos puede ser el 50% o más de todo el código escrito).

FastAPI proporciona varias herramientas para ayudarte a manejar la **Seguridad** de manera fácil, rápida y estándar, sin tener que estudiar y aprender todas las especificaciones de seguridad.

Pero primero, vamos a revisar algunos pequeños conceptos.

¿Con prisa?

Si no te importan ninguno de estos términos y solo necesitas agregar seguridad con autenticación basada en nombre de usuario y contraseña *ahora mismo*, salta a los siguientes capítulos.

OAuth2

OAuth2 es una especificación que define varias maneras de manejar la autenticación y autorización.

Es una especificación bastante extensa y cubre varios casos de uso complejos.

Incluye formas de autenticarse usando un "tercero".

Eso es lo que todos los sistemas con "iniciar sesión con Facebook, Google, X (Twitter), GitHub" utilizan internamente.

OAuth 1

Hubo un OAuth 1, que es muy diferente de OAuth2, y más complejo, ya que incluía especificaciones directas sobre cómo encriptar la comunicación.

No es muy popular o usado hoy en día.

OAuth2 no especifica cómo encriptar la comunicación, espera que tengas tu aplicación servida con HTTPS.

> **Consejo**
> En la sección sobre **deployment** verás cómo configurar HTTPS de forma gratuita, usando Traefik y Let's Encrypt.

OpenID Connect

OpenID Connect es otra especificación, basada en **OAuth2**.

Solo extiende OAuth2 especificando algunas cosas que son relativamente ambiguas en OAuth2, para intentar hacerla más interoperable.

Por ejemplo, el login de Google usa OpenID Connect (que internamente usa OAuth2).

Pero el login de Facebook no soporta OpenID Connect. Tiene su propia versión de OAuth2.

OpenID (no "OpenID Connect")

Hubo también una especificación "OpenID". Que intentaba resolver lo mismo que **OpenID Connect**, pero no estaba basada en OAuth2.

Entonces, era un sistema completo adicional.

No es muy popular o usado hoy en día.

OpenAPI

OpenAPI (anteriormente conocido como Swagger) es la especificación abierta para construir APIs (ahora parte de la Linux Foundation).

FastAPI se basa en **OpenAPI**.

Eso es lo que hace posible tener múltiples interfaces de documentación interactiva automática, generación de código, etc.

OpenAPI tiene una forma de definir múltiples "esquemas" de seguridad.

Al usarlos, puedes aprovechar todas estas herramientas basadas en estándares, incluidos estos sistemas de documentación interactiva.

OpenAPI define los siguientes esquemas de seguridad:

- `apiKey`: una clave específica de la aplicación que puede provenir de:
- Un parámetro de query.
- Un header.
- Una cookie.
- `http`: sistemas de autenticación HTTP estándar, incluyendo:
- `bearer`: un header `Authorization` con un valor de `Bearer` más un token. Esto se hereda de OAuth2.
- Autenticación básica HTTP.
- Digest HTTP, etc.
- `oauth2`: todas las formas de OAuth2 para manejar la seguridad (llamadas "flujos").
- Varios de estos flujos son apropiados para construir un proveedor de autenticación OAuth 2.0 (como Google, Facebook, X (Twitter), GitHub, etc.):
    - `implicit`
        - `clientCredentials`
        - `authorizationCode`
- Pero hay un "flujo" específico que puede usarse perfectamente para manejar la autenticación directamente en la misma aplicación:
    - `password`: algunos de los próximos capítulos cubrirán ejemplos de esto.
- `openIdConnect`: tiene una forma de definir cómo descubrir automáticamente los datos de autenticación OAuth2.
- Este descubrimiento automático es lo que se define en la especificación de OpenID Connect.

Consejo

Integrar otros proveedores de autenticación/autorización como Google, Facebook, X (Twitter), GitHub, etc. también es posible y relativamente fácil.

El problema más complejo es construir un proveedor de autenticación/autorización como esos, pero **FastAPI** te da las

herramientas para hacerlo fácilmente, mientras hace el trabajo pesado por ti.

Utilidades de FastAPI

FastAPI proporciona varias herramientas para cada uno de estos esquemas de seguridad en el módulo `fastapi.security` que simplifican el uso de estos mecanismos de seguridad.

En los siguientes capítulos verás cómo agregar seguridad a tu API usando esas herramientas proporcionadas por **FastAPI**.

Y también verás cómo se integra automáticamente en el sistema de documentación interactiva.

- # Seguridad - Primeros pasos

  Imaginemos que tienes tu API de **backend** en algún dominio.

  Y tienes un **frontend** en otro dominio o en un path diferente del mismo dominio (o en una aplicación móvil).

  Y quieres tener una forma para que el frontend se autentique con el backend, usando un **username** y **password**.

  Podemos usar **OAuth2** para construir eso con **FastAPI**.

  Pero vamos a ahorrarte el tiempo de leer la larga especificación completa solo para encontrar esos pequeños fragmentos de información que necesitas.

  Usemos las herramientas proporcionadas por **FastAPI** para manejar la seguridad.

  ## Cómo se ve

  Primero solo usemos el código y veamos cómo funciona, y luego volveremos para entender qué está sucediendo.

  ## Crea main.py

  Copia el ejemplo en un archivo `main.py`:

```python
from typing import Annotated

from fastapi import Depends, FastAPI
from fastapi.security import OAuth2PasswordBearer

app = FastAPI()

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

@app.get("/items/")
async def read_items(token: Annotated[str,
Depends(oauth2_scheme)]):
    return {"token": token}
```

Ejecútalo

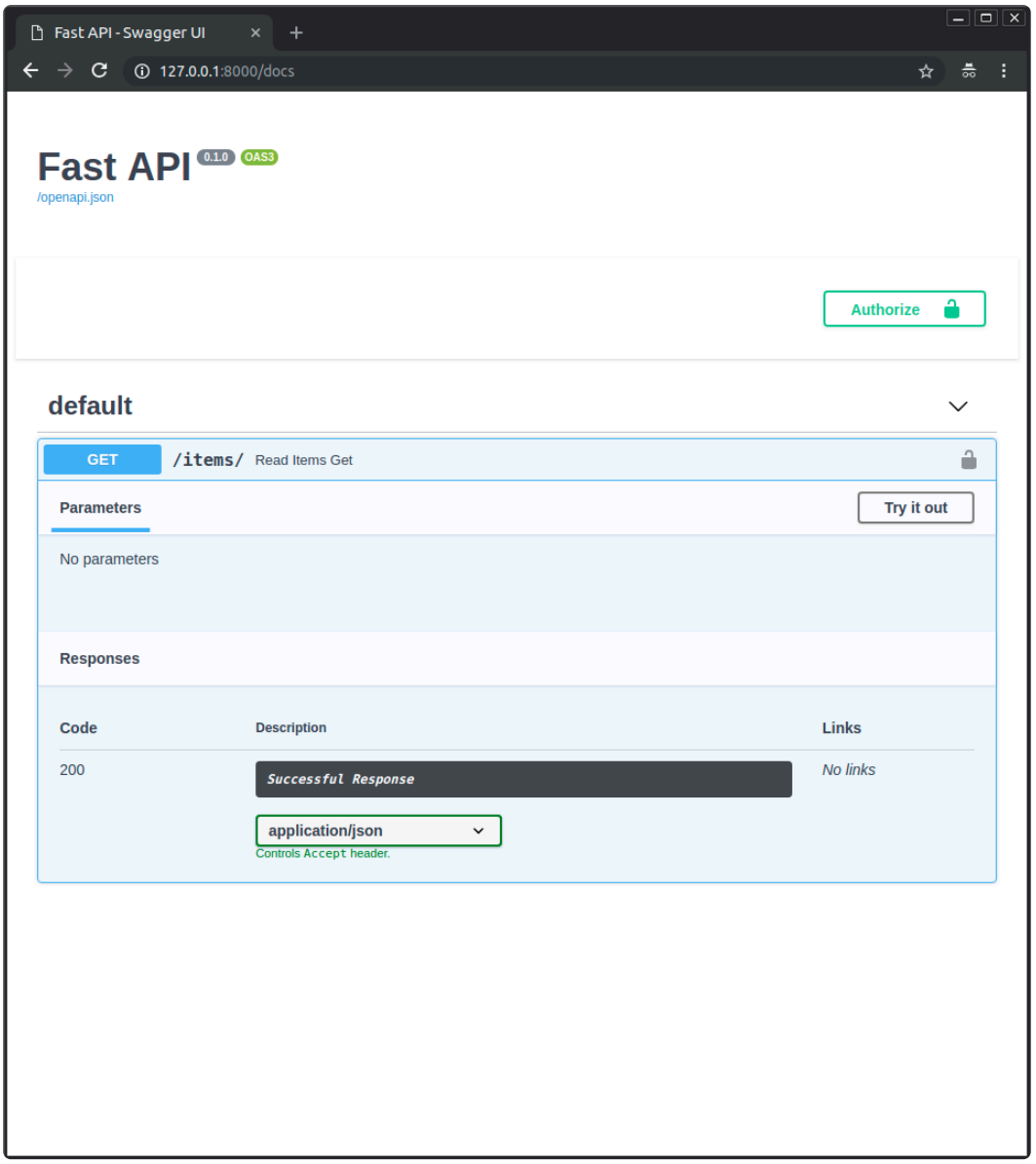
> **Nota**
> El paquete `python-multipart` se instala automáticamente con **FastAPI** cuando ejecutas el comando `uv add "fastapi[standard]"`.
>
> Sin embargo, si usas el comando `uv add fastapi`, el paquete `python-multipart` no se incluye por defecto.
>
> Para instalarlo manualmente, agrégalo a tu proyecto con:
>
> ```
> $ uv add python-multipart
> ```
>
> Esto se debe a que **OAuth2** utiliza "form data" para enviar el `username` y `password`.

Ejecuta el ejemplo con:

Revisa

Ve a la documentación interactiva en: http://127.0.0.1:8000/docs.

Verás algo así:

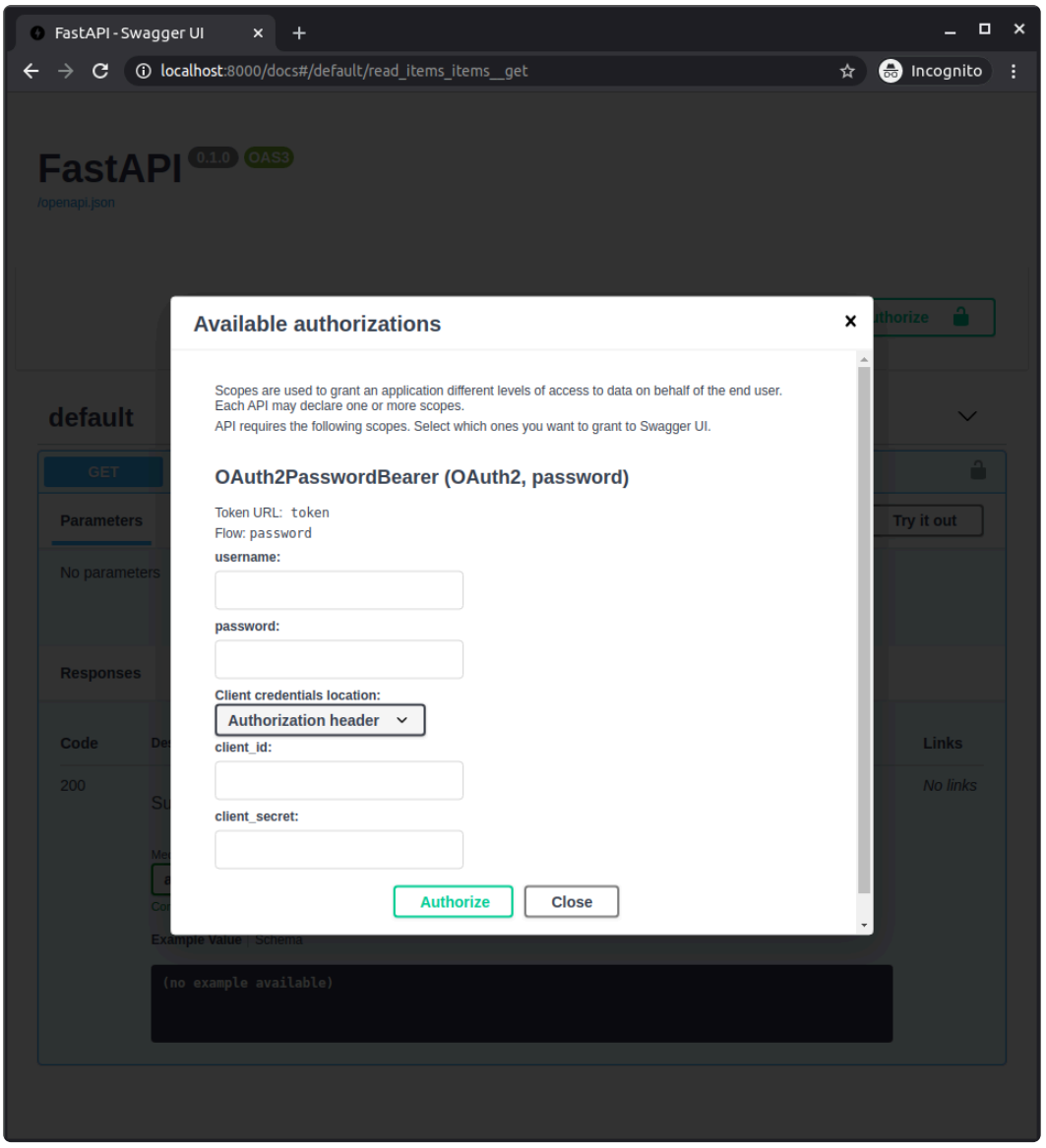


¡Botón de autorización!

Ya tienes un nuevo y brillante botón de "Authorize".

Y tu *path operation* tiene un pequeño candado en la esquina superior derecha que puedes pulsar.

Y si lo haces, tendrás un pequeño formulario de autorización para escribir un `username` y `password` (y otros campos opcionales):



/// note | Nota

No importa lo que escribas en el formulario, aún no funcionará. Pero llegaremos allí.

///

Esto por supuesto no es el frontend para los usuarios finales, pero es una gran herramienta automática para documentar interactivamente toda tu API.

Puede ser utilizada por el equipo de frontend (que también puedes ser tú mismo).

Puede ser utilizada por aplicaciones y sistemas de terceros.

Y también puede ser utilizada por ti mismo, para depurar, revisar y probar la misma aplicación.

El flujo password

Ahora retrocedamos un poco y entendamos qué es todo eso.

El "flujo" `password` es una de las formas ("flujos") definidas en OAuth2, para manejar la seguridad y la autenticación.

OAuth2 fue diseñado para que el backend o la API pudieran ser independientes del servidor que autentica al usuario.

Pero en este caso, la misma aplicación de **FastAPI** manejará la API y la autenticación.

Así que, revisémoslo desde ese punto de vista simplificado:

- El usuario escribe el `username` y `password` en el frontend, y presiona `Enter`.
- El frontend (ejecutándose en el navegador del usuario) envía ese `username` y `password` a una URL específica en nuestra API (declarada con `tokenUrl="token"`).
- La API verifica ese `username` y `password`, y responde con un "token" (no hemos implementado nada de esto aún).
    - Un "token" es solo un string con algún contenido que podemos usar luego para verificar a este usuario.
        - Normalmente, un token se establece para que expire después de algún tiempo.
        - Así que, el usuario tendrá que volver a iniciar sesión más adelante.
    - Y si el token es robado, el riesgo es menor. No es como una llave permanente que funcionará para siempre (en la mayoría de los casos).
- El frontend almacena temporalmente ese token en algún lugar.
- El usuario hace clic en el frontend para ir a otra sección de la aplicación web frontend.
- El frontend necesita obtener más datos de la API.

- Pero necesita autenticación para ese endpoint específico.
    - Así que, para autenticarse con nuestra API, envía un `header` `Authorization` con un valor de `Bearer` más el token.
    - Si el token contiene `foobar`, el contenido del `header` `Authorization` sería: `Bearer foobar`.

OAuth2PasswordBearer de FastAPI

FastAPI proporciona varias herramientas, en diferentes niveles de abstracción, para implementar estas funcionalidades de seguridad.

En este ejemplo vamos a usar **OAuth2**, con el flujo **Password**, usando un token **Bearer**. Hacemos eso utilizando la clase `OAuth2PasswordBearer`.

> **Nota**
> Un token "bearer" no es la única opción.
>
> Pero es la mejor para nuestro caso de uso.
>
> Y podría ser la mejor para la mayoría de los casos de uso, a menos que seas un experto en OAuth2 y sepas exactamente por qué hay otra opción que se adapta mejor a tus necesidades.
>
> En ese caso, **FastAPI** también te proporciona las herramientas para construirlo.

Cuando creamos una instance de la clase `OAuth2PasswordBearer` pasamos el parámetro `tokenUrl`. Este parámetro contiene la URL que el cliente (el frontend corriendo en el navegador del usuario) usará para enviar el `username` y `password` a fin de obtener un token.

```python
from typing import Annotated

from fastapi import Depends, FastAPI
from fastapi.security import OAuth2PasswordBearer

app = FastAPI()

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

@app.get("/items/")
async def read_items(token: Annotated[str,
Depends(oauth2_scheme)]):
    return {"token": token}
```

Consejo

Aquí `tokenUrl="token"` se refiere a una URL relativa `token` que aún no hemos creado. Como es una URL relativa, es equivalente a `./token`.

Porque estamos usando una URL relativa, si tu API estuviera ubicada en `https://example.com/`, entonces se referiría a `https://example.com/token`. Pero si tu API estuviera ubicada en `https://example.com/api/v1/`, entonces se referiría a `https://example.com/api/v1/token`.

Usar una URL relativa es importante para asegurarse de que tu aplicación siga funcionando incluso en un caso de uso avanzado como Detrás de un Proxy.

Este parámetro no crea ese endpoint / *path operation*, pero declara que la URL `/token` será la que el cliente deberá usar para obtener el token. Esa información se usa en OpenAPI, y luego en los sistemas de documentación interactiva del API.

Pronto también crearemos la path operation real.

Nota

Si eres un "Pythonista" muy estricto, tal vez no te guste el estilo del nombre del parámetro `tokenUrl` en lugar de `token_url`.

Eso es porque está usando el mismo nombre que en la especificación de OpenAPI. Para que si necesitas investigar más sobre cualquiera de estos esquemas de seguridad, puedas simplemente copiarlo y pegarlo para encontrar más información al respecto.

La variable `oauth2_scheme` es una instancia de `OAuth2PasswordBearer`, pero también es un "callable".

Podría ser llamada como:

```
oauth2_scheme(some, parameters)
```

Así que, puede usarse con `Depends`.

Úsalo

Ahora puedes pasar ese `oauth2_scheme` en una dependencia con `Depends`.

```
from typing import Annotated

from fastapi import Depends, FastAPI
from fastapi.security import OAuth2PasswordBearer

app = FastAPI()

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

@app.get("/items/")
async def read_items(token: Annotated[str,
Depends(oauth2_scheme)]):
    return {"token": token}
```

Esta dependencia proporcionará un `str` que se asigna al parámetro `token` de la *path operation function*.

FastAPI sabrá que puede usar esta dependencia para definir un "security scheme" en el esquema OpenAPI (y en la documentación automática de la API).

> **Detalles técnicos**
> **FastAPI** sabrá que puede usar la clase `OAuth2PasswordBearer` (declarada en una dependencia) para definir el esquema de seguridad en OpenAPI porque hereda de `fastapi.security.oauth2.OAuth2`, que a su vez hereda de `fastapi.security.base.SecurityBase`.
>
> Todas las utilidades de seguridad que se integran con OpenAPI (y la documentación automática de la API) heredan de `SecurityBase`, así es como **FastAPI** puede saber cómo integrarlas en OpenAPI.

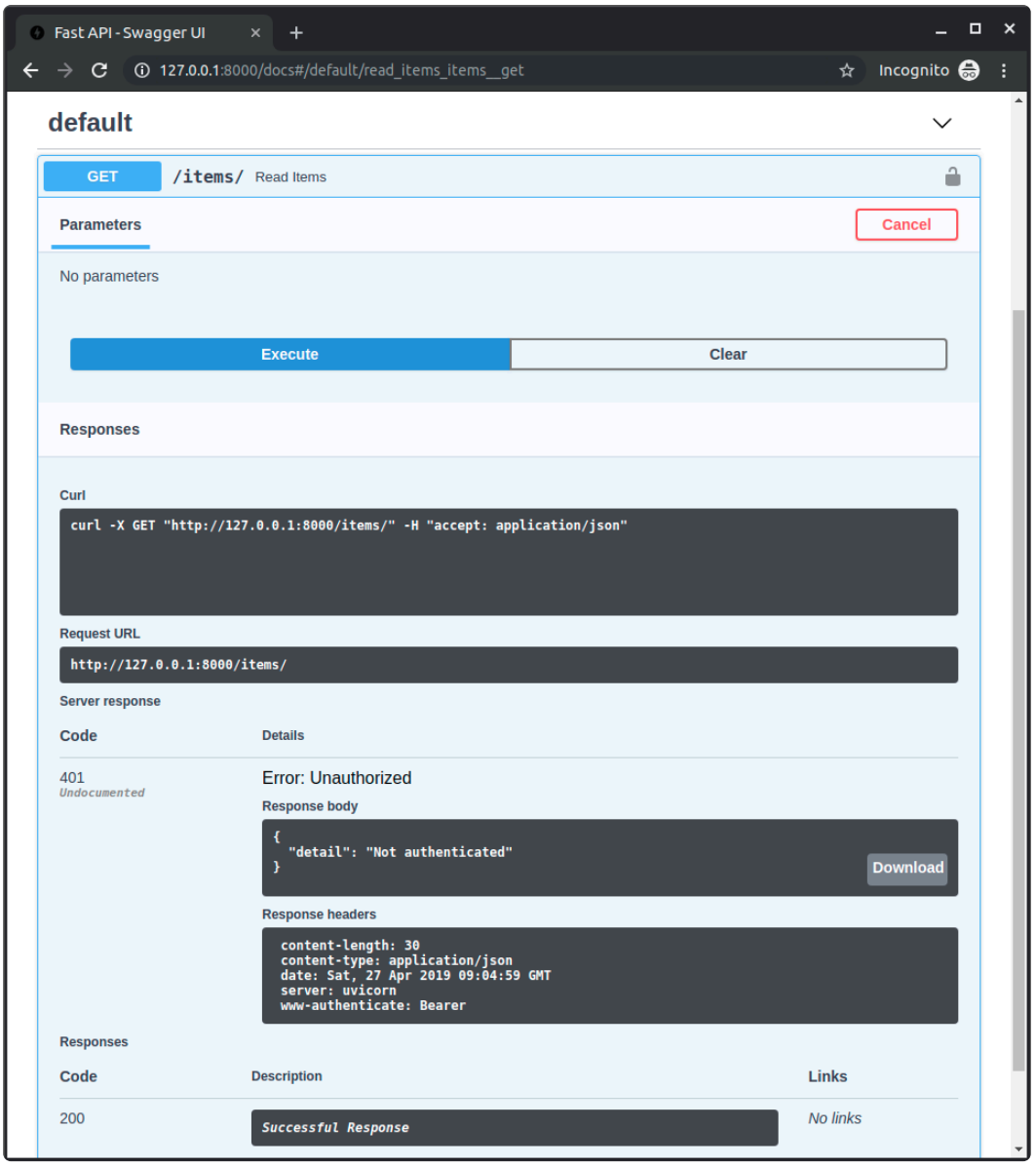
Lo que hace

Irá y buscará en el request ese header `Authorization`, verificará si el valor es `Bearer` más algún token, y devolverá el token como un `str`.

Si no ve un header `Authorization`, o el valor no tiene un token `Bearer`, responderá directamente con un error de código de estado 401 (`UNAUTHORIZED`).

Ni siquiera tienes que verificar si el token existe para devolver un error. Puedes estar seguro de que si tu función se ejecuta, tendrá un `str` en ese token.

Puedes probarlo ya en los docs interactivos:



Todavía no estamos verificando la validez del token, pero ya es un comienzo.

Resumen

Así que, en solo 3 o 4 líneas adicionales, ya tienes alguna forma primitiva de seguridad.

Obtener Usuario Actual

En el capítulo anterior, el sistema de seguridad (que se basa en el sistema de inyección de dependencias) le estaba dando a la *path operation function* un `token` como un `str`:

```
from typing import Annotated

from fastapi import Depends, FastAPI
```

```python
from fastapi.security import OAuth2PasswordBearer

app = FastAPI()

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

@app.get("/items/")
async def read_items(token: Annotated[str,
Depends(oauth2_scheme)]):
    return {"token": token}
```

Pero eso aún no es tan útil.

Vamos a hacer que nos dé el usuario actual.

Crear un modelo de usuario

Primero, vamos a crear un modelo de usuario con Pydantic.

De la misma manera que usamos Pydantic para declarar cuerpos, podemos usarlo en cualquier otra parte:

```python
from typing import Annotated

from fastapi import Depends, FastAPI
from fastapi.security import OAuth2PasswordBearer
from pydantic import BaseModel

app = FastAPI()

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

class User(BaseModel):
    username: str
    email: str | None = None
    full_name: str | None = None
    disabled: bool | None = None

def fake_decode_token(token):
    return User(
        username=token + "fakedecoded",
```

```
    email="john@example.com", full_name="John Doe"
    )

async def get_current_user(token: Annotated[str,
Depends(oauth2_scheme)]):
    user = fake_decode_token(token)
    return user

@app.get("/users/me")
async def read_users_me(current_user: Annotated[User,
Depends(get_current_user)]):
    return current_user
```

Crear una dependencia get_current_user

Vamos a crear una dependencia `get_current_user`.

¿Recuerdas que las dependencias pueden tener sub-dependencias?

`get_current_user` tendrá una dependencia con el mismo `oauth2_scheme` que creamos antes.

De la misma manera que estábamos haciendo antes en la *path operation* directamente, nuestra nueva dependencia `get_current_user` recibirá un `token` como un `str` de la sub-dependencia `oauth2_scheme`:

```
from typing import Annotated

from fastapi import Depends, FastAPI
from fastapi.security import OAuth2PasswordBearer
from pydantic import BaseModel

app = FastAPI()

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

class User(BaseModel):
```

```
    username: str
    email: str | None = None
    full_name: str | None = None
    disabled: bool | None = None

def fake_decode_token(token):
    return User(
        username=token + "fakedecoded",
email="john@example.com", full_name="John Doe"
    )

async def get_current_user(token: Annotated[str,
Depends(oauth2_scheme)]):
    user = fake_decode_token(token)
    return user

@app.get("/users/me")
async def read_users_me(current_user: Annotated[User,
Depends(get_current_user)]):
    return current_user
```

Obtener el usuario

`get_current_user` usará una función de utilidad (falsa) que creamos, que toma un token como un `str` y devuelve nuestro modelo de Pydantic `User`:

```
from typing import Annotated

from fastapi import Depends, FastAPI
from fastapi.security import OAuth2PasswordBearer
from pydantic import BaseModel

app = FastAPI()

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

class User(BaseModel):
    username: str
    email: str | None = None
```

```
    full_name: str | None = None
    disabled: bool | None = None

def fake_decode_token(token):
    return User(
        username=token + "fakedecoded",
email="john@example.com", full_name="John Doe"
    )

async def get_current_user(token: Annotated[str,
Depends(oauth2_scheme)]):
    user = fake_decode_token(token)
    return user

@app.get("/users/me")
async def read_users_me(current_user: Annotated[User,
Depends(get_current_user)]):
    return current_user
```

Inyectar al usuario actual

Entonces ahora podemos usar el mismo `Depends` con nuestro `get_current_user` en la *path operation*:

```
from typing import Annotated

from fastapi import Depends, FastAPI
from fastapi.security import OAuth2PasswordBearer
from pydantic import BaseModel

app = FastAPI()

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

class User(BaseModel):
    username: str
    email: str | None = None
    full_name: str | None = None
    disabled: bool | None = None
```

```python
def fake_decode_token(token):
    return User(
        username=token + "fakedecoded",
email="john@example.com", full_name="John Doe"
    )

async def get_current_user(token: Annotated[str,
Depends(oauth2_scheme)]):
    user = fake_decode_token(token)
    return user

@app.get("/users/me")
async def read_users_me(current_user: Annotated[User,
Depends(get_current_user)]):
    return current_user
```

Ten en cuenta que declaramos el tipo de `current_user` como el modelo de Pydantic `User`.

Esto nos ayudará dentro de la función con todo el autocompletado y chequeo de tipos.

> **Consejo**
> Tal vez recuerdes que los request bodies también se declaran con modelos de Pydantic.
>
> Aquí **FastAPI** no se confundirá porque estás usando `Depends`.

> **Consejo**
> El modo en que este sistema de dependencias está diseñado nos permite tener diferentes dependencias (diferentes "dependables") que todas devuelven un modelo `User`.
>
> No estamos restringidos a tener solo una dependencia que pueda devolver ese tipo de datos.

Otros modelos

Ahora puedes obtener el usuario actual directamente en las *path operation functions* y manejar los mecanismos de seguridad a nivel de **Dependency Injection**, usando `Depends`.

Y puedes usar cualquier modelo o datos para los requisitos de seguridad (en este caso, un modelo de Pydantic `User`).

Pero no estás limitado a usar algún modelo de datos, clase o tipo específico.

¿Quieres tener un `id` y `email` y no tener ningún `username` en tu modelo? Claro. Puedes usar estas mismas herramientas.

¿Quieres solo tener un `str`? ¿O solo un `dict`? ¿O un instance de clase modelo de base de datos directamente? Todo funciona de la misma manera.

¿En realidad no tienes usuarios que inicien sesión en tu aplicación sino robots, bots u otros sistemas, que solo tienen un token de acceso? Una vez más, todo funciona igual.

Usa cualquier tipo de modelo, cualquier tipo de clase, cualquier tipo de base de datos que necesites para tu aplicación. **FastAPI** te cubre con el sistema de inyección de dependencias.

Tamaño del código

Este ejemplo podría parecer extenso. Ten en cuenta que estamos mezclando seguridad, modelos de datos, funciones de utilidad y *path operations* en el mismo archivo.

Pero aquí está el punto clave.

El tema de seguridad e inyección de dependencias se escribe una vez.

Y puedes hacerlo tan complejo como desees. Y aún así, tenerlo escrito solo una vez, en un solo lugar. Con toda la

flexibilidad.

Pero puedes tener miles de endpoints (*path operations*) usando el mismo sistema de seguridad.

Y todos ellos (o cualquier porción de ellos que quieras) pueden aprovechar la reutilización de estas dependencias o cualquier otra dependencia que crees.

Y todas estas miles de *path operations* pueden ser tan pequeñas como 3 líneas:

```
from typing import Annotated

from fastapi import Depends, FastAPI
from fastapi.security import OAuth2PasswordBearer
from pydantic import BaseModel

app = FastAPI()

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

class User(BaseModel):
    username: str
    email: str | None = None
    full_name: str | None = None
    disabled: bool | None = None

def fake_decode_token(token):
    return User(
        username=token + "fakedecoded",
email="john@example.com", full_name="John Doe"
    )

async def get_current_user(token: Annotated[str,
Depends(oauth2_scheme)]):
    user = fake_decode_token(token)
    return user

@app.get("/users/me")
async def read_users_me(current_user: Annotated[User,
```

```
Depends(get_current_user)]):
    return current_user
```

Resumen

Ahora puedes obtener el usuario actual directamente en tu *path operation function*.

Ya estamos a mitad de camino.

Solo necesitamos agregar una *path operation* para que el usuario/cliente envíe realmente el `username` y `password`.

Eso es lo que viene a continuación.

Simple OAuth2 con Password y Bearer

Ahora vamos a construir a partir del capítulo anterior y agregar las partes faltantes para tener un flujo de seguridad completo.

Obtener el username y password

Vamos a usar las utilidades de seguridad de **FastAPI** para obtener el `username` y `password`.

OAuth2 especifica que cuando se utiliza el "password flow" (que estamos usando), el cliente/usuario debe enviar campos `username` y `password` como form data.

Y la especificación dice que los campos deben llamarse así. Por lo que `user-name` o `email` no funcionarían.

Pero no te preocupes, puedes mostrarlo como quieras a tus usuarios finales en el frontend.

Y tus modelos de base de datos pueden usar cualquier otro nombre que desees.

Pero para la *path operation* de inicio de sesión, necesitamos usar estos nombres para ser compatibles con la especificación (y poder, por ejemplo, utilizar el sistema de documentación integrada de la API).

La especificación también establece que el `username` y `password` deben enviarse como form data (por lo que no hay JSON aquí).

scope

La especificación también indica que el cliente puede enviar otro campo del formulario llamado " `scope` ".

El nombre del campo del formulario es `scope` (en singular), pero en realidad es un string largo con "scopes" separados por espacios.

Cada "scope" es simplemente un string (sin espacios).

Normalmente se utilizan para declarar permisos de seguridad específicos, por ejemplo:

- `users:read` o `users:write` son ejemplos comunes.
- `instagram_basic` es usado por Facebook / Instagram.
- `https://www.googleapis.com/auth/drive` es usado por Google.

> **Nota**
> En OAuth2 un "scope" es solo un string que declara un permiso específico requerido.
>
> No importa si tiene otros caracteres como `:` o si es una URL.
>
> Esos detalles son específicos de la implementación.
>
> Para OAuth2 son solo strings.

Código para obtener el username y password

Ahora vamos a usar las utilidades proporcionadas por **FastAPI** para manejar esto.

OAuth2PasswordRequestForm

Primero, importa `OAuth2PasswordRequestForm`, y úsalo como una dependencia con `Depends` en la *path operation* para `/token`:

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException,
status
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from pydantic import BaseModel

fake_users_db = {
    "johndoe": {
        "username": "johndoe",
        "full_name": "John Doe",
        "email": "johndoe@example.com",
        "hashed_password": "fakehashedsecret",
        "disabled": False,
    },
    "alice": {
        "username": "alice",
        "full_name": "Alice Wonderson",
        "email": "alice@example.com",
        "hashed_password": "fakehashedsecret2",
        "disabled": True,
    },
}

app = FastAPI()

def fake_hash_password(password: str):
```

```
    return "fakehashed" + password

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

class User(BaseModel):
    username: str
    email: str | None = None
    full_name: str | None = None
    disabled: bool | None = None

class UserInDB(User):
    hashed_password: str

def get_user(db, username: str):
    if username in db:
        user_dict = db[username]
        return UserInDB(**user_dict)

def fake_decode_token(token):
    # This doesn't provide any security at all
    # Check the next version
    user = get_user(fake_users_db, token)
    return user

async def get_current_user(token: Annotated[str,
Depends(oauth2_scheme)]):
    user = fake_decode_token(token)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Not authenticated",
            headers={"WWW-Authenticate": "Bearer"},
        )
    return user

async def get_current_active_user(
    current_user: Annotated[User,
Depends(get_current_user)],
):
    if current_user.disabled:
        raise HTTPException(status_code=400,
detail="Inactive user")
```

```
    return current_user

@app.post("/token")
async def login(form_data:
Annotated[OAuth2PasswordRequestForm, Depends()]):
    user_dict = fake_users_db.get(form_data.username)
    if not user_dict:
        raise HTTPException(status_code=400,
detail="Incorrect username or password")
    user = UserInDB(**user_dict)
    hashed_password =
fake_hash_password(form_data.password)
    if not hashed_password == user.hashed_password:
        raise HTTPException(status_code=400,
detail="Incorrect username or password")

    return {"access_token": user.username,
"token_type": "bearer"}

@app.get("/users/me")
async def read_users_me(
    current_user: Annotated[User,
Depends(get_current_active_user)],
):
    return current_user
```

`OAuth2PasswordRequestForm` es una dependencia de clase que declara un body de formulario con:

- El `username`.
- El `password`.
- Un campo opcional `scope` como un string grande, compuesto por strings separados por espacios.
- Un `grant_type` opcional.

Consejo

La especificación de OAuth2 en realidad *requiere* un campo `grant_type` con un valor fijo de `password`, pero `OAuth2PasswordRequestForm` no lo obliga.

Si necesitas imponerlo, utiliza `OAuth2PasswordRequestFormStrict` en lugar de `OAuth2PasswordRequestForm`.

- Un `client_id` opcional (no lo necesitamos para nuestro ejemplo).
- Un `client_secret` opcional (no lo necesitamos para nuestro ejemplo).

Nota

`OAuth2PasswordRequestForm` no es una clase especial para **FastAPI** como lo es `OAuth2PasswordBearer`.

`OAuth2PasswordBearer` hace que **FastAPI** sepa que es un esquema de seguridad. Así que se añade de esa manera a OpenAPI.

Pero `OAuth2PasswordRequestForm` es solo una dependencia de clase que podrías haber escrito tú mismo, o podrías haber declarado parámetros de `Form` directamente.

Pero como es un caso de uso común, se proporciona directamente por **FastAPI**, solo para facilitarlo.

Usa el form data

Consejo

La instance de la clase de dependencia `OAuth2PasswordRequestForm` no tendrá un atributo `scope` con el string largo separado por espacios, en su lugar, tendrá un atributo `scopes` con la lista real de strings para cada scope enviado.

No estamos usando `scopes` en este ejemplo, pero la funcionalidad está ahí si la necesitas.

Ahora, obtén los datos del usuario desde la base de datos (falsa), usando el `username` del campo del formulario.

Si no existe tal usuario, devolvemos un error diciendo "Incorrect username or password".

Para el error, usamos la excepción `HTTPException`:

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException,
status
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from pydantic import BaseModel

fake_users_db = {
    "johndoe": {
        "username": "johndoe",
        "full_name": "John Doe",
        "email": "johndoe@example.com",
        "hashed_password": "fakehashedsecret",
        "disabled": False,
    },
    "alice": {
        "username": "alice",
        "full_name": "Alice Wonderson",
        "email": "alice@example.com",
        "hashed_password": "fakehashedsecret2",
        "disabled": True,
    },
}

app = FastAPI()

def fake_hash_password(password: str):
    return "fakehashed" + password

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

class User(BaseModel):
```

```
    username: str
    email: str | None = None
    full_name: str | None = None
    disabled: bool | None = None

class UserInDB(User):
    hashed_password: str

def get_user(db, username: str):
    if username in db:
        user_dict = db[username]
        return UserInDB(**user_dict)

def fake_decode_token(token):
    # This doesn't provide any security at all
    # Check the next version
    user = get_user(fake_users_db, token)
    return user

async def get_current_user(token: Annotated[str,
Depends(oauth2_scheme)]):
    user = fake_decode_token(token)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Not authenticated",
            headers={"WWW-Authenticate": "Bearer"},
        )
    return user

async def get_current_active_user(
    current_user: Annotated[User,
Depends(get_current_user)],
):
    if current_user.disabled:
        raise HTTPException(status_code=400,
detail="Inactive user")
    return current_user

@app.post("/token")
async def login(form_data:
Annotated[OAuth2PasswordRequestForm, Depends()]):
```

```
    user_dict = fake_users_db.get(form_data.username)
    if not user_dict:
        raise HTTPException(status_code=400,
detail="Incorrect username or password")
    user = UserInDB(**user_dict)
    hashed_password =
fake_hash_password(form_data.password)
    if not hashed_password == user.hashed_password:
        raise HTTPException(status_code=400,
detail="Incorrect username or password")

    return {"access_token": user.username,
"token_type": "bearer"}

@app.get("/users/me")
async def read_users_me(
    current_user: Annotated[User,
Depends(get_current_active_user)],
):
    return current_user
```

Revisa el password

En este punto tenemos los datos del usuario de nuestra base de datos, pero no hemos revisado el password.

Primero pongamos esos datos en el modelo `UserInDB` de Pydantic.

Nunca deberías guardar passwords en texto plano, así que, usaremos el sistema de hash de passwords (falso).

Si los passwords no coinciden, devolvemos el mismo error.

Hashing de passwords

"Hacer hash" significa: convertir algún contenido (un password en este caso) en una secuencia de bytes (solo un string) que parece un galimatías.

Siempre que pases exactamente el mismo contenido (exactamente el mismo password) obtienes exactamente el mismo galimatías.

Pero no puedes convertir del galimatías al password.

Por qué usar hashing de passwords

Si tu base de datos es robada, el ladrón no tendrá los passwords en texto plano de tus usuarios, solo los hashes.

Entonces, el ladrón no podrá intentar usar esos mismos passwords en otro sistema (como muchos usuarios usan el mismo password en todas partes, esto sería peligroso).

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException,
status
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from pydantic import BaseModel

fake_users_db = {
    "johndoe": {
        "username": "johndoe",
        "full_name": "John Doe",
        "email": "johndoe@example.com",
        "hashed_password": "fakehashedsecret",
        "disabled": False,
    },
    "alice": {
        "username": "alice",
        "full_name": "Alice Wonderson",
        "email": "alice@example.com",
        "hashed_password": "fakehashedsecret2",
        "disabled": True,
    },
}

app = FastAPI()
```

```python
def fake_hash_password(password: str):
    return "fakehashed" + password

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

class User(BaseModel):
    username: str
    email: str | None = None
    full_name: str | None = None
    disabled: bool | None = None

class UserInDB(User):
    hashed_password: str

def get_user(db, username: str):
    if username in db:
        user_dict = db[username]
        return UserInDB(**user_dict)

def fake_decode_token(token):
    # This doesn't provide any security at all
    # Check the next version
    user = get_user(fake_users_db, token)
    return user

async def get_current_user(token: Annotated[str,
Depends(oauth2_scheme)]):
    user = fake_decode_token(token)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Not authenticated",
            headers={"WWW-Authenticate": "Bearer"},
        )
    return user

async def get_current_active_user(
    current_user: Annotated[User,
Depends(get_current_user)],
):
    if current_user.disabled:
```

```
        raise HTTPException(status_code=400,
detail="Inactive user")
    return current_user

@app.post("/token")
async def login(form_data:
Annotated[OAuth2PasswordRequestForm, Depends()]):
    user_dict = fake_users_db.get(form_data.username)
    if not user_dict:
        raise HTTPException(status_code=400,
detail="Incorrect username or password")
    user = UserInDB(**user_dict)
    hashed_password =
fake_hash_password(form_data.password)
    if not hashed_password == user.hashed_password:
        raise HTTPException(status_code=400,
detail="Incorrect username or password")

    return {"access_token": user.username,
"token_type": "bearer"}

@app.get("/users/me")
async def read_users_me(
    current_user: Annotated[User,
Depends(get_current_active_user)],
):
    return current_user
```

Sobre **user_dict

`UserInDB(**user_dict)` significa:

Pasa las claves y valores de `user_dict` directamente como argumentos clave-valor, equivalente a:

```
UserInDB(
    username = user_dict["username"],
    email = user_dict["email"],
    full_name = user_dict["full_name"],
    disabled = user_dict["disabled"],
```

```
    hashed_password = user_dict["hashed_password"],
)
```

> **Nota**
> Para una explicación más completa de `**user_dict` revisa en la documentación para **Extra Models**.

Devolver el token

El response del endpoint `token` debe ser un objeto JSON.

Debe tener un `token_type`. En nuestro caso, como estamos usando tokens "Bearer", el tipo de token debe ser " `bearer` ".

Y debe tener un `access_token`, con un string que contenga nuestro token de acceso.

Para este ejemplo simple, vamos a ser completamente inseguros y devolver el mismo `username` como el token.

> **Consejo**
> En el próximo capítulo, verás una implementación segura real, con hashing de passwords y tokens JWT.
>
> Pero por ahora, enfoquémonos en los detalles específicos que necesitamos.

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException,
status
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from pydantic import BaseModel

fake_users_db = {
    "johndoe": {
        "username": "johndoe",
        "full_name": "John Doe",
```

```
        "email": "johndoe@example.com",
        "hashed_password": "fakehashedsecret",
        "disabled": False,
    },
    "alice": {
        "username": "alice",
        "full_name": "Alice Wonderson",
        "email": "alice@example.com",
        "hashed_password": "fakehashedsecret2",
        "disabled": True,
    },
}

app = FastAPI()

def fake_hash_password(password: str):
    return "fakehashed" + password

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

class User(BaseModel):
    username: str
    email: str | None = None
    full_name: str | None = None
    disabled: bool | None = None

class UserInDB(User):
    hashed_password: str

def get_user(db, username: str):
    if username in db:
        user_dict = db[username]
        return UserInDB(**user_dict)

def fake_decode_token(token):
    # This doesn't provide any security at all
    # Check the next version
    user = get_user(fake_users_db, token)
    return user

async def get_current_user(token: Annotated[str,
Depends(oauth2_scheme)]):
```

```
    user = fake_decode_token(token)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Not authenticated",
            headers={"WWW-Authenticate": "Bearer"},
        )
    return user

async def get_current_active_user(
    current_user: Annotated[User,
Depends(get_current_user)],
):
    if current_user.disabled:
        raise HTTPException(status_code=400,
detail="Inactive user")
    return current_user

@app.post("/token")
async def login(form_data:
Annotated[OAuth2PasswordRequestForm, Depends()]):
    user_dict = fake_users_db.get(form_data.username)
    if not user_dict:
        raise HTTPException(status_code=400,
detail="Incorrect username or password")
    user = UserInDB(**user_dict)
    hashed_password =
fake_hash_password(form_data.password)
    if not hashed_password == user.hashed_password:
        raise HTTPException(status_code=400,
detail="Incorrect username or password")

    return {"access_token": user.username,
"token_type": "bearer"}

@app.get("/users/me")
async def read_users_me(
    current_user: Annotated[User,
Depends(get_current_active_user)],
):
    return current_user
```

Consejo

De acuerdo con la especificación, deberías devolver un JSON con un `access_token` y un `token_type`, igual que en este ejemplo.

Esto es algo que tienes que hacer tú mismo en tu código, y asegurarte de usar esas claves JSON.

Es casi lo único que tienes que recordar hacer correctamente tú mismo, para ser compatible con las especificaciones.

Para el resto, **FastAPI** lo maneja por ti.

Actualizar las dependencias

Ahora vamos a actualizar nuestras dependencias.

Queremos obtener el `current_user` *solo* si este usuario está activo.

Entonces, creamos una dependencia adicional `get_current_active_user` que a su vez utiliza `get_current_user` como dependencia.

Ambas dependencias solo devolverán un error HTTP si el usuario no existe, o si está inactivo.

Así que, en nuestro endpoint, solo obtendremos un usuario si el usuario existe, fue autenticado correctamente, y está activo:

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException,
status
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from pydantic import BaseModel

fake_users_db = {
    "johndoe": {
```

```
        "username": "johndoe",
        "full_name": "John Doe",
        "email": "johndoe@example.com",
        "hashed_password": "fakehashedsecret",
        "disabled": False,
    },
    "alice": {
        "username": "alice",
        "full_name": "Alice Wonderson",
        "email": "alice@example.com",
        "hashed_password": "fakehashedsecret2",
        "disabled": True,
    },
}

app = FastAPI()

def fake_hash_password(password: str):
    return "fakehashed" + password

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

class User(BaseModel):
    username: str
    email: str | None = None
    full_name: str | None = None
    disabled: bool | None = None

class UserInDB(User):
    hashed_password: str

def get_user(db, username: str):
    if username in db:
        user_dict = db[username]
        return UserInDB(**user_dict)

def fake_decode_token(token):
    # This doesn't provide any security at all
    # Check the next version
    user = get_user(fake_users_db, token)
    return user
```

```
async def get_current_user(token: Annotated[str,
Depends(oauth2_scheme)]):
    user = fake_decode_token(token)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Not authenticated",
            headers={"WWW-Authenticate": "Bearer"},
        )
    return user

async def get_current_active_user(
    current_user: Annotated[User,
Depends(get_current_user)],
):
    if current_user.disabled:
        raise HTTPException(status_code=400,
detail="Inactive user")
    return current_user

@app.post("/token")
async def login(form_data:
Annotated[OAuth2PasswordRequestForm, Depends()]):
    user_dict = fake_users_db.get(form_data.username)
    if not user_dict:
        raise HTTPException(status_code=400,
detail="Incorrect username or password")
    user = UserInDB(**user_dict)
    hashed_password =
fake_hash_password(form_data.password)
    if not hashed_password == user.hashed_password:
        raise HTTPException(status_code=400,
detail="Incorrect username or password")

    return {"access_token": user.username,
"token_type": "bearer"}

@app.get("/users/me")
async def read_users_me(
    current_user: Annotated[User,
Depends(get_current_active_user)],
```

```
):
    return current_user
```

> **Nota**
> El header adicional `WWW-Authenticate` con el valor `Bearer` que estamos devolviendo aquí también es parte de la especificación.
>
> Cualquier código de estado HTTP (error) 401 "UNAUTHORIZED" se supone que también debe devolver un header `WWW-Authenticate`.
>
> En el caso de tokens bearer (nuestro caso), el valor de ese header debe ser `Bearer`.
>
> De hecho, puedes omitir ese header extra y aún funcionaría.
>
> Pero se proporciona aquí para cumplir con las especificaciones.
>
> Además, podría haber herramientas que lo esperen y lo usen (ahora o en el futuro) y eso podría ser útil para ti o tus usuarios, ahora o en el futuro.
>
> Ese es el beneficio de los estándares...

Verlo en acción

Abre la documentación interactiva: http://127.0.0.1:8000/docs.

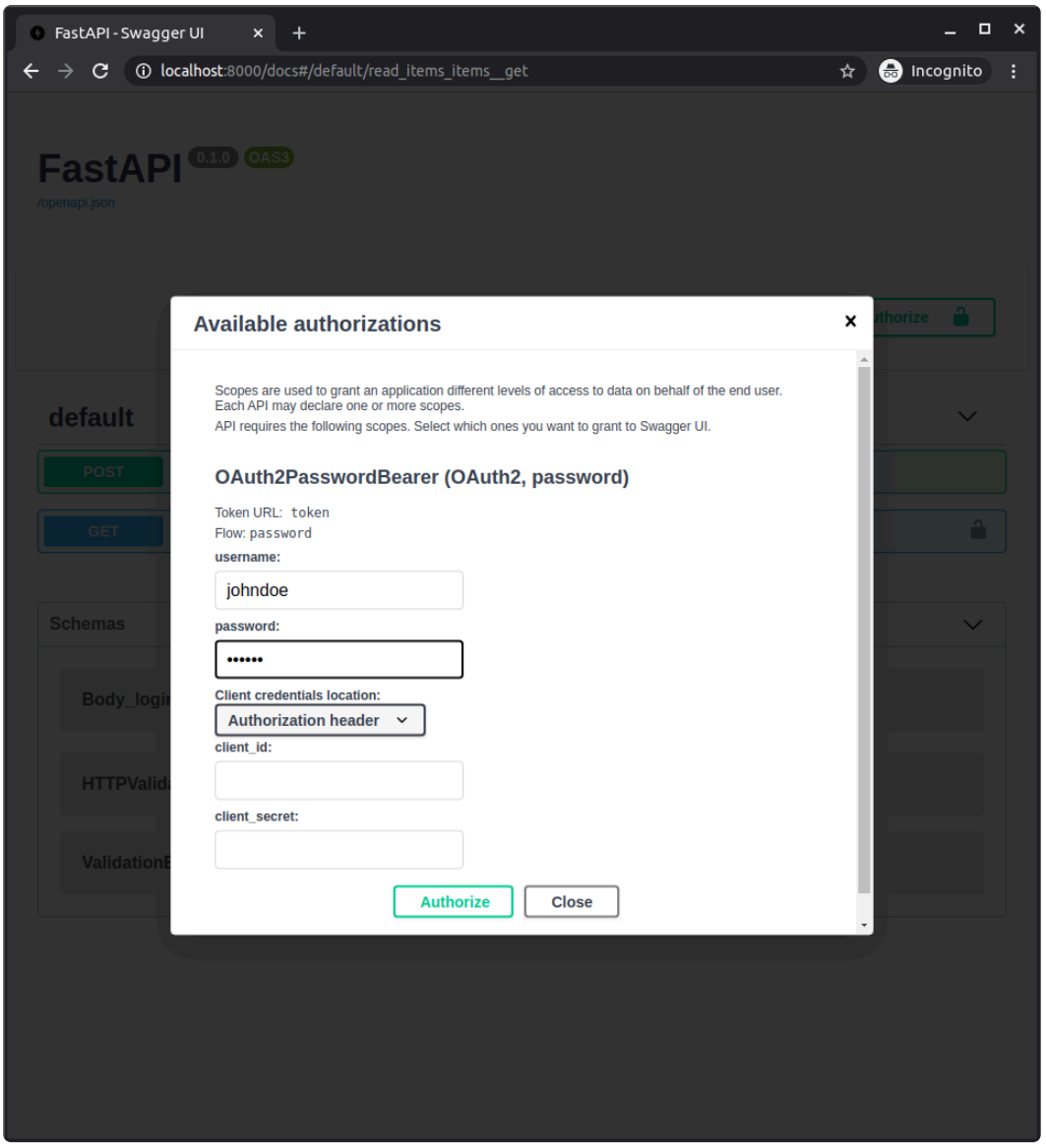
Autenticar

Haz clic en el botón "Authorize".

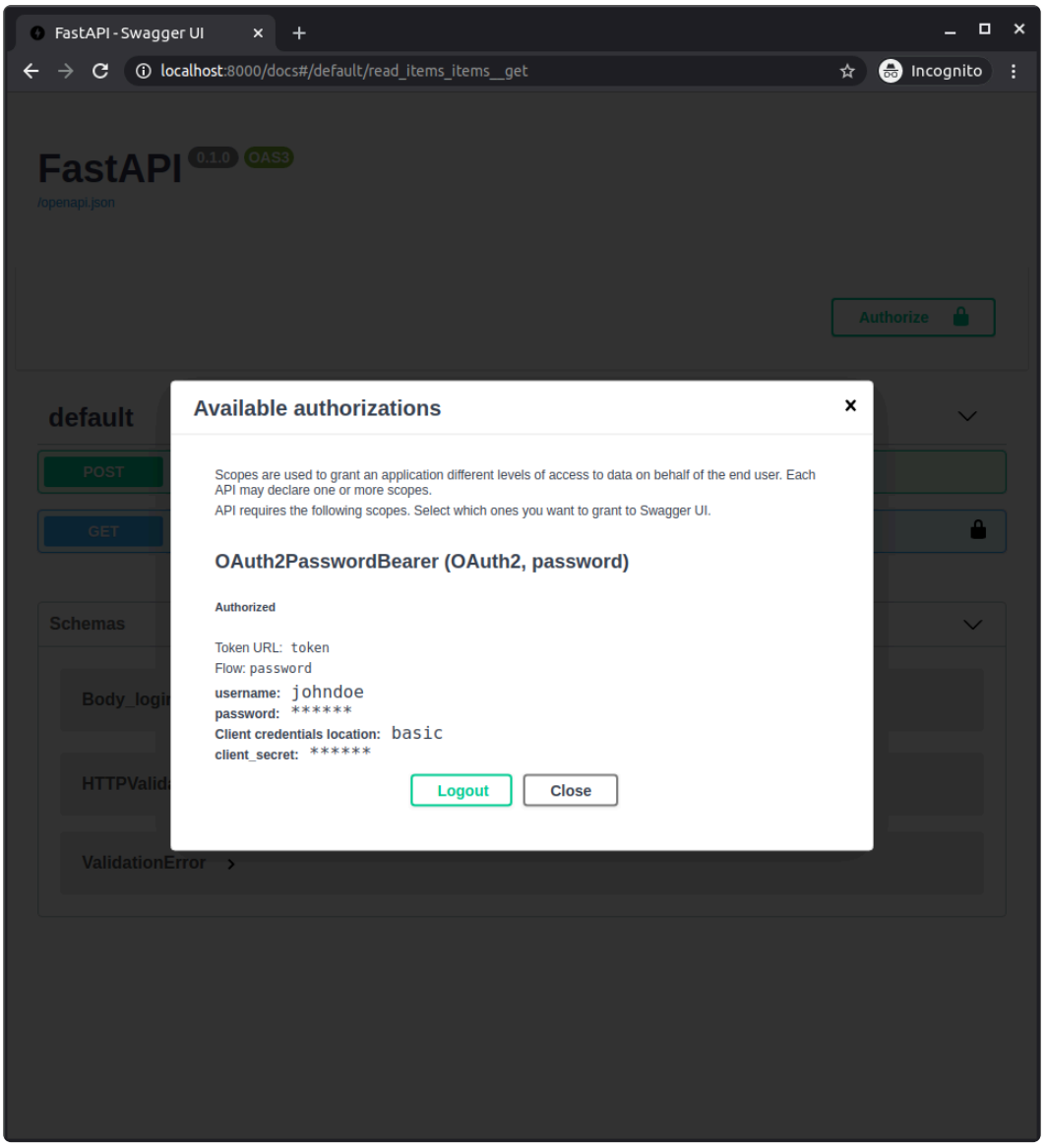
Usa las credenciales:

Usuario: `johndoe`

Contraseña: `secret`



Después de autenticarte en el sistema, lo verás así:

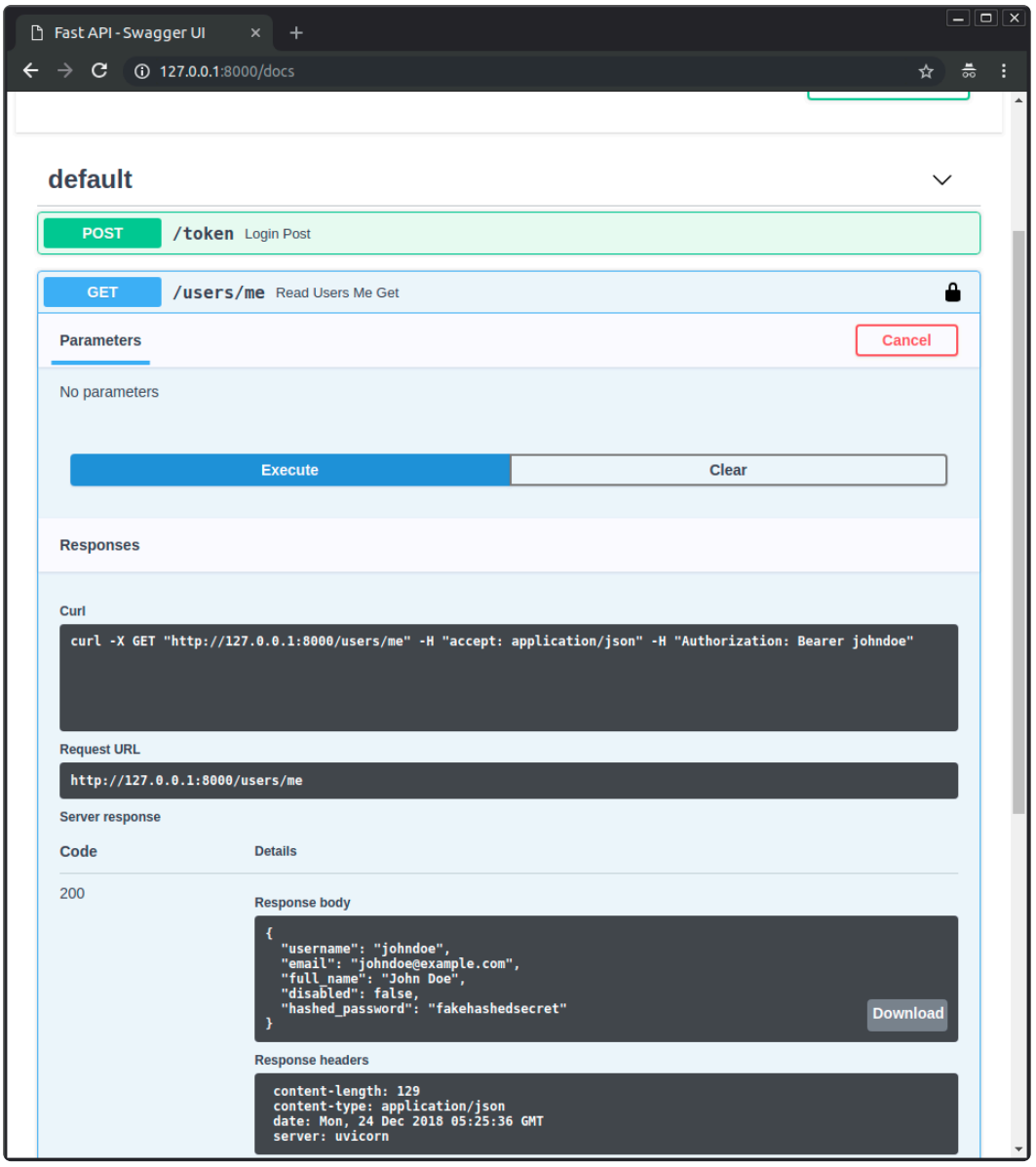


Obtener tus propios datos de usuario

Ahora usa la operación `GET` con la path `/users/me`.

Obtendrás los datos de tu usuario, como:

```
{
  "username": "johndoe",
  "email": "johndoe@example.com",
  "full_name": "John Doe",
  "disabled": false,
  "hashed_password": "fakehashedsecret"
}
```



Si haces clic en el icono de candado y cierras sesión, y luego intentas la misma operación nuevamente, obtendrás un error HTTP 401 de:

```
{
  "detail": "Not authenticated"
}
```

Usuario inactivo

Ahora prueba con un usuario inactivo, autentícate con:

Usuario: `alice`

Contraseña: `secret2`

Y trata de usar la operación `GET` con la path `/users/me`.

Obtendrás un error de "Usuario inactivo", como:

```
{
  "detail": "Inactive user"
}
```

Recapitulación

Ahora tienes las herramientas para implementar un sistema de seguridad completo basado en `username` y `password` para tu API.

Usando estas herramientas, puedes hacer que el sistema de seguridad sea compatible con cualquier base de datos y con cualquier modelo de usuario o de datos.

El único detalle que falta es que en realidad no es "seguro" aún.

En el próximo capítulo verás cómo usar un paquete de hashing de passwords seguro y tokens JWT.

OAuth2 con Password (y hashing), Bearer con tokens JWT

Ahora que tenemos todo el flujo de seguridad, hagamos que la aplicación sea realmente segura, usando tokens JWT y hashing de contraseñas seguras.

Este código es algo que puedes usar realmente en tu aplicación, guardar los hashes de las contraseñas en tu base de datos, etc.

Vamos a empezar desde donde lo dejamos en el capítulo anterior e incrementarlo.

Acerca de JWT

JWT significa "JSON Web Tokens".

Es un estándar para codificar un objeto JSON en un string largo y denso sin espacios. Se ve así:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

No está encriptado, por lo que cualquiera podría recuperar la información de los contenidos.

Pero está firmado. Así que, cuando recibes un token que has emitido, puedes verificar que realmente lo emitiste.

De esta manera, puedes crear un token con una expiración de, digamos, 1 semana. Y luego, cuando el usuario regresa al día siguiente con el token, sabes que el usuario todavía está registrado en tu sistema.

Después de una semana, el token estará expirado y el usuario no estará autorizado y tendrá que iniciar sesión nuevamente para obtener un nuevo token. Y si el usuario (o un tercero) intenta modificar el token para cambiar la expiración, podrás descubrirlo, porque las firmas no coincidirían.

Si quieres jugar con tokens JWT y ver cómo funcionan, revisa https://jwt.io.

Instalar PyJWT

Necesitamos instalar `PyJWT` para generar y verificar los tokens JWT en Python.

Añade `pyjwt` a tu proyecto:

> **Nota**

Si planeas usar algoritmos de firma digital como RSA o ECDSA, deberías instalar la dependencia del paquete de criptografía `pyjwt[crypto]`.

Puedes leer más al respecto en la documentación de instalación de PyJWT.

Hashing de contraseñas

"Hacer hashing" significa convertir algún contenido (una contraseña en este caso) en una secuencia de bytes (solo un string) que parece un galimatías.

Siempre que pases exactamente el mismo contenido (exactamente la misma contraseña) obtienes exactamente el mismo galimatías.

Pero no puedes convertir del galimatías de nuevo a la contraseña.

Por qué usar hashing de contraseñas

Si tu base de datos es robada, el ladrón no tendrá las contraseñas en texto claro de tus usuarios, solo los hashes.

Por lo tanto, el ladrón no podrá intentar usar esa contraseña en otro sistema (como muchos usuarios usan la misma contraseña en todas partes, esto sería peligroso).

Instalar pwdlib

pwdlib es un gran paquete de Python para manejar hashes de contraseñas.

Soporta muchos algoritmos de hashing seguros y utilidades para trabajar con ellos.

El algoritmo recomendado es "Argon2".

Añade `pwdlib` con Argon2 a tu proyecto:

> **Consejo**
> Con `pwdlib`, incluso podrías configurarlo para poder leer contraseñas creadas por **Django**, un plug-in de seguridad de **Flask** u otros muchos.
>
> Así, podrías, por ejemplo, compartir los mismos datos de una aplicación de Django en una base de datos con una aplicación de FastAPI. O migrar gradualmente una aplicación de Django usando la misma base de datos.
>
> Y tus usuarios podrían iniciar sesión desde tu aplicación Django o desde tu aplicación **FastAPI**, al mismo tiempo.

Hash y verificación de contraseñas

Importa las herramientas que necesitamos de `pwdlib`.

Crea un instance PasswordHash con configuraciones recomendadas: se usará para hacer el hash y verificar las contraseñas.

> **Consejo**
> pwdlib también soporta el algoritmo de hashing bcrypt pero no incluye algoritmos legacy; para trabajar con hashes desactualizados, se recomienda usar el paquete passlib.
>
> Por ejemplo, podrías usarlo para leer y verificar contraseñas generadas por otro sistema (como Django) pero hacer hash de cualquier contraseña nueva con un algoritmo diferente como Argon2 o Bcrypt.
>
> Y ser compatible con todos ellos al mismo tiempo.

Crea una función de utilidad para hacer el hash de una contraseña que venga del usuario.

Y otra utilidad para verificar si una contraseña recibida coincide con el hash almacenado.

Y otra más para autenticar y devolver un usuario.

```
from datetime import datetime, timedelta, timezone
from typing import Annotated

import jwt
from fastapi import Depends, FastAPI, HTTPException,
status
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from jwt.exceptions import InvalidTokenError
from pwdlib import PasswordHash
from pydantic import BaseModel

# to get a string like this run:
# openssl rand -hex 32
SECRET_KEY =
"09d25e094faa6ca2556c818166b7a9563b93f7099f6f0f4caa6cf
63b88e8d3e7"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

fake_users_db = {
    "johndoe": {
        "username": "johndoe",
        "full_name": "John Doe",
        "email": "johndoe@example.com",
        "hashed_password":
"$argon2id$v=19$m=65536,t=3,p=4$wagCPXjifgvUFBzq4hqe3w
$CYaIb8sB+wtD+Vu/P4uod1+Qof8h+1g7bbDlBID48Rc",
        "disabled": False,
    }
}

class Token(BaseModel):
    access_token: str
    token_type: str

class TokenData(BaseModel):
    username: str | None = None
```

```python
class User(BaseModel):
    username: str
    email: str | None = None
    full_name: str | None = None
    disabled: bool | None = None

class UserInDB(User):
    hashed_password: str

password_hash = PasswordHash.recommended()

DUMMY_HASH = password_hash.hash("dummypassword")

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

app = FastAPI()

def verify_password(plain_password, hashed_password):
    return password_hash.verify(plain_password,
hashed_password)

def get_password_hash(password):
    return password_hash.hash(password)

def get_user(db, username: str):
    if username in db:
        user_dict = db[username]
        return UserInDB(**user_dict)

def authenticate_user(fake_db, username: str,
password: str):
    user = get_user(fake_db, username)
    if not user:
        verify_password(password, DUMMY_HASH)
        return False
    if not verify_password(password,
user.hashed_password):
        return False
    return user

def create_access_token(data: dict, expires_delta:
timedelta | None = None):
```

```
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.now(timezone.utc) +
expires_delta
    else:
        expire = datetime.now(timezone.utc) +
timedelta(minutes=15)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY,
algorithm=ALGORITHM)
    return encoded_jwt

async def get_current_user(token: Annotated[str,
Depends(oauth2_scheme)]):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, SECRET_KEY,
algorithms=[ALGORITHM])
        username = payload.get("sub")
        if username is None:
            raise credentials_exception
        token_data = TokenData(username=username)
    except InvalidTokenError:
        raise credentials_exception
    user = get_user(fake_users_db,
username=token_data.username)
    if user is None:
        raise credentials_exception
    return user

async def get_current_active_user(
    current_user: Annotated[User,
Depends(get_current_user)],
):
    if current_user.disabled:
        raise HTTPException(status_code=400,
detail="Inactive user")
    return current_user
```

```
@app.post("/token")
async def login_for_access_token(
    form_data: Annotated[OAuth2PasswordRequestForm,
Depends()],
) -> Token:
    user = authenticate_user(fake_users_db,
form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    access_token_expires =
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = create_access_token(
        data={"sub": user.username},
expires_delta=access_token_expires
    )
    return Token(access_token=access_token,
token_type="bearer")

@app.get("/users/me/")
async def read_users_me(
    current_user: Annotated[User,
Depends(get_current_active_user)],
) -> User:
    return current_user

@app.get("/users/me/items/")
async def read_own_items(
    current_user: Annotated[User,
Depends(get_current_active_user)],
):
    return [{"item_id": "Foo", "owner":
current_user.username}]
```

Cuando `authenticate_user` se llama con un nombre de usuario que no existe en la base de datos, aun así ejecutamos `verify_password` contra un hash ficticio.

Esto asegura que el endpoint tarda aproximadamente la misma cantidad de tiempo en responder tanto si el nombre de usuario es válido como si no, previniendo los **timing attacks** que podrían usarse para enumerar nombres de usuario existentes.

> **Nota**
> Si revisas la nueva (falsa) base de datos `fake_users_db`, verás cómo se ve ahora la contraseña con hash: `"$argon2id$v=19$m=65536,t=3,p=4$wagCPXjifgvUFBzq4hqe3w$CYaIb8sB+wtD+Vu/P4uod1+Qof8h+1g7bbDlBID48Rc"`.

Manejo de tokens JWT

Importa los módulos instalados.

Crea una clave secreta aleatoria que se usará para firmar los tokens JWT.

Para generar una clave secreta segura al azar usa el comando:

Y copia el resultado a la variable `SECRET_KEY` (no uses la del ejemplo).

Crea una variable `ALGORITHM` con el algoritmo usado para firmar el token JWT y configúralo a `"HS256"`.

Crea una variable para la expiración del token.

Define un Modelo de Pydantic que se usará en el endpoint de token para el response.

Crea una función de utilidad para generar un nuevo token de acceso.

```
from datetime import datetime, timedelta, timezone
from typing import Annotated

import jwt
from fastapi import Depends, FastAPI, HTTPException,
```

```
status
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from jwt.exceptions import InvalidTokenError
from pwdlib import PasswordHash
from pydantic import BaseModel

# to get a string like this run:
# openssl rand -hex 32
SECRET_KEY =
"09d25e094faa6ca2556c818166b7a9563b93f7099f6f0f4caa6cf
63b88e8d3e7"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

fake_users_db = {
    "johndoe": {
        "username": "johndoe",
        "full_name": "John Doe",
        "email": "johndoe@example.com",
        "hashed_password":
"$argon2id$v=19$m=65536,t=3,p=4$wagCPXjifgvUFBzq4hqe3w
$CYaIb8sB+wtD+Vu/P4uod1+Qof8h+1g7bbDlBID48Rc",
        "disabled": False,
    }
}

class Token(BaseModel):
    access_token: str
    token_type: str

class TokenData(BaseModel):
    username: str | None = None

class User(BaseModel):
    username: str
    email: str | None = None
    full_name: str | None = None
    disabled: bool | None = None

class UserInDB(User):
    hashed_password: str
```

```
password_hash = PasswordHash.recommended()

DUMMY_HASH = password_hash.hash("dummypassword")

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

app = FastAPI()

def verify_password(plain_password, hashed_password):
    return password_hash.verify(plain_password,
hashed_password)

def get_password_hash(password):
    return password_hash.hash(password)

def get_user(db, username: str):
    if username in db:
        user_dict = db[username]
        return UserInDB(**user_dict)

def authenticate_user(fake_db, username: str,
password: str):
    user = get_user(fake_db, username)
    if not user:
        verify_password(password, DUMMY_HASH)
        return False
    if not verify_password(password,
user.hashed_password):
        return False
    return user

def create_access_token(data: dict, expires_delta:
timedelta | None = None):
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.now(timezone.utc) +
expires_delta
    else:
        expire = datetime.now(timezone.utc) +
timedelta(minutes=15)
    to_encode.update({"exp": expire})
```

```
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY,
algorithm=ALGORITHM)
    return encoded_jwt

async def get_current_user(token: Annotated[str,
Depends(oauth2_scheme)]):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, SECRET_KEY,
algorithms=[ALGORITHM])
        username = payload.get("sub")
        if username is None:
            raise credentials_exception
        token_data = TokenData(username=username)
    except InvalidTokenError:
        raise credentials_exception
    user = get_user(fake_users_db,
username=token_data.username)
    if user is None:
        raise credentials_exception
    return user

async def get_current_active_user(
    current_user: Annotated[User,
Depends(get_current_user)],
):
    if current_user.disabled:
        raise HTTPException(status_code=400,
detail="Inactive user")
    return current_user

@app.post("/token")
async def login_for_access_token(
    form_data: Annotated[OAuth2PasswordRequestForm,
Depends()],
) -> Token:
    user = authenticate_user(fake_users_db,
form_data.username, form_data.password)
```

```
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    access_token_expires =
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = create_access_token(
        data={"sub": user.username},
expires_delta=access_token_expires
    )
    return Token(access_token=access_token,
token_type="bearer")

@app.get("/users/me/")
async def read_users_me(
    current_user: Annotated[User,
Depends(get_current_active_user)],
) -> User:
    return current_user

@app.get("/users/me/items/")
async def read_own_items(
    current_user: Annotated[User,
Depends(get_current_active_user)],
):
    return [{"item_id": "Foo", "owner":
current_user.username}]
```

Actualizar las dependencias

Actualiza `get_current_user` para recibir el mismo token que antes, pero esta vez, usando tokens JWT.

Decodifica el token recibido, verifícalo y devuelve el usuario actual.

Si el token es inválido, devuelve un error HTTP de inmediato.

```python
from datetime import datetime, timedelta, timezone
from typing import Annotated

import jwt
from fastapi import Depends, FastAPI, HTTPException,
status
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from jwt.exceptions import InvalidTokenError
from pwdlib import PasswordHash
from pydantic import BaseModel

# to get a string like this run:
# openssl rand -hex 32
SECRET_KEY =
"09d25e094faa6ca2556c818166b7a9563b93f7099f6f0f4caa6cf
63b88e8d3e7"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

fake_users_db = {
    "johndoe": {
        "username": "johndoe",
        "full_name": "John Doe",
        "email": "johndoe@example.com",
        "hashed_password":
"$argon2id$v=19$m=65536,t=3,p=4$wagCPXjifgvUFBzq4hqe3w
$CYaIb8sB+wtD+Vu/P4uod1+Qof8h+1g7bbDlBID48Rc",
        "disabled": False,
    }
}

class Token(BaseModel):
    access_token: str
    token_type: str

class TokenData(BaseModel):
    username: str | None = None

class User(BaseModel):
    username: str
```

```
    email: str | None = None
    full_name: str | None = None
    disabled: bool | None = None

class UserInDB(User):
    hashed_password: str

password_hash = PasswordHash.recommended()

DUMMY_HASH = password_hash.hash("dummypassword")

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

app = FastAPI()

def verify_password(plain_password, hashed_password):
    return password_hash.verify(plain_password,
hashed_password)

def get_password_hash(password):
    return password_hash.hash(password)

def get_user(db, username: str):
    if username in db:
        user_dict = db[username]
        return UserInDB(**user_dict)

def authenticate_user(fake_db, username: str,
password: str):
    user = get_user(fake_db, username)
    if not user:
        verify_password(password, DUMMY_HASH)
        return False
    if not verify_password(password,
user.hashed_password):
        return False
    return user

def create_access_token(data: dict, expires_delta:
timedelta | None = None):
    to_encode = data.copy()
    if expires_delta:
```

```
        expire = datetime.now(timezone.utc) +
expires_delta
    else:
        expire = datetime.now(timezone.utc) +
timedelta(minutes=15)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY,
algorithm=ALGORITHM)
    return encoded_jwt

async def get_current_user(token: Annotated[str,
Depends(oauth2_scheme)]):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, SECRET_KEY,
algorithms=[ALGORITHM])
        username = payload.get("sub")
        if username is None:
            raise credentials_exception
        token_data = TokenData(username=username)
    except InvalidTokenError:
        raise credentials_exception
    user = get_user(fake_users_db,
username=token_data.username)
    if user is None:
        raise credentials_exception
    return user

async def get_current_active_user(
    current_user: Annotated[User,
Depends(get_current_user)],
):
    if current_user.disabled:
        raise HTTPException(status_code=400,
detail="Inactive user")
    return current_user

@app.post("/token")
```

```python
async def login_for_access_token(
    form_data: Annotated[OAuth2PasswordRequestForm,
Depends()],
) -> Token:
    user = authenticate_user(fake_users_db,
form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    access_token_expires =
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = create_access_token(
        data={"sub": user.username},
expires_delta=access_token_expires
    )
    return Token(access_token=access_token,
token_type="bearer")

@app.get("/users/me/")
async def read_users_me(
    current_user: Annotated[User,
Depends(get_current_active_user)],
) -> User:
    return current_user

@app.get("/users/me/items/")
async def read_own_items(
    current_user: Annotated[User,
Depends(get_current_active_user)],
):
    return [{"item_id": "Foo", "owner":
current_user.username}]
```

Actualizar la path operation /token

Crea un `timedelta` con el tiempo de expiración del token.

Crea un verdadero token de acceso JWT y devuélvelo.

```python
from datetime import datetime, timedelta, timezone
from typing import Annotated

import jwt
from fastapi import Depends, FastAPI, HTTPException,
status
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from jwt.exceptions import InvalidTokenError
from pwdlib import PasswordHash
from pydantic import BaseModel

# to get a string like this run:
# openssl rand -hex 32
SECRET_KEY =
"09d25e094faa6ca2556c818166b7a9563b93f7099f6f0f4caa6cf
63b88e8d3e7"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

fake_users_db = {
    "johndoe": {
        "username": "johndoe",
        "full_name": "John Doe",
        "email": "johndoe@example.com",
        "hashed_password":
"$argon2id$v=19$m=65536,t=3,p=4$wagCPXjifgvUFBzq4hqe3w
$CYaIb8sB+wtD+Vu/P4uod1+Qof8h+1g7bbDlBID48Rc",
        "disabled": False,
    }
}

class Token(BaseModel):
    access_token: str
    token_type: str

class TokenData(BaseModel):
    username: str | None = None

class User(BaseModel):
    username: str
```

```
    email: str | None = None
    full_name: str | None = None
    disabled: bool | None = None

class UserInDB(User):
    hashed_password: str

password_hash = PasswordHash.recommended()

DUMMY_HASH = password_hash.hash("dummypassword")

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

app = FastAPI()

def verify_password(plain_password, hashed_password):
    return password_hash.verify(plain_password,
hashed_password)

def get_password_hash(password):
    return password_hash.hash(password)

def get_user(db, username: str):
    if username in db:
        user_dict = db[username]
        return UserInDB(**user_dict)

def authenticate_user(fake_db, username: str,
password: str):
    user = get_user(fake_db, username)
    if not user:
        verify_password(password, DUMMY_HASH)
        return False
    if not verify_password(password,
user.hashed_password):
        return False
    return user

def create_access_token(data: dict, expires_delta:
timedelta | None = None):
    to_encode = data.copy()
    if expires_delta:
```

```
        expire = datetime.now(timezone.utc) +
expires_delta
    else:
        expire = datetime.now(timezone.utc) +
timedelta(minutes=15)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY,
algorithm=ALGORITHM)
    return encoded_jwt

async def get_current_user(token: Annotated[str,
Depends(oauth2_scheme)]):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, SECRET_KEY,
algorithms=[ALGORITHM])
        username = payload.get("sub")
        if username is None:
            raise credentials_exception
        token_data = TokenData(username=username)
    except InvalidTokenError:
        raise credentials_exception
    user = get_user(fake_users_db,
username=token_data.username)
    if user is None:
        raise credentials_exception
    return user

async def get_current_active_user(
    current_user: Annotated[User,
Depends(get_current_user)],
):
    if current_user.disabled:
        raise HTTPException(status_code=400,
detail="Inactive user")
    return current_user

@app.post("/token")
```

```
async def login_for_access_token(
    form_data: Annotated[OAuth2PasswordRequestForm,
Depends()],
) -> Token:
    user = authenticate_user(fake_users_db,
form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    access_token_expires =
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = create_access_token(
        data={"sub": user.username},
expires_delta=access_token_expires
    )
    return Token(access_token=access_token,
token_type="bearer")

@app.get("/users/me/")
async def read_users_me(
    current_user: Annotated[User,
Depends(get_current_active_user)],
) -> User:
    return current_user

@app.get("/users/me/items/")
async def read_own_items(
    current_user: Annotated[User,
Depends(get_current_active_user)],
):
    return [{"item_id": "Foo", "owner":
current_user.username}]
```

Detalles técnicos sobre el "sujeto" sub de JWT

La especificación de JWT dice que hay una clave `sub`, con el sujeto del token.

Es opcional usarlo, pero ahí es donde pondrías la identificación del usuario, por lo que lo estamos usando aquí.

JWT podría ser usado para otras cosas aparte de identificar un usuario y permitirle realizar operaciones directamente en tu API.

Por ejemplo, podrías identificar un "coche" o un "artículo de blog".

Luego, podrías agregar permisos sobre esa entidad, como "conducir" (para el coche) o "editar" (para el blog).

Y luego, podrías darle ese token JWT a un usuario (o bot), y ellos podrían usarlo para realizar esas acciones (conducir el coche, o editar el artículo del blog) sin siquiera necesitar tener una cuenta, solo con el token JWT que tu API generó para eso.

Usando estas ideas, JWT puede ser utilizado para escenarios mucho más sofisticados.

En esos casos, varias de esas entidades podrían tener el mismo ID, digamos `foo` (un usuario `foo`, un coche `foo`, y un artículo del blog `foo`).

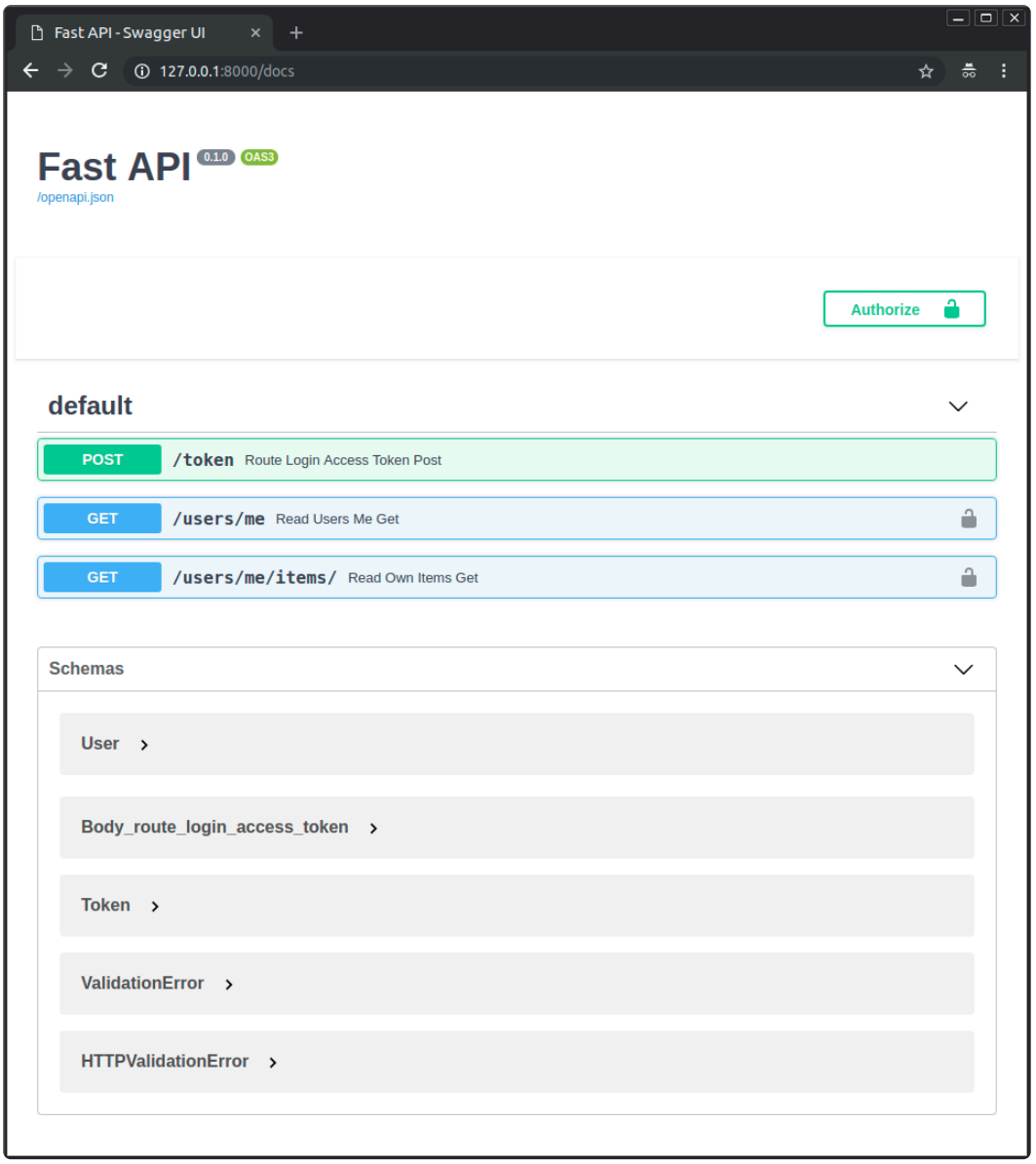
Entonces, para evitar colisiones de ID, cuando crees el token JWT para el usuario, podrías prefijar el valor de la clave `sub`, por ejemplo, con `username:`. Así, en este ejemplo, el valor de `sub` podría haber sido: `username:johndoe`.

Lo importante a tener en cuenta es que la clave `sub` debería tener un identificador único a lo largo de toda la aplicación, y debería ser un string.

Revisa

Ejecuta el servidor y ve a la documentación: http://127.0.0.1:8000/docs.

Verás la interfaz de usuario como:

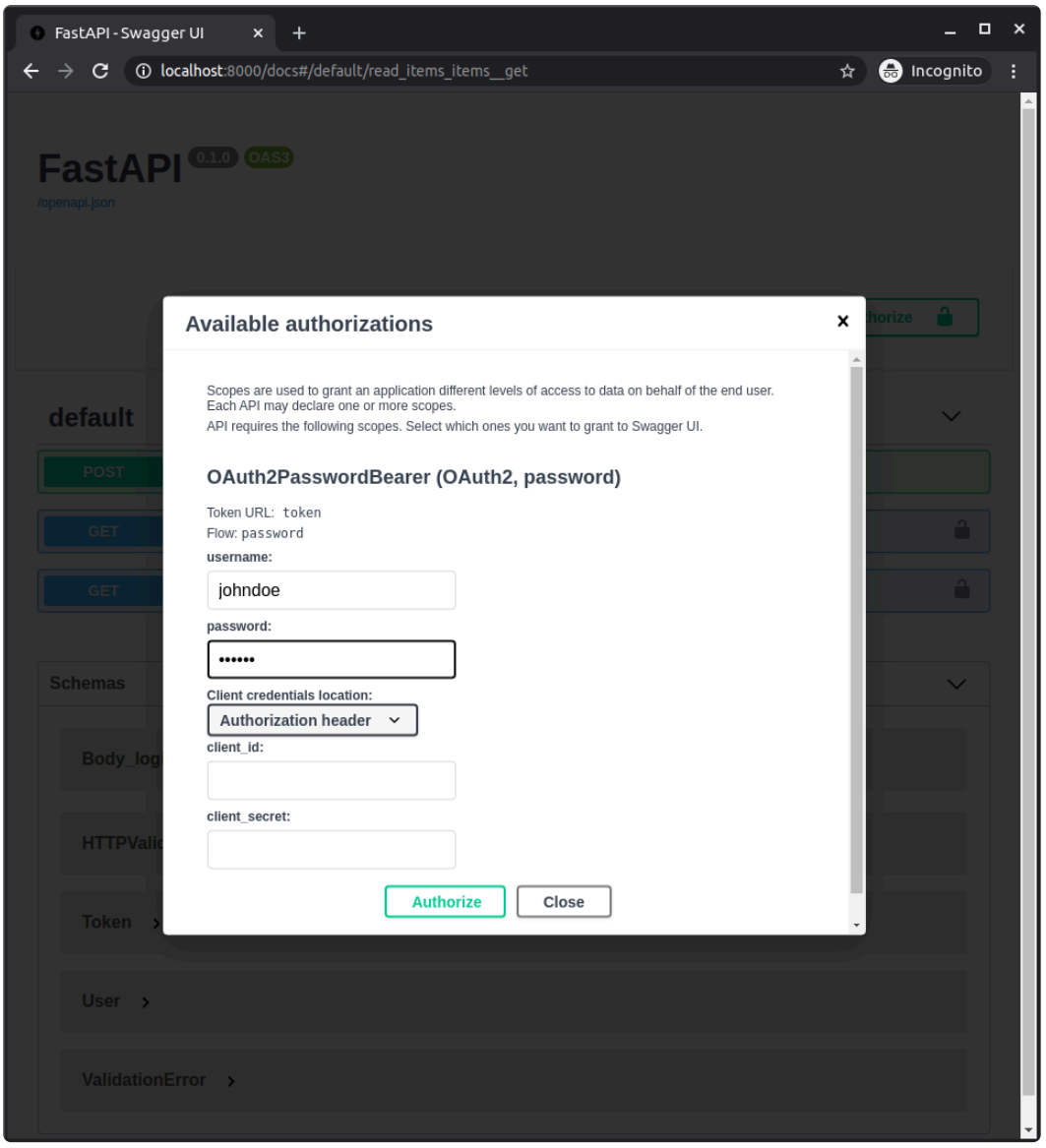


Autoriza la aplicación de la misma manera que antes.

Usando las credenciales:

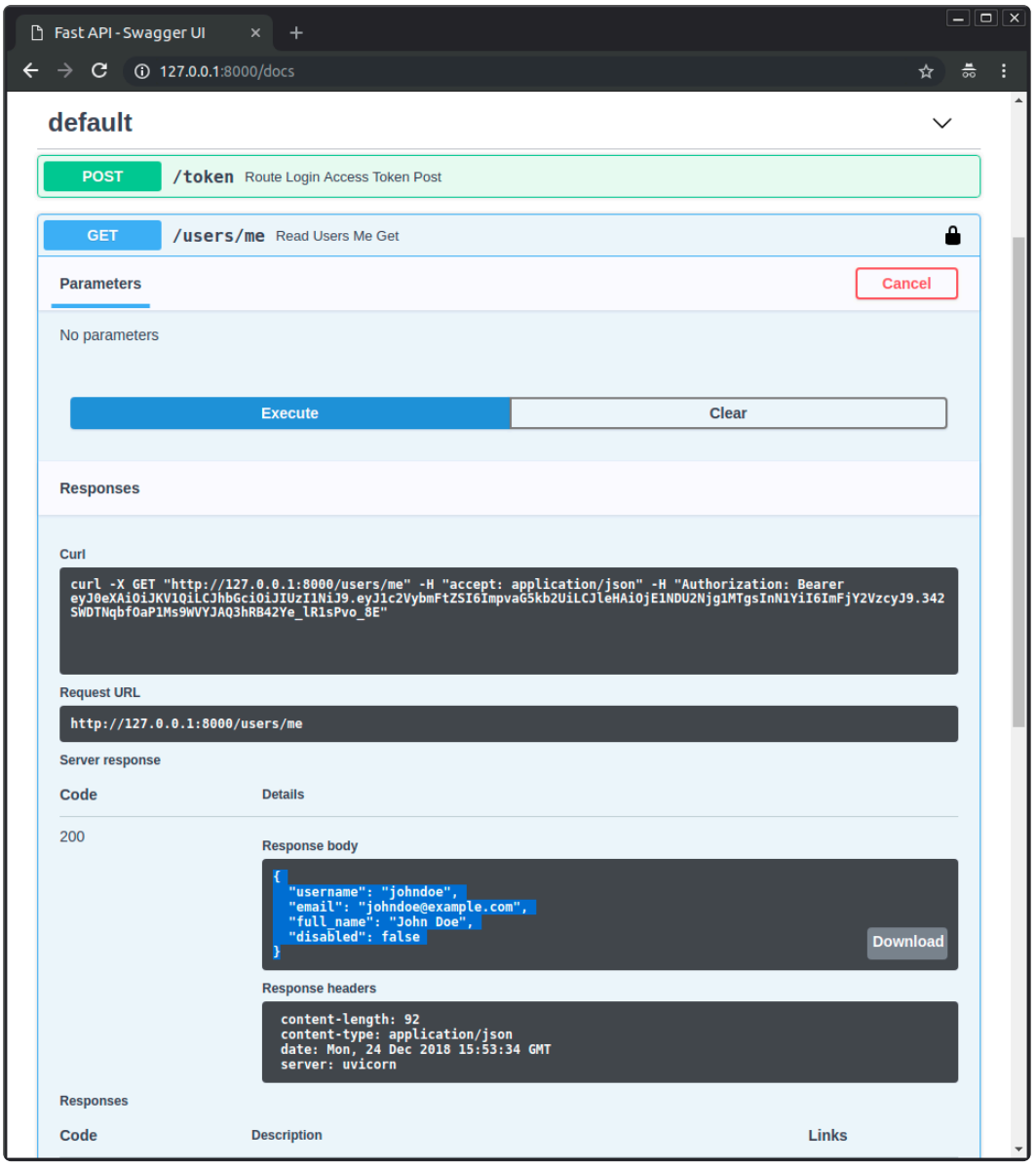
Usuario: `johndoe` Contraseña: `secret`

> **Consejo**
> Observa que en ninguna parte del código está la contraseña en texto claro " `secret` ", solo tenemos la versión con hash.

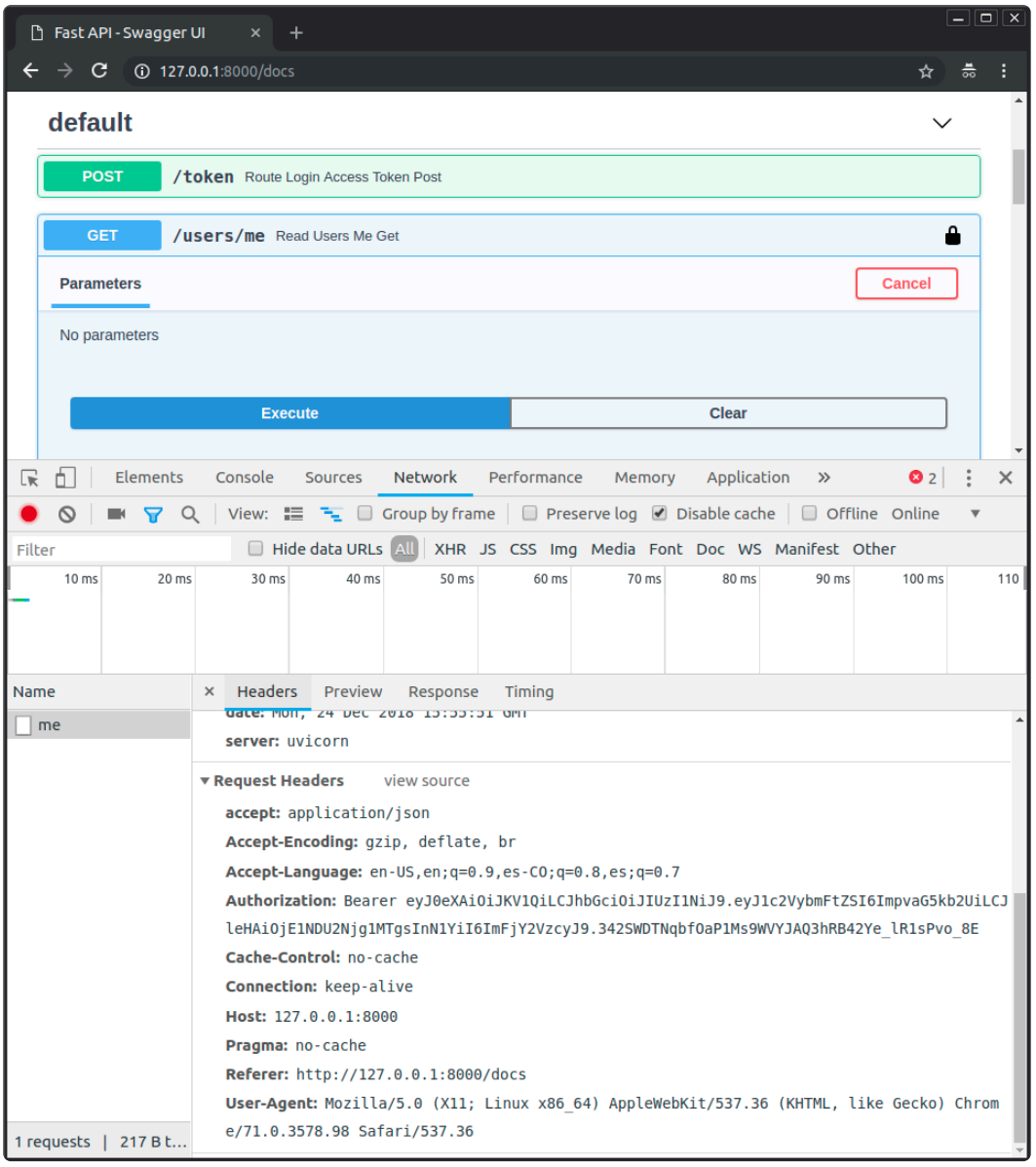


Llama al endpoint `/users/me/`, obtendrás el response como:

```
{
  "username": "johndoe",
  "email": "johndoe@example.com",
  "full_name": "John Doe",
  "disabled": false
}
```



Si abres las herramientas de desarrollador, podrías ver cómo los datos enviados solo incluyen el token, la contraseña solo se envía en la primera request para autenticar al usuario y obtener ese token de acceso, pero no después:



Nota

Observa el header `Authorization`, con un valor que comienza con `Bearer`.

Uso avanzado con scopes

OAuth2 tiene la noción de "scopes".

Puedes usarlos para agregar un conjunto específico de permisos a un token JWT.

Luego, puedes darle este token directamente a un usuario o a un tercero, para interactuar con tu API con un conjunto de restricciones.

Puedes aprender cómo usarlos y cómo están integrados en **FastAPI** más adelante en la **Guía de Usuario Avanzada**.

Resumen

Con lo que has visto hasta ahora, puedes configurar una aplicación **FastAPI** segura usando estándares como OAuth2 y JWT.

En casi cualquier framework el manejo de la seguridad se convierte en un tema bastante complejo rápidamente.

Muchos paquetes que lo simplifican tienen que hacer muchos compromisos con el modelo de datos, la base de datos y las funcionalidades disponibles. Y algunos de estos paquetes que simplifican las cosas demasiado en realidad tienen fallos de seguridad en el fondo.

FastAPI no hace ningún compromiso con ninguna base de datos, modelo de datos o herramienta.

Te da toda la flexibilidad para elegir aquellas que se ajusten mejor a tu proyecto.

Y puedes usar directamente muchos paquetes bien mantenidos y ampliamente usados como `pwdlib` y `PyJWT`, porque **FastAPI** no requiere mecanismos complejos para integrar paquetes externos.

Pero te proporciona las herramientas para simplificar el proceso tanto como sea posible sin comprometer la flexibilidad, la robustez o la seguridad.

Y puedes usar e implementar protocolos seguros y estándar, como OAuth2 de una manera relativamente simple.

Puedes aprender más en la **Guía de Usuario Avanzada** sobre cómo usar "scopes" de OAuth2, para un sistema de

permisos más detallado, siguiendo estos mismos estándares. OAuth2 con scopes es el mecanismo utilizado por muchos grandes proveedores de autenticación, como Facebook, Google, GitHub, Microsoft, X (Twitter), etc. para autorizar aplicaciones de terceros para interactuar con sus APIs en nombre de sus usuarios.

- # Middleware

Puedes añadir middleware a las aplicaciones de **FastAPI**.

Un "middleware" es una función que trabaja con cada **request** antes de que sea procesada por cualquier *path operation* específica. Y también con cada **response** antes de devolverla.

- Toma cada **request** que llega a tu aplicación.
- Puede entonces hacer algo a esa **request** o ejecutar cualquier código necesario.
- Luego pasa la **request** para que sea procesada por el resto de la aplicación (por alguna *path operation*).
- Después toma la **response** generada por la aplicación (por alguna *path operation*).
- Puede hacer algo a esa **response** o ejecutar cualquier código necesario.
- Luego devuelve la **response**.

> **Detalles Técnicos**
> Si tienes dependencias con `yield`, el código de salida se ejecutará *después* del middleware.
>
> Si hubiera tareas en segundo plano (cubiertas en la sección Tareas en segundo plano, lo verás más adelante), se ejecutarán *después* de todo el middleware.

Crear un middleware

Para crear un middleware usas el decorador `@app.middleware("http")` encima de una función.

La función middleware recibe:

- La `request`.
- Una función `call_next` que recibirá la `request` como parámetro.
    - Esta función pasará la `request` a la correspondiente *path operation*.
        - Luego devuelve la `response` generada por la correspondiente *path operation*.
- Puedes entonces modificar aún más la `response` antes de devolverla.

```Python
import time

from fastapi import FastAPI, Request

app = FastAPI()

@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start_time = time.perf_counter()
    response = await call_next(request)
    process_time = time.perf_counter() - start_time
    response.headers["X-Process-Time"] = str(process_time)
    return response
```

Consejo

Ten en cuenta que los custom proprietary headers se pueden añadir usando el prefijo `X-`.

Pero si tienes custom headers que deseas que un cliente en un navegador pueda ver, necesitas añadirlos a tus configuraciones de CORS (CORS (Cross-Origin Resource Sharing)) usando el parámetro `expose_headers` documentado en la documentación de CORS de Starlette.

Detalles Técnicos

También podrías usar `from starlette.requests import Request`.

FastAPI lo proporciona como una conveniencia para ti, el desarrollador. Pero viene directamente de Starlette.

Antes y después de la response

Puedes añadir código que se ejecute con la `request`, antes de que cualquier *path operation* la reciba.

Y también después de que se genere la `response`, antes de devolverla.

Por ejemplo, podrías añadir un custom header `X-Process-Time` que contenga el tiempo en segundos que tomó procesar la request y generar una response:

```
import time

from fastapi import FastAPI, Request

app = FastAPI()

@app.middleware("http")
async def add_process_time_header(request: Request,
call_next):
    start_time = time.perf_counter()
    response = await call_next(request)
    process_time = time.perf_counter() - start_time
    response.headers["X-Process-Time"] = str(process_time)
    return response
```

Consejo

Aquí usamos `time.perf_counter()` en lugar de `time.time()` porque puede ser más preciso para estos casos de uso.

Orden de ejecución con múltiples middlewares

Cuando añades múltiples middlewares usando ya sea el decorador `@app.middleware()` o el método `app.add_middleware()`, cada nuevo middleware envuelve la aplicación, formando un stack. El último middleware añadido es el *más externo*, y el primero es el *más interno*.

En el camino de la request, el middleware *más externo* se ejecuta primero.

En el camino de la response, se ejecuta al final.

Por ejemplo:

```
app.add_middleware(MiddlewareA)
app.add_middleware(MiddlewareB)
```

Esto da como resultado el siguiente orden de ejecución:

- **Request**: MiddlewareB → MiddlewareA → ruta
- **Response**: ruta → MiddlewareA → MiddlewareB

Este comportamiento de apilamiento asegura que los middlewares se ejecuten en un orden predecible y controlable.

Otros middlewares

Más adelante puedes leer sobre otros middlewares en la Guía del Usuario Avanzado: Middleware Avanzado.

Leerás sobre cómo manejar CORS con un middleware en la siguiente sección.

CORS (Cross-Origin Resource Sharing)

CORS o "Cross-Origin Resource Sharing" se refiere a situaciones en las que un frontend que se ejecuta en un navegador tiene código JavaScript que se comunica con un backend, y el backend está en un "origen" diferente al frontend.

Origen

Un origen es la combinación de protocolo (`http` , `https`), dominio (`myapp.com` , `localhost` , `localhost.tiangolo.com`) y puerto (`80` , `443` , `8080`).

Así que, todos estos son orígenes diferentes:

- `http://localhost`
- `https://localhost`
- `http://localhost:8080`

Aunque todos están en `localhost` , usan protocolos o puertos diferentes, por lo tanto, son "orígenes" diferentes.

Pasos

Entonces, digamos que tienes un frontend corriendo en tu navegador en `http://localhost:8080` , y su JavaScript está tratando de comunicarse con un backend corriendo en `http://localhost` (porque no especificamos un puerto, el navegador asumirá el puerto por defecto `80`).

Entonces, el navegador enviará un request HTTP `OPTIONS` al backend `:80` , y si el backend envía los headers apropiados autorizando la comunicación desde este origen diferente (`http://localhost:8080`), entonces el navegador `:8080` permitirá que el JavaScript en el frontend envíe su request al backend `:80` .

Para lograr esto, el backend `:80` debe tener una lista de "orígenes permitidos".

En este caso, la lista tendría que incluir `http://localhost:8080` para que el frontend `:8080` funcione correctamente.

Comodines

También es posible declarar la lista como `"*"` (un "comodín") para decir que todos están permitidos.

Pero eso solo permitirá ciertos tipos de comunicación, excluyendo todo lo que implique credenciales: Cookies, headers de autorización como los utilizados con Bearer Tokens, etc.

Así que, para que todo funcione correctamente, es mejor especificar explícitamente los orígenes permitidos.

Usa CORSMiddleware

Puedes configurarlo en tu aplicación **FastAPI** usando el `CORSMiddleware`.

- Importa `CORSMiddleware`.
- Crea una lista de orígenes permitidos (como strings).
- Agrégalo como un "middleware" a tu aplicación **FastAPI**.

También puedes especificar si tu backend permite:

- Credenciales (headers de autorización, cookies, etc).
- Métodos HTTP específicos (`POST`, `PUT`) o todos ellos con el comodín `"*"`.
- Headers HTTP específicos o todos ellos con el comodín `"*"`.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

origins = [
    "http://localhost.tiangolo.com",
    "https://localhost.tiangolo.com",
    "http://localhost",
    "http://localhost:8080",
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
```

```
    allow_headers=["*"],
)

@app.get("/")
async def main():
    return {"message": "Hello World"}
```

Los parámetros predeterminados utilizados por la implementación de `CORSMiddleware` son restrictivos por defecto, por lo que necesitarás habilitar explícitamente orígenes, métodos o headers particulares para que los navegadores estén permitidos de usarlos en un contexto de Cross-Domain.

Se admiten los siguientes argumentos:

- `allow_origins` - Una lista de orígenes que deberían estar permitidos para hacer requests cross-origin. Por ejemplo, `['https://example.org', 'https://www.example.org']`. Puedes usar `['*']` para permitir cualquier origen.
- `allow_origin_regex` - Una cadena regex para coincidir con orígenes que deberían estar permitidos para hacer requests cross-origin. por ejemplo, `'https://.*\.example\.org'`.
- `allow_methods` - Una lista de métodos HTTP que deberían estar permitidos para requests cross-origin. Por defecto es `['GET']`. Puedes usar `['*']` para permitir todos los métodos estándar.
- `allow_headers` - Una lista de headers de request HTTP que deberían estar soportados para requests cross-origin. Por defecto es `[]`. Puedes usar `['*']` para permitir todos los headers. Los headers `Accept`, `Accept-Language`, `Content-Language` y `Content-Type` siempre están permitidos para requests CORS simples.
- `allow_credentials` - Indica que las cookies deberían estar soportadas para requests cross-origin. Por defecto es `False`. Ninguno de `allow_origins`, `allow_methods` y `allow_headers` puede establecerse a `['*']` si `allow_credentials` está

configurado a `True`. Todos deben ser especificados explícitamente.

- `expose_headers` - Indica cualquier header de response que debería ser accesible para el navegador. Por defecto es `[]`.
- `max_age` - Establece un tiempo máximo en segundos para que los navegadores almacenen en caché los responses CORS. Por defecto es `600`.

El middleware responde a dos tipos particulares de request HTTP...

Requests de preflight CORS

Estos son cualquier request `OPTIONS` con headers `Origin` y `Access-Control-Request-Method`.

En este caso, el middleware interceptará el request entrante y responderá con los headers CORS adecuados, y un response `200` o `400` con fines informativos.

Requests simples

Cualquier request con un header `Origin`. En este caso, el middleware pasará el request a través de lo normal, pero incluirá los headers CORS adecuados en el response.

Más info

Para más información sobre CORS, revisa la documentación de CORS de Mozilla.

> **Detalles Técnicos**
> También podrías usar `from starlette.middleware.cors import CORSMiddleware`.
>
> **FastAPI** proporciona varios middlewares en `fastapi.middleware` como una conveniencia para ti, el

desarrollador. Pero la mayoría de los middlewares disponibles provienen directamente de Starlette.

- # Bases de Datos SQL (Relacionales)

FastAPI no requiere que uses una base de datos SQL (relacional). Pero puedes utilizar **cualquier base de datos** que desees.

Aquí veremos un ejemplo usando SQLModel.

SQLModel está construido sobre SQLAlchemy y Pydantic. Fue creado por el mismo autor de **FastAPI** para ser la combinación perfecta para aplicaciones de FastAPI que necesiten usar **bases de datos SQL**.

> **Consejo**
> Puedes usar cualquier otro paquete de bases de datos SQL o NoSQL que quieras (en algunos casos llamadas "ORMs"), FastAPI no te obliga a usar nada.

Como SQLModel se basa en SQLAlchemy, puedes usar fácilmente **cualquier base de datos soportada** por SQLAlchemy (lo que las hace también soportadas por SQLModel), como:

- PostgreSQL
- MySQL
- SQLite
- Oracle
- Microsoft SQL Server, etc.

En este ejemplo, usaremos **SQLite**, porque utiliza un solo archivo y Python tiene soporte integrado. Así que puedes copiar este ejemplo y ejecutarlo tal cual.

Más adelante, para tu aplicación en producción, es posible que desees usar un servidor de base de datos como **PostgreSQL**.

> **Consejo**

Hay un generador de proyectos oficial con **FastAPI** y **PostgreSQL** que incluye un frontend y más herramientas: https://github.com/fastapi/full-stack-fastapi-template

Este es un tutorial muy simple y corto, si deseas aprender sobre bases de datos en general, sobre SQL o más funcionalidades avanzadas, ve a la documentación de SQLModel.

Instalar SQLModel

Añade `sqlmodel` a tu proyecto:

```
uv add sqlmodel
```

Crear la App con un Solo Modelo

Primero crearemos la versión más simple de la aplicación con un solo modelo de **SQLModel**.

Más adelante la mejoraremos aumentando la seguridad y versatilidad con **múltiples modelos** a continuación. 😳

Crear Modelos

Importa `SQLModel` y crea un modelo de base de datos:

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel,
create_engine, select

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str
```

```
# Code below omitted
```

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel,
create_engine, select

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)
    secret_name: str

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url,
connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

@app.post("/heroes/")
def create_hero(hero: Hero, session: SessionDep) -> Hero:
    session.add(hero)
    session.commit()
```

```
    session.refresh(hero)
    return hero

@app.get("/heroes/")
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,
) -> list[Hero]:
    heroes =
session.exec(select(Hero).offset(offset).limit(limit)).all
()
    return heroes

@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: SessionDep) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero
not found")
    return hero

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero
not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

La clase `Hero` es muy similar a un modelo de Pydantic (de hecho, en el fondo, realmente *es un modelo de Pydantic*).

Hay algunas diferencias:

- `table=True` le dice a SQLModel que este es un *modelo de tabla*, que debe representar una **tabla** en la base de datos SQL, no es solo un *modelo de datos* (como lo sería cualquier otra clase regular de Pydantic).

- `Field(primary_key=True)` le dice a SQLModel que `id` es la **clave primaria** en la base de datos SQL (puedes aprender más sobre claves primarias de SQL en la documentación de SQLModel).
  Nota: Usamos `int | None` para el campo de clave primaria para que en el código Python podamos *crear un objeto sin un `id`* (`id=None`), asumiendo que la base de datos lo *generará al guardar*. SQLModel entiende que la base de datos proporcionará el `id` y *define la columna como un `INTEGER` no nulo* en el esquema de la base de datos. Consulta la documentación de SQLModel sobre claves primarias para más detalles.
- `Field(index=True)` le dice a SQLModel que debe crear un **índice SQL** para esta columna, lo que permitirá búsquedas más rápidas en la base de datos cuando se lean datos filtrados por esta columna.
  SQLModel sabrá que algo declarado como `str` será una columna SQL de tipo `TEXT` (o `VARCHAR` , dependiendo de la base de datos).

Crear un Engine

Un `engine` de SQLModel (en el fondo, realmente es un `engine` de SQLAlchemy) es lo que **mantiene las conexiones** a la base de datos.

Tendrías **un solo objeto** `engine` para todo tu código para conectar a la misma base de datos.

```
# Code above omitted

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url,
connect_args=connect_args)
```

```
# Code below omitted
```

Usar `check_same_thread=False` permite a FastAPI usar la misma base de datos SQLite en diferentes hilos. Esto es necesario ya que **una sola request** podría usar **más de un hilo** (por ejemplo, en dependencias).

No te preocupes, con la forma en que está estructurado el código, nos aseguraremos de usar **una sola *session* de SQLModel por request** más adelante, esto es realmente lo que intenta lograr el `check_same_thread`.

Crear las Tablas

Luego añadimos una función que usa `SQLModel.metadata.create_all(engine)` para **crear las tablas** para todos los *modelos de tabla*.

```
# Code above omitted

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

# Code below omitted
```

Crear una Dependencia de Session

Una `Session` es lo que almacena los **objetos en memoria** y lleva un seguimiento de cualquier cambio necesario en los datos, luego **usa el** `engine` para comunicarse con la base de datos.

Crearemos una **dependencia de FastAPI** con `yield` que proporcionará una nueva `Session` para cada request. Esto es lo que asegura que usemos una sola session por request.

Luego creamos una dependencia `Annotated` `SessionDep` para simplificar el resto del código que usará esta dependencia.

```
# Code above omitted

def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]

# Code below omitted
```

Crear Tablas de Base de Datos al Arrancar

Crearemos las tablas de la base de datos cuando arranque la aplicación.

```
# Code above omitted

app = FastAPI()

@app.on_event("startup")
def on_startup():
    create_db_and_tables()

# Code below omitted
```

Aquí creamos las tablas en un evento de inicio de la aplicación.

Para producción probablemente usarías un script de migración que se ejecuta antes de iniciar tu aplicación.

> **Consejo**
> SQLModel tendrá utilidades de migración envolviendo Alembic, pero por ahora, puedes usar Alembic directamente.

Crear un Hero

Debido a que cada modelo de SQLModel también es un modelo de Pydantic, puedes usarlo en las mismas **anotaciones de tipos** que

podrías usar en modelos de Pydantic.

Por ejemplo, si declaras un parámetro de tipo `Hero`, será leído desde el **JSON body**.

De la misma manera, puedes declararlo como el **tipo de retorno** de la función, y luego la forma de los datos aparecerá en la interfaz automática de documentación de la API.

```
# Code above omitted

@app.post("/heroes/")
def create_hero(hero: Hero, session: SessionDep) -> Hero:
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero

# Code below omitted
```

Aquí usamos la dependencia `SessionDep` (una `Session`) para añadir el nuevo `Hero` a la instance `Session`, comiteamos los cambios a la base de datos, refrescamos los datos en el `hero` y luego lo devolvemos.

Leer Heroes

Podemos **leer** `Hero` s de la base de datos usando un `select()`. Podemos incluir un `limit` y `offset` para paginar los resultados.

```
# Code above omitted

@app.get("/heroes/")
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,
) -> list[Hero]:
    heroes =
session.exec(select(Hero).offset(offset).limit(limit)).all
```

```
()
    return heroes

# Code below omitted
```

Leer Un Hero

Podemos **leer** un único `Hero`.

```
# Code above omitted

@app.get("/heroes/{hero_id}")
def read_hero(hero_id: int, session: SessionDep) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero
not found")
    return hero

# Code below omitted
```

Eliminar un Hero

También podemos **eliminar** un `Hero`.

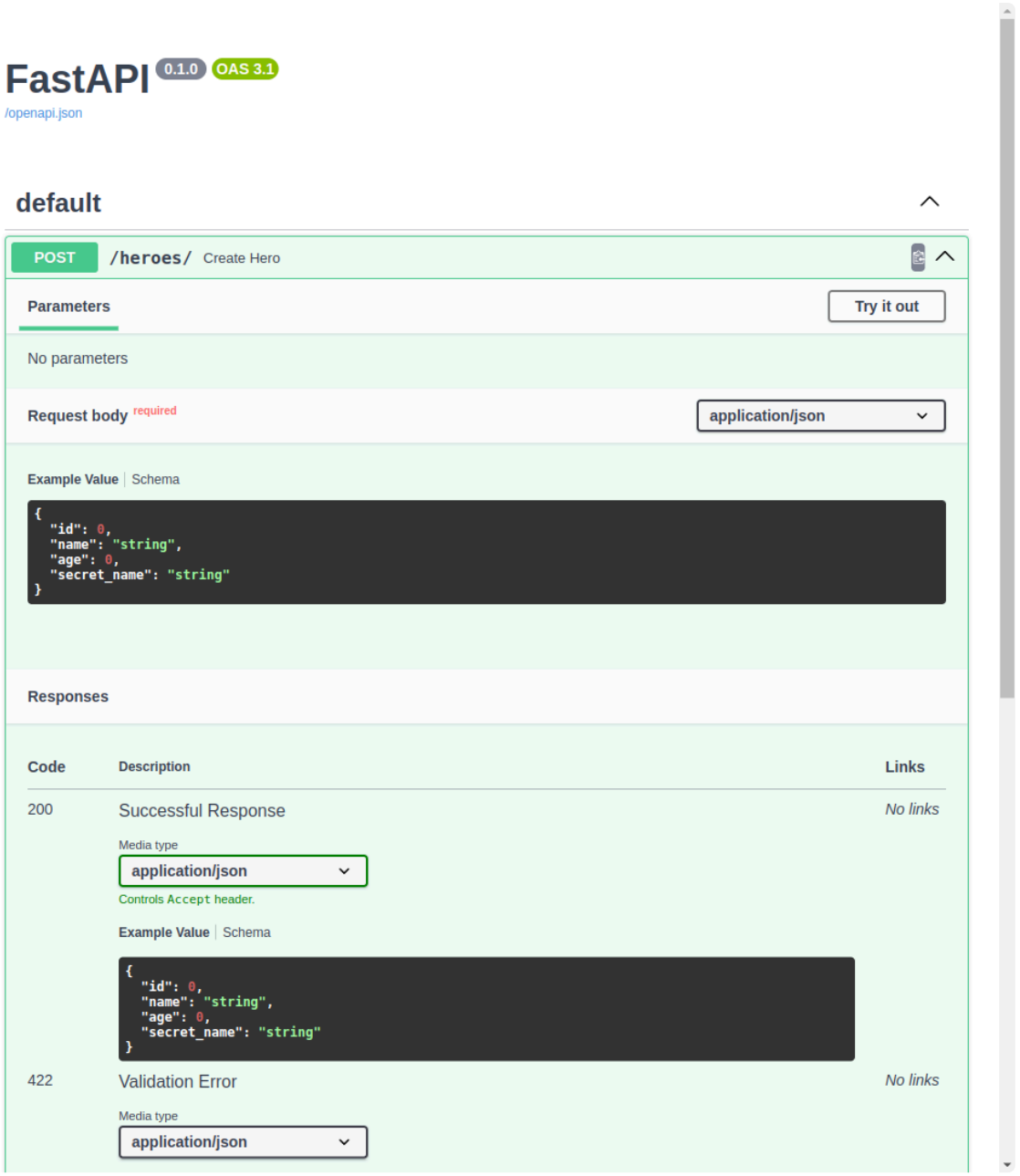
```
# Code above omitted

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero
not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

Ejecutar la App

Puedes ejecutar la aplicación:

Luego dirígete a la interfaz de `/docs`, verás que **FastAPI** está usando estos **modelos** para **documentar** la API, y los usará para **serializar** y **validar** los datos también.



Actualizar la App con Múltiples Modelos

Ahora vamos a **refactorizar** un poco esta aplicación para aumentar la **seguridad** y la **versatilidad**.

Si revisas la aplicación anterior, en la interfaz verás que, hasta ahora, permite al cliente decidir el `id` del `Hero` a crear.

No deberíamos permitir que eso suceda, podrían sobrescribir un `id` que ya tenemos asignado en la base de datos. Decidir el `id` debería ser tarea del **backend** o la **base de datos**, **no del cliente**.

Además, creamos un `secret_name` para el héroe, pero hasta ahora, lo estamos devolviendo en todas partes, eso no es muy **secreto**...

Arreglaremos estas cosas añadiendo unos **modelos extra**. Aquí es donde SQLModel brillará.

Crear Múltiples Modelos

En **SQLModel**, cualquier clase de modelo que tenga `table=True` es un **modelo de tabla**.

Y cualquier clase de modelo que no tenga `table=True` es un **modelo de datos**, estos son en realidad solo modelos de Pydantic (con un par de pequeñas funcionalidades extra).

Con SQLModel, podemos usar **herencia** para **evitar duplicar** todos los campos en todos los casos.

HeroBase - la clase base

Comencemos con un modelo `HeroBase` que tiene todos los **campos que son compartidos** por todos los modelos:

- `name`
- `age`

```
# Code above omitted

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

# Code below omitted
```

Hero - el modelo de tabla

Luego, crearemos `Hero`, el *modelo de tabla* real, con los **campos extra** que no siempre están en los otros modelos:

- `id`
- `secret_name`

Debido a que `Hero` hereda de `HeroBase`, **también** tiene los **campos** declarados en `HeroBase`, por lo que todos los campos para `Hero` son:

- `id`
- `name`
- `age`
- `secret_name`

```
# Code above omitted

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

# Code below omitted
```

HeroPublic - el modelo de datos público

A continuación, creamos un modelo `HeroPublic`, este es el que será **devuelto** a los clientes de la API.

Tiene los mismos campos que `HeroBase`, por lo que no incluirá `secret_name`.

Por fin, la identidad de nuestros héroes está protegida!

También vuelve a declarar `id: int`. Al hacer esto, estamos haciendo un **contrato** con los clientes de la API, para que siempre puedan esperar que el `id` esté allí y sea un `int` (nunca será `None`).

> **Consejo**
>
> Tener el modelo de retorno asegurando que un valor siempre esté disponible y siempre sea `int` (no `None`) es muy útil para los clientes de la API, pueden escribir código mucho más simple teniendo esta certeza.
>
> Además, los **clientes generados automáticamente** tendrán interfaces más simples, para que los desarrolladores que se comuniquen con tu API puedan tener una experiencia mucho mejor trabajando con tu API.

Todos los campos en `HeroPublic` son los mismos que en `HeroBase`, con `id` declarado como `int` (no `None`):

- `id`
- `name`
- `age`

```
# Code above omitted

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

# Code below omitted
```

HeroCreate - el modelo de datos para crear un héroe

Ahora creamos un modelo `HeroCreate`, este es el que **validará** los datos de los clientes.

Tiene los mismos campos que `HeroBase`, y también tiene `secret_name`.

Ahora, cuando los clientes **crean un nuevo héroe**, enviarán el `secret_name`, se almacenará en la base de datos, pero esos nombres secretos no se devolverán en la API a los clientes.

> **Consejo**
> Esta es la forma en la que manejarías **contraseñas**. Recíbelas, pero no las devuelvas en la API.
>
> También **hashea** los valores de las contraseñas antes de almacenarlos, **nunca los almacenes en texto plano**.

Los campos de `HeroCreate` son:

- `name`
- `age`
- `secret_name`

```
# Code above omitted

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str
```

```
# Code below omitted
```

HeroUpdate - el modelo de datos para actualizar un héroe

No teníamos una forma de **actualizar un héroe** en la versión anterior de la aplicación, pero ahora con **múltiples modelos**, podemos hacerlo.

El *modelo de datos* `HeroUpdate` es algo especial, tiene **todos los mismos campos** que serían necesarios para crear un nuevo héroe, pero todos los campos son **opcionales** (todos tienen un valor por defecto). De esta forma, cuando actualices un héroe, puedes enviar solo los campos que deseas actualizar.

Debido a que todos los **campos realmente cambian** (el tipo ahora incluye `None` y ahora tienen un valor por defecto de `None`), necesitamos **volver a declararlos**.

Realmente no necesitamos heredar de `HeroBase` porque estamos volviendo a declarar todos los campos. Lo dejaré heredando solo por consistencia, pero esto no es necesario. Es más una cuestión de gusto personal. 🤷

Los campos de `HeroUpdate` son:

- `name`
- `age`
- `secret_name`

```
# Code above omitted

class HeroBase(SQLModel):
    name: str = Field(index=True)
    age: int | None = Field(default=None, index=True)

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
```

```
    secret_name: str

class HeroPublic(HeroBase):
    id: int

class HeroCreate(HeroBase):
    secret_name: str

class HeroUpdate(HeroBase):
    name: str | None = None
    age: int | None = None
    secret_name: str | None = None

# Code below omitted
```

Crear con HeroCreate y devolver un HeroPublic

Ahora que tenemos **múltiples modelos**, podemos actualizar las partes de la aplicación que los usan.

Recibimos en la request un *modelo de datos* `HeroCreate`, y a partir de él, creamos un *modelo de tabla* `Hero`.

Este nuevo *modelo de tabla* `Hero` tendrá los campos enviados por el cliente, y también tendrá un `id` generado por la base de datos.

Luego devolvemos el mismo *modelo de tabla* `Hero` tal cual desde la función. Pero como declaramos el `response_model` con el *modelo de datos* `HeroPublic`, **FastAPI** usará `HeroPublic` para validar y serializar los datos.

```
# Code above omitted

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: SessionDep):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero
```

```
# Code below omitted
```

Leer Heroes con HeroPublic

Podemos hacer lo mismo que antes para **leer** `Hero` s, nuevamente, usamos `response_model=list[HeroPublic]` para asegurar que los datos se validen y serialicen correctamente.

```
# Code above omitted

@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,
):
    heroes =
session.exec(select(Hero).offset(offset).limit(limit)).all
()
    return heroes

# Code below omitted
```

Leer Un Hero con HeroPublic

Podemos **leer** un único héroe:

```
# Code above omitted

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero
not found")
    return hero
```

```
# Code below omitted
```

Actualizar un Hero con HeroUpdate

Podemos **actualizar un héroe**. Para esto usamos una operación HTTP `PATCH`.

Y en el código, obtenemos un `dict` con todos los datos enviados por el cliente, **solo los datos enviados por el cliente**, excluyendo cualquier valor que estaría allí solo por ser valores por defecto. Para hacerlo usamos `exclude_unset=True`. Este es el truco principal.

Luego usamos `hero_db.sqlmodel_update(hero_data)` para actualizar el `hero_db` con los datos de `hero_data`.

```
# Code above omitted

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate, session:
SessionDep):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero
not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

# Code below omitted
```

Eliminar un Hero de Nuevo

Eliminar un héroe se mantiene prácticamente igual.

No satisfaremos el deseo de refactorizar todo en este punto.

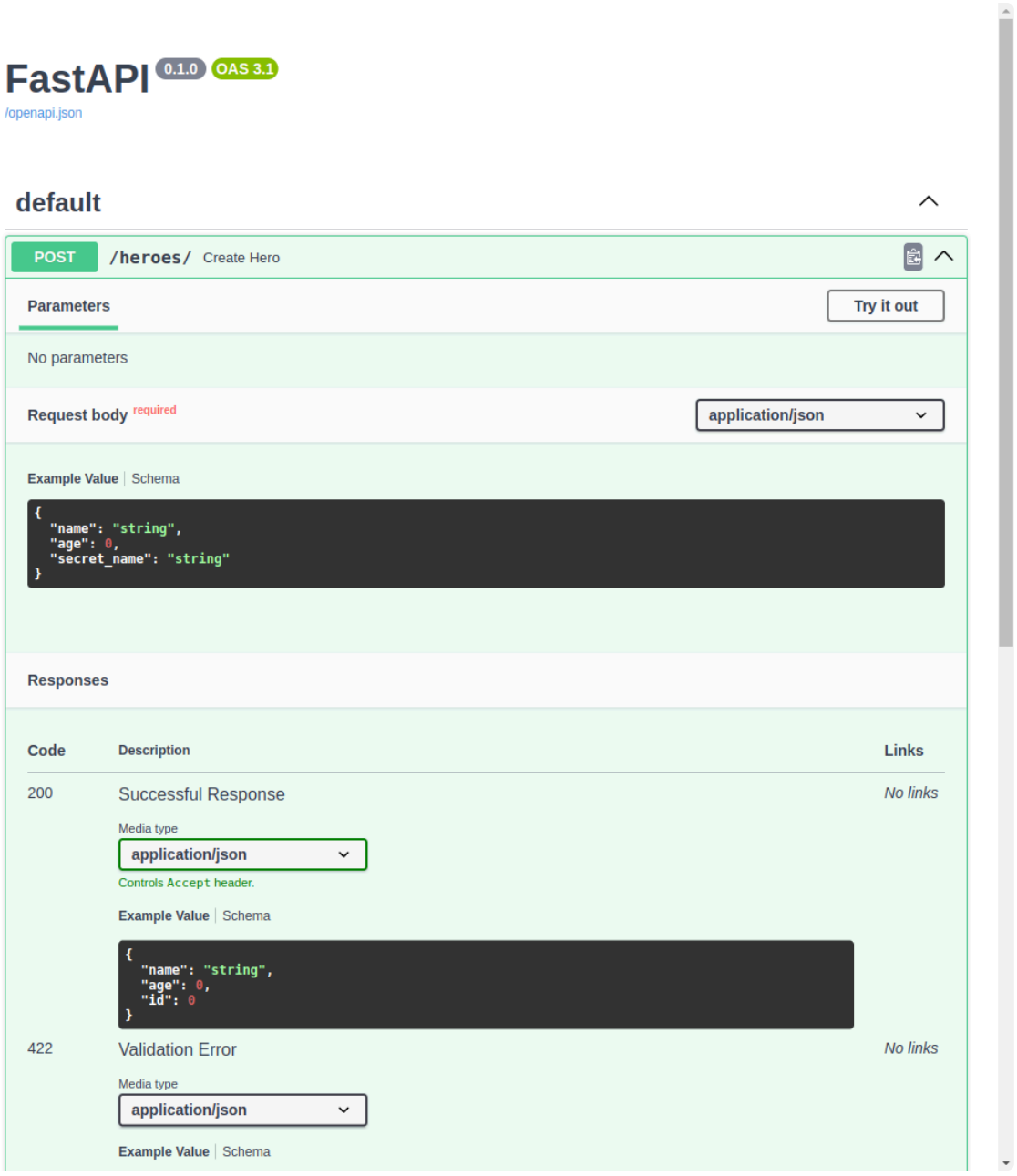
```
# Code above omitted

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero
not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

Ejecutar la App de Nuevo

Puedes ejecutar la aplicación de nuevo:

Si vas a la interfaz de `/docs` de la API, verás que ahora está actualizada, y no esperará recibir el `id` del cliente al crear un héroe, etc.



Resumen

Puedes usar **SQLModel** para interactuar con una base de datos SQL y simplificar el código con *modelos de datos* y *modelos de tablas*.

Puedes aprender mucho más en la documentación de **SQLModel**, hay un mini tutorial más largo sobre el uso de SQLModel con **FastAPI**.

Aplicaciones más grandes - Múltiples archivos

Si estás construyendo una aplicación o una API web, rara vez podrás poner todo en un solo archivo.

FastAPI proporciona una herramienta conveniente para estructurar tu aplicación manteniendo toda la flexibilidad.

> **Nota**
> Si vienes de Flask, esto sería el equivalente a los Blueprints de Flask.

Un ejemplo de estructura de archivos

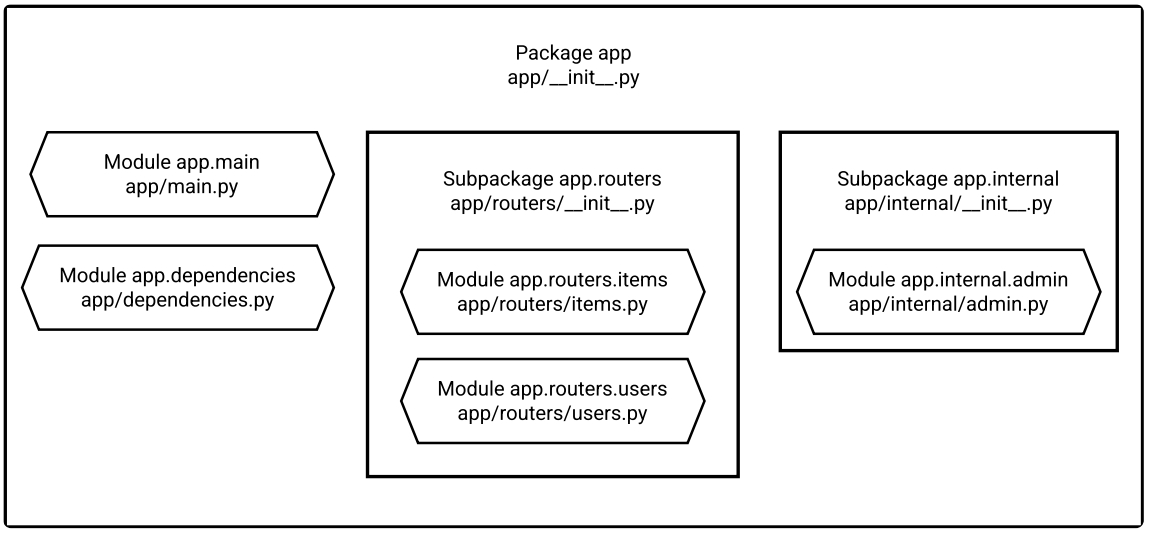
Digamos que tienes una estructura de archivos como esta:

```
.
├── app
│   ├── __init__.py
│   ├── main.py
│   ├── dependencies.py
│   └── routers
│   │   ├── __init__.py
│   │   ├── items.py
│   │   └── users.py
│   └── internal
│       ├── __init__.py
│       └── admin.py
```

> **Consejo**
> Hay varios archivos `__init__.py`: uno en cada directorio o subdirectorio.
>
> Esto es lo que permite importar código de un archivo a otro.
>
> Por ejemplo, en `app/main.py` podrías tener una línea como:
>
> ```
> from app.routers import items
> ```

- El directorio `app` contiene todo. Y tiene un archivo vacío `app/__init__.py`, por lo que es un "paquete de Python" (una colección de "módulos de Python"): `app`.

- Contiene un archivo `app/main.py`. Como está dentro de un paquete de Python (un directorio con un archivo `__init__.py`), es un "módulo" de ese paquete: `app.main`.
- También hay un archivo `app/dependencies.py`, al igual que `app/main.py`, es un "módulo": `app.dependencies`.
- Hay un subdirectorio `app/routers/` con otro archivo `__init__.py`, por lo que es un "subpaquete de Python": `app.routers`.
- El archivo `app/routers/items.py` está dentro de un paquete, `app/routers/`, por lo que es un submódulo: `app.routers.items`.
- Lo mismo con `app/routers/users.py`, es otro submódulo: `app.routers.users`.
- También hay un subdirectorio `app/internal/` con otro archivo `__init__.py`, por lo que es otro "subpaquete de Python": `app.internal`.
- Y el archivo `app/internal/admin.py` es otro submódulo: `app.internal.admin`.



La misma estructura de archivos con comentarios:

```
.
├── app                  # "app" es un paquete de Python
│   ├── __init__.py      # este archivo hace que "app" sea
un "paquete de Python"
│   ├── main.py          # módulo "main", por ejemplo
import app.main
```

```
|   ├── dependencies.py  # módulo "dependencies", por
ejemplo import app.dependencies
|   └── routers          # "routers" es un "subpaquete de
Python"
|   |   ├── __init__.py  # hace que "routers" sea un
"subpaquete de Python"
|   |   ├── items.py     # submódulo "items", por ejemplo
import app.routers.items
|   |   └── users.py     # submódulo "users", por ejemplo
import app.routers.users
|   └── internal         # "internal" es un "subpaquete de
Python"
|       ├── __init__.py  # hace que "internal" sea un
"subpaquete de Python"
|       └── admin.py     # submódulo "admin", por ejemplo
import app.internal.admin
```

APIRouter

Digamos que el archivo dedicado solo a manejar usuarios es el submódulo en `/app/routers/users.py`.

Quieres tener las *path operations* relacionadas con tus usuarios separadas del resto del código, para mantenerlo organizado.

Pero todavía es parte de la misma aplicación/web API de **FastAPI** (es parte del mismo "paquete de Python").

Puedes crear las *path operations* para ese módulo usando `APIRouter`.

Importa APIRouter

Lo importas y creas una "instance" de la misma manera que lo harías con la clase `FastAPI`:

```python
from fastapi import APIRouter

router = APIRouter()
```

```python
@router.get("/users/", tags=["users"])
async def read_users():
    return [{"username": "Rick"}, {"username": "Morty"}]

@router.get("/users/me", tags=["users"])
async def read_user_me():
    return {"username": "fakecurrentuser"}

@router.get("/users/{username}", tags=["users"])
async def read_user(username: str):
    return {"username": username}
```

Path operations con APIRouter

Y luego lo usas para declarar tus *path operations*.

Úsalo de la misma manera que usarías la clase `FastAPI`:

```python
@router.get("/users/", tags=["users"])
...

@router.get("/users/me", tags=["users"])
...

@router.get("/users/{username}", tags=["users"])
...
```

Puedes pensar en `APIRouter` como una clase "mini `FastAPI`".

Se soportan todas las mismas opciones.

Todos los mismos `parameters`, `responses`, `dependencies`, `tags`, etc.

> **Consejo**
> En este ejemplo, la variable se llama `router`, pero puedes nombrarla como quieras.

Vamos a incluir este `APIRouter` en la aplicación principal de `FastAPI`, pero primero, revisemos las dependencias y otro `APIRouter`.

Dependencias

Vemos que vamos a necesitar algunas dependencias usadas en varios lugares de la aplicación.

Así que las ponemos en su propio módulo `dependencies` (`app/dependencies.py`).

Ahora utilizaremos una dependencia simple para leer un header `X-Token` personalizado:

```
from typing import Annotated

from fastapi import Header, HTTPException

async def get_token_header(x_token: Annotated[str,
Header()]):
    if x_token != "fake-super-secret-token":
        raise HTTPException(status_code=400, detail="X-
Token header invalid")

async def get_query_token(token: str):
    if token != "jessica":
        raise HTTPException(status_code=400, detail="No
Jessica token provided")
```

> **Consejo**
> Estamos usando un header inventado para simplificar este ejemplo.
>
> Pero en casos reales obtendrás mejores resultados usando las utilidades de Seguridad integradas.

Otro módulo con APIRouter

Digamos que también tienes los endpoints dedicados a manejar "items" de tu aplicación en el módulo `app/routers/items.py`.

Tienes *path operations* para:

- `/items/`
- `/items/{item_id}`

Es toda la misma estructura que con `app/routers/users.py`.

Pero queremos ser más inteligentes y simplificar un poco el código.

Sabemos que todas las *path operations* en este módulo tienen el mismo:

- Prefijo de path: `/items`.
- `tags`: (solo una etiqueta: `items`).
- `responses` extra.
- `dependencies`: todas necesitan esa dependencia `X-Token` que creamos.

Entonces, en lugar de agregar todo eso a cada *path operation*, podemos agregarlo al `APIRouter`.

```
from fastapi import APIRouter, Depends, HTTPException

from ..dependencies import get_token_header

router = APIRouter(
    prefix="/items",
    tags=["items"],
    dependencies=[Depends(get_token_header)],
    responses={404: {"description": "Not found"}},
)

fake_items_db = {"plumbus": {"name": "Plumbus"}, "gun":
{"name": "Portal Gun"}}

@router.get("/")
async def read_items():
    return fake_items_db
```

```
@router.get("/{item_id}")
async def read_item(item_id: str):
    if item_id not in fake_items_db:
        raise HTTPException(status_code=404, detail="Item
not found")
    return {"name": fake_items_db[item_id]["name"],
"item_id": item_id}

@router.put(
    "/{item_id}",
    tags=["custom"],
    responses={403: {"description": "Operation
forbidden"}},
)
async def update_item(item_id: str):
    if item_id != "plumbus":
        raise HTTPException(
            status_code=403, detail="You can only update
the item: plumbus"
        )
    return {"item_id": item_id, "name": "The great
Plumbus"}
```

Como el path de cada *path operation* tiene que empezar con `/`, como en:

```
@router.get("/{item_id}")
async def read_item(item_id: str):
    ...
```

...el prefijo no debe incluir un `/` final.

Así que, el prefijo en este caso es `/items`.

También podemos agregar una lista de `tags` y `responses` extra que se aplicarán a todas las *path operations* incluidas en este router.

Y podemos agregar una lista de `dependencies` que se añadirá a todas las *path operations* en el router y se ejecutarán/solucionarán

por cada request que les haga.

> **Consejo**
> Nota que, al igual que dependencias en decoradores de *path operations*, ningún valor será pasado a tu *path operation function*.

El resultado final es que los paths de item son ahora:

- `/items/`
- `/items/{item_id}`

...como pretendíamos.

- Serán marcados con una lista de tags que contiene un solo string `"items"`.
    - Estos "tags" son especialmente útiles para los sistemas de documentación interactiva automática (usando OpenAPI).
- Todos incluirán las `responses` predefinidas.
- Todas estas *path operations* tendrán la lista de `dependencies` evaluadas/ejecutadas antes de ellas.
    - Si también declaras dependencias en una *path operation* específica, **también se ejecutarán**.
        - Las dependencias del router se ejecutan primero, luego las `dependencies` en el decorador, y luego las dependencias de parámetros normales.
        - También puedes agregar dependencias de `Security` con `scopes`.

> **Consejo**
> Tener `dependencies` en el `APIRouter` puede ser usado, por ejemplo, para requerir autenticación para un grupo completo de *path operations*. Incluso si las dependencias no son añadidas individualmente a cada una de ellas.

> **Consejo**
> Los parámetros `prefix`, `tags`, `responses`, y `dependencies` son (como en muchos otros casos) solo una funcionalidad de **FastAPI** para ayudarte a evitar la duplicación de código.

Importa las dependencias

Este código vive en el módulo `app.routers.items`, el archivo `app/routers/items.py`.

Y necesitamos obtener la función de dependencia del módulo `app.dependencies`, el archivo `app/dependencies.py`.

Así que usamos un import relativo con `..` para las dependencias:

```
from ..dependencies import get_token_header
```

Cómo funcionan los imports relativos

> **Consejo**
> Si sabes perfectamente cómo funcionan los imports, continúa a la siguiente sección abajo.

Un solo punto `.`, como en:

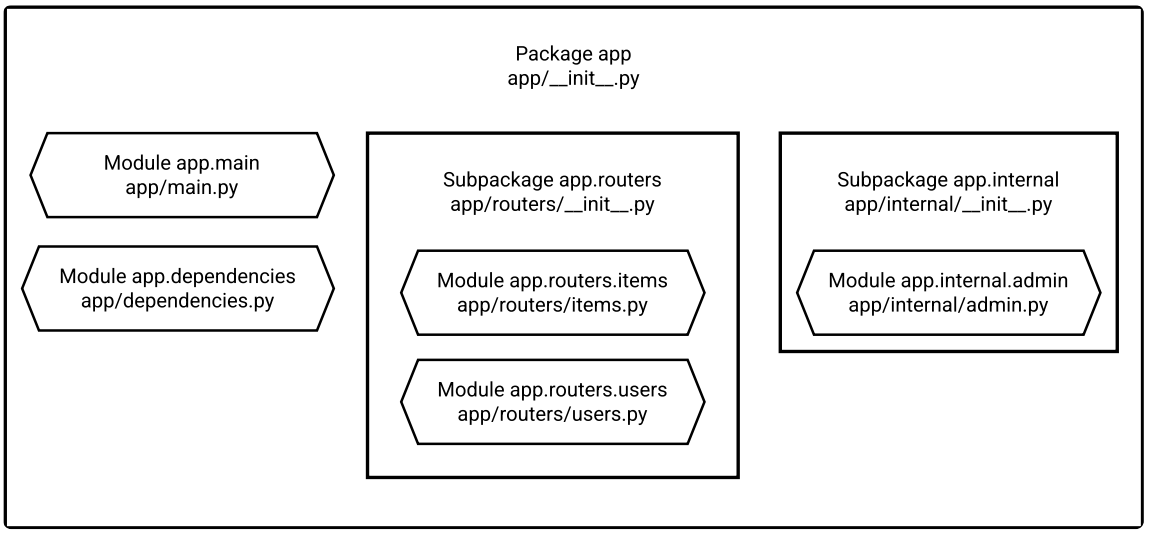
```
from .dependencies import get_token_header
```

significaría:

- Partiendo en el mismo paquete en el que este módulo (el archivo `app/routers/items.py`) habita (el directorio `app/routers/`)...
- busca el módulo `dependencies` (un archivo imaginario en `app/routers/dependencies.py`)...
- y de él, importa la función `get_token_header`.

Pero ese archivo no existe, nuestras dependencias están en un archivo en `app/dependencies.py`.

Recuerda cómo se ve nuestra estructura de aplicación/archivo:



Los dos puntos `..`, como en:

```
from ..dependencies import get_token_header
```

significan:

- Partiendo en el mismo paquete en el que este módulo (el archivo `app/routers/items.py`) habita (el directorio `app/routers/`)...
- ve al paquete padre (el directorio `app/`)...
- y allí, busca el módulo `dependencies` (el archivo en `app/dependencies.py`)...
- y de él, importa la función `get_token_header`.

¡Eso funciona correctamente!

De la misma manera, si hubiéramos usado tres puntos `...`, como en:

```
from ...dependencies import get_token_header
```

eso significaría:

- Partiendo en el mismo paquete en el que este módulo (el archivo `app/routers/items.py`) habita (el directorio `app/routers/`)...
- ve al paquete padre (el directorio `app/`)...
- luego ve al paquete padre de ese paquete (no hay paquete padre, `app` es el nivel superior)...
- y allí, busca el módulo `dependencies` (el archivo en `app/dependencies.py`)...
- y de él, importa la función `get_token_header`.

Eso se referiría a algún paquete arriba de `app/`, con su propio archivo `__init__.py`, etc. Pero no tenemos eso. Así que, eso lanzaría un error en nuestro ejemplo.

Pero ahora sabes cómo funciona, para que puedas usar imports relativos en tus propias apps sin importar cuán complejas sean.

Agrega algunos tags, responses, y dependencies personalizados

No estamos agregando el prefijo `/items` ni los `tags=["items"]` a cada *path operation* porque los hemos añadido al `APIRouter`.

Pero aún podemos agregar *más* `tags` que se aplicarán a una *path operation* específica, y también algunas `responses` extra específicas para esa *path operation*:

```
@router.put(
    "/{item_id}",
    tags=["custom"],
    responses={403: {"description": "Operation
forbidden"}},
)
...
```

> **Consejo**
>
> Esta última path operation tendrá la combinación de tags: `["items", "custom"]`.
>
> Y también tendrá ambas responses en la documentación, una para `404` y otra para `403`.

El FastAPI principal

Ahora, veamos el módulo en `app/main.py`.

Aquí es donde importas y usas la clase `FastAPI`.

Este será el archivo principal en tu aplicación que conecta todo.

Y como la mayor parte de tu lógica ahora vivirá en su propio módulo específico, el archivo principal será bastante simple.

Importa FastAPI

Importas y creas una clase `FastAPI` como normalmente.

Y podemos incluso declarar dependencias globales que se combinarán con las dependencias para cada `APIRouter`:

```
from fastapi import Depends, FastAPI

from .dependencies import get_query_token,
get_token_header
from .internal import admin
from .routers import items, users

app = FastAPI(dependencies=[Depends(get_query_token)])

app.include_router(users.router)
app.include_router(items.router)
app.include_router(
    admin.router,
    prefix="/admin",
    tags=["admin"],
    dependencies=[Depends(get_token_header)],
```

```
    responses={418: {"description": "I'm a teapot"}},
)

@app.get("/")
async def root():
    return {"message": "Hello Bigger Applications!"}
```

Importa el APIRouter

Ahora importamos los otros submódulos que tienen `APIRouter` s:

```
from .dependencies import get_query_token,
get_token_header
from .internal import admin
from .routers import items, users
```

Como los archivos `app/routers/users.py` y `app/routers/items.py` son submódulos que son parte del mismo paquete de Python `app`, podemos usar un solo punto `.` para importarlos usando "imports relativos".

Cómo funciona el import

La sección:

```
from .routers import items, users
```

significa:

- Partiendo en el mismo paquete en el que este módulo (el archivo `app/main.py`) habita (el directorio `app/`)...
- busca el subpaquete `routers` (el directorio en `app/routers/`)...
- y de él, importa el submódulo `items` (el archivo en `app/routers/items.py`) y `users` (el archivo en

`app/routers/users.py`)...

El módulo `items` tendrá una variable `router` (`items.router`). Este es el mismo que creamos en el archivo `app/routers/items.py`, es un objeto `APIRouter`.

Y luego hacemos lo mismo para el módulo `users`.

También podríamos importarlos así:

```
from app.routers import items, users
```

> **Nota**
> La primera versión es un "import relativo":
>
> ```
> from .routers import items, users
> ```
>
> La segunda versión es un "import absoluto":
>
> ```
> from app.routers import items, users
> ```
>
> Para aprender más sobre Paquetes y Módulos de Python, lee la documentación oficial de Python sobre Módulos.

Evita colisiones de nombres

Estamos importando el submódulo `items` directamente, en lugar de importar solo su variable `router`.

Esto se debe a que también tenemos otra variable llamada `router` en el submódulo `users`.

Si hubiéramos importado uno después del otro, como:

```
from .routers.items import router
from .routers.users import router
```

el `router` de `users` sobrescribiría el de `items` y no podríamos usarlos al mismo tiempo.

Así que, para poder usar ambos en el mismo archivo, importamos los submódulos directamente:

```
from .routers import items, users
```

Incluye los APIRouters para users y items

Ahora, incluyamos los `router` s de los submódulos `users` y `items` :

```
app.include_router(users.router)
app.include_router(items.router)
```

> **Nota**
> `users.router` contiene el `APIRouter` dentro del archivo `app/routers/users.py` .
>
> Y `items.router` contiene el `APIRouter` dentro del archivo `app/routers/items.py` .

Con `app.include_router()` podemos agregar cada `APIRouter` a la aplicación principal de `FastAPI` .

Incluirá todas las rutas de ese router como parte de ella.

> **Detalles Técnicos**
> FastAPI mantiene activo el `APIRouter` original y sus `APIRoute` s cuando el router se incluye en la aplicación principal.
>
> Eso significa que las subclases personalizadas de `APIRouter` y `APIRoute` aún pueden participar después de incluir el router.

> **Consejo**

No tienes que preocuparte por el rendimiento al incluir routers.

Esto está diseñado para ser liviano y evitar añadir sobrecarga a cada request.

Así que no afectará el rendimiento.

Incluye un APIRouter con un prefix, tags, responses, y dependencies personalizados

Ahora, imaginemos que tu organización te dio el archivo `app/internal/admin.py`.

Contiene un `APIRouter` con algunas *path operations* de administración que tu organización comparte entre varios proyectos.

Para este ejemplo será súper simple. Pero digamos que porque está compartido con otros proyectos en la organización, no podemos modificarlo y agregar un `prefix`, `dependencies`, `tags`, etc. directamente al `APIRouter`:

```
from fastapi import APIRouter

router = APIRouter()

@router.post("/")
async def update_admin():
    return {"message": "Admin getting schwifty"}
```

Pero aún queremos configurar un `prefix` personalizado al incluir el `APIRouter` para que todas sus *path operations* comiencen con `/admin`, queremos asegurarlo con las `dependencies` que ya tenemos para este proyecto, y queremos incluir `tags` y `responses`.

Podemos declarar todo eso sin tener que modificar el `APIRouter` original pasando esos parámetros a `app.include_router()`:

```

app.include_router(
```

```
    admin.router,
    prefix="/admin",
    tags=["admin"],
    dependencies=[Depends(get_token_header)],
    responses={418: {"description": "I'm a teapot"}},
)
```

De esa manera, el `APIRouter` original permanecerá sin modificar, por lo que aún podemos compartir ese mismo archivo `app/internal/admin.py` con otros proyectos en la organización.

El resultado es que, en nuestra app, cada una de las *path operations* del módulo `admin` tendrá:

- El prefix `/admin`.
- El tag `admin`.
- La dependencia `get_token_header`.
- La response `418`. 🍵

Pero eso solo afectará a ese `APIRouter` en nuestra app, no en ningún otro código que lo utilice.

Así, por ejemplo, otros proyectos podrían usar el mismo `APIRouter` con un método de autenticación diferente.

Incluye una path operation

También podemos agregar *path operations* directamente a la app de `FastAPI`.

Aquí lo hacemos... solo para mostrar que podemos 🤷:

```
@app.get("/")
async def root():
    return {"message": "Hello Bigger Applications!"}
```

y funcionará correctamente, junto con todas las otras *path operations* añadidas con `app.include_router()`.

> **Detalles Muy Técnicos**
> **Nota**: este es un detalle muy técnico que probablemente puedes **simplemente omitir**.
>
> ---
>
> Los `APIRouter` s no están "montados", no están aislados del resto de la aplicación.
>
> Esto se debe a que queremos incluir sus *path operations* en el esquema de OpenAPI y las interfaces de usuario.
>
> FastAPI mantiene los routers y path operations originales activos, y combina los prefijos del router, dependencias, tags, responses y otros metadatos al manejar requests y generar OpenAPI.

Configura el entrypoint en pyproject.toml

Como tu objeto `app` de FastAPI vive en `app/main.py`, puedes configurar el `entrypoint` en tu archivo `pyproject.toml` así:

```toml
[tool.fastapi]
entrypoint = "app.main:app"
```

que es equivalente a importar como:

```
from app.main import app
```

De esa manera el comando `fastapi` sabrá dónde encontrar tu app.

> **Nota**
> También podrías pasar la ruta al comando, como:

```
$ uv run fastapi dev app/main.py
```

Pero tendrías que recordar pasar la ruta correcta cada vez que llames al comando `fastapi`.

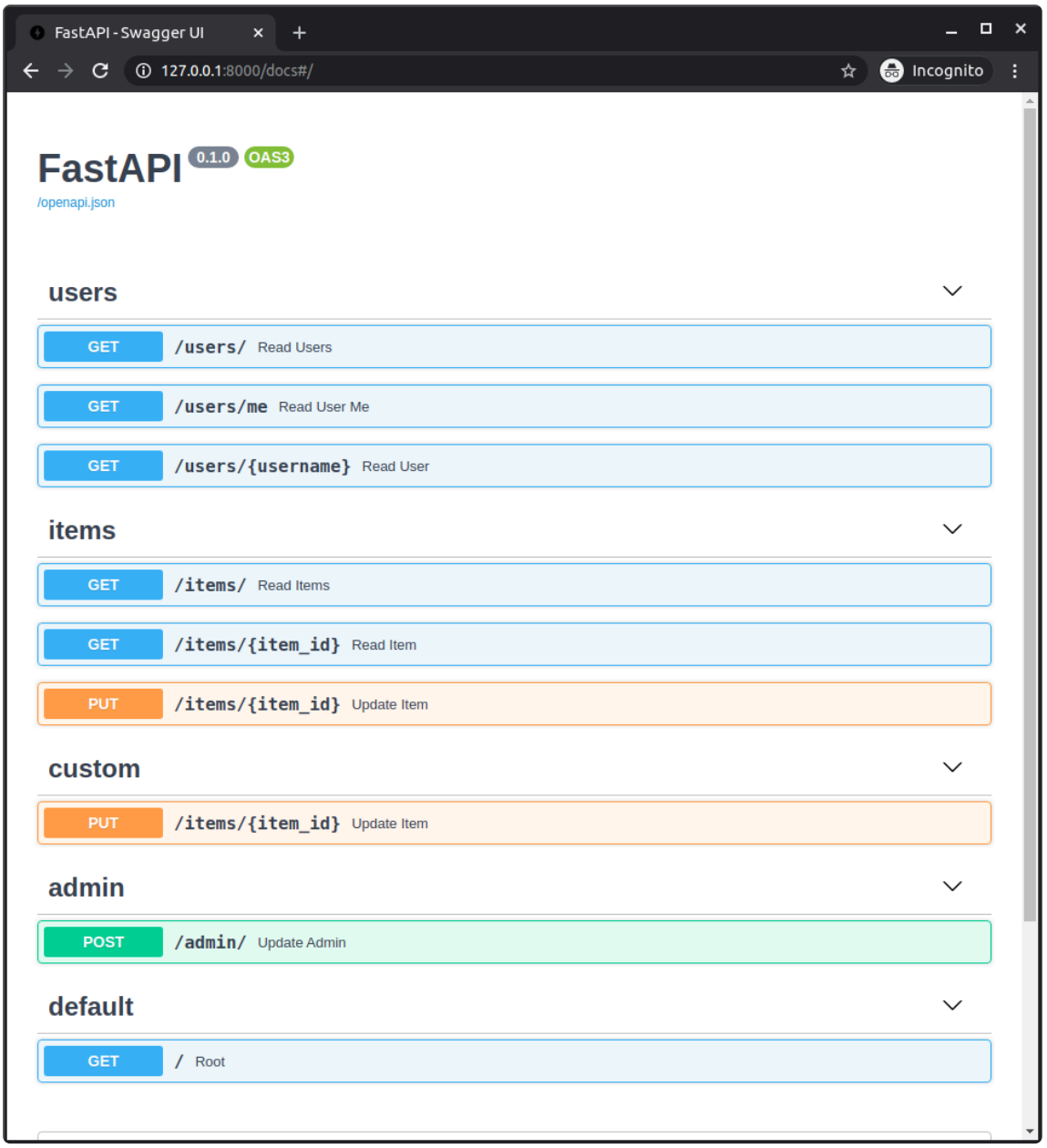
Además, otras herramientas podrían no ser capaces de encontrarla, por ejemplo la Extensión de VS Code o FastAPI Cloud, así que se recomienda usar el `entrypoint` en `pyproject.toml`.

Revisa la documentación automática de la API

Ahora, ejecuta tu app:

Y abre la documentación en http://127.0.0.1:8000/docs.

Verás la documentación automática de la API, incluyendo los paths de todos los submódulos, usando los paths correctos (y prefijos) y los tags correctos:



Incluye el mismo router múltiples veces con diferentes prefix

También puedes usar `.include_router()` múltiples veces con el *mismo* router usando diferentes prefijos.

Esto podría ser útil, por ejemplo, para exponer la misma API bajo diferentes prefijos, por ejemplo, `/api/v1` y `/api/latest`.

Este es un uso avanzado que quizás no necesites realmente, pero está allí en caso de que lo necesites.

Incluye un APIRouter en otro

De la misma manera que puedes incluir un `APIRouter` en una aplicación `FastAPI`, puedes incluir un `APIRouter` en otro `APIRouter` usando:

```
router.include_router(other_router)
```

Puedes hacerlo antes o después de incluir `router` en la app de `FastAPI`. FastAPI seguirá incluyendo las *path operations* de `other_router` en el ruteo y en OpenAPI.

Lo mismo aplica a las *path operations* añadidas después a los routers. También serán visibles a través de la inclusión anterior.

⚠ Detalles Técnicos
Evita mutar directamente `router.routes` después de incluir un router. FastAPI trata la inclusión de routers como “en vivo”, así que el router original y sus rutas siguen formando parte del ruteo y de la generación de OpenAPI.

Usa APIs documentadas como los decoradores de *path operations* y `.include_router()` para agregar rutas y routers.

Trata `router.routes` como un árbol de rutas de nivel bajo que puede contener definiciones de rutas y routers incluidos, y evita depender de él como una lista plana de *path operations* finales.

Transmitir JSON Lines

Podrías tener una secuencia de datos que quieras enviar en un " **stream** ", podrías hacerlo con **JSON Lines**.

✎ Nota
Añadido en FastAPI 0.134.0.

¿Qué es un Stream?

Hacer " **Streaming** " de datos significa que tu app empezará a enviar ítems de datos al cliente sin esperar a que toda la secuencia

de ítems esté lista.

Entonces, enviará el primer ítem, el cliente lo recibirá y empezará a procesarlo, y tú podrías seguir produciendo el siguiente ítem.

Incluso podría ser un stream infinito, donde sigues enviando datos.

JSON Lines

En estos casos, es común enviar "**JSON Lines**", que es un formato donde envías un objeto JSON por línea.

Una response tendría un tipo de contenido `application/jsonl` (en lugar de `application/json`) y el response body sería algo como:

```
{"name": "Plumbus", "description": "A multi-purpose household device."}
{"name": "Portal Gun", "description": "A portal opening device."}
{"name": "Meeseeks Box", "description": "A box that summons a Meeseeks."}
```

Es muy similar a un array JSON (equivalente de una list de Python), pero en lugar de estar envuelto en `[]` y tener `,` entre los ítems, tiene **un objeto JSON por línea**, separados por un carácter de nueva línea.

> **Nota**
> El punto importante es que tu app podrá producir cada línea a su turno, mientras el cliente consume las líneas anteriores.

> **Detalles técnicos**
> Como cada objeto JSON estará separado por una nueva línea, no pueden contener caracteres de nueva línea literales en su contenido, pero sí pueden contener nuevas líneas escapadas (`\n`), lo cual es parte del estándar JSON.

Pero normalmente no tendrás que preocuparte por eso, se hace automáticamente, sigue leyendo.

Casos de uso

Podrías usar esto para hacer stream de datos desde un servicio de **AI LLM**, desde **logs** o **telemetry**, o desde otros tipos de datos que puedan estructurarse en ítems **JSON**.

> **Consejo**
> Si quieres hacer stream de datos binarios, por ejemplo video o audio, Revisa la guía avanzada: Transmitir datos.

Transmitir JSON Lines con FastAPI

Para transmitir JSON Lines con FastAPI puedes, en lugar de usar `return` en tu *path operation function*, usar `yield` para producir cada ítem a su turno.

```
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None

items = [
    Item(name="Plumbus", description="A multi-purpose household device."),
    Item(name="Portal Gun", description="A portal opening device."),
    Item(name="Meeseeks Box", description="A box that summons a Meeseeks."),
]
```

```python
@app.get("/items/stream")
async def stream_items() -> AsyncIterable[Item]:
    for item in items:
        yield item

@app.get("/items/stream-no-async")
def stream_items_no_async() -> Iterable[Item]:
    for item in items:
        yield item

@app.get("/items/stream-no-annotation")
async def stream_items_no_annotation():
    for item in items:
        yield item

@app.get("/items/stream-no-async-no-annotation")
def stream_items_no_async_no_annotation():
    for item in items:
        yield item
```

Si cada ítem JSON que quieres enviar de vuelta es de tipo `Item` (un modelo de Pydantic) y es una función async, puedes declarar el tipo de retorno como `AsyncIterable[Item]`:

```python
class Item(BaseModel):
    name: str
    description: str | None


@app.get("/items/stream")
async def stream_items() -> AsyncIterable[Item]:
...
```

Si declaras el tipo de retorno, FastAPI lo usará para **validar** los datos, **documentarlos** en OpenAPI, **filtrarlos** y **serializarlos** usando Pydantic.

Consejo

Como Pydantic lo serializará en el lado de **Rust**, obtendrás un **rendimiento** mucho mayor que si no declaras un tipo de retorno.

path operation functions no-async

También puedes usar funciones `def` regulares (sin `async`), y usar `yield` de la misma forma.

FastAPI se asegurará de que se ejecute correctamente para que no bloquee el event loop.

Como en este caso la función no es async, el tipo de retorno correcto sería `Iterable[Item]`:

```
# Code above omitted

...
def stream_items_no_async() -> Iterable[Item]:
...

# Code below omitted
```

Sin tipo de retorno

También puedes omitir el tipo de retorno. Entonces FastAPI usará `jsonable_encoder` para convertir los datos a algo que se pueda serializar a JSON y luego enviarlo como JSON Lines.

```
async def stream_items_no_annotation():
...
```

Server-Sent Events (SSE)

FastAPI también tiene soporte de primera clase para Server-Sent Events (SSE), que son bastante similares pero con un par de

detalles extra. Puedes aprender sobre ellos en el siguiente capítulo: Eventos enviados por el servidor (SSE).

- # Server-Sent Events (SSE)

Puedes enviar datos en streaming al cliente usando **Server-Sent Events** (SSE).

Esto es similar a Stream JSON Lines, pero usa el formato `text/event-stream`, que los navegadores soportan de forma nativa con la `EventSource` API.

> **Nota**
> Añadido en FastAPI 0.135.0.

¿Qué son los Server-Sent Events?

SSE es un estándar para hacer streaming de datos desde el servidor al cliente sobre HTTP.

Cada evento es un pequeño bloque de texto con "campos" como `data`, `event`, `id` y `retry`, separados por líneas en blanco.

Se ve así:

```
data: {"name": "Portal Gun", "price": 999.99}

data: {"name": "Plumbus", "price": 32.99}
```

SSE se usa comúnmente para streaming de chat de IA, notificaciones en vivo, logs y observabilidad, y otros casos donde el servidor envía actualizaciones al cliente.

> **Consejo**
> Si quieres hacer streaming de datos binarios, por ejemplo video o audio, Revisa la guía avanzada: Stream Data.

Streaming de SSE con FastAPI

Para hacer streaming de SSE con FastAPI, usa `yield` en tu path operation function y establece `response_class=EventSourceResponse`.

import `EventSourceResponse` de `fastapi.sse`:

```
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from fastapi.sse import EventSourceResponse
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None

items = [
    Item(name="Plumbus", description="A multi-purpose household device."),
    Item(name="Portal Gun", description="A portal opening device."),
    Item(name="Meeseeks Box", description="A box that summons a Meeseeks."),
]

@app.get("/items/stream", response_class=EventSourceResponse)
async def sse_items() -> AsyncIterable[Item]:
    for item in items:
        yield item

@app.get("/items/stream-no-async", response_class=EventSourceResponse)
def sse_items_no_async() -> Iterable[Item]:
    for item in items:
        yield item

@app.get("/items/stream-no-annotation", response_class=EventSourceResponse)
```

```
async def sse_items_no_annotation():
    for item in items:
        yield item

@app.get("/items/stream-no-async-no-annotation",
response_class=EventSourceResponse)
def sse_items_no_async_no_annotation():
    for item in items:
        yield item
```

Cada ítem producido con `yield` se codifica como JSON y se envía en el campo `data:` de un evento SSE.

Si declaras el tipo de retorno como `AsyncIterable[Item]`, FastAPI lo usará para **validar**, **documentar** y **serializar** los datos usando Pydantic.

```
class Item(BaseModel):
    name: str
    description: str | None
```

> **Consejo**
> Como Pydantic lo serializará en el lado de **Rust**, obtendrás un **rendimiento** mucho mayor que si no declaras un tipo de retorno.

No async path operation functions

También puedes usar funciones `def` normales (sin `async`), y usar `yield` de la misma manera.

FastAPI se asegurará de ejecutarlo correctamente para que no bloquee el event loop.

Como en este caso la función no es async, el tipo de retorno correcto sería `Iterable[Item]`:

```
def sse_items_no_async() -> Iterable[Item]:
...
```

Sin tipo de retorno

También puedes omitir el tipo de retorno. FastAPI usará el `jsonable_encoder` para convertir los datos y enviarlos.

```
async def sse_items_no_annotation():
...
```

ServerSentEvent

Si necesitas configurar campos SSE como `event`, `id`, `retry` o `comment`, puedes hacer `yield` de objetos `ServerSentEvent` en lugar de datos simples.

import `ServerSentEvent` de `fastapi.sse`:

```
from fastapi.sse import EventSourceResponse,
ServerSentEvent
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float

items = [
    Item(name="Plumbus", price=32.99),
    Item(name="Portal Gun", price=999.99),
    Item(name="Meeseeks Box", price=49.99),
]
```

```
@app.get("/items/stream",
response_class=EventSourceResponse)
async def stream_items() ->
AsyncIterable[ServerSentEvent]:
    yield ServerSentEvent(comment="stream of item
updates")
    for i, item in enumerate(items):
        yield ServerSentEvent(data=item,
event="item_update", id=str(i + 1), retry=5000)
```

El campo `data` siempre se codifica como JSON. Puedes pasar cualquier valor que pueda serializarse como JSON, incluidos modelos de Pydantic.

Datos sin procesar

Si necesitas enviar datos **sin** codificarlos a JSON, usa `raw_data` en lugar de `data`.

Esto es útil para enviar texto preformateado, líneas de log, o valores especiales de "centinela" como `[DONE]`.

```
from collections.abc import AsyncIterable

from fastapi import FastAPI
from fastapi.sse import EventSourceResponse,
ServerSentEvent

app = FastAPI()

@app.get("/logs/stream",
response_class=EventSourceResponse)
async def stream_logs() -> AsyncIterable[ServerSentEvent]:
    logs = [
        "2025-01-01 INFO  Application started",
        "2025-01-01 DEBUG Connected to database",
        "2025-01-01 WARN  High memory usage detected",
    ]
```

```
    for log_line in logs:
        yield ServerSentEvent(raw_data=log_line)
```

> **Nota**
> `data` y `raw_data` son mutuamente excluyentes. Solo puedes establecer uno de ellos en cada `ServerSentEvent`.

Reanudar con Last-Event-ID

Cuando un navegador se reconecta después de una caída de la conexión, envía el último `id` recibido en el header `Last-Event-ID`.

Puedes leerlo como un parámetro de header y usarlo para reanudar el stream desde donde el cliente se quedó:

```
from collections.abc import AsyncIterable
from typing import Annotated

from fastapi import FastAPI, Header
from fastapi.sse import EventSourceResponse,
ServerSentEvent
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float

items = [
    Item(name="Plumbus", price=32.99),
    Item(name="Portal Gun", price=999.99),
    Item(name="Meeseeks Box", price=49.99),
]

@app.get("/items/stream",
response_class=EventSourceResponse)
async def stream_items(
    last_event_id: Annotated[int | None, Header()] = None,
) -> AsyncIterable[ServerSentEvent]:
```

```
    start = last_event_id + 1 if last_event_id is not None
else 0
    for i, item in enumerate(items):
        if i < start:
            continue
        yield ServerSentEvent(data=item, id=str(i))
```

SSE con POST

SSE funciona con **cualquier método HTTP**, no solo con `GET`.

Esto es útil para protocolos como MCP que hacen streaming de SSE sobre `POST`:

```
from collections.abc import AsyncIterable

from fastapi import FastAPI
from fastapi.sse import EventSourceResponse,
ServerSentEvent
from pydantic import BaseModel

app = FastAPI()

class Prompt(BaseModel):
    text: str

@app.post("/chat/stream",
response_class=EventSourceResponse)
async def stream_chat(prompt: Prompt) ->
AsyncIterable[ServerSentEvent]:
    words = prompt.text.split()
    for word in words:
        yield ServerSentEvent(data=word, event="token")
    yield ServerSentEvent(raw_data="[DONE]", event="done")
```

Detalles técnicos

FastAPI implementa algunas mejores prácticas de SSE desde el primer momento.

- Enviar un comentario de **"keep alive"** `ping` cada 15 segundos cuando no ha habido ningún mensaje, para evitar que algunos proxies cierren la conexión, como se sugiere en la Especificación HTML: Server-Sent Events.
- Configurar el header `Cache-Control: no-cache` para **evitar el almacenamiento en caché** del stream.
- Configurar un header especial `X-Accel-Buffering: no` para **evitar el buffering** en algunos proxies como Nginx.

No tienes que hacer nada, funciona tal cual viene. 😳

- # Tareas en Segundo Plano

Puedes definir tareas en segundo plano para que se ejecuten *después* de devolver un response.

Esto es útil para operaciones que necesitan ocurrir después de un request, pero para las que el cliente realmente no necesita esperar a que la operación termine antes de recibir el response.

Esto incluye, por ejemplo:

- Notificaciones por email enviadas después de realizar una acción:
- Como conectarse a un servidor de email y enviar un email tiende a ser "lento" (varios segundos), puedes devolver el response de inmediato y enviar la notificación por email en segundo plano.
- Procesamiento de datos:
- Por ejemplo, supongamos que recibes un archivo que debe pasar por un proceso lento, puedes devolver un response de "Accepted" (HTTP 202) y procesar el archivo en segundo plano.

Usando BackgroundTasks

Primero, importa `BackgroundTasks` y define un parámetro en tu *path operation function* con una declaración de tipo de `BackgroundTasks`:

```python
from fastapi import BackgroundTasks, FastAPI

app = FastAPI()

def write_notification(email: str, message=""):
    with open("log.txt", mode="w") as email_file:
        content = f"notification for {email}: {message}"
        email_file.write(content)

@app.post("/send-notification/{email}")
async def send_notification(email: str, background_tasks:
BackgroundTasks):
    background_tasks.add_task(write_notification, email,
message="some notification")
    return {"message": "Notification sent in the
background"}
```

FastAPI creará el objeto de tipo `BackgroundTasks` por ti y lo pasará como ese parámetro.

Crea una función de tarea

Crea una función para que se ejecute como la tarea en segundo plano.

Es solo una función estándar que puede recibir parámetros.

Puede ser una función `async def` o una función normal `def`, **FastAPI** sabrá cómo manejarla correctamente.

En este caso, la función de tarea escribirá en un archivo (simulando el envío de un email).

Y como la operación de escritura no usa `async` y `await`, definimos la función con un `def` normal:

```python

def write_notification(email: str, message=""):
    with open("log.txt", mode="w") as email_file:
        content = f"notification for {email}: {message}"
```

```
        email_file.write(content)
```

Agrega la tarea en segundo plano

Dentro de tu *path operation function*, pasa tu función de tarea al objeto de *tareas en segundo plano* con el método `.add_task()`:

```
    ...
    background_tasks.add_task(write_notification, email,
message="some notification")
    ...
```

`.add_task()` recibe como argumentos:

- Una función de tarea para ejecutar en segundo plano (`write_notification`).
- Cualquier secuencia de argumentos que deba pasarse a la función de tarea en orden (`email`).
- Cualquier argumento de palabras clave que deba pasarse a la función de tarea (`message="some notification"`).

Inyección de Dependencias

Usar `BackgroundTasks` también funciona con el sistema de inyección de dependencias, puedes declarar un parámetro de tipo `BackgroundTasks` en varios niveles: en una *path operation function*, en una dependencia (dependable), en una sub-dependencia, etc.

FastAPI sabe qué hacer en cada caso y cómo reutilizar el mismo objeto, de modo que todas las tareas en segundo plano se combinan y ejecutan en segundo plano después:

```
from typing import Annotated

from fastapi import BackgroundTasks, Depends, FastAPI
```

```python
app = FastAPI()

def write_log(message: str):
    with open("log.txt", mode="a") as log:
        log.write(message)

def get_query(background_tasks: BackgroundTasks, q: str |
None = None):
    if q:
        message = f"found query: {q}\n"
        background_tasks.add_task(write_log, message)
    return q

@app.post("/send-notification/{email}")
async def send_notification(
    email: str, background_tasks: BackgroundTasks, q:
Annotated[str, Depends(get_query)]
):
    message = f"message to {email}\n"
    background_tasks.add_task(write_log, message)
    return {"message": "Message sent"}
```

En este ejemplo, los mensajes se escribirán en el archivo `log.txt` *después* de que se envíe el response.

Si hay un query en el request, se escribirá en el log en una tarea en segundo plano.

Y luego otra tarea en segundo plano generada en la *path operation function* escribirá un mensaje usando el parámetro de path `email`.

Detalles Técnicos

La clase `BackgroundTasks` proviene directamente de `starlette.background`.

Se importa/incluye directamente en FastAPI para que puedas importarla desde `fastapi` y evitar importar accidentalmente la

alternativa `BackgroundTask` (sin la `s` al final) de `starlette.background`.

Al usar solo `BackgroundTasks` (y no `BackgroundTask`), es posible usarla como un parámetro de *path operation function* y dejar que **FastAPI** maneje el resto por ti, tal como cuando usas el objeto `Request` directamente.

Todavía es posible usar `BackgroundTask` solo en FastAPI, pero debes crear el objeto en tu código y devolver una `Response` de Starlette incluyéndolo.

Puedes ver más detalles en la documentación oficial de Starlette sobre Background Tasks.

Advertencia

Si necesitas realizar una computación intensa en segundo plano y no necesariamente necesitas que se ejecute por el mismo proceso (por ejemplo, no necesitas compartir memoria, variables, etc.), podrías beneficiarte del uso de otras herramientas más grandes como Celery.

Tienden a requerir configuraciones más complejas, un gestor de cola de mensajes/trabajos, como RabbitMQ o Redis, pero te permiten ejecutar tareas en segundo plano en múltiples procesos, y especialmente, en múltiples servidores.

Pero si necesitas acceder a variables y objetos de la misma app de **FastAPI**, o necesitas realizar pequeñas tareas en segundo plano (como enviar una notificación por email), simplemente puedes usar `BackgroundTasks`.

Resumen

Importa y usa `BackgroundTasks` con parámetros en *path operation functions* y dependencias para agregar tareas en segundo plano.

Metadata y URLs de documentación

Puedes personalizar varias configuraciones de metadata en tu aplicación **FastAPI**.

Metadata para la API

Puedes establecer los siguientes campos que se usan en la especificación OpenAPI y en las interfaces automáticas de documentación de la API:

Parámetro	Tipo	Descripción
`title`	`str`	El título de la API
`sumary`	`str`	Un resumen corto de la API. Disponible desde OpenAPI 3.1.0, FastAPI 0.99.0.
`description`	`str`	Una breve descripción de la API. Puede usar Markdown.
`version`	`str`	La versión de la API. Esta es la versión de tu propia aplicación, no de OpenAPI. Por ejemplo, `2.5.0`.
`terms_of_service`	`str`	Una URL a los Términos de Servicio para la API. Si se proporciona, debe ser una URL.
`contact`	`dict`	La información de contacto para la API expuesta. Puede contener varios campos. `name` `str` El nombre identificativo de la persona/organización de contacto. `url` `str` La URL que apunta a la información de contacto. DEBE tener el formato de una URL. `email` `str` La dirección de correo electrónico de la persona/organización de contacto. DEBE tener el formato de una dirección de correo.

Parámetro	Tipo	Descripción
`license_info`	`dict`	`name` `str` **REQUERIDO** (si se establece un `license_info`). El nombre de la licencia utilizada para la API. `identifier` `str` Una expresión de licencia SPDX para la API. El campo `identifier` es mutuamente excluyente del campo `url`. Disponible desde OpenAPI 3.1.0, FastAPI 0.99.0. `url` `str` Una URL a la licencia utilizada para la API. DEBE tener el formato de una URL.

Puedes configurarlos de la siguiente manera:

```
from fastapi import FastAPI

description = """
ChimichangApp API helps you do awesome stuff. 🚀

## Items

You can **read items**.

## Users

You will be able to:

* **Create users** (_not implemented_).
* **Read users** (_not implemented_).
"""

app = FastAPI(
    title="ChimichangApp",
    description=description,
    summary="Deadpool's favorite app. Nuff said.",
    version="0.0.1",
    terms_of_service="http://example.com/terms/",
    contact={
```

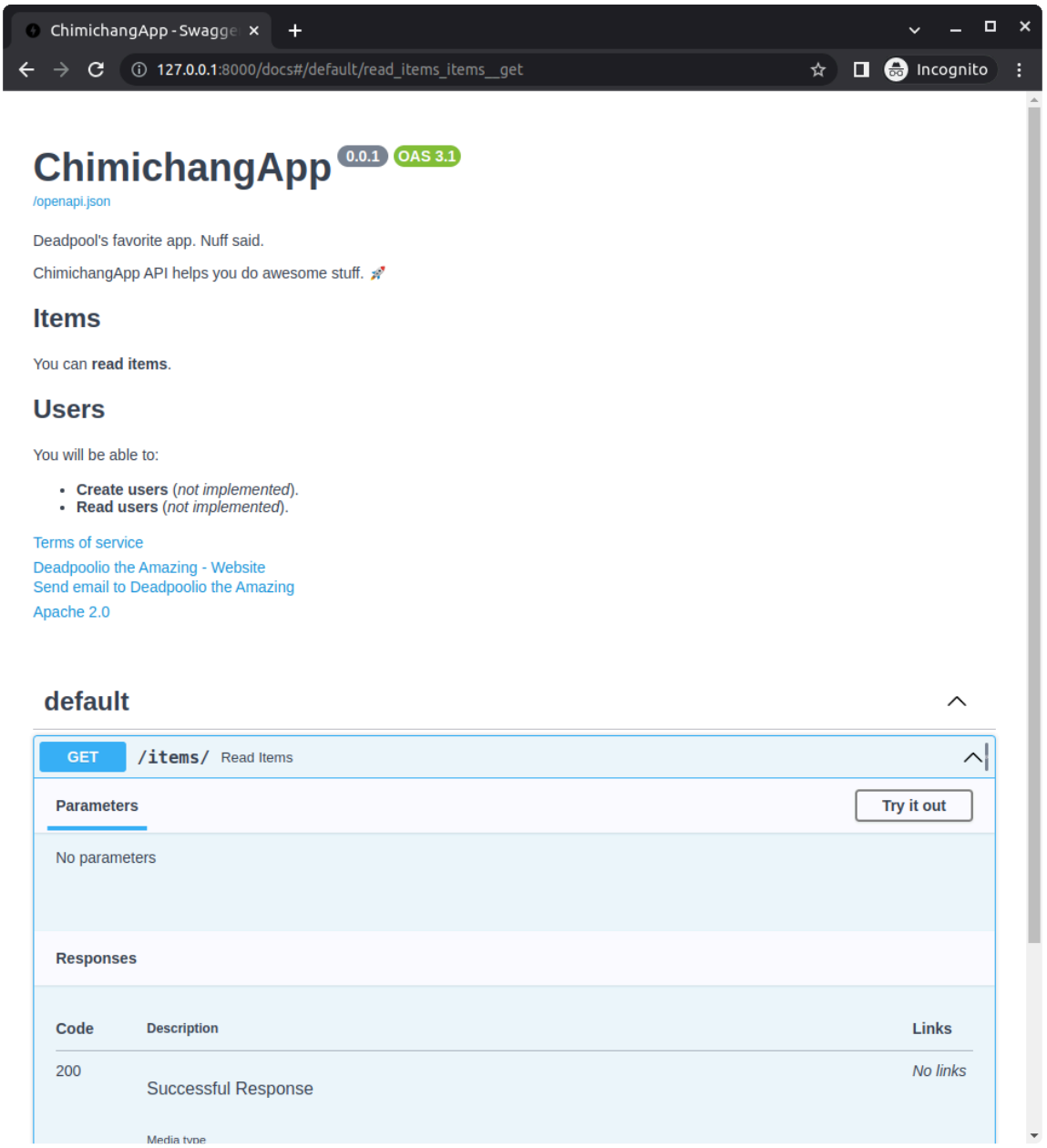
```
        "name": "Deadpoolio the Amazing",
        "url": "http://x-force.example.com/contact/",
        "email": "dp@x-force.example.com",
    },
    license_info={
        "name": "Apache 2.0",
        "url": "https://www.apache.org/licenses/LICENSE-
2.0.html",
    },
)

@app.get("/items/")
async def read_items():
    return [{"name": "Katana"}]
```

Consejo

Puedes escribir Markdown en el campo `description` y se mostrará en el resultado.

Con esta configuración, la documentación automática de la API se vería así:



Identificador de licencia

Desde OpenAPI 3.1.0 y FastAPI 0.99.0, también puedes establecer la `license_info` con un `identifier` en lugar de una `url`.

Por ejemplo:

```
...
    license_info={
        "name": "Apache 2.0",
        "identifier": "Apache-2.0",
    },
...
```

Metadata para etiquetas

También puedes agregar metadata adicional para las diferentes etiquetas usadas para agrupar tus path operations con el parámetro `openapi_tags`.

Este toma una list que contiene un diccionario para cada etiqueta.

Cada diccionario puede contener:

- `name` (**requerido**): un `str` con el mismo nombre de etiqueta que usas en el parámetro `tags` en tus *path operations* y `APIRouter` s.
- `description`: un `str` con una breve descripción de la etiqueta. Puede tener Markdown y se mostrará en la interfaz de documentación.
- `externalDocs`: un `dict` que describe documentación externa con:
    - `description`: un `str` con una breve descripción para la documentación externa.
        - `url` (**requerido**): un `str` con la URL para la documentación externa.

Crear metadata para etiquetas

Probemos eso en un ejemplo con etiquetas para `users` y `items`.

Crea metadata para tus etiquetas y pásala al parámetro `openapi_tags`:

```
tags_metadata = [
    {
        "name": "users",
        "description": "Operations with users. The
**login** logic is also here.",
    },
    {
        "name": "items",
```

```
        "description": "Manage items. So _fancy_ they have
their own docs.",
        "externalDocs": {
            "description": "Items external docs",
            "url": "https://fastapi.tiangolo.com/",
        },
    },
]

app = FastAPI(openapi_tags=tags_metadata)
```

Nota que puedes utilizar Markdown dentro de las descripciones, por ejemplo "login" se mostrará en negrita (**login**) y "fancy" se mostrará en cursiva (*fancy*).

> **Consejo**
> No tienes que agregar metadata para todas las etiquetas que uses.

Usar tus etiquetas

Usa el parámetro `tags` con tus *path operations* (y `APIRouter` s) para asignarlas a diferentes etiquetas:

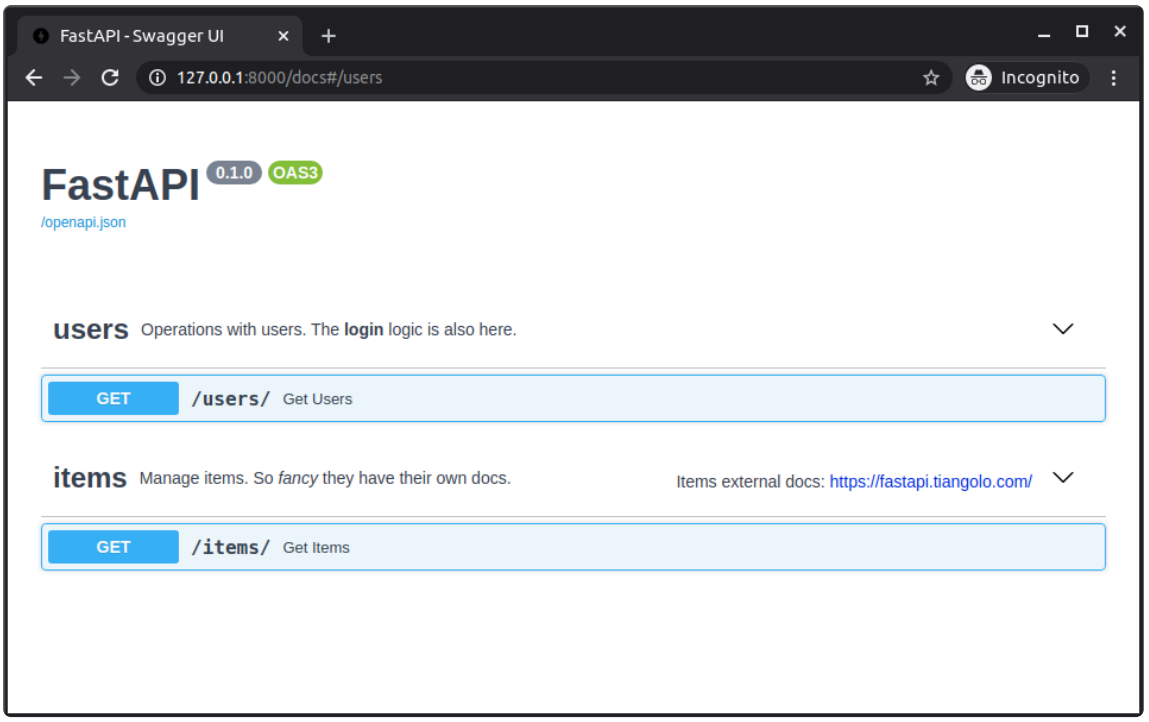
```
@app.get("/users/", tags=["users"])
...

@app.get("/items/", tags=["items"])
...
```

> **Nota**
> Lee más sobre etiquetas en Configuración de Path Operation.

Revisa la documentación

Ahora, si revisas la documentación, mostrará toda la metadata adicional:



Orden de las etiquetas

El orden de cada diccionario de metadata de etiqueta también define el orden mostrado en la interfaz de documentación.

Por ejemplo, aunque `users` iría después de `items` en orden alfabético, se muestra antes porque agregamos su metadata como el primer diccionario en la list.

URL de OpenAPI

Por defecto, el esquema OpenAPI se sirve en `/openapi.json`.

Pero puedes configurarlo con el parámetro `openapi_url`.

Por ejemplo, para configurarlo para que se sirva en `/api/v1/openapi.json`:

```
from fastapi import FastAPI

app = FastAPI(openapi_url="/api/v1/openapi.json")
```

```
@app.get("/items/")
async def read_items():
    return [{"name": "Foo"}]
```

Si quieres deshabilitar el esquema OpenAPI completamente, puedes establecer `openapi_url=None`, eso también deshabilitará las interfaces de usuario de documentación que lo usan.

URLs de documentación

Puedes configurar las dos interfaces de usuario de documentación incluidas:

- **Swagger UI**: servida en `/docs`.
    - Puedes establecer su URL con el parámetro `docs_url`.
        - Puedes deshabilitarla estableciendo `docs_url=None`.
- **ReDoc**: servida en `/redoc`.
    - Puedes establecer su URL con el parámetro `redoc_url`.
        - Puedes deshabilitarla estableciendo `redoc_url=None`.

Por ejemplo, para configurar Swagger UI para que se sirva en `/documentation` y deshabilitar ReDoc:

```
from fastapi import FastAPI

app = FastAPI(docs_url="/documentation", redoc_url=None)

@app.get("/items/")
async def read_items():
    return [{"name": "Foo"}]
```

- # Frontend

Puedes servir apps frontend estáticas con `app.frontend()` (o `router.frontend()`).

Esto es útil para herramientas de frontend que generan archivos estáticos, como React con Vite, TanStack Router, Astro, Vue, Svelte, Angular, Solid y otras.

Con estas herramientas, normalmente tienes un paso que construye el frontend, con un comando como:

```
npm run build
```

Eso generaría un directorio como `./dist/` con tus archivos frontend.

Puedes usar `app.frontend()` para servir ese directorio siguiendo las convenciones que necesitan estos frameworks frontend.

FastAPI revisa primero las *path operations*. Los archivos frontend se revisan solo si ninguna ruta normal coincide, así que tu API no se verá afectada.

Sirve un Frontend

Después de construir tu frontend, por ejemplo con `npm run build`, pon los archivos generados en un directorio, por ejemplo, `dist`.

La estructura de tu proyecto podría verse así:

```
.
├── pyproject.toml
├── app
│   ├── __init__.py
│   └── main.py
└── dist
    ├── index.html
    └── assets
        └── app.js
```

Luego sírvelo con `app.frontend()`:

```python
from fastapi import FastAPI

app = FastAPI()

app.frontend("/", directory="dist")
```

Con esto, un request a `/assets/app.js` puede servir `dist/assets/app.js`.

Si también tienes una *path operation* de **FastAPI**, la *path operation* gana.

Routing del lado del cliente

Muchas apps frontend, incluidas las **single-page apps** (SPAs), usan routing del lado del cliente. Un path como `/dashboard/settings` podría no ser un archivo real, pero el framework se encargaría de manejarlo.

Entonces, si se accede a esa URL directamente (en lugar de navegar por la app), el backend debería servir la app frontend desde `index.html`, para que el framework frontend pueda manejar el routing del lado del cliente.

Para eso, usa `fallback="index.html"`:

```python
from fastapi import FastAPI

app = FastAPI()

app.frontend("/", directory="dist", fallback="index.html")
```

FastAPI usa este fallback solo para requests `GET` y `HEAD` que aceptan HTML explícitamente con `Accept: text/html` o `Accept: application/xhtml+xml`, como normalmente hacen los requests de navegación del navegador. Los archivos faltantes como JavaScript, CSS e imágenes siguen devolviendo `404`.

Los requests con otros métodos, como `POST` o `PUT`, a paths que solo coinciden con el fallback del frontend también devuelven `404`. Las *path operations* normales de **FastAPI** siguen teniendo mayor prioridad que las rutas frontend.

> **Consejo**
> Por defecto, `fallback` tiene un valor de `fallback="auto"`. En la mayoría de los casos no necesitarás especificar `fallback`. Lee más abajo para los detalles.

Esto es lo que querrías con muchas apps frontend que usan routing del lado del cliente, por ejemplo, React con TanStack Router, Vue, Angular, SvelteKit o Solid.

Página 404 personalizada

También puedes servir una página estática `404.html` para paths frontend faltantes:

```
from fastapi import FastAPI

app = FastAPI()

app.frontend("/", directory="dist", fallback="404.html")
```

Esa response mantiene un código de estado `404`.

En este caso, **FastAPI** no servirá `index.html` para paths frontend faltantes. En su lugar, devolverá el archivo `404.html`.

> **Consejo**
> Por defecto, `fallback` tiene un valor de `fallback="auto"`. Con esto, si se encuentra un archivo `404.html`, se usará automáticamente como fallback.
>
> Así que normalmente puedes omitir el argumento `fallback`.

Esto es útil con herramientas de frontend que generan archivos HTML estáticos para cada página, como Astro.

Fallback automático

Por defecto, `app.frontend()` usa `fallback="auto"`.

Si hay un archivo `404.html` en el directorio frontend, los paths frontend faltantes sirven ese archivo con código de estado `404`.

De lo contrario, si hay un archivo `index.html`, los paths faltantes de navegación del navegador sirven `index.html`, que es lo que muchas apps frontend con routing del lado del cliente esperan.

Así que, en la mayoría de los casos, puedes usar `app.frontend("/", directory="dist")` sin especificar el argumento `fallback`.

```
from fastapi import FastAPI

app = FastAPI()

app.frontend("/", directory="dist")
```

Desactiva el fallback

Si no quieres servir un archivo fallback para paths frontend faltantes, usa `fallback=None`:

```
from fastapi import FastAPI

app = FastAPI()

app.frontend("/", directory="dist", fallback=None)
```

Entonces los paths frontend faltantes devuelven el `404` normal.

Revisa el directorio

Por defecto, `app.frontend()` usa `check_dir="auto"`.

Cuando la variable de entorno `FASTAPI_ENV` se configura como `development`, **FastAPI** solo muestra una advertencia si falta el directorio de salida del build del frontend. El comando `fastapi dev` configura esta variable de entorno por ti si todavía no está configurada. Esto te permite iniciar el backend antes de construir o iniciar el frontend durante el desarrollo.

En cualquier otro entorno, **FastAPI** lanza un error cuando se crea la app. Esto ayuda a detectar errores de configuración temprano antes de desplegar una app sin sus archivos frontend.

También puedes configurar `check_dir=True` para revisar siempre el directorio cuando se crea la app.

Si tus archivos frontend se crean más tarde, por ejemplo mediante un paso de build separado después de crear el objeto app, configura `check_dir=False`:

```
from fastapi import FastAPI

app = FastAPI()

app.frontend("/", directory="dist", check_dir=False)
```

Con `check_dir=False`, **FastAPI** no revisará el directorio cuando se cree la app. Si el directorio configurado todavía falta cuando se maneja un request, **FastAPI** lanzará un error en ese momento.

Úsalo con APIRouter

También puedes agregar archivos frontend a un `APIRouter` e incluirlo con un prefijo:

```
from fastapi import APIRouter, FastAPI

app = FastAPI()
router = APIRouter()
```

```
router.frontend("/", directory="dist",
fallback="index.html")
app.include_router(router, prefix="/app")
```

En este ejemplo, los paths frontend se sirven bajo `/app`.

Cualquier *path operation* regular en la app seguirá teniendo prioridad, incluso en otros routers.

Dependencias y Middleware

Las responses frontend se ejecutan dentro de la aplicación **FastAPI** normal, así que el middleware HTTP se aplica a ellas.

Las dependencias de la app, de un `APIRouter` y de `include_router()` también se aplican a las responses frontend. Esto puede ser útil para proteger un frontend con autenticación por cookie o similar.

Las dependencias también pueden modificar headers de response y agregar tareas en background, como con las *path operations* normales.

Solo salida estática del build

`app.frontend()` sirve archivos ya generados por tu build del frontend.

No ejecuta renderizado del lado del servidor. Es para frameworks frontend que generan archivos estáticos, no para frameworks que necesitan renderizado dinámico en el servidor para cada request.

Archivos Estáticos

Puedes servir archivos estáticos automáticamente desde un directorio utilizando `StaticFiles`.

Consejo

Si necesitas alojar un frontend, usa `app.frontend()` en su lugar, lee sobre ello en Frontend.

`app.frontend()` usa `StaticFiles` por debajo, con varias ventajas adicionales para frontends, como manejar el routing del lado del cliente.

Usa StaticFiles

- Importa `StaticFiles`.
- "Monta" una instance de `StaticFiles()` en un path específico.

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

app = FastAPI()

app.mount("/static", StaticFiles(directory="static"),
name="static")
```

Detalles Técnicos

También podrías usar `from starlette.staticfiles import StaticFiles`.

FastAPI proporciona el mismo `starlette.staticfiles` como `fastapi.staticfiles` solo como una conveniencia para ti, el desarrollador. Pero en realidad viene directamente de Starlette.

Qué es "Montar"

"Montar" significa agregar una aplicación completa "independiente" en un path específico, que luego se encargará de manejar todos los sub-paths.

Esto es diferente a usar un `APIRouter`, ya que una aplicación montada es completamente independiente. El OpenAPI y la documentación de tu aplicación principal no incluirán nada de la aplicación montada, etc.

Puedes leer más sobre esto en la Guía de Usuario Avanzada.

Detalles

El primer `"/static"` se refiere al sub-path en el que esta "sub-aplicación" será "montada". Por lo tanto, cualquier path que comience con `"/static"` será manejado por ella.

El `directory="static"` se refiere al nombre del directorio que contiene tus archivos estáticos.

El `name="static"` le da un nombre que puede ser utilizado internamente por **FastAPI**.

Todos estos parámetros pueden ser diferentes a " `static` ", ajústalos según las necesidades y detalles específicos de tu propia aplicación.

Más info

Para más detalles y opciones revisa la documentación de Starlette sobre Archivos Estáticos.

Pruebas

Gracias a Starlette, escribir pruebas para aplicaciones de **FastAPI** es fácil y agradable.

Está basado en HTTPX, que a su vez está diseñado basado en Requests, por lo que es muy familiar e intuitivo.

Con él, puedes usar pytest directamente con **FastAPI**.

Usando TestClient

> **Nota**
> Para usar `TestClient`, primero instala `httpx`.
>
> Añádelo a tu proyecto:

```
$ uv add httpx
```

Importa `TestClient`.

Crea un `TestClient` pasándole tu aplicación de **FastAPI**.

Crea funciones con un nombre que comience con `test_` (esta es la convención estándar de `pytest`).

Usa el objeto `TestClient` de la misma manera que con `httpx`.

Escribe statements `assert` simples con las expresiones estándar de Python que necesites revisar (otra vez, estándar de `pytest`).

```
from fastapi import FastAPI
from fastapi.testclient import TestClient

app = FastAPI()

@app.get("/")
async def read_main():
    return {"msg": "Hello World"}

client = TestClient(app)

def test_read_main():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"msg": "Hello World"}
```

Consejo

Nota que las funciones de prueba son `def` normales, no `async def`.

Y las llamadas al cliente también son llamadas normales, sin usar `await`.

Esto te permite usar `pytest` directamente sin complicaciones.

Detalles técnicos

También podrías usar `from starlette.testclient import TestClient`.

FastAPI proporciona el mismo `starlette.testclient` como `fastapi.testclient` solo por conveniencia para ti, el desarrollador. Pero proviene directamente de Starlette.

Consejo

Si quieres llamar a funciones `async` en tus pruebas además de enviar requests a tu aplicación FastAPI (por ejemplo, funciones asincrónicas de bases de datos), echa un vistazo a las Pruebas Asincrónicas en el tutorial avanzado.

Separando pruebas

En una aplicación real, probablemente tendrías tus pruebas en un archivo diferente.

Y tu aplicación de **FastAPI** también podría estar compuesta de varios archivos/módulos, etc.

Archivo de aplicación FastAPI

Digamos que tienes una estructura de archivos como se describe en Aplicaciones Más Grandes:

```
.
├── app
│   ├── __init__.py
│   └── main.py
```

En el archivo `main.py` tienes tu aplicación de **FastAPI**:

```
from fastapi import FastAPI

app = FastAPI()
```

```python
@app.get("/")
async def read_main():
    return {"msg": "Hello World"}
```

Archivo de prueba

Entonces podrías tener un archivo `test_main.py` con tus pruebas. Podría estar en el mismo paquete de Python (el mismo directorio con un archivo `__init__.py`):

```
.
├── app
│   ├── __init__.py
│   ├── main.py
│   └── test_main.py
```

Debido a que este archivo está en el mismo paquete, puedes usar imports relativos para importar el objeto `app` desde el módulo `main` (`main.py`):

```python
from fastapi.testclient import TestClient

from .main import app

client = TestClient(app)

def test_read_main():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"msg": "Hello World"}
```

...y tener el código para las pruebas tal como antes.

Pruebas: ejemplo extendido

Ahora extendamos este ejemplo y añade más detalles para ver cómo escribir pruebas para diferentes partes.

Archivo de aplicación FastAPI extendido

Continuemos con la misma estructura de archivos que antes:

```
.
├── app
│   ├── __init__.py
│   ├── main.py
│   └── test_main.py
```

Digamos que ahora el archivo `main.py` con tu aplicación de **FastAPI** tiene algunas otras **path operations**.

Tiene una operación `GET` que podría devolver un error.

Tiene una operación `POST` que podría devolver varios errores.

Ambas *path operations* requieren un `X-Token` header.

```
from typing import Annotated

from fastapi import FastAPI, Header, HTTPException
from pydantic import BaseModel

fake_secret_token = "coneofsilence"

fake_db = {
    "foo": {"id": "foo", "title": "Foo", "description":
"There goes my hero"},
    "bar": {"id": "bar", "title": "Bar", "description":
"The bartenders"},
}

app = FastAPI()

class Item(BaseModel):
    id: str
    title: str
    description: str | None = None

@app.get("/items/{item_id}", response_model=Item)
```

```
async def read_main(item_id: str, x_token: Annotated[str,
Header()]):
    if x_token != fake_secret_token:
        raise HTTPException(status_code=400,
detail="Invalid X-Token header")
    if item_id not in fake_db:
        raise HTTPException(status_code=404, detail="Item
not found")
    return fake_db[item_id]

@app.post("/items/")
async def create_item(item: Item, x_token: Annotated[str,
Header()]) -> Item:
    if x_token != fake_secret_token:
        raise HTTPException(status_code=400,
detail="Invalid X-Token header")
    if item.id in fake_db:
        raise HTTPException(status_code=409, detail="Item
already exists")
    fake_db[item.id] = item.model_dump()
    return item
```

Archivo de prueba extendido

Podrías entonces actualizar `test_main.py` con las pruebas extendidas:

```
from fastapi.testclient import TestClient

from .main import app

client = TestClient(app)

def test_read_item():
    response = client.get("/items/foo", headers={"X-
Token": "coneofsilence"})
    assert response.status_code == 200
    assert response.json() == {
        "id": "foo",
        "title": "Foo",
```

```
        "description": "There goes my hero",
    }

def test_read_item_bad_token():
    response = client.get("/items/foo", headers={"X-
Token": "hailhydra"})
    assert response.status_code == 400
    assert response.json() == {"detail": "Invalid X-Token
header"}

def test_read_nonexistent_item():
    response = client.get("/items/baz", headers={"X-
Token": "coneofsilence"})
    assert response.status_code == 404
    assert response.json() == {"detail": "Item not found"}

def test_create_item():
    response = client.post(
        "/items/",
        headers={"X-Token": "coneofsilence"},
        json={"id": "foobar", "title": "Foo Bar",
"description": "The Foo Barters"},
    )
    assert response.status_code == 200
    assert response.json() == {
        "id": "foobar",
        "title": "Foo Bar",
        "description": "The Foo Barters",
    }

def test_create_item_bad_token():
    response = client.post(
        "/items/",
        headers={"X-Token": "hailhydra"},
        json={"id": "bazz", "title": "Bazz",
"description": "Drop the bazz"},
    )
    assert response.status_code == 400
    assert response.json() == {"detail": "Invalid X-Token
header"}

def test_create_existing_item():
```

```
    response = client.post(
        "/items/",
        headers={"X-Token": "coneofsilence"},
        json={
            "id": "foo",
            "title": "The Foo ID Stealers",
            "description": "There goes my stealer",
        },
    )
    assert response.status_code == 409
    assert response.json() == {"detail": "Item already
exists"}
```

Cada vez que necesites que el cliente pase información en el request y no sepas cómo, puedes buscar (Googlear) cómo hacerlo en `httpx`, o incluso cómo hacerlo con `requests`, dado que el diseño de HTTPX está basado en el diseño de Requests.

Luego simplemente haces lo mismo en tus pruebas.

Por ejemplo:

- Para pasar un parámetro de *path* o *query*, añádelo a la URL misma.
- Para pasar un cuerpo JSON, pasa un objeto de Python (por ejemplo, un `dict`) al parámetro `json`.
- Si necesitas enviar *Form Data* en lugar de JSON, usa el parámetro `data` en su lugar.
- Para pasar *headers*, usa un `dict` en el parámetro `headers`.
- Para *cookies*, un `dict` en el parámetro `cookies`.

Para más información sobre cómo pasar datos al backend (usando `httpx` o el `TestClient`) revisa la documentación de HTTPX.

Nota

Ten en cuenta que el `TestClient` recibe datos que pueden ser convertidos a JSON, no modelos de Pydantic.

Si tienes un modelo de Pydantic en tu prueba y quieres enviar sus datos a la aplicación durante las pruebas, puedes usar el

`jsonable_encoder` descrito en Codificador Compatible con JSON.

Ejecútalo

Después de eso, solo necesitas instalar `pytest`.

Añádelo a tu proyecto:

```
uv add pytest
```

Detectará los archivos y pruebas automáticamente, ejecutará las mismas y te reportará los resultados.

Ejecuta las pruebas con:

```
uv run pytest
```

- ## Depuración

Puedes conectar el depurador en tu editor, por ejemplo con Visual Studio Code o PyCharm.

Llama a uvicorn

En tu aplicación de FastAPI, importa y ejecuta `uvicorn` directamente:

```
import uvicorn
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    a = "a"
    b = "b" + a
    return {"hello world": b}
```

```
if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Acerca de __name__ == "__main__"

El objetivo principal de `__name__ == "__main__"` es tener algo de código que se ejecute cuando tu archivo es llamado con:

pero no es llamado cuando otro archivo lo importa, como en:

```
from myapp import app
```

Más detalles

Supongamos que tu archivo se llama `myapp.py`.

Si lo ejecutas con:

entonces la variable interna `__name__` en tu archivo, creada automáticamente por Python, tendrá como valor el string `"__main__"`.

Así que, la sección:

```
uvicorn.run(app, host="0.0.0.0", port=8000)
```

se ejecutará.

Esto no ocurrirá si importas ese módulo (archivo).

Entonces, si tienes otro archivo `importer.py` con:

```
from myapp import app

# Algún código adicional
```

en ese caso, la variable creada automáticamente `__name__` dentro de `myapp.py` no tendrá el valor `"__main__"`.

Así que, la línea:

```
uvicorn.run(app, host="0.0.0.0", port=8000)
```

no se ejecutará.

> **Nota**
> Para más información, revisa la documentación oficial de Python.

Ejecuta tu código con tu depurador

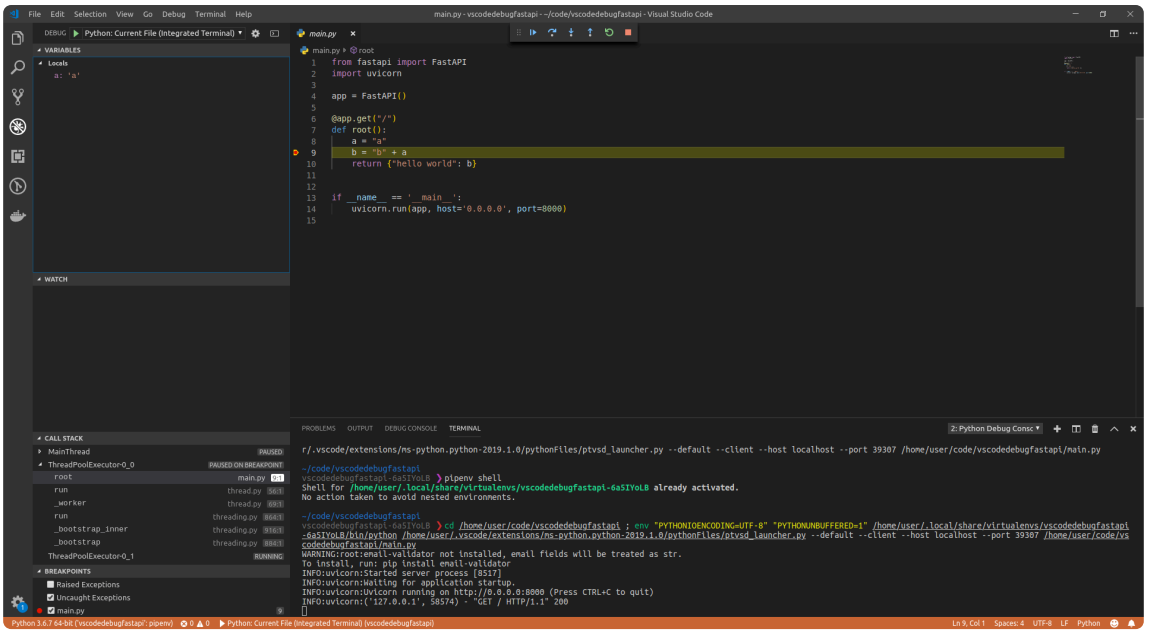
Dado que estás ejecutando el servidor Uvicorn directamente desde tu código, puedes llamar a tu programa de Python (tu aplicación FastAPI) directamente desde el depurador.

Por ejemplo, en Visual Studio Code, puedes:

- Ir al panel de "Debug".
- "Add configuration...".
- Seleccionar "Python"
- Ejecutar el depurador con la opción "`Python: Current File (Integrated Terminal)`".

Luego, iniciará el servidor con tu código **FastAPI**, deteniéndose en tus puntos de interrupción, etc.

Así es como podría verse:

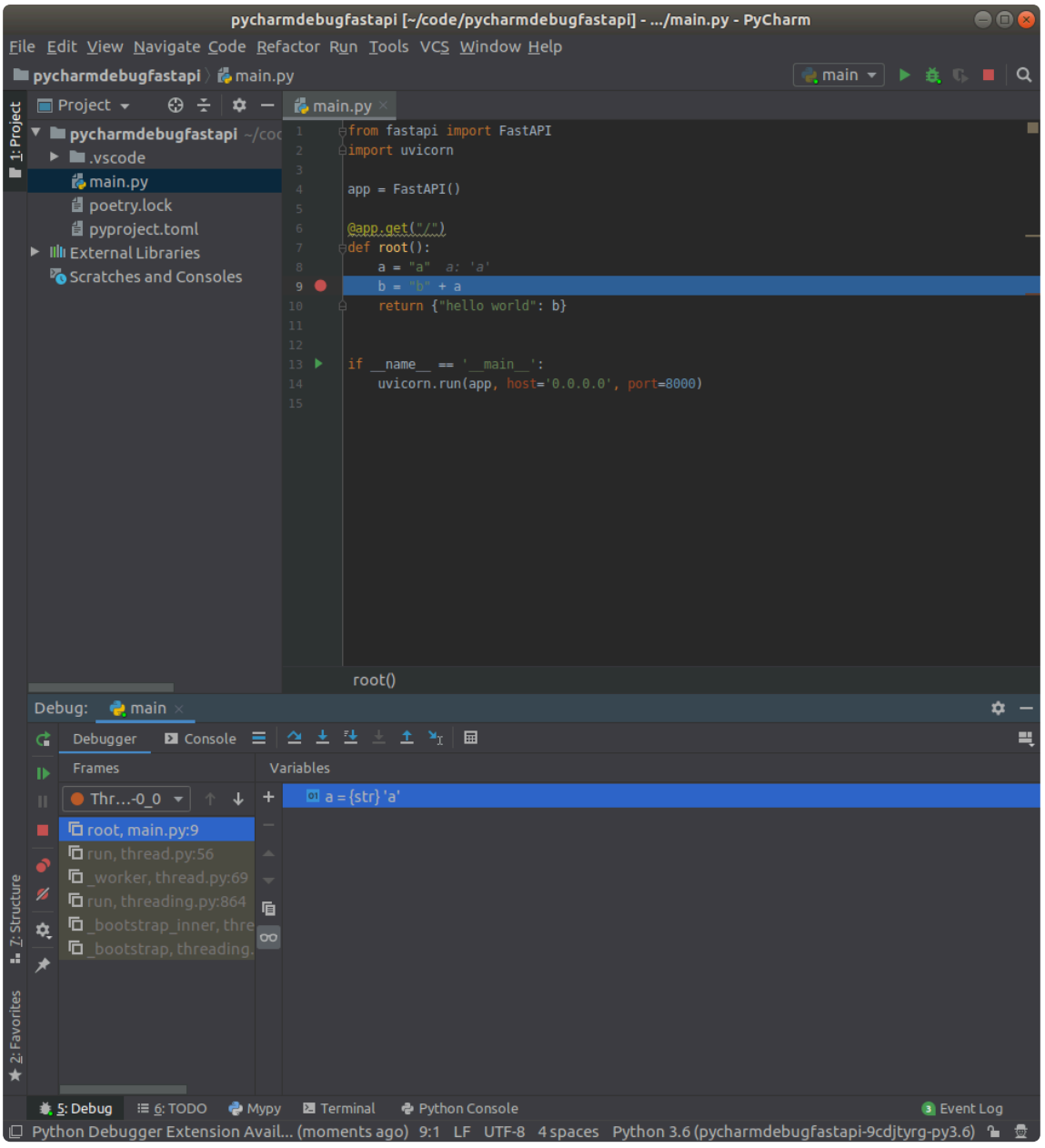


Si usas PyCharm, puedes:

- Abrir el menú "Run".
- Seleccionar la opción "Debug...".
- Luego aparece un menú contextual.
- Selecciona el archivo para depurar (en este caso, `main.py`).

Luego, iniciará el servidor con tu código **FastAPI**, deteniéndose en tus puntos de interrupción, etc.

Así es como podría verse:


pycharmdebugfastapi [~/code/pycharmdebugfastapi] - .../main.py - PyCharm
File Edit View Navigate Code Refactor Run Tools VCS Window Help
pycharmdebugfastapi main.py
main
Project
main.py
pycharmdebugfastapi ~/coc
.vscode
main.py
poetry.lock
pyproject.toml
External Libraries
Scratches and Consoles
1 from fastapi import FastAPI
2 import uvicorn
3
4 app = FastAPI()
5
6 @app.get("/")
7 def root():
8 a = "a" a: 'a'
9 b = "b" + a
10 return {"hello world": b}
11
12
13 if __name__ == '__main__':
14 uvicorn.run(app, host='0.0.0.0', port=8000)
15
root()
Debug: main
Debugger Console
Frames
Thr...-0_0
root, main.py:9
run, thread.py:56
_worker, thread.py:69
run, threading.py:864
_bootstrap_inner, thre
_bootstrap, threading.
Variables
a = {str} 'a'
1: Project
7: Structure
2: Favorites
5: Debug
6: TODO
Mypy
Terminal
Python Console
Event Log
Python Debugger Extension Avail... (moments ago) 9:1 LF UTF-8 4 spaces Python 3.6 (pycharmdebugfastapi-9cdjtyrg-py3.6)